第1章 Python語言與運算思維基礎

- 1-1 程式與程式邏輯
- 1-2 Python語言、運算思維和Thonny
- 1-3 下載與安裝Thonny
- 1-4 使用Thonny建立第一個Python程式
- 1-5 Thonny基本使用與程式除錯



1-1 程式與程式邏輯

- 1-1-1 認識程式與程式設計
- 1-1-2 程式邏輯的基礎

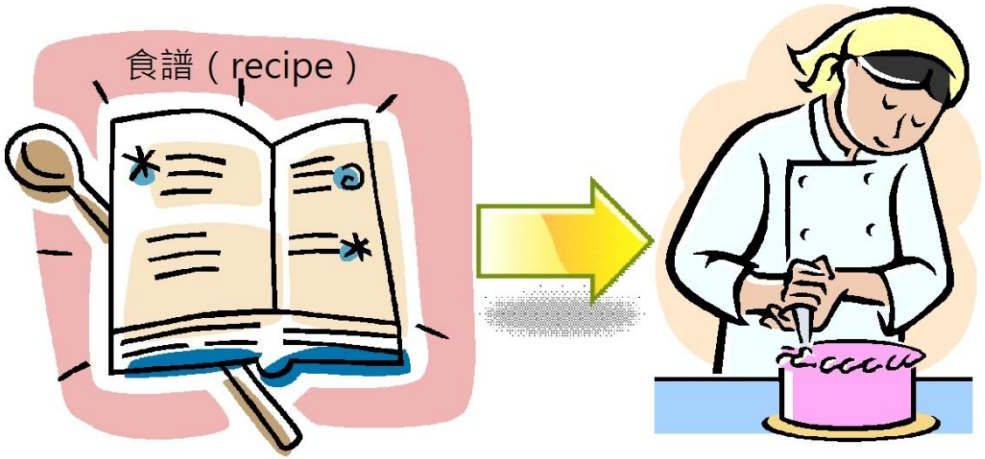


1-1-1 認識程式與程式設計

- 從太陽昇起的一天開始，手機鬧鐘響起叫你起床，順手查看LINE或在Facebook按讚，上課前交作業寄送電子郵件、打一篇文章，或休閒時玩玩遊戲，想想看，你有哪一天沒有作這些事。
- 這些事就是在執行「程式」（Programs）或稱為「電腦程式」（Computer Programs），不要懷疑，程式早已融入你的生活，而且在日常生活中，大部分人也早已無法離開程式。

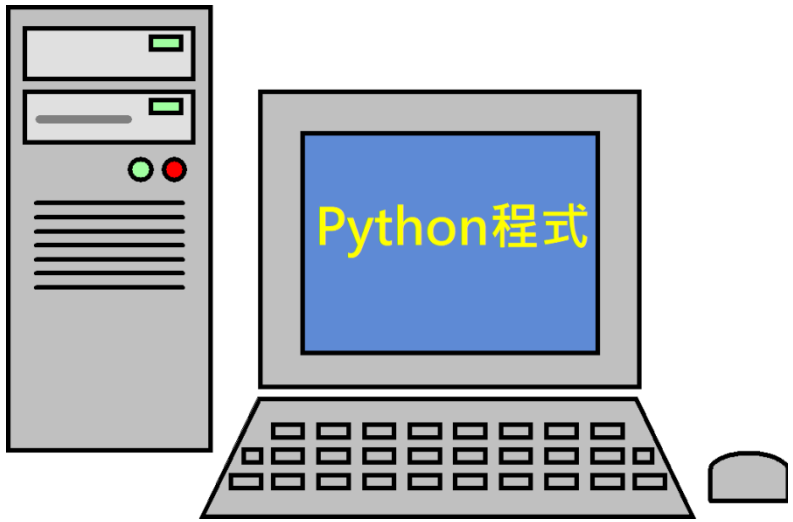
1-1-1 認識程式與程式設計

- 電腦是一種硬體（Hardware）；程式是軟體（Software），我們需要透過程式的軟體來指示電腦作什麼事，例如：打卡、按讚和回應LINE等，電腦的工作就是正確執行程式來完成我們所需的工作，如同烘焙蛋糕的食譜（Recipe），可以告訴我們製作蛋糕的步驟，如下圖所示：



1-1-1 認識程式與程式設計

- 程式就是電腦的食譜，可以下達指令告訴電腦如何打卡、按讚、回應LINE、收發電子郵件、打一篇文章或玩遊戲。而程式設計（Programming）的主要工作，就是在建立電腦可以執行的程式，在本書是建立電腦上執行的Python程式，如下圖所示：



1-1-2 程式邏輯的基礎

- 我們使用程式語言的目的是撰寫程式碼來建立程式，所以需要使用電腦的程式邏輯（Program Logic）來撰寫程式碼，如此電腦才能執行程式碼來解決我們的問題，因為電腦才是真正的「目標執行者」（Target Executer），負責執行你寫的程式；並不是你的大腦在執行程式。
- 讀者可能會問撰寫程式碼執行程式設計（Programming）很困難嗎？事實上，如果你能夠一步一步詳細列出活動流程、引導問路人到達目的地、走迷宮、使用自動購票機買票或從地圖上找出最短路徑，就表示你一定可以撰寫程式碼。

1-1-2 程式邏輯的基礎

人類的邏輯：目標執行者是人類

- 因為目標執行者是人類，對於人類來說，只需檢視地圖，即可輕鬆寫下開車從高速公路北上到台北市大安森林公園的步驟，如下所示：
  - Step 1：中山高速公路向北開。
  - Step 2：下圓山交流道（建國高架橋）。
  - Step 3：下建國高架橋（仁愛路）。
  - Step 4：直行建國南路，在紅綠燈右轉仁愛路。
  - Step 5：左轉新生南路。

1-1-2 程式邏輯的基礎

電腦的程式邏輯：目標執行者是電腦

- 對於目標執行者電腦來說，人類邏輯的步驟描述太不明確，我們需要提供更多資訊給電腦，才能讓電腦開車到達目的地，如下所示：
  - 從哪裡開始開車（起點）？中山高速公路需向北開幾公里到達圓山交流道？
  - 如何分辨已經到了圓山交流道？如何從交流道下來？
  - 在建國高架橋上開幾公里可以到達仁愛路出口？如何下去？
  - 直行建國南路幾公里可以看到紅綠燈？左轉或右轉？
  - 開多少公里可以看到新生南路？如何左轉？接著需要如何開？如何停車？

1-2 認識Python、運算思維和Thonny

- 1-2-1 談談運算思維與演算法
- 1-2-2 認識Python語言
- 1-2-3 Thonny整合開發環境



1-2-1 談談運算思維與演算法-運算思維

- 基本上，運算思維可以分成五大領域，如下所示：
  - 抽象化（abstraction）：思考不同層次的問題解決步驟。
  - 演算法（algorithms）：將解決問題的工作思考成一序列可行且有限的步驟。
  - 分割問題（decomposition）：了解在處理大型問題時，我們需要將大型問題分割成小問題的集合，然後各個擊破來一一解決。
  - 樣式識別（pattern recognition）：察覺新問題是否和之前已解決問題之間擁有關係，可以讓我們直接使用已知或現成的解決方法來解決問題。
  - 歸納（generalization）：了解已解決的問題可能是用來解決其他或更大範圍問題的關鍵。

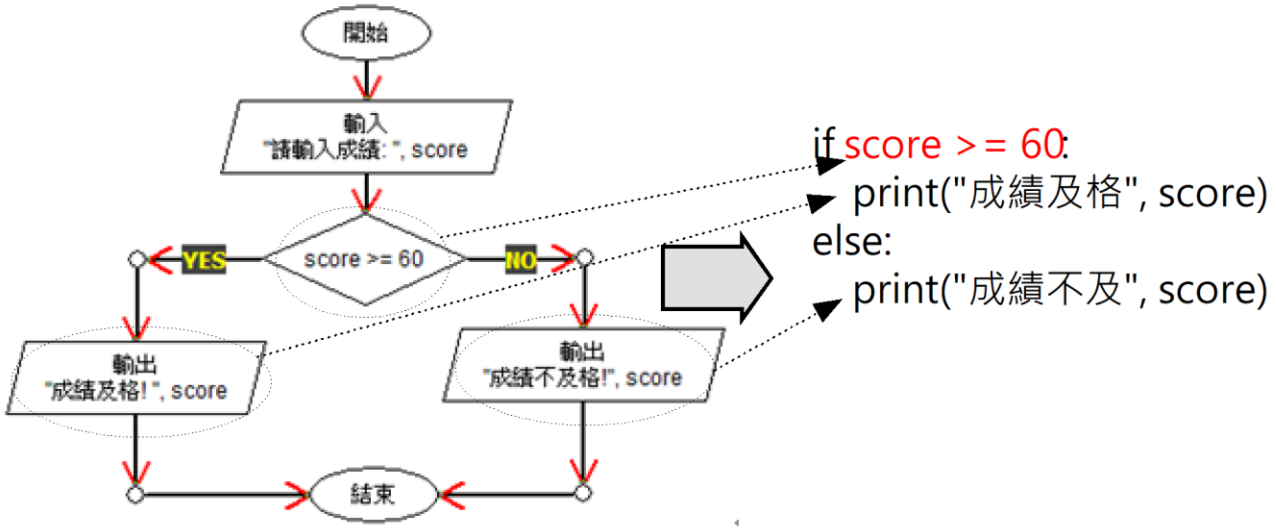
1-2-1 談談運算思維與演算法-演算法

- 演算法（Algorithms）簡單的說就是一張食譜（Recipe），提供一組一步接著一步（Step-by-step）的詳細過程，包含動作和順序，可以將食材烹調成美味的食物，例如：在第1-1-1節說明的蛋糕製作，製作蛋糕的食譜就是一個演算法，如下圖所示：



1-2-1 談談運算思維與演算法-使用流程圖描述演算法

- 演算法可以使用文字描述或圖形化方式來描述，圖形化方式就是流程圖（Flow Chart），流程圖是使用標準圖示符號來描述執行過程，以各種不同形狀的圖示表示不同的操作，箭頭線標示流程執行的方向，當畫出流程圖的執行過程後，就可以轉換撰寫成特定語言的程式碼，例如：Python語言，如下圖所示：



1-2-2 認識Python語言

- Python語言是Guido Van Rossum開發的一種通用用途（General Purpose）的程式語言，這是擁有優雅語法和高可讀性程式碼的程式語言，可以讓我們開發GUI視窗程式、Web應用程式、系統管理工作、財務分析、大數據資料分析和人工智慧等各種不同的應用程式。
- Python語言兩個版本：Python 2和Python 3，在本書說明的是Python 3語言。

1-2-3 Thonny整合開發環境

- Thonny是愛沙尼亞Tartu大學開發，一套完全針對「初學者」開發的免費Python整合開發環境，其主要特點如下所示：
  - Thonny支援Python和MicroPython語言。
  - Thonny支援自動程式碼完成和括號提示，可以幫助初學者輸入正確的Python程式碼。
  - Thonny使用即時高亮度提示程式碼錯誤，並且提供協助說明和程式碼除錯，可以讓我們一步一步執行程式碼來進行程式除錯。

1-3 下載與安裝Thonny

- Thonny跨平台支援Windows、MacOS和Linux作業系統，可以在Thonny官方網站免費下載最新版本（Thonny本身就是使用Python開發）。
- 方法一：在官網自行下載和安裝Thonny
- 方法二：下載安裝本書客製化WinPython可攜式套件

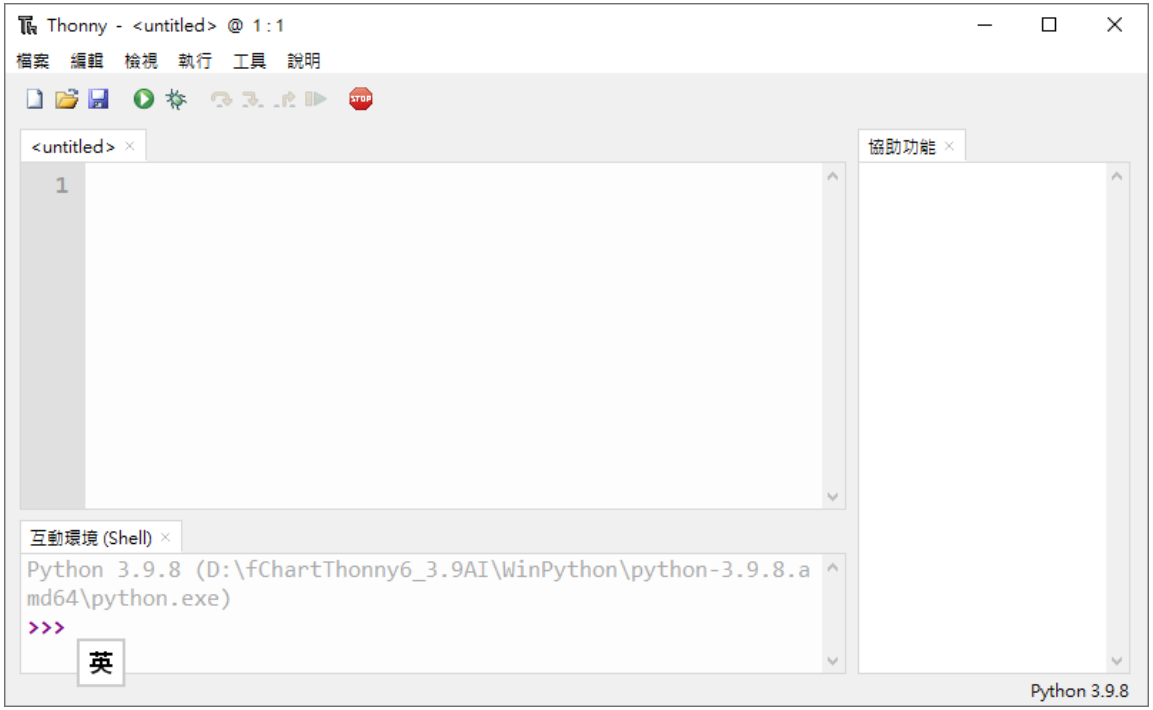
1-4 使用Thonny建立第一個Python程式

- 1-4-1 建立第一個Python程式
- 1-4-2 使用Python互動環境



1-4-1 建立第一個Python程式-Step 1

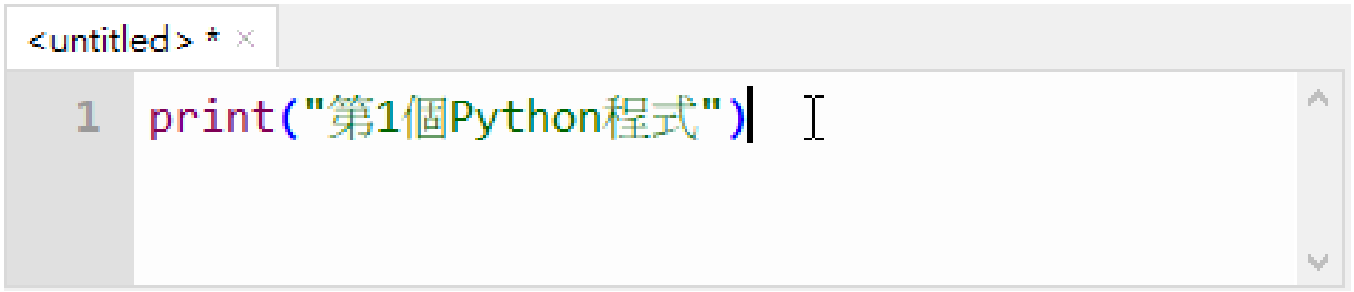
- Step 1：請在fChart主選單執行【Thonny Python IDE】命令（自行安裝請執行「開始→Thonny → Thonny」命令或桌面【Thonny】捷徑），即可啟動Thonny開發環境看到簡潔的開發介面。



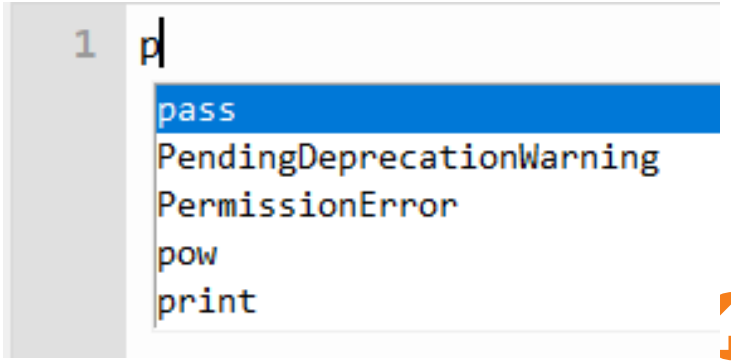
1-4-1 建立第一個Python程式-Step 2

- Step 2：在編輯器的【<untitled>】標籤輸入第一個Python程式，如下所示：

  print("第1個Python程式")

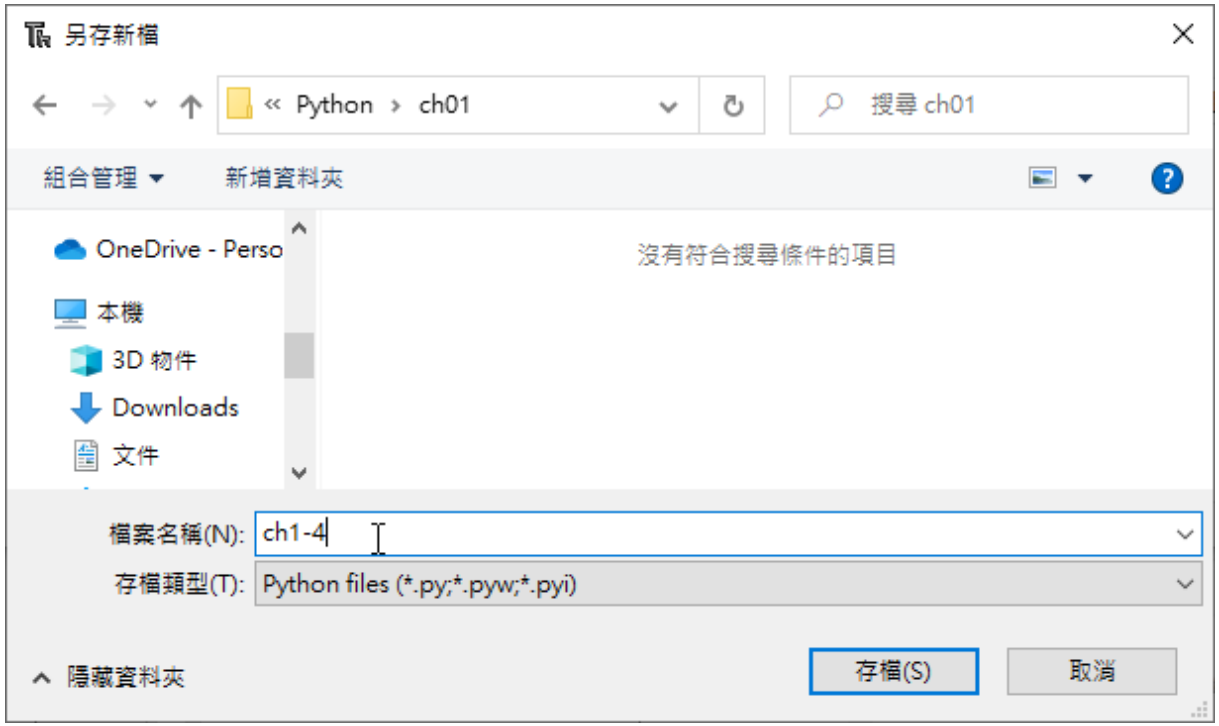


Thonny支援自動程式碼完成，請輸入p後按Tab鍵啟用，有時需等一下，如右所示：



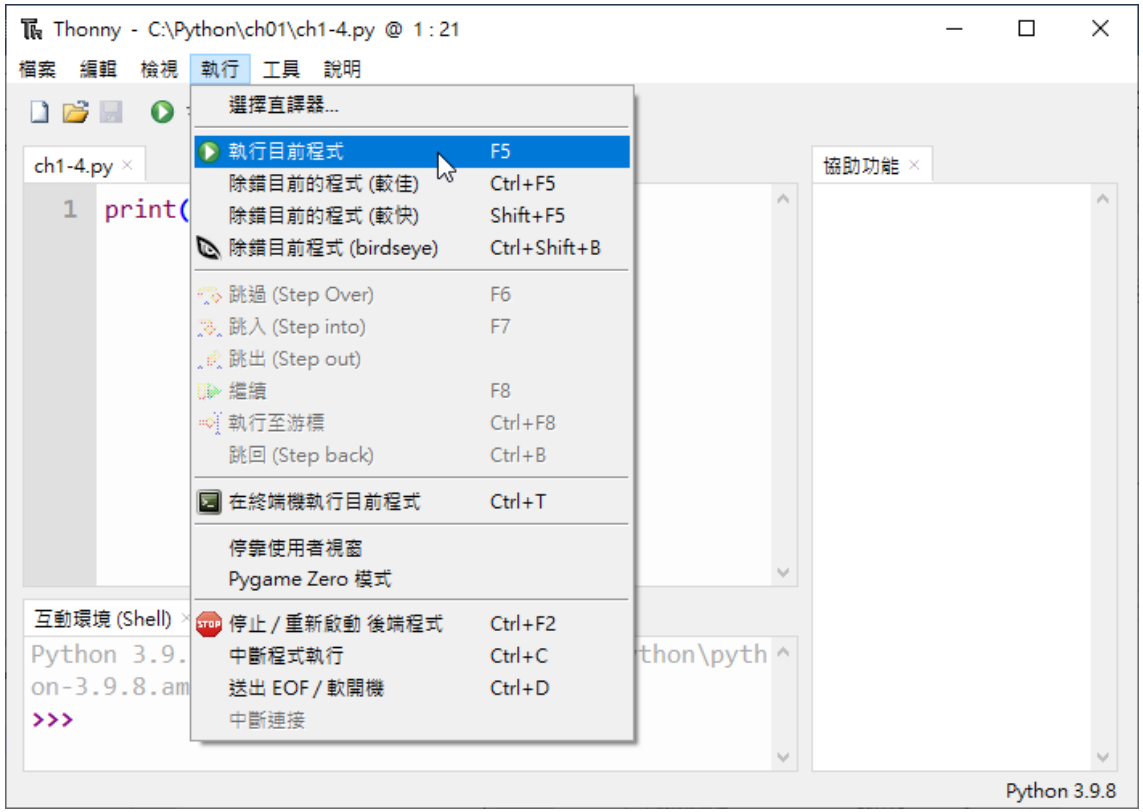
1-4-1 建立第一個Python程式-Step 3

- Step 3：執行「檔案→儲存檔案」命令或按工具列的【儲存檔案】鈕，可以看到「另存新檔」對話方塊，請切換至「/Python/ch01」目錄，輸入【ch1-4】，按【存檔】鈕儲存成ch1-4.py程式。



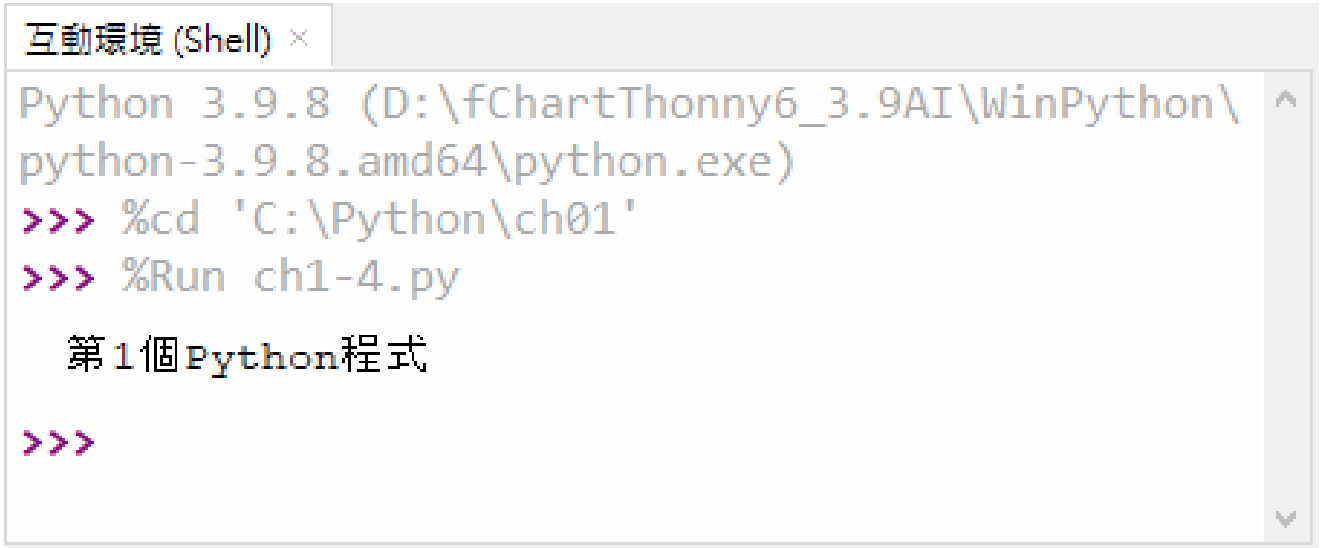
1-4-1 建立第一個Python程式-Step 4

- Step 4：可以看到標籤名稱已經改成檔案名稱，然後執行「執行→執行目前程式」命令，或按工具列綠色箭頭圖示的【執行目前程式】鈕（也可按F5鍵）來執行Python程式。



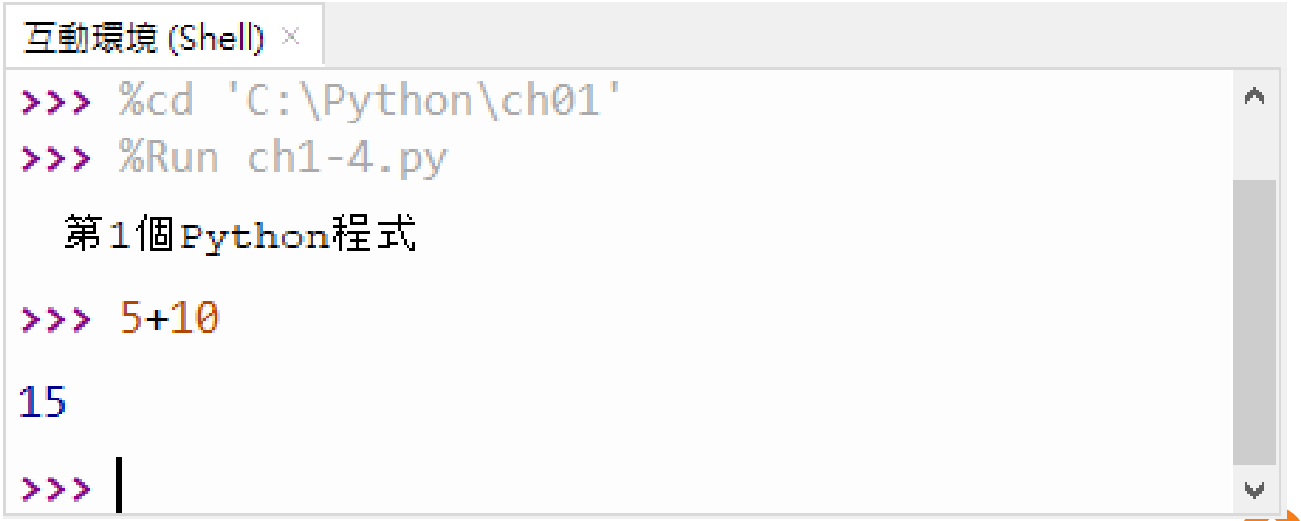
1-4-1 建立第一個Python程式-Step 5

- Step 5：可以在下方「互動環境 (Shell)」視窗看到Python程式的執行結果。



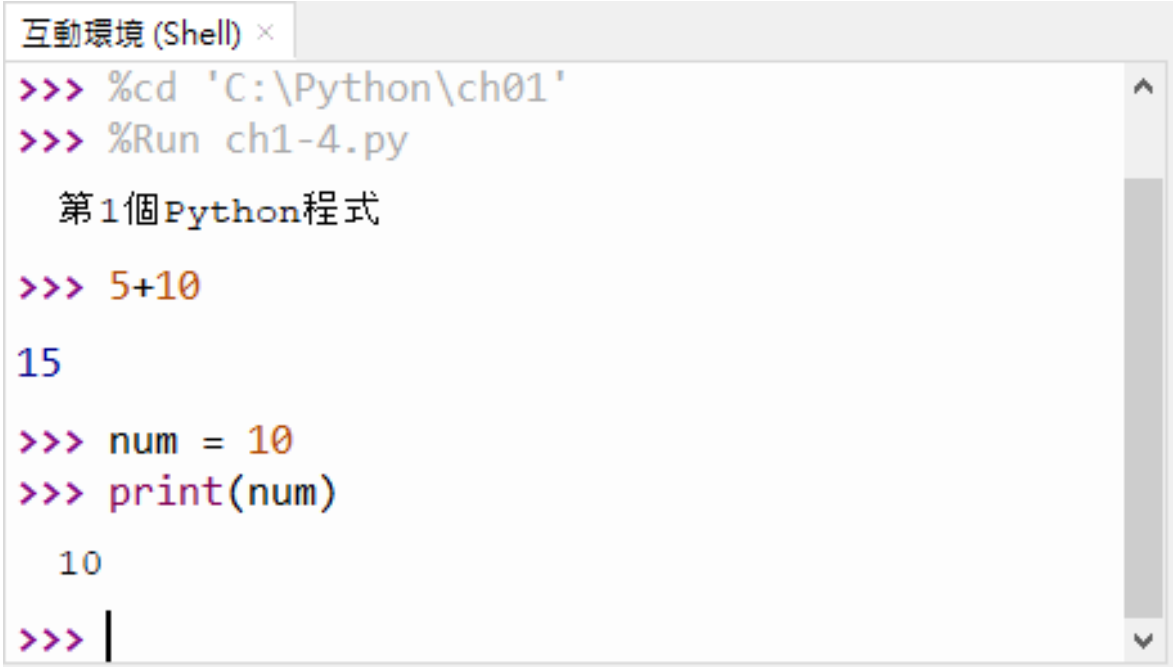
1-4-2 使用Python互動環境

- 在Thonny開發介面下方的「互動環境 (Shell)」視窗就是REPL交談模式，REPL（Read-Eval-Print Loop）是循環「讀取-評估-輸出」的互動程式開發環境，可以讓我們直接在「>>>」提示文字後輸入Python程式碼來馬上執行程式碼，例如：輸入5+10，按Enter鍵，可以馬上看到執行結果15，如下圖所示：



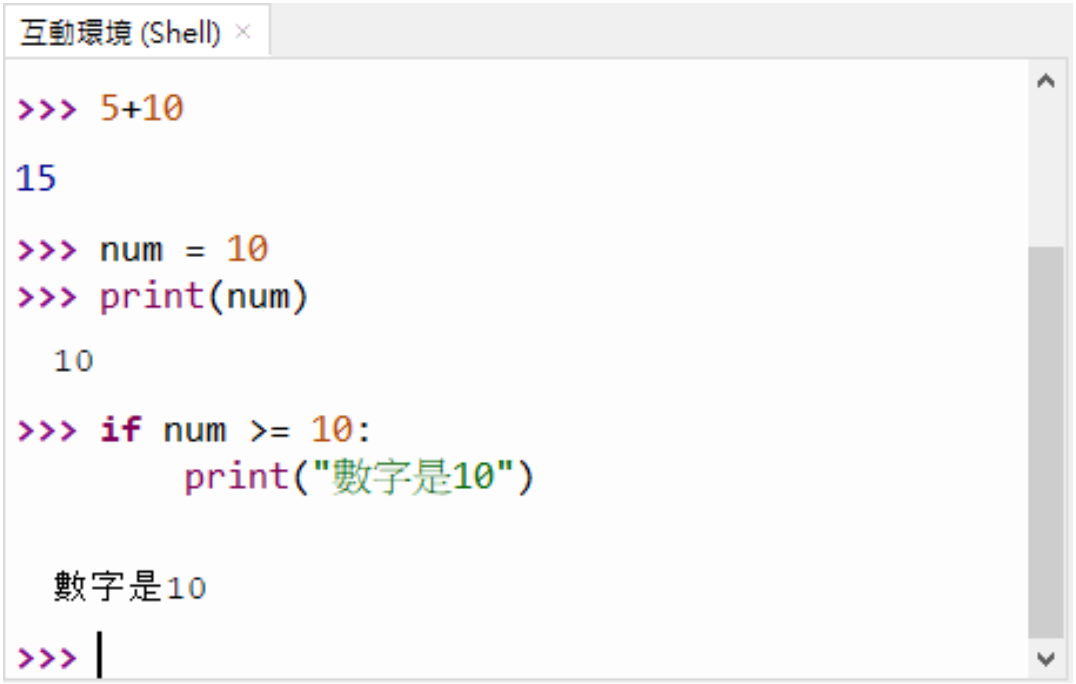
1-4-2 使用Python互動環境

- 同樣的，我們可以定義變數num = 10，然後輸入print()函式來顯示變數num的值，如下圖所示：



1-4-2 使用Python互動環境

- 如果是輸入程式區塊，例如：if條件敘述，請在輸入if num >= 10:後（最後是「:」冒號），按Enter鍵，就會換行且自動縮排4個空白字元，然後按二次Enter鍵來執行程式碼，可以看到執行結果，如下圖所示：



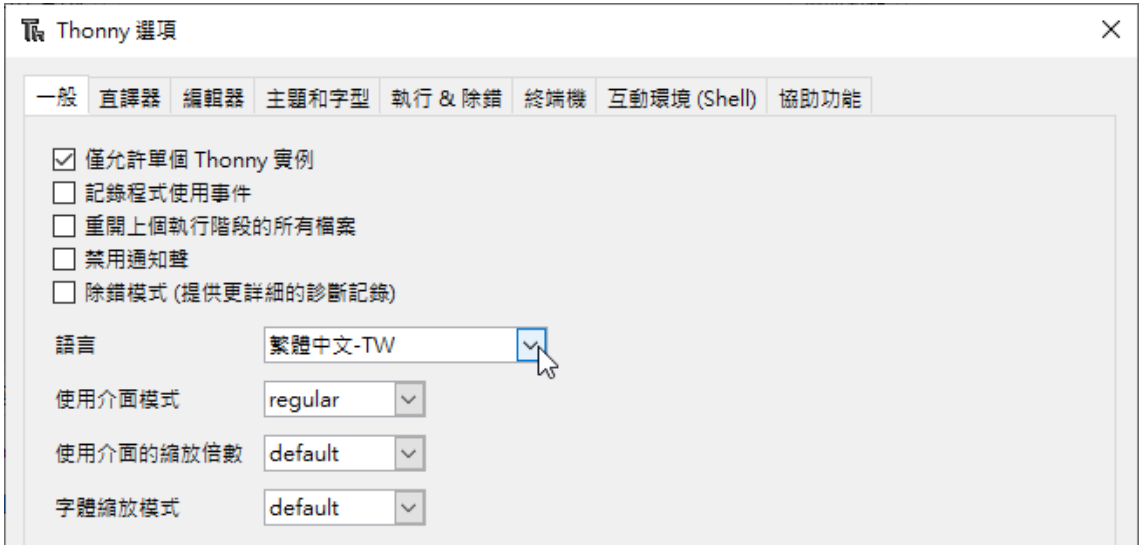
1-5 Thonny基本使用與程式除錯

- 1-5-1 更改Thonny選項
- 1-5-2 使用Thonny進行程式除錯



1-5-1 更改Thonny選項

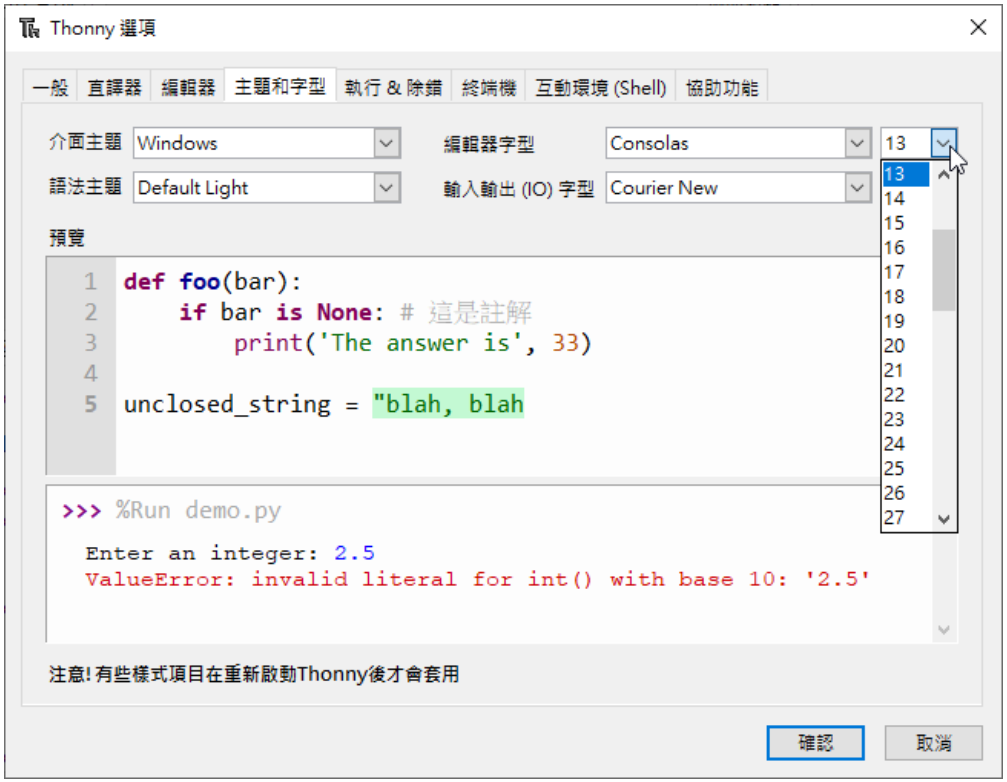
- 當啟動Thonny後，請執行「工具→選項」命令，可以看到「Thonny選項」對話方塊，請切換標籤來設定所需的選項。
- 切換Thonny介面的語言
- 在【一般】標籤可以切換Thonny介面的語言，預設是【繁體中文-TW】，如下圖所示：



1-5-1 更改Thonny選項

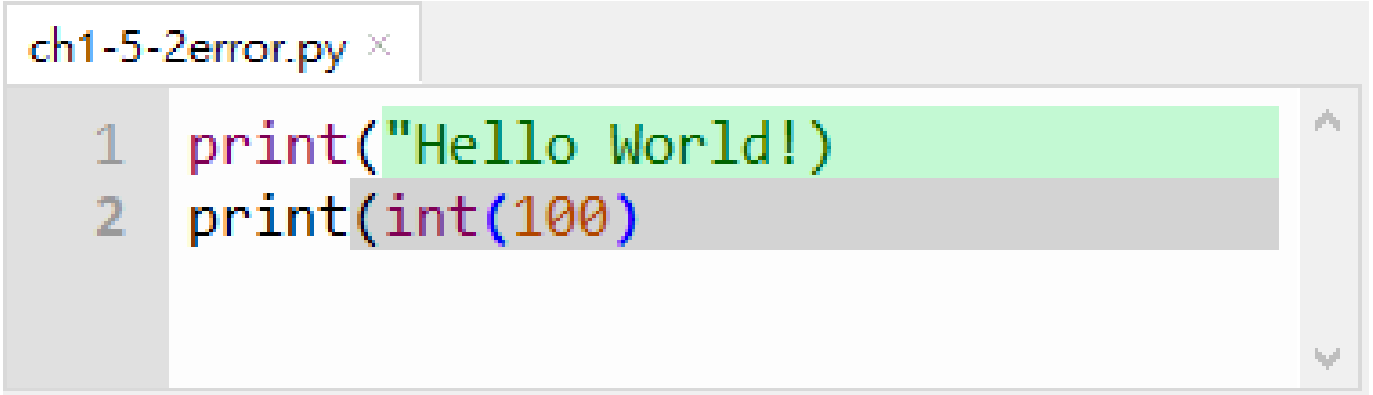
更改Thonny佈景主題和字型尺寸

- 選【主題和字型】標籤，可以更改Thonny外觀的主題和編輯器的字型與尺寸，如下圖所示：



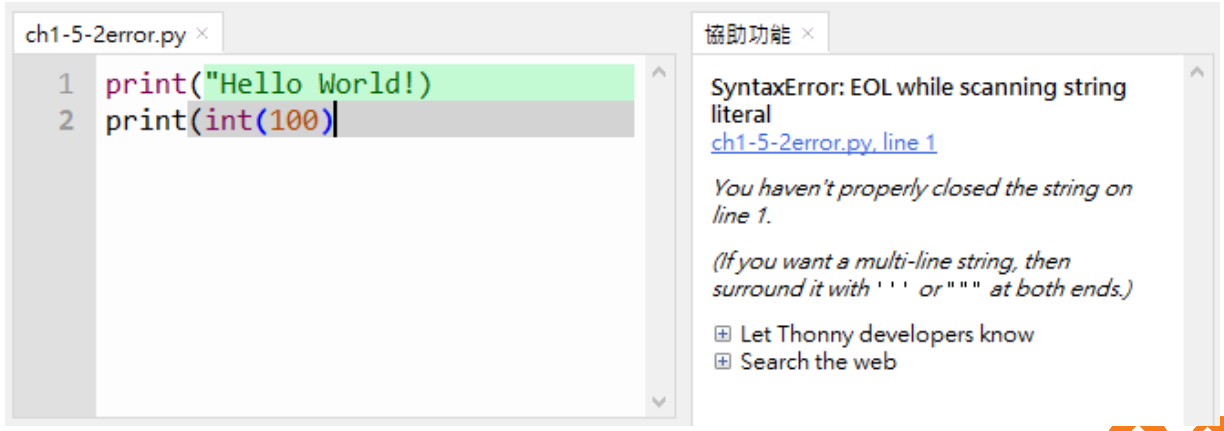
1-5-2 使用Thonny進行程式除錯
語法錯誤與協助說明

- 語法錯誤（Syntax Error）是指輸入程式碼不符合Python語法規則，例如：Python程式：ch1-5-2error.py有語法錯誤的2行程式碼，在第1行程式碼忘了最後的雙引號，Thonny是使用即時高亮度綠色來標示語法錯誤；第2行少了右括號，Thonny是使用灰色來標示，如下圖所示：



1-5-2 使用Thonny進行程式除錯
語法錯誤與協助說明

- 當執行上述語法錯誤的程式碼後，在右邊的「協助功能」視窗就會顯示語法錯誤的協助說明：You haven’t properly closed the string on line 1.（即在第1行的字串少了最後的雙引號），如果沒有看到此視窗，請執行「檢視>協助功能」命令來切換顯示此視窗。
- 請在第1行的字串最後加上雙引號後，再執行一次Python程式，可以看到「協助功能」視窗顯示第2行少了右括號，如下圖所示：



1-5-2 使用Thonny進行程式除錯

Thonny除錯器

- Thonny內建除錯器（Debugger），可以讓我們一行一行逐步執行程式碼來找出程式錯誤。例如：Python程式ch1-5-2.py可以顯示「#」號的三角形，程式需要顯示5行三角形，但執行結果只顯示4行三角形，如下所示：

```
>>> %Run ch1-5-2.py
#
##
###
####
```

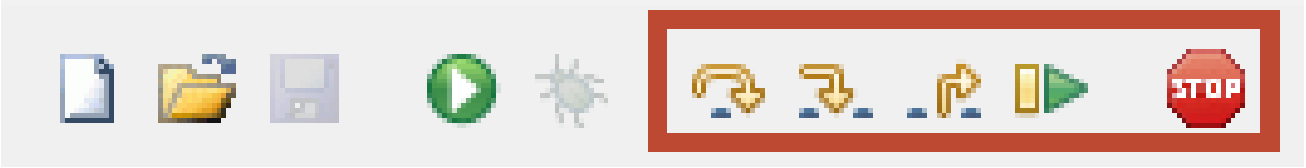
1-5-2 使用Thonny進行程式除錯

Thonny除錯器

- 在Thonny上方工具列提供除錯所需的相關按鈕，小蟲圖示鈕就是開始除錯，如下圖所示：

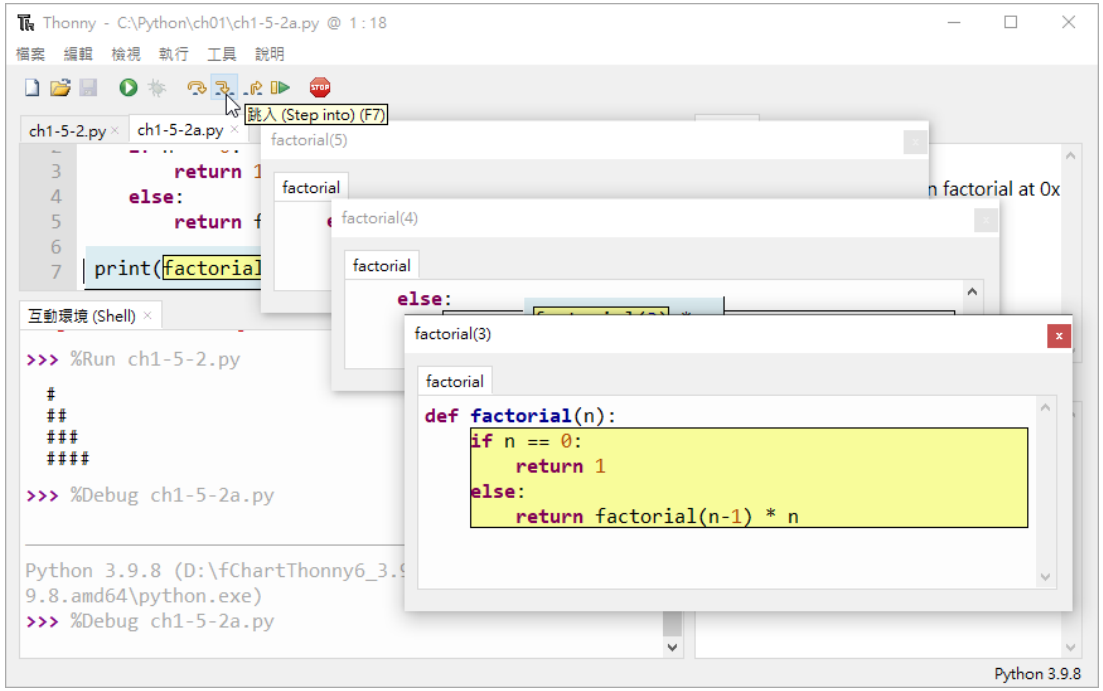


- 按下小蟲圖示鈕（或按Ctrl-F5鍵），Thonny就進入一步一步執行的除錯模式，在之後是除錯的相關按鈕，如下圖所示：



1-5-2 使用Thonny進行程式除錯 視覺化顯示函式呼叫的執行過程

- Python程式：ch1-5-2a.py的factorial()函數是遞迴階層函數（N!），當在Thonny使用除錯模式執行ch1-5-2a.py時，請持續按F7鍵，可以看到視覺化顯示整個factorial()函數的呼叫過程，首先呼叫factorial(5)函數，如下圖所示：



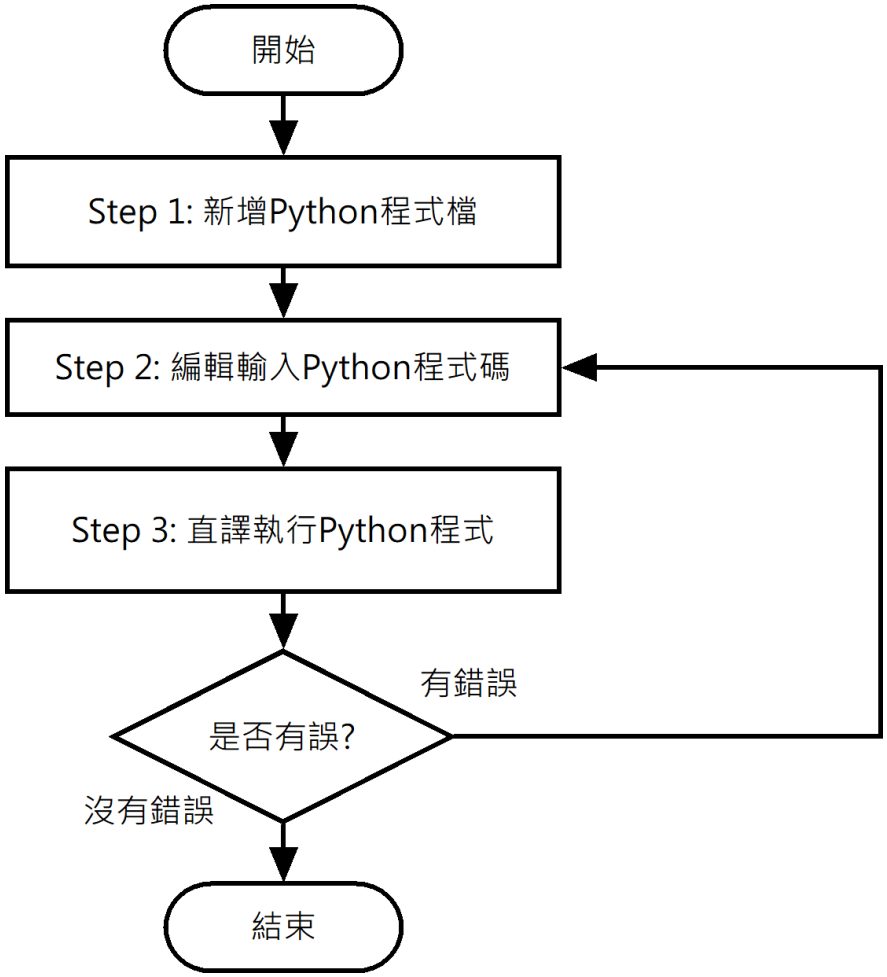
第2章 寫出和認識Python程式

- 2-1 開發Python程式的基本步驟
- 2-2 編輯現存的Python程式
- 2-3 建立第二個Python程式的加法運算
- 2-4 看看Python程式的內容
- 2-5 Python文字值
- 2-6 Python寫作風格



2-1 開發Python程式的基本步驟

- 在第1章成功建立和執行第1個Python程式後，我們可以了解使用Thonny整合開發環境開發Python程式的基本步驟，如右圖所示：



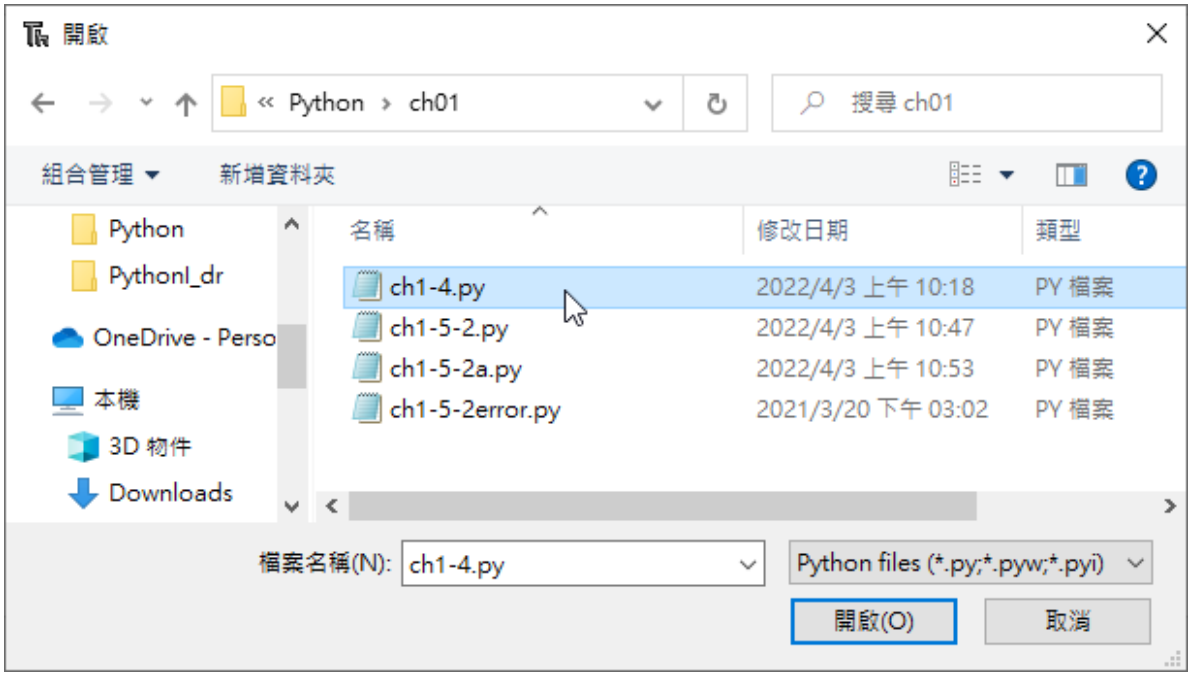
2-2 編輯現存的Python程式

- 2-2-1 編輯現存的Python程式檔
- 2-2-2 在Python程式輸出數值和字串



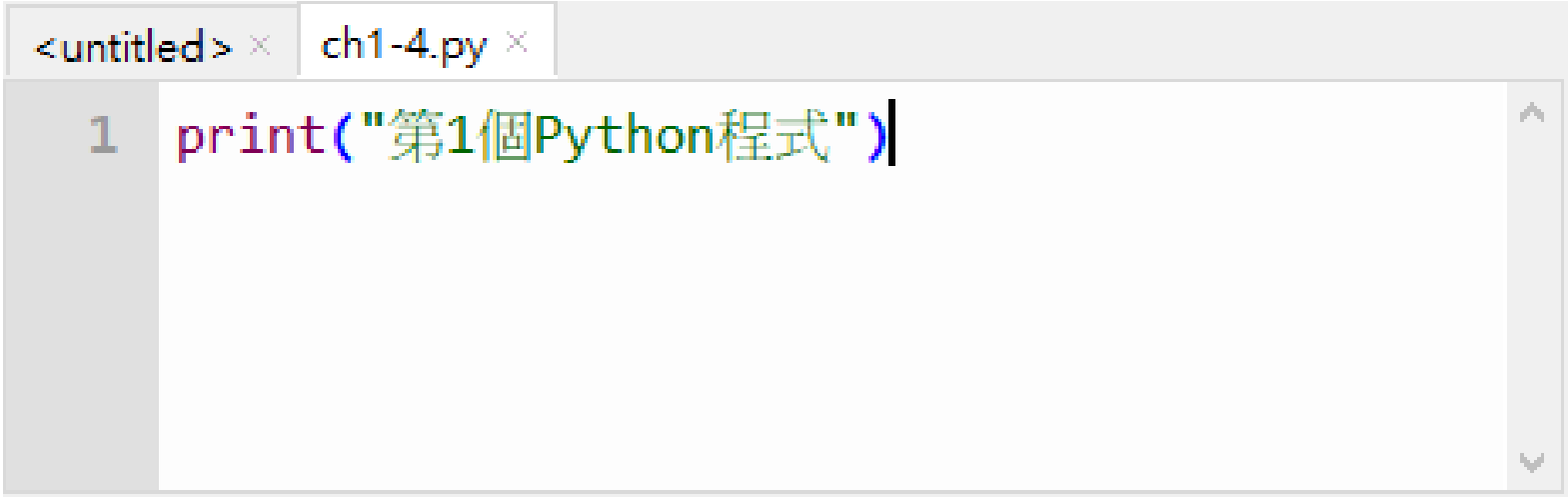
2-2-1 編輯現存的Python程式檔 另存新檔和再輸入一行新的程式

- Step 1：請啟動Thonny執行「檔案→開啟舊檔」命令，在「開啟」對話方塊切換至「/Python/ch01」目錄後，選【ch1-4.py】，按【開啟】鈕。



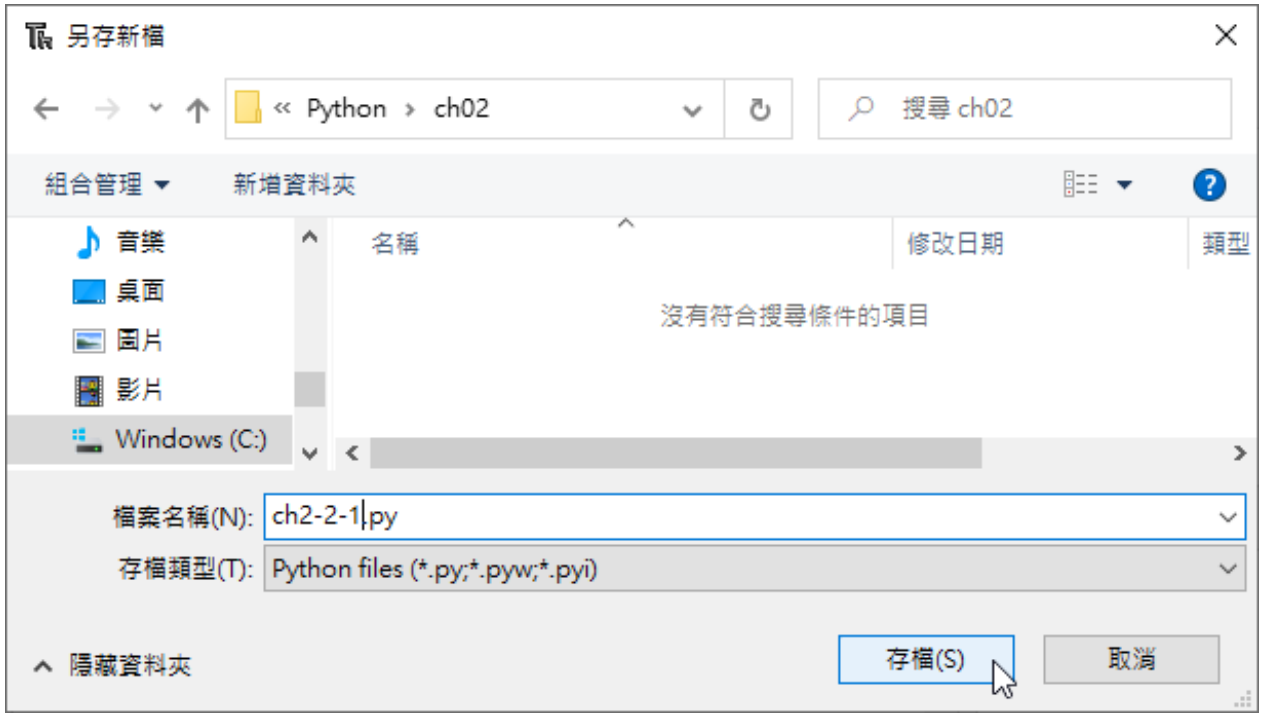
2-2-1 編輯現存的Python程式檔 另存新檔和再輸入一行新的程式

- Step 2：可以看到標籤頁顯示載入的Python程式碼檔（點選檔名標籤後的【X】圖示可關閉檔案），如下圖所示：



2-2-1 編輯現存的Python程式檔 另存新檔和再輸入一行新的程式

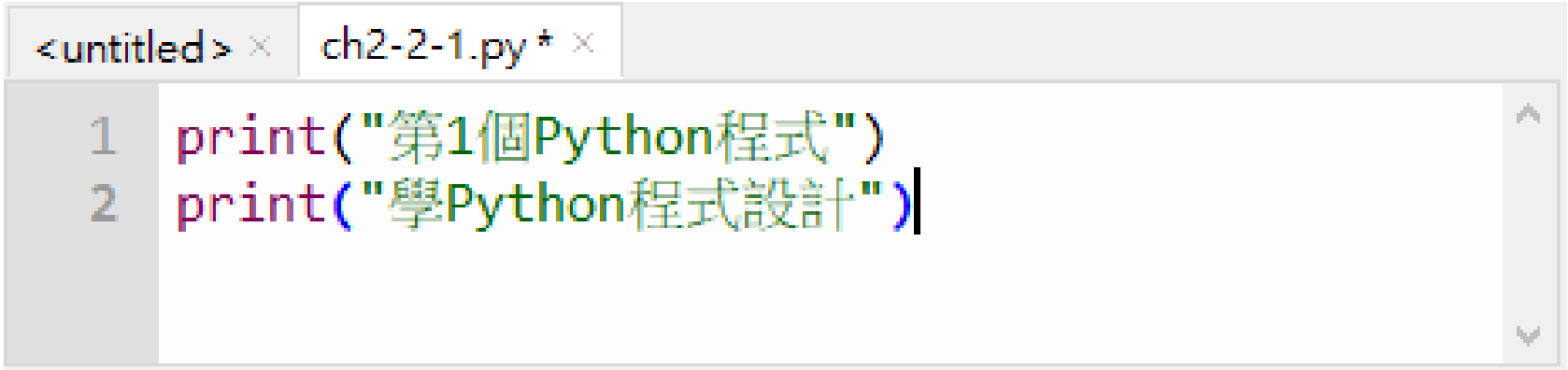
- Step 3：請執行「檔案→另存新檔」命令和切換至「ch02」目錄後，輸入檔案名稱【ch2-2-1.py】，按【存檔】鈕另存程式檔至其他目錄。



2-2-1 編輯現存的Python程式檔
另存新檔和再輸入一行新的程式

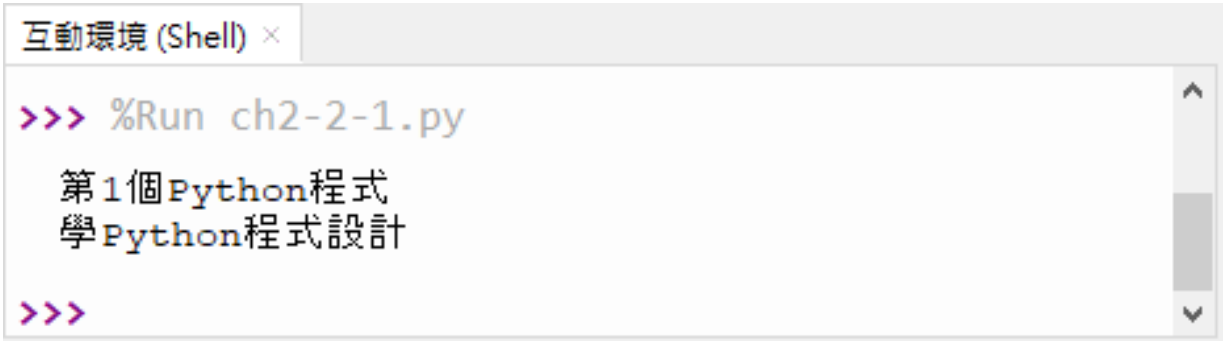
- Step 4：可以看到標籤名稱改為新檔名，接著在print()這一行最後，點選作為插入點後按Enter鍵，輸入第2行程式碼，如下所示：

  print("學Python程式設計")



2-2-1 編輯現存的Python程式檔 另存新檔和再輸入一行新的程式

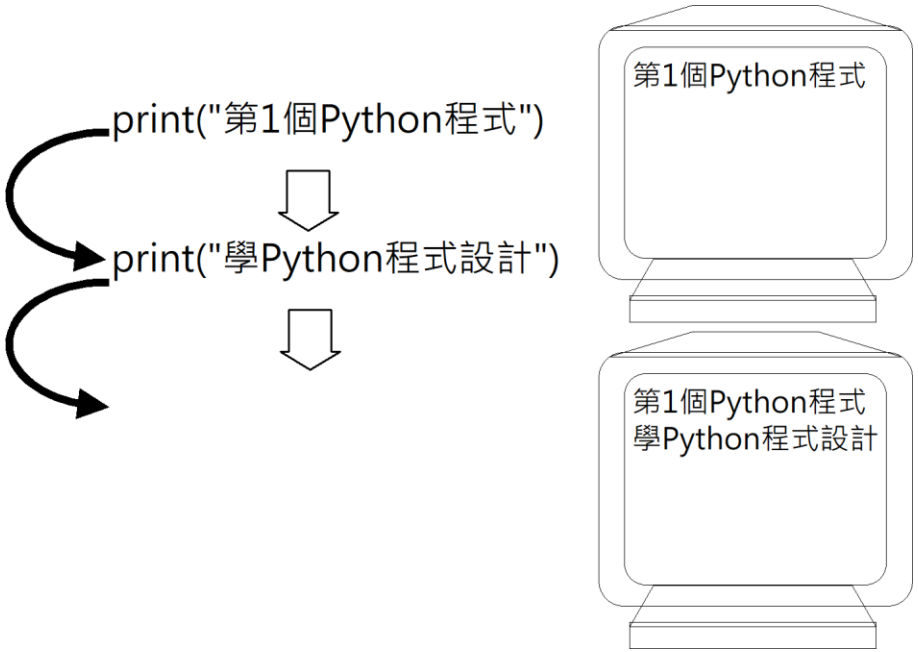
- Step 5：請執行「執行→執行目前程式」命令，或按工具列綠色箭頭圖示的【執行目前程式】鈕（也可按F5鍵）執行Python程式，可以看到執行結果，如下圖所示：



- 上述執行結果和第1章的第1個程式相同都是輸出文字內容，ch1-4.py輸出一行文字內容，本節ch2-2-1.py輸出2行文字內容，因為我們在Python程式多加了一行print()程式碼。

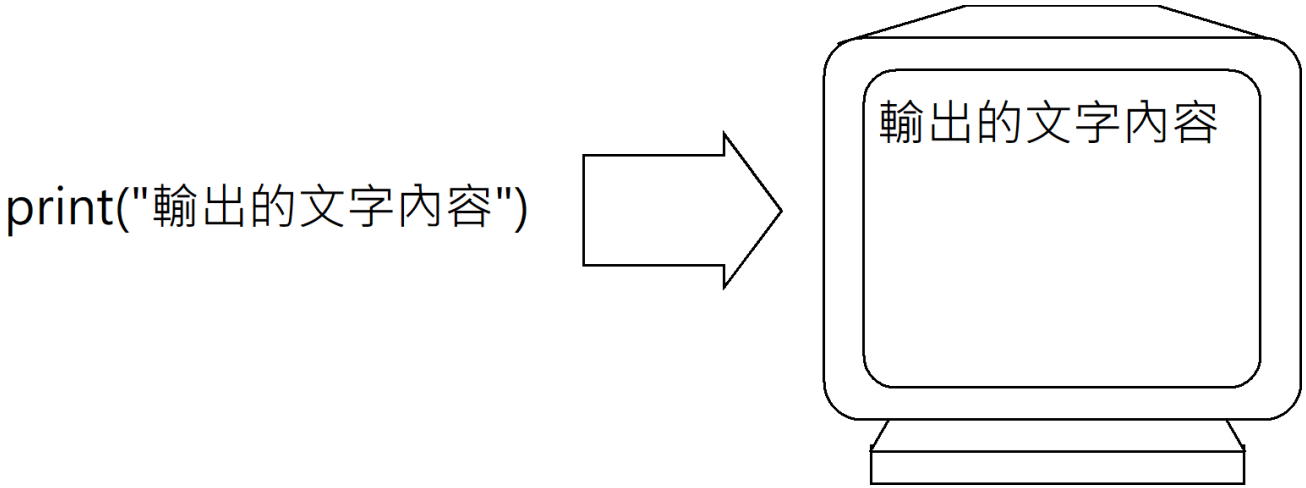
2-2-1 編輯現存的Python程式檔 循序執行

- 循序執行（Sequential Run）是電腦程式預設的執行方式，也就是一個程式敘述跟著一個程式敘述來依序的執行，在Python程式主要是使用換行來分隔程式成為一個一個程式敘述，即每一行是一個程式敘述，以ch2-2-1.py為例，共有2行2個程式敘述，如下圖所示：



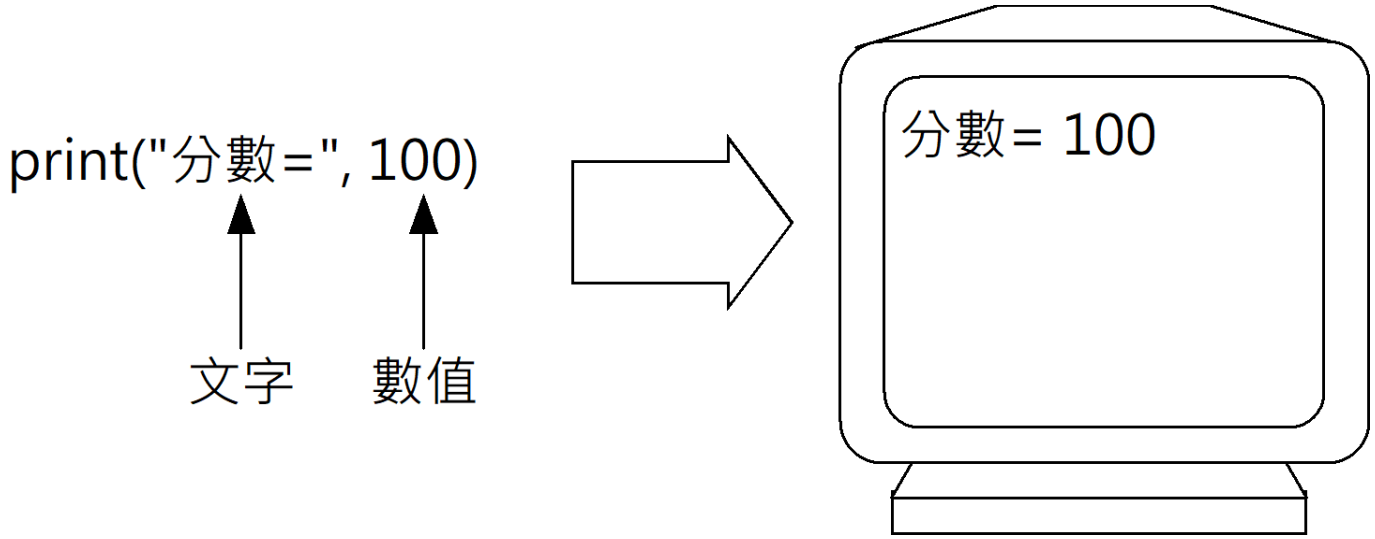
2-2-2 在Python程式輸出數值和字串

- Python程式是使用print()函數在電腦螢幕輸出執行結果的文字內容或數值，print()函數的基本語法，如下圖所示：



2-2-2 在Python程式輸出數值和字串

- 如果需要輸出數值或第3章的變數值，請使用「,」號分隔多個輸出資料，其語法如下圖所示：



2-3 建立第二個Python程式的加法運算

- 2-3-1 建立第二個Python程式
- 2-3-2 認識主控台輸出
- 2-3-3 Python主控台輸出：print()函數



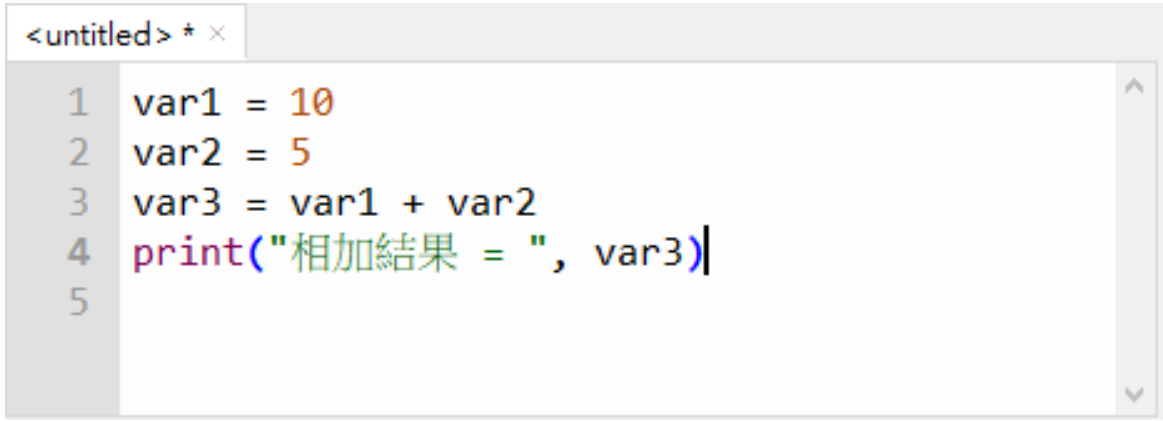
2-3-1 建立第二個Python程式

- Step 1：請啟動Thonny，執行「檔案→開新檔案」命令新增名為【<untitled>】標籤的全新Python程式檔（如果已有【<untitled>】標籤，請直接在此標籤輸入Python程式碼）。

2-3-1 建立第二個Python程式

- Step 2：在標籤頁輸入Python程式碼，var1~var3是變數，在使用「=」指定var1和var2變數值後，執行加法運算，最後輸出執行結果，如下圖所示：

```
var1 = 10
var2 = 5
var3 = var1 + var2
print("相加結果 = ", var3)
```



2-3-1 建立第二個Python程式

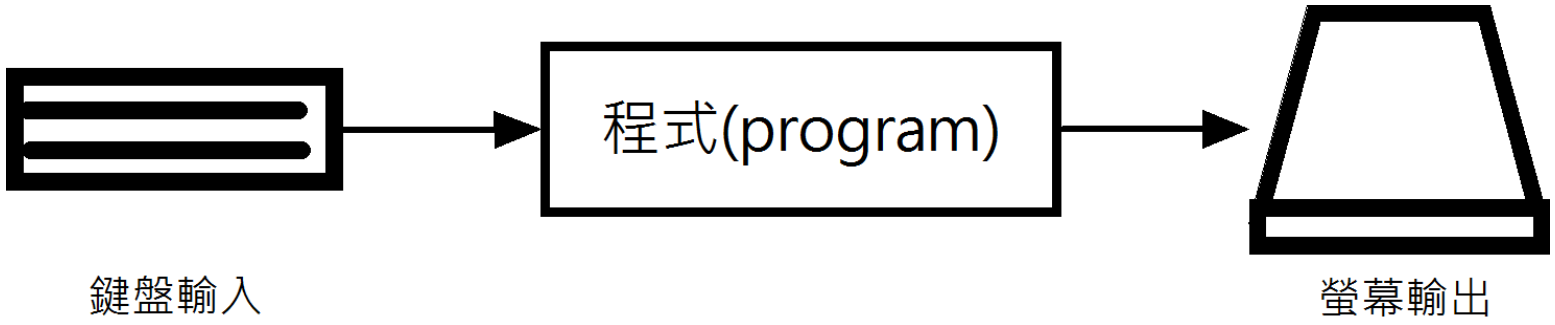
- Step 3：執行「檔案→儲存檔案」命令，可以看到「另存新檔」對話方塊，請切換至「/Python/ch02」目錄，輸入【ch2-3-1】，按【存檔】鈕儲存成ch2-3-1.py程式。
- Step 4：請執行「執行→執行目前程式」命令，或按工具列綠色箭頭圖示的【執行目前程式】鈕（也可按F5鍵）執行Python程式，可以看到執行結果是加法運算結果15，如下圖所示：

```
>>> %Run ch2-3-1.py
  相加結果 =  15
```

2-3-2 認識主控台輸出

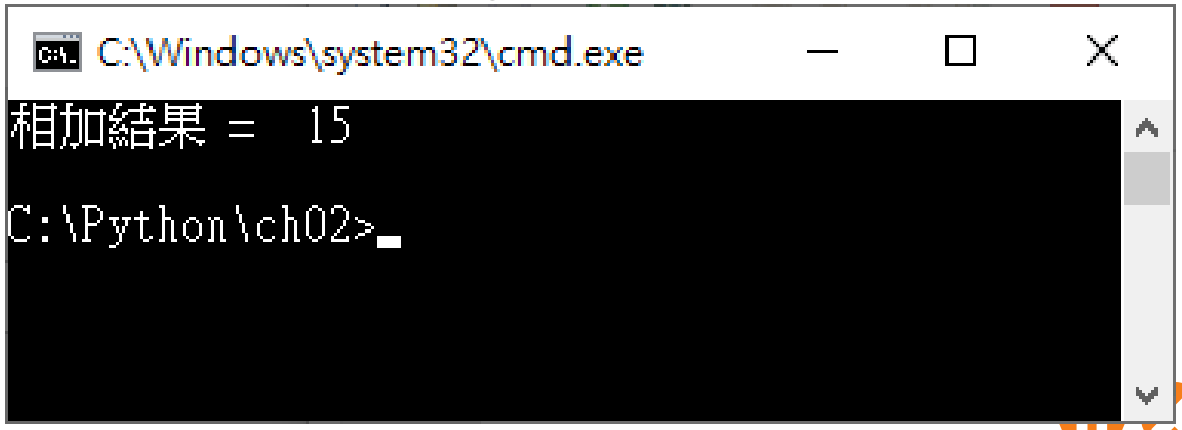
主控台輸入與輸出

- Python語言建立的主控台應用程式（Console Application），就是在Windows作業系統的「命令提示字元」視窗執行的程式，最常使用的標準輸入裝置是鍵盤；標準輸出裝置是電腦螢幕，即主控台輸入與輸出（Console Input and Output，Console I/O），如下圖所示：



2-3-2 認識主控台輸出
在終端機執行Python程式

- Thonny開發環境也可以在終端機執行Python程式，也就是在Windows作業系統的「命令提示字元」視窗執行Python程式，事實上，真正的主控台應用程式是在終端機執行的程式，Linux作業系統稱為終端機；在Windows作業系統就是「命令提示字元」視窗。
- 請啟動Thonny開啟ch2-3-1.py程式後，執行「執行→在終端機執行目前程式」命令，可以開啟Windows作業系統的「命令提示字元」視窗，看到Python程式的執行結果，如下圖所示：



2-3-2 認識主控台輸出

在輸出結果顯示換行：使用「\n」新行字元

- 在print()函數輸出的文字內容因為是輸出至終端機，「\n」新行字元就是換行，換句話說，我們只需在輸出字串中加上新行字元「\n」，就可以在輸出至螢幕時顯示換行，如下所示：

```
print("學Python程式\n分數=", 100)
```

2-3-3 Python主控台輸出：print()函數

- Python輸出函數print()可以將「,」逗號分隔的資料輸出顯示在螢幕上，這些分隔資料稱為函數引數（Arguments）或參數（Parameters），為了方便說明，在本書都使用參數，其基本語法如下所示：

  print(項目1 [,項目2...], sep=" ", end="\n")

  •項目1和項目2等參數：這些是使用「,」號分隔的輸出內容，可以一次輸出多個項目。

  •sep參數：分隔字元預設1個空白字元，如果輸出多個項目，即在每一個項目之間加上1個空白字元，此參數需在項目1~n之後。

  •end參數：結束字元是輸出最後加上的字元，預設是"\n"新行字元，此參數需在項目1~n之後。

2-4 看看Python程式的內容

- Python程式的副檔名是「.py」，其基本結構是匯入模組、全域變數、函數和程式敘述所組成，如下所示：

```
import 模組
全域變數
def 函數名稱1(參數列) :
    程式敘述1~N
...
def 函數名稱N(參數列) :
    程式敘述1~N
程式敘述1~N
```

2-4 看看Python程式的內容
import 模組

- 因為Python本身只提供簡單語法和少數內建函數，大部分功能都是透過Python標準函式庫的模組或第三方套件所提供，即其他程式語言的函式庫。例如：當Python程式需要使用三角函數，我們可以匯入math模組，如下所示：

```
import math
```

- 當Python程式匯入math模組，在Python程式碼就可以呼叫此模組的sin()、cos()和tan()三角函數，如下所示：

```
math.sin(x)
math.cos(x)
math.tan(x)
```

2-4 看看Python程式的內容

全域變數與函數

- Python函數是使用def關鍵字來建立，函數（Functions）是一個獨立程式片段，可以完成指定工作，這是由函數名稱、參數列和縮排的程式區塊所組成。
- 在函數外定義的變數稱為全域變數，Python程式檔案的所有程式碼都可以存取此變數值，對比函數中使用的區域變數。關於全域變數、區域變數和函數的說明請參閱第6章。

2-4 看看Python程式的內容

程式敘述1~N

- 在Python程式檔案中沒有縮排的程式敘述，這些程式敘述就是其他程式語言的主程式，當直譯執行Python程式時，就是從第1行沒有縮排的程式敘述開始，執行到最後1行沒有縮排的程式敘述為止。
- 例如：Python程式ch2-3-1.py沒有匯入模組、全域變數和函數，當直譯器執行Python程式時，就是從第1行var1 = 10的程式碼開始執行，直到執行到最後第4行呼叫print()函數輸出運算結果為止。

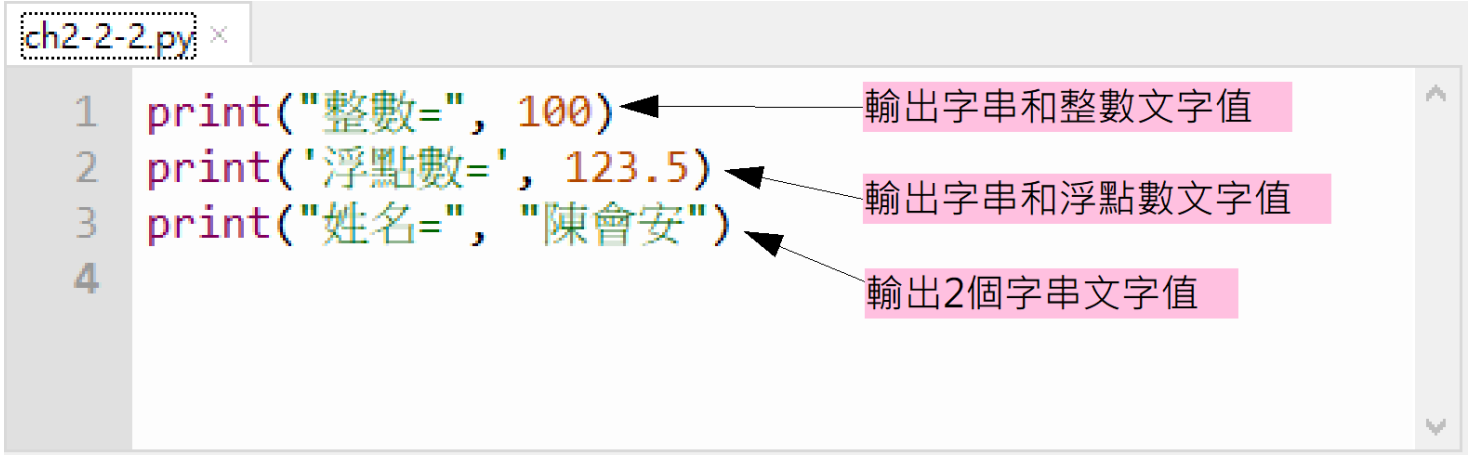
2-5 Python文字值

- 2-5-1 字串文字值
- 2-5-2 數值文字值
- 2-5-3 布林文字值



2-5 Python文字值

- Python文字值（Literals、也稱字面值）或稱常數值（Constants），這是一種文字表面顯示的值，即撰寫程式碼時直接使用鍵盤輸入的值。在Python程式：ch2-2-2.py輸出的整數100、浮點數123.5和字串"陳會安"等，都是文字值，如下圖所示：

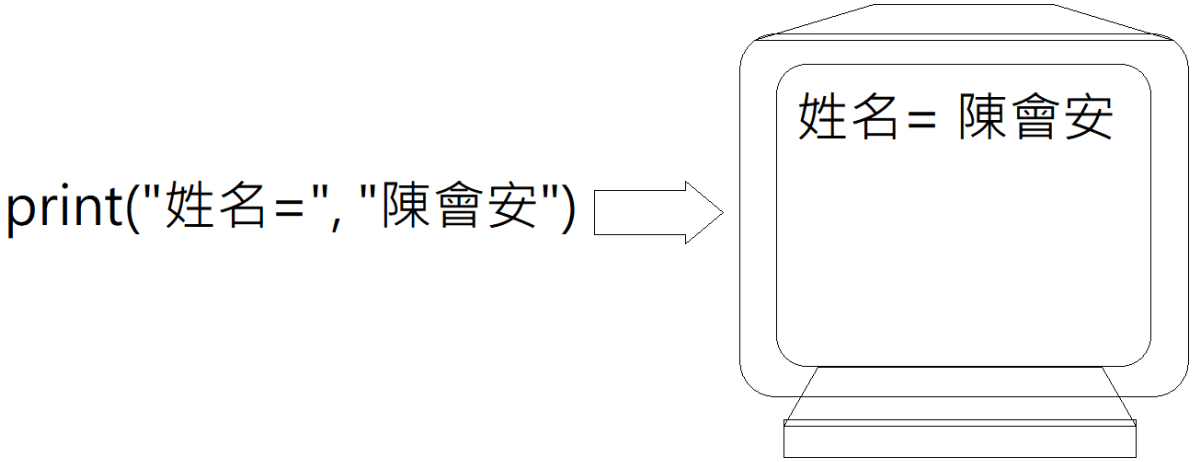


2-5-1 字串文字值-說明

- Python字串文字值（String Literals）就是字串，字串是0或多個使用「'」單引號或「"」雙引號括起的一序列Unicode字元，如下所示：

  "學習Python語言程式設計"

  'Hello World!'



2-5-1 字串文字值

Escape逸出字元（Escape Characters）

- Python提供Escape逸出字元來輸入特殊字元，這是一些使用「\」符號開頭的字元，如下表所示：

Escape逸出字元	說明
\b	Backspace，即Backspace鍵
\f	FF，Form feed換頁字元
\n	LF（Line Feed）換行或NL（New Line）新行字元
\r	Carriage Return，即Enter鍵
\t	定位字元，即Tab鍵
\'	「'」單引號
\"	「"」雙引號
\\	「\」符號

2-5-1 字串文字值
使用八進位和十六進位值表示ASCII字元

- ASCII字元也可以使用Escape逸出字元來表示，即「\x」字串開頭的2個十六進位數字或「\」字串開頭3個八進位數字來表示ASCII字元碼，如下所示：

  '\x61'

  '\101'

- 上述表示法，筆者整理如下表所示：

ASCII字元碼	說明
\N	N是八進位值的字元常數，例如：\040空白字元
\xN	N是十六進位值的字元常數，例如：\x20空白字元

2-5-2 數值文字值

整數文字值

- 整數文字值是指資料是整數值，沒有小數點，其資料長度可以是任何長度，視記憶體空間而定。例如：一些整數文字值的範例，如下所示：

  1　100　122　56789

- 上述整數值是10進位值，也是我們習慣使用的數字系統，Python語言支援二進位、八進位和十六進位的數字系統，此時的數值需要加上數字系統的字首（十進位並不需要），如下表所示：

數字系統	字首	範例（十進位值）
二進位	0b或0B	0b1101011（107）
八進位	0o或0O	0o15（13）
十六進位	0x或0X	0xFB（253）

2-5-2 數值文字值

浮點數文字值

- 浮點數文字值是指數值資料是整數加上小數，其精確度可以到小數點下15位，基本上，整數和浮點數的差異就是小數點，5是整數；5.0是浮點數，例如：一些浮點數文字值的範例（Python程式：ch2-5-2a.py），如下所示：

  1.0

  55.22

2-5-3 布林文字值

- Python語言的布林（Boolean）文字值是使用True和False關鍵字來表示（Python程式：ch2-5-3.py），如下所示：

```
True
False
```

2-6 Python寫作風格

- 2-6-1 程式敘述
- 2-6-2 程式區塊
- 2-6-3 程式註解
- 2-6-4 太長的程式碼



2-6-1 程式敘述

- Python程式是使用程式敘述（Statements）所組成，一行程式敘述如同英文的一個句子，內含多個運算式、運算子或關鍵字，這就是Python直譯器可以執行的程式碼。

程式敘述的範例

- 一些Python程式敘述的範例，如下所示：

```
b = 10
c = 2
a = b * c
print("第一個Python程式")
```

- 上述第1行和第2行程式碼是指定變數初值，第3行是指定敘述的運算式，第4行是呼叫print()函數。

2-6-1 程式敘述

「;」分號

- 大部分程式語言：C/C++、Java和C#等語言的「;」分號代表程式敘述的結束，告訴直譯器/編譯器已經到達程式敘述的最後，請注意！Python語言並不需要在程式敘述最後加上「;」分號，如果習慣在程式敘述最後加上「;」分號，也不會有錯誤，如下所示：

```
d = 5;
```

- 在Python語言使用「;」分號的主要目的是在同一行程式碼撰寫多個程式敘述，如下所示：

```
b = 10; c = 4; a = b * c
```

2-6-2 程式區塊

- 程式區塊（Blocks）是由多個程式敘述所組成，大部分程式語言是使用"{"和"}"大括號（Braces）包圍來建立程式區塊。Python語言的程式區塊是使用縮排，當多行程式敘述擁有相同數量的空白字元縮排時，就屬於同一個程式區塊，通常是使用「4」個空白字元或1個Tab鍵。
- Python程式區塊是從第1個縮排的程式敘述開始，到第1個沒有縮排的程式敘述的前一行為止，如下所示：

```
for i in range(1, 11):
    print(i)
    if i == 5 :
        break
print("迴圈結束")
```

2-6-3 程式註解

- 程式註解（Comments）是程式中十分重要的部分，可以提供程式內容的進一步說明，良好的註解文字不但能夠了解程式目的，並且在程式維護上，也可以提供更多的資訊。
- 基本上，程式註解是給程式設計者閱讀的內容，Python直譯器在直譯原始程式碼時會忽略註解文字和多餘的空白字元。

2-6-3 程式註解
Python語言的單行註解

- Python語言的單行註解是在程式中以「#」符號開始的行，或程式行位在「#」符號後的文字內容都是註解文字，如下所示：

```
# 顯示訊息
print("第一個Python程式")  # 顯示訊息
```

2-6-3 程式註解
Python語言的多行註解

- Python語言的程式註解可以跨過很多行，這是使用「"""」和「"""」符號（3個「"」符號）或「'''」和「'''」符號（3個「'」符號）括起的文字內容，例如：我們可以在Python程式開頭加上程式檔名稱的註解文字，如下所示：

```
''' Python程式:
    檔名: ch2-6.py '''
```

- 上述註解文字是位在「'''」和「'''」符號中的文字內容。使用「"""」符號的多行註解，如下所示：

```
""" ---------------------------
    程式範例: ch2-6.py
    --------------------------- """
```

2-6-4 太長的程式碼

- Python語言的程式碼行如果太長，基於程式編排的需求，太長的程式碼並不容易閱讀，我們可以分割成多行來編排。請在程式碼該行的最後加上「\」符號（Line Splicing），將程式碼分成數行來編排，如下所示：

```
sum = 1 + 2 + \
      3 + 4 + \
      5
```

第3章 變數、運算式與運算子

- 3-1 程式語言的變數
- 3-2 在程式使用變數
- 3-3 變數的資料型態和型態轉換函數
- 3-4 讓使用者輸入變數值
- 3-5 認識運算式和運算子
- 3-6 在程式使用運算子



3-1 程式語言的變數

- 3-1-1 認識變數
- 3-1-2 使用變數前的準備工作
- 3-1-3 Python語言的命名規則

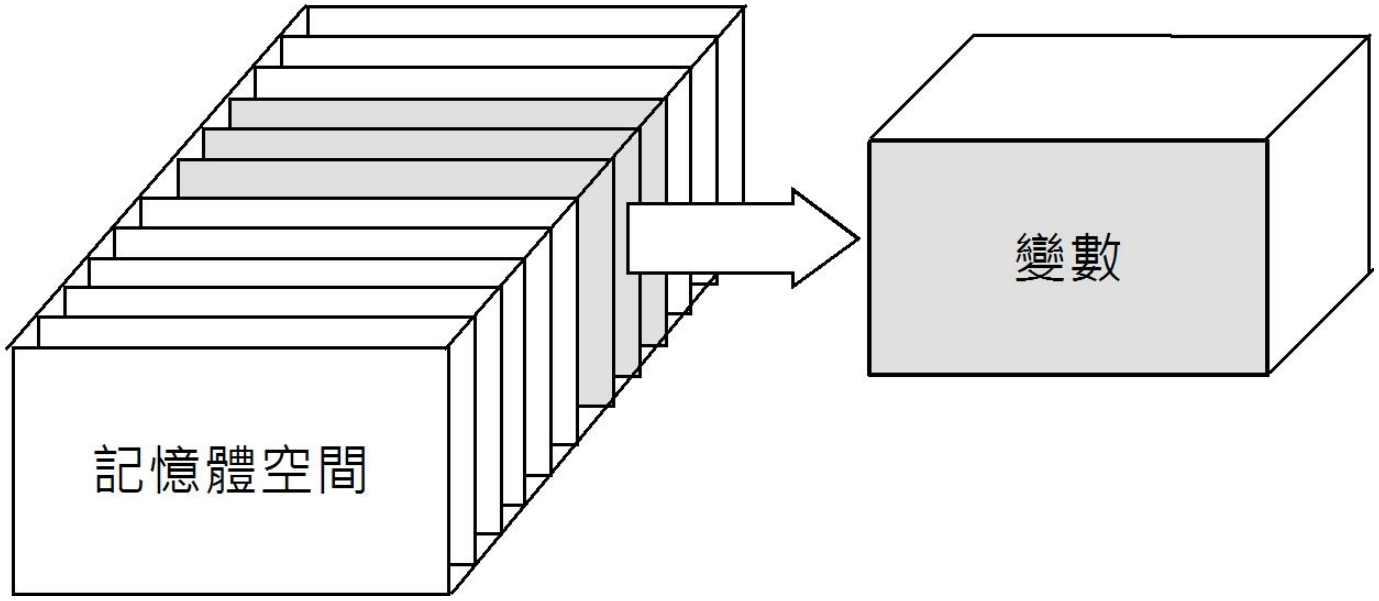


3-1-1 認識變數– 說明

- 電腦程式在執行時常常需要記住一些資料，所以在程式語言會提供一個地方，用來記得執行時的一些資料，這個地方就是「變數」（variables）。
- 例如：去商店買東西時，為了比較價格，我們會記下商品價格，同樣的，程式是使用變數儲存這些執行時需記住的資料，也就是將這些值儲存至變數，當變數擁有儲存值後，就可以在需要的地方取出變數值，例如：執行數學運算和比較等。

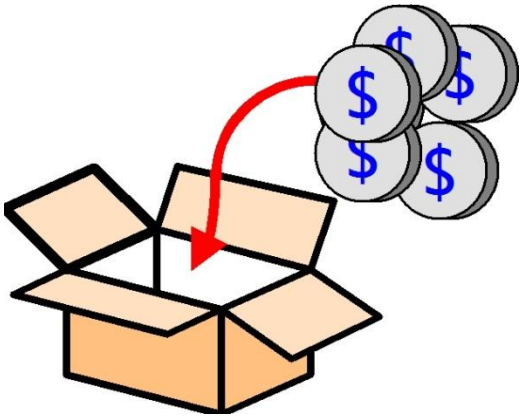
3-1-1 認識變數– 變數是儲存在哪裡

- 問題是，這些需記住的資料是儲存在哪裡，答案就是電腦的「記憶體」（memory），變數是一個名稱，用來代表電腦記憶體空間的一個位址，如下圖所示：



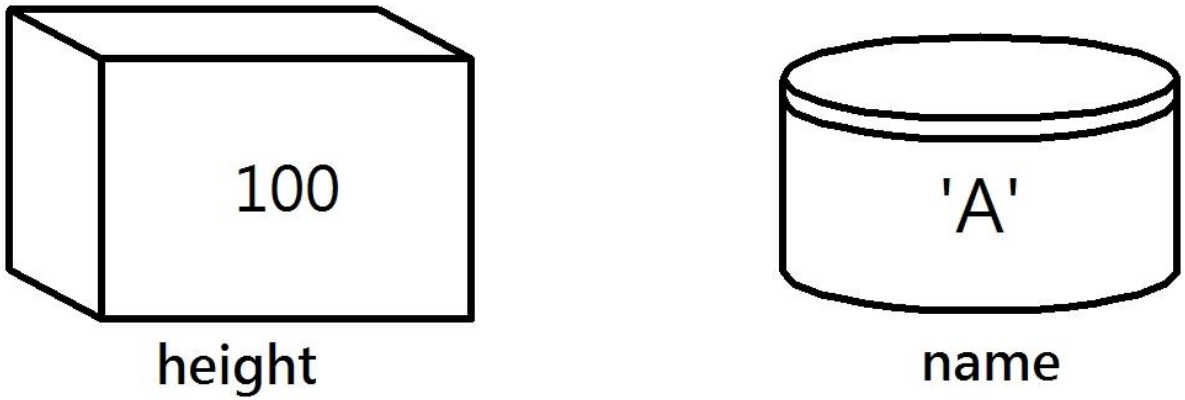
3-1-1 認識變數– 變數的基本操作

- 變數只有兩種操作，如下所示：
  - **在變數存入新值**：指定變數成為一個全新值，我們並不能如同盒子一般，只取出部分金額。因為變數只能指定成一個新值，如果需要減掉一個值，其操作是先讀取變數值，在減掉後，再將變數指定成最後運算結果的新值。
  - **讀取變數值**：取得目前變數的值，讀取變數值，並不會更改變數目前儲存的值。



3-1-2 使用變數前的準備工作– 說明

- 程式語言的變數如同是一個擁有名稱的盒子，能夠暫時儲存程式執行時所需的資料，也就是記住這些資料。方形和圓柱形的兩個盒子，盒子名稱是變數名稱height和name，在盒子儲存的資料100和'A'是常數值。現在回到盒子本身，盒子形狀和尺寸決定儲存的資料種類，對比程式語言，形狀和尺寸是變數的「資料型態」（data types），如下圖所示：



3-1-2 使用變數前的準備工作– 內容

- 如果程式語言是靜態語言，例如：C/C++和Java，當變數指定資料型態後，就表示只能儲存這種型態的資料，如同圓形盒子放不進相同直徑的方形物品，我們只能放進方形盒子。Python是動態語言，變數的資料型態是可以更改的，當變數指定成其他資料型態的文字值時，變數的資料型態也會一併更改成文字值的資料型態。
- 所以，程式語言在使用變數前，需要2項準備工作，如下所示：
  - 替變數命名：即上述name和height等變數名稱。
  - 決定變數的資料型態：即變數儲存什麼樣的值，即整數或字串等。

3-1-3 Python語言的命名規則

- 程式設計者在程式碼自行命名的元素，稱為識別字（Identifier），例如：變數名稱，關鍵字（Keywords）是一些對直譯器/編譯器來說擁有特殊意義的名稱，在命名時，我們需要避開這些名稱。
- 識別字名稱（Identifier Names）是指Python語言的變數、函數、類別或其他識別字的名稱，程式設計者在撰寫程式時，需要替這些識別字命名。

3-1-3 Python語言的命名規則

- Python語言的命名規則，如下所示：
  - 名稱是一個合法識別字，識別字是使用英文字母或「_」底線開頭（不可以使用數字開頭），不限長度，包含字母、數字和底線「_」字元組成的名稱。
  - 名稱區分英文字母大小寫，例如：total、Total和TOTAL屬於不同的識別字。
  - 名稱不能使用Python關鍵字，因為這些字對於直譯器擁有特殊意義。Python語言的關鍵字可以在互動環境輸入help("keywords")指令來查詢。
  - 有效範圍（Scope）是指在有效範圍的程式碼中名稱必須是唯一，例如：在程式中可以使用相同的變數名稱，不過變數名稱需要位在不同的範圍，詳細的範圍說明請參閱第6-6-1節。

3-2 在程式使用變數

- 3-2-1 指定和輸出變數值
- 3-2-2 指定成其他文字值
- 3-2-3 指定成其他變數值



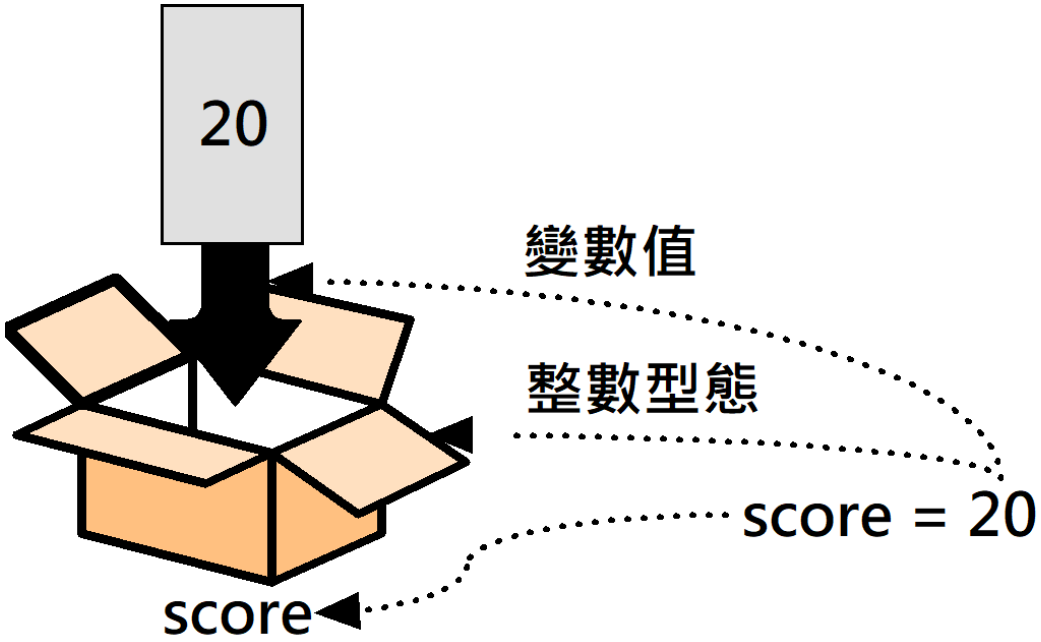
3-2 在程式使用變數

- Python變數不用預先宣告，當需要變數時，直接指定變數值即可，如果習慣宣告變數，可以在程式開頭將變數指定成None，None關鍵字表示變數並沒有值，如下所示：

```
score = None
```

3-2-1 指定和輸出變數值

- Python在使用指定敘述指定變數值，變數名稱如同是一個盒子，指定的變數值就是將值放入盒子，和決定盒子的資料型態，如下圖所示：



3-2-1 指定和輸出變數值

- 當使用「=」等號指定變數score的值20，就是將文字值20放入盒子，同時決定變數的資料型態是整數，如下所示：

  score = 20

- 上述程式碼指定變數score值是20，變數score因為指定文字值20，所以記得文字值20且決定是整數型態，其基本語法如下所示：

  變數 = 資料

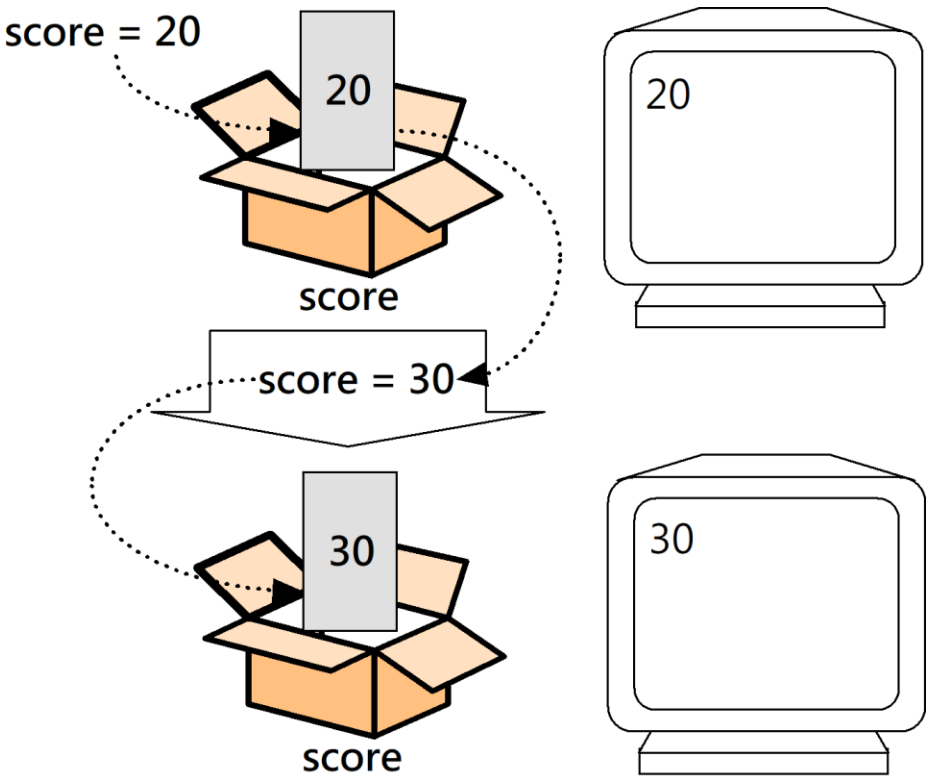
- 上述「=」等號就是指定敘述，可以：

「將右邊資料的文字值指定給左邊的變數，在左邊變數儲存的值就是右邊資料的文字值。」

3-2-2 指定成其他文字值

- 變數是執行程式時暫存儲存資料的地方，當建立Python變數後，例如：變數score和指定值20後，我們可以隨時再次使用指定敘述「=」等號來更改變數值，如下所示：

score = 30



3-2-2 指定成其他文字值

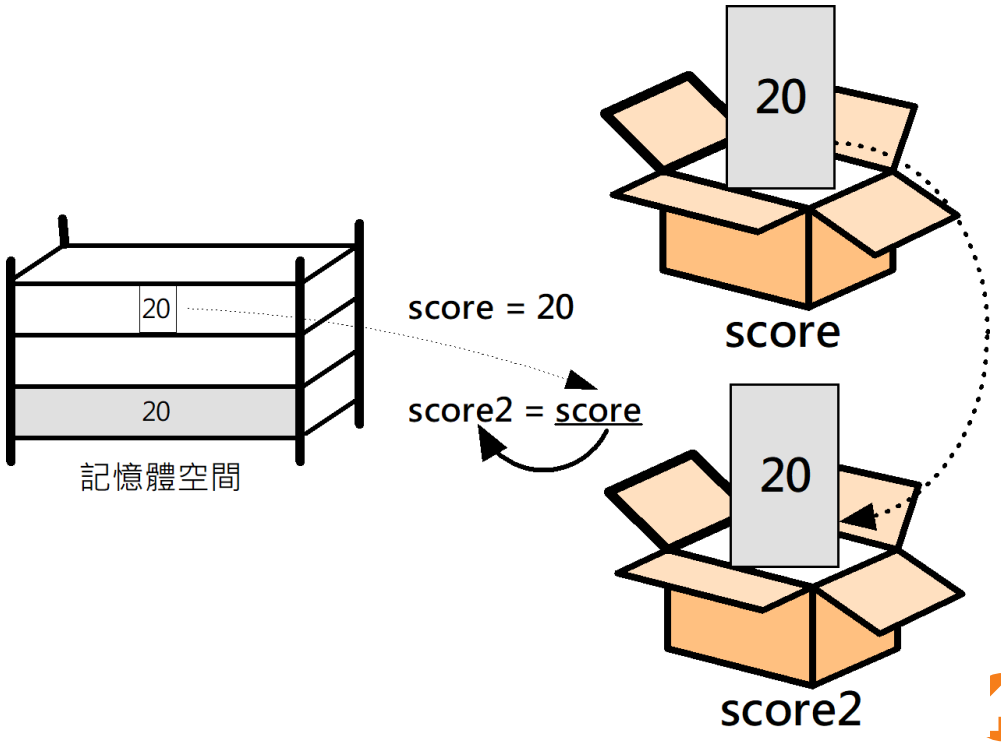
- Python變數也可以隨時更改成其他資料型態的文字值，例如：浮點數98.5，此時不只更改score變數的值，同時更改資料型態成為浮點數，如下所示：

```
score = 98.5
```

3-2-3 指定成其他變數值

- 變數除了可以使用指定敘述「=」等號更新成其他文字值，我們還可以將變數指定成其他變數值，就是更改成其他變數儲存的文字值，例如：在建立整數變數score的值20後，使用指定敘述來指定變數score2的值是score變數儲存的值，如下所示：

```
score = 20
score2 = score
```



3-3 變數的資料型態和型態轉換函數

- 3-3-1 取得變數的資料型態
- 3-3-2 資料型態轉換函數



3-3 變數的資料型態和型態轉換函數

- Python變數儲存的值決定變數目前的資料型態，當指定變數的文字值時，就決定變數的資料型態，例如：在第2-5節的字串、數值和布林文字值，就是對應字串（String）、數值（Number）和布林（Boolean）資料型態。
- 除此之外，Python還提供多種容器型態（Contains Type），例如：串列（List）、元組（Tuple）合（Set）和字典（Dictionary），進一步字串和容器型態的說明請參閱第7章。

3-3-1 取得變數的資料型態

- Python可以使用type()函數取得目前變數的資料型態，如下所示：

```
score = 20
print("變數score值是:", score, type(score))
```

- 上述變數score是整數，可以呼叫type(a)取得變數a資料型態的物件<class 'int'>，請注意！Python所有東西都是class類別的物件。

3-3-2 資料型態轉換函數

- Python不會自動轉換變數/文字值的資料型態，我們需要自行使用內建型態轉換函數來轉換變數/文字值成所需的資料型態，如下表所示：

型態轉換函數	說明
str()	將任何資料型態的參數轉換成字串型態
int()	將參數轉換成整數資料型態，參數如果是字串，字串內容只能是數字，如果是浮點數，轉換成整數會損失精確度
float()	將參數轉換成浮點數資料型態，如果是字串，字串內容只可以是數字和小數點

3-4 讓使用者輸入變數值

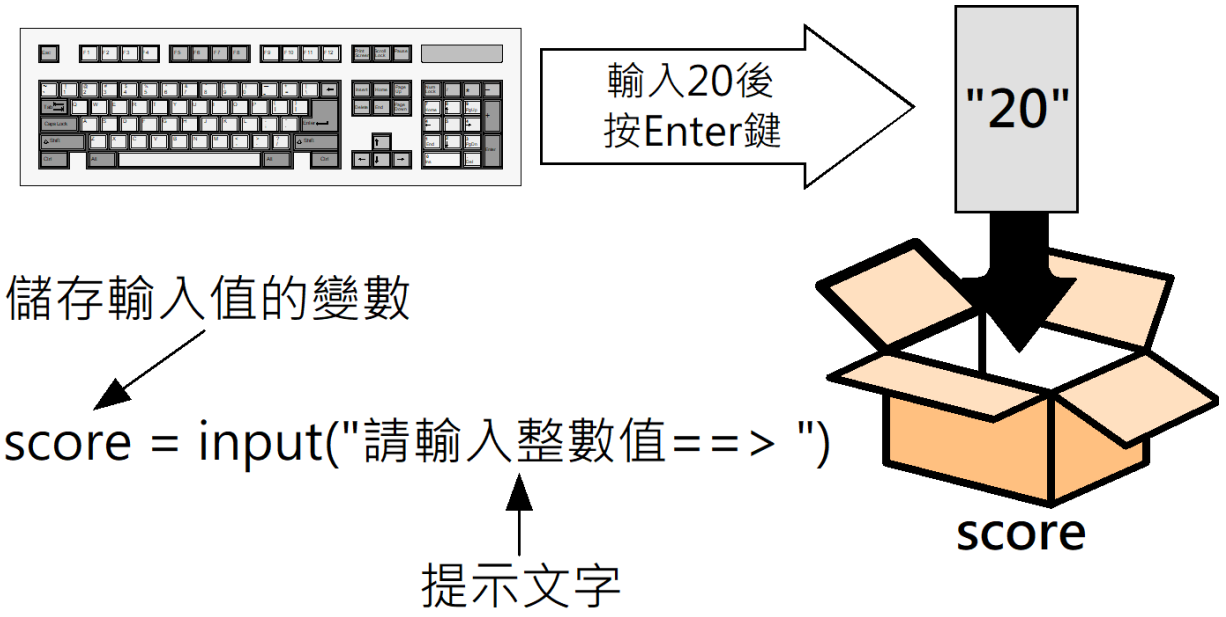
從鍵盤輸入字串資料

- Python可以讓使用者以鍵盤輸入變數值，這就是第2-3-2節的主控台輸入，我們是使用input()函數讓使用者輸入字串文字值。
- 當變數值可以讓使用者自行使用鍵盤來輸入時，我們建立的Python程式就擁有更多的彈性，因為變數存入的值是在執行Python程式時，才讓使用者自行從鍵盤輸入，而不是撰寫Python程式碼來指定其值。

3-4 讓使用者輸入變數值
從鍵盤輸入字串資料

- Python程式是呼叫input()函數來輸入字串資料型態的資料，如下所示：

```
score = input("請輸入整數值==> ")  # 輸入字串文字值
```



3-4 讓使用者輸入變數值

從鍵盤輸入數值資料

- 請注意！input()函數只能輸入字串文字值，我們需要使用第3-3-2節的int()和float()資料型態轉換函數來轉換成整數變數score2和浮點數變數score3，如下所示：

```
score2 = int(score)
score3 = float(score)
```

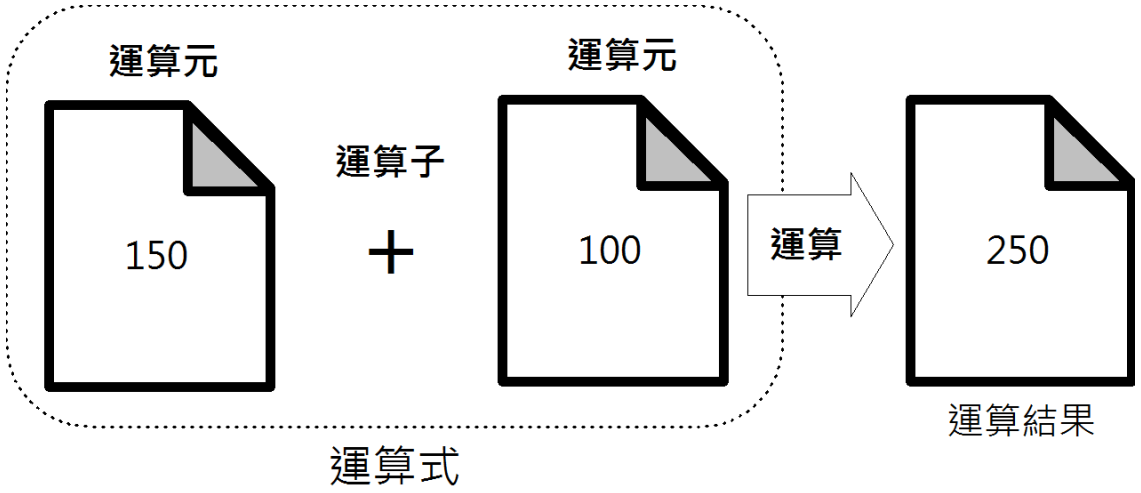
3-5 認識運算式和運算子

- 3-5-1 關於運算式
- 3-5-2 輸出運算式的運算結果
- 3-5-3 執行不同種類運算元的運算
- 3-5-4 讓使用者輸入值來執行運算



3-5-1 關於運算式

- 運算式（Expressions）是由一序列運算子（Operators）和運算元（Operands）所組成，可以在程式中執行所需的運算任務（即執行資料處理），如下圖所示：



3-5-1 關於運算式

- Python運算子依運算元的個數分成二種，如下所示：
  - 單元運算子（Unary Operator）：只有一個運算元，例如：正號或負號，如下所示：

    -15

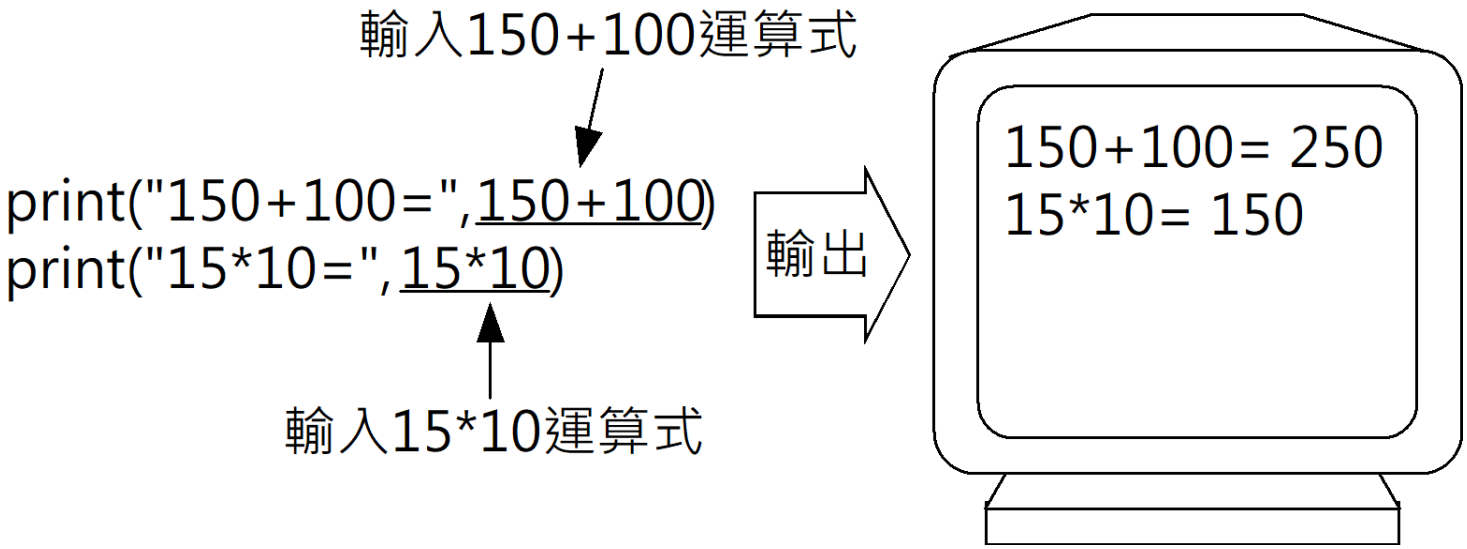
    +10
  - 二元運算子（Binary Operator）：擁有位在左右的兩個運算元，Python運算子大部分是二元運算子，如下所示：

    5 + 10

    10 - 2

3-5-2 輸出運算式的運算結果

- Python的print()函數可以在電腦螢幕輸出執行結果，同樣的，我們可以輸出運算式的運算結果，如下圖所示：



3-5-3 執行不同種類運算元的運算
2個運算元都是變數

- Python加法運算式的2個運算元可以是2個變數，例如：計算分數的總和，如下所示：

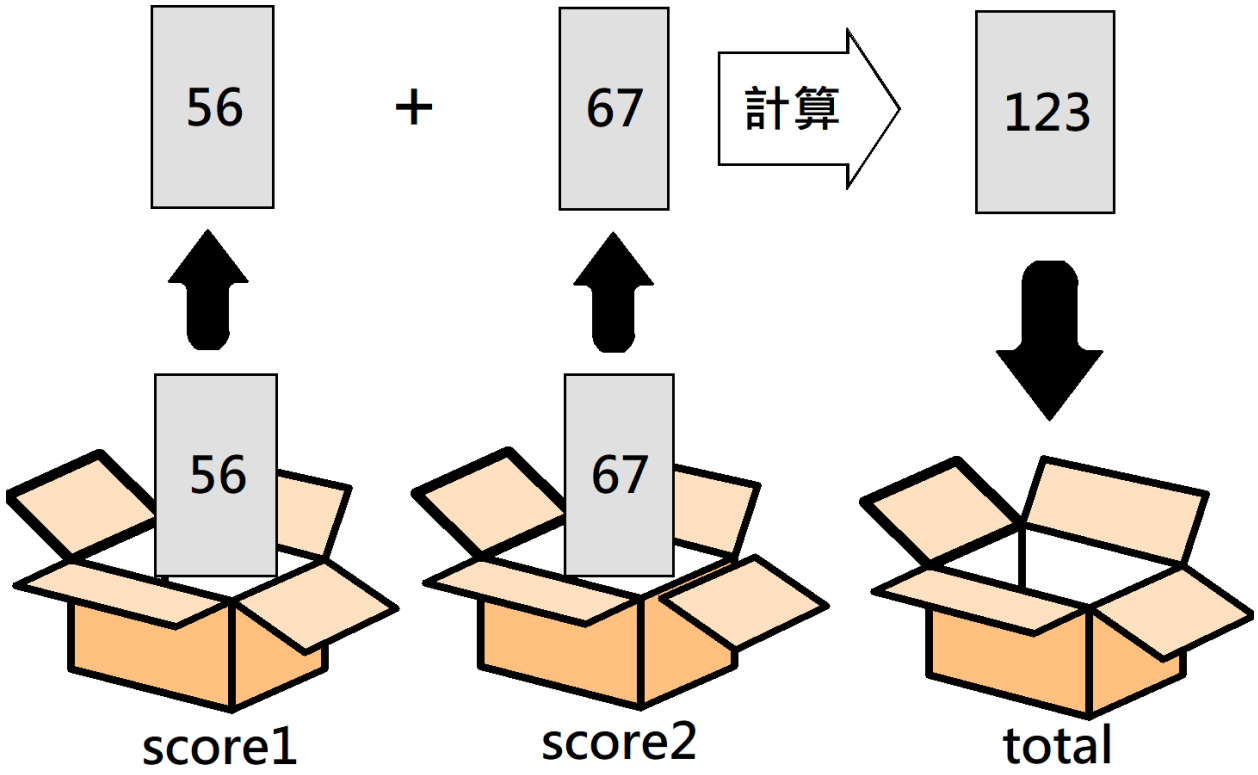
```
score1 = 56
score2 = 67
total = score1 + score2   # 加法運算式
```

- 上述運算式「score1+score2」的2個運算元都是變數，「total = score1+score2」運算式的意義是：

「取出變數score1儲存的值56，和取出變數score2儲存的值67後，將2個常數值相加56+67後，再將運算結果123存入變數total。」

3-5-3 執行不同種類運算元的運算

2個運算元都是變數



3-5-3 執行不同種類運算元的運算
1個運算元是變數；1個運算元是文字值

- Python加法運算式的2個運算元可以其中一個是變數；另一個是文字值，例如：調整變數score1的分數，將它加10分，如下所示：

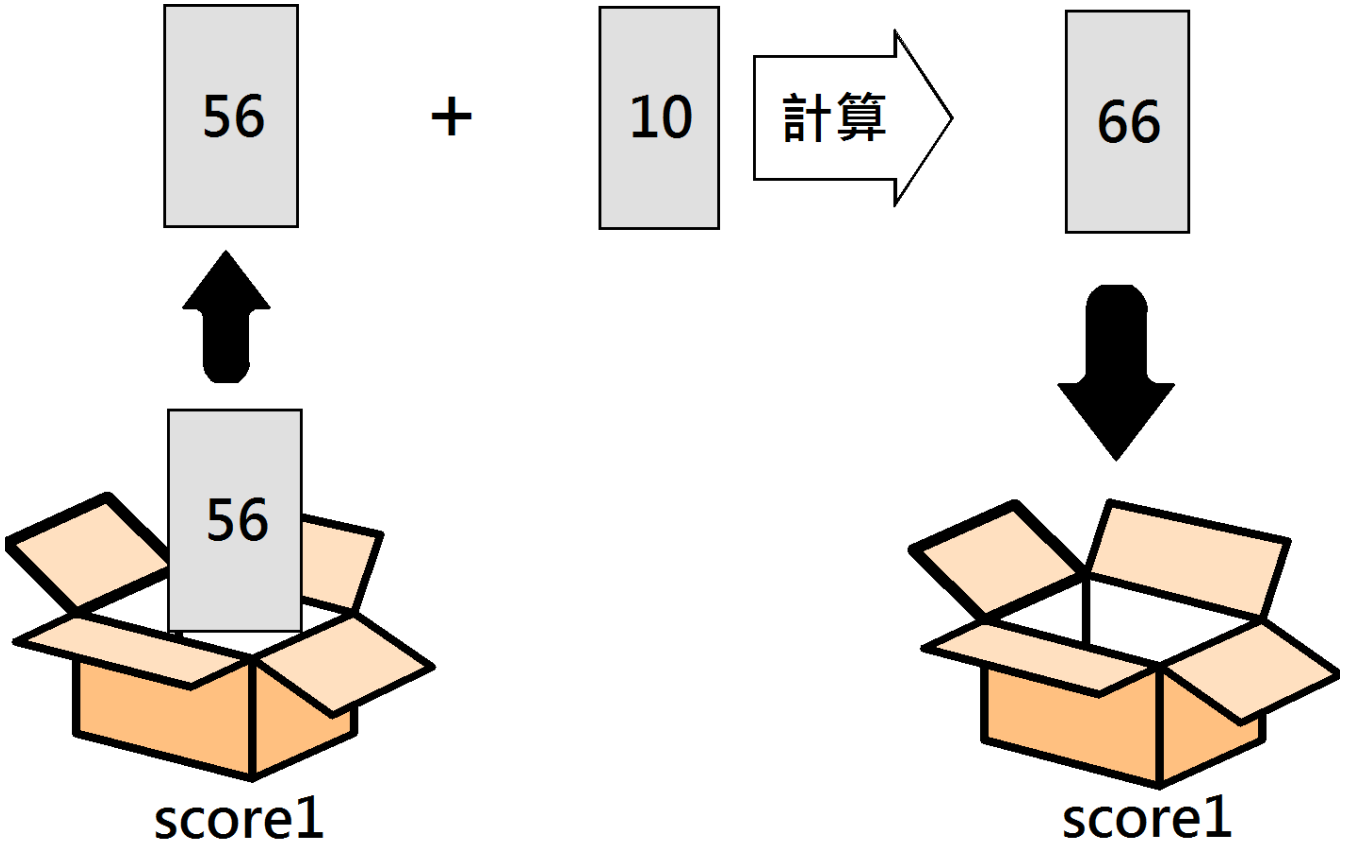
```
score1 = 56
score1 = score1 + 10   # 加法運算式
```

- 上述運算式「score1+10」的第1個運算元是變數；第2個是文字值，「score1 = score1+10」運算式的意義是：

「取出變數score1儲存的值56，加上文字值10後，再將運算結果56+10=66存入變數score1。」

3-5-3 執行不同種類運算元的運算

1個運算元是變數；1個運算元是文字值



3-5-4 讓使用者輸入值來執行運算

- 當運算式的運算元是變數，我們只需更改變數值，就可以產生不同的運算結果，如下所示：

```
total = score1 + score2   # 加法運算式
```

- 上述運算元變數score1和score2的值如果不同，total變數的運算結果就會不同，Python程式可以使用input()函數，讓使用者自行輸入變數值來執行運算，只需輸入不同值，就可以得到不同的加法運算結果。

3-6 在程式使用運算子

- 3-6-1 運算子的優先順序
- 3-6-2 算術運算子
- 3-6-3 指定運算子
- 3-6-4 更多的指定敘述



3-6-1 運算子的優先順序

優先順序（Precedence）

- Python運算子的優先順序決定運算式中運算子的執行順序，可以讓擁有多運算子的運算式得到相同的運算結果。
- 當同一Python運算式使用多個運算子時，為了讓運算式能夠得到相同的運算結果，運算式是以運算子預設的優先順序來進行運算，也就是熟知的「先乘除後加減」口訣，如下所示：

  a + b * 2

- 上述運算式因為運算子的優先順序「*」大於「+」，所以先計算b*2後才和a相加。如果需要，在運算式可以使用括號推翻預設的運算子優先順序，例如：改變上述運算式的運算順序，先執行加法運算後，才是乘法，如下所示：

  (a + b) * 2

3-6-1 運算子的優先順序

Python運算子的優先順序

運算子	說明
()	括號運算子
**	指數運算子
~	位元運算子NOT
+、-	正號、負號
*、/、//、%	算術運算子的乘法、除法、整數除法和餘數
+、-	算術運算子加法和減法
<<、>>	位元運算子左移和右移
&	位元運算子AND
^	位元運算子XOR
|	位元運算子OR
in、not in、is、is not、<、<=、>、>=、<>、!=、==	成員、識別和關係運算子小於、小於等於、大於、大於等於、不等於和等於
not	邏輯運算子NOT
and	邏輯運算子AND
or	邏輯運算子OR

3-6-2 算術運算子

- Python算術運算子（Arithmetic Operators）可以建立數學的算術運算式（Arithmetic Expressions），其說明如下表所示：

運算子	說明	運算式範例
-	負號	-7
+	正號	+7
*	乘法	7 * 2 = 14
/	除法	7 / 2 = 3.5
//	整數除法	7 // 2 = 3
%	餘數	7 % 2 = 1
+	加法	7 + 2 = 9
-	減法	7 – 2 = 5
**	指數	2 ** 3 = 8

3-6-3 指定運算子

- 指定運算式（Assignment Expressions）就是指定敘述，這是使用「=」等號指定運算子來建立運算式，請注意！這是指定或稱為指派；並沒有相等或等於的意思，其目的如下所以：
- 「將右邊運算元或運算式運算結果的文字值，存入位在左邊的變數。」
- 在指定運算子「=」等號左邊是用來指定值的變數；右邊可以是變數、文字值或運算式，在本章之前已經說明很多現成的範例。

3-6-3 指定運算子

- 在這一節準備說明Python指定運算式的簡化寫法，其條件如下所示：
  - 在指定運算子「=」等號的右邊是二元運算式，擁有2個運算元。
  - 在指定運算子「=」等號的左邊的變數和第1個運算元相同。
- 例如：滿足上述條件的指定運算式，如下所示：

```
x = x + y
```

- 可以改用「+=」運算子來改寫此運算式，就是指定運算式的簡化寫法，如下所示：

```
x += y
```

3-6-3 指定運算子

- 各種簡化或稱縮寫表示的指定運算式和運算子說明，如下表所示：

指定運算子	範例	相當的運算式	說明
=	x = y	N/A	指定敘述
+=	x+ = y	x = x + y	加法
-=	x -= y	x = x - y	減法
*=	x *= y	x = x * y	乘法
/=	x /= y	x = x / y	除法
//=	x //= y	x = x // y	整數除法
%=	x %= y	x = x % y	餘數
**=	x **= y	x = x ** y	指數

3-6-4 更多的指定敘述

多重指定敘述：ch3-6-4.py

•多重指定敘述（Multiple Assignments）可以在一行指定敘述同時指定多個變數值，如下所示：

```
score1 = score2 = score3 = 25
print(score1, score2, score3)
```

同時指定敘述：ch3-6-4a.py

•同時指定敘述（Simultaneous Assignments）的「=」等號左右邊是使用「,」逗號分隔的多個變數和值，如下所示：

```
x, y = 1, 2
print("X =", x, "Y =", y)
```

第4章 條件判斷

- 4-1 你的程式可以走不同的路
- 4-2 關係運算子與條件運算式
- 4-3 if單選條件敘述
- 4-4 if/else二選一條件敘述
- 4-5 if/elif/else多選一條件敘述
- 4-6 在條件敘述使用邏輯運算子



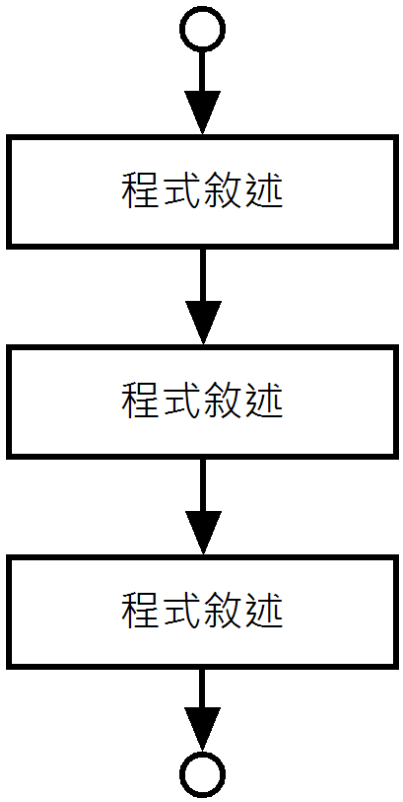
4-1 你的程式可以走不同的路

- 4-1-1 循序結構
- 4-1-2 選擇結構
- 4-1-3 重複結構



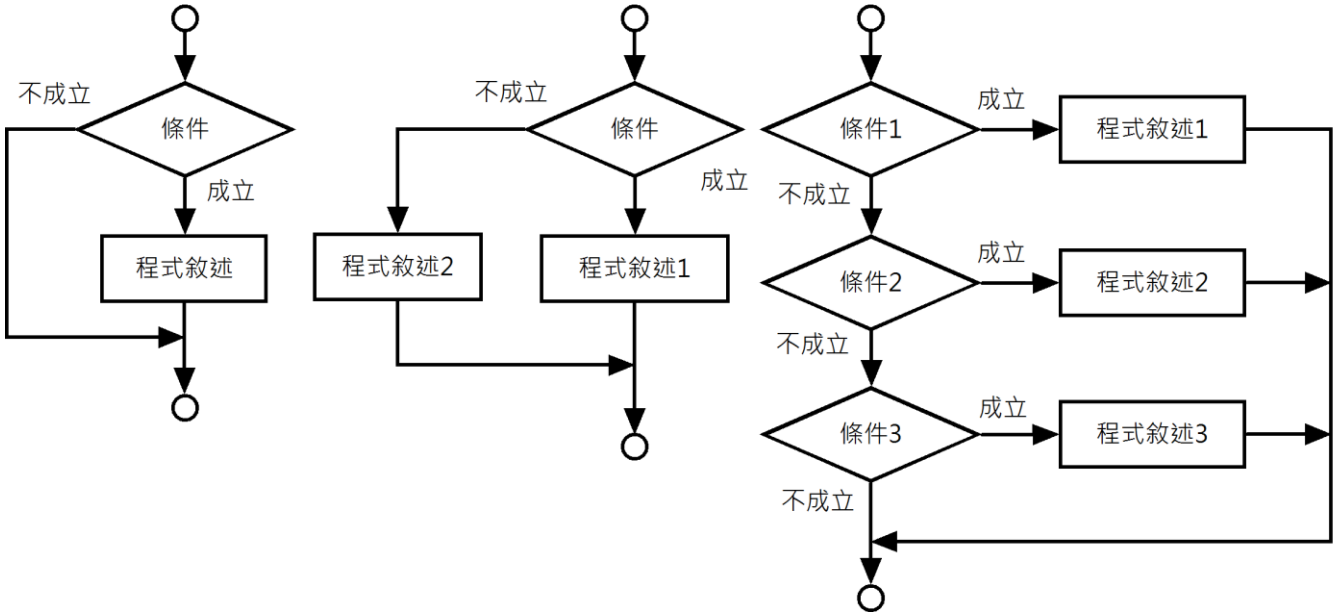
4-1-1 循序結構

- 循序結構（Sequential）是程式預設的執行方式，也就是一個敘述接著一個敘述依序的執行（在流程圖上方和下方的連接符號是控制結構的單一進入點和離開點，循序結構只有一種積木），如下圖所示：



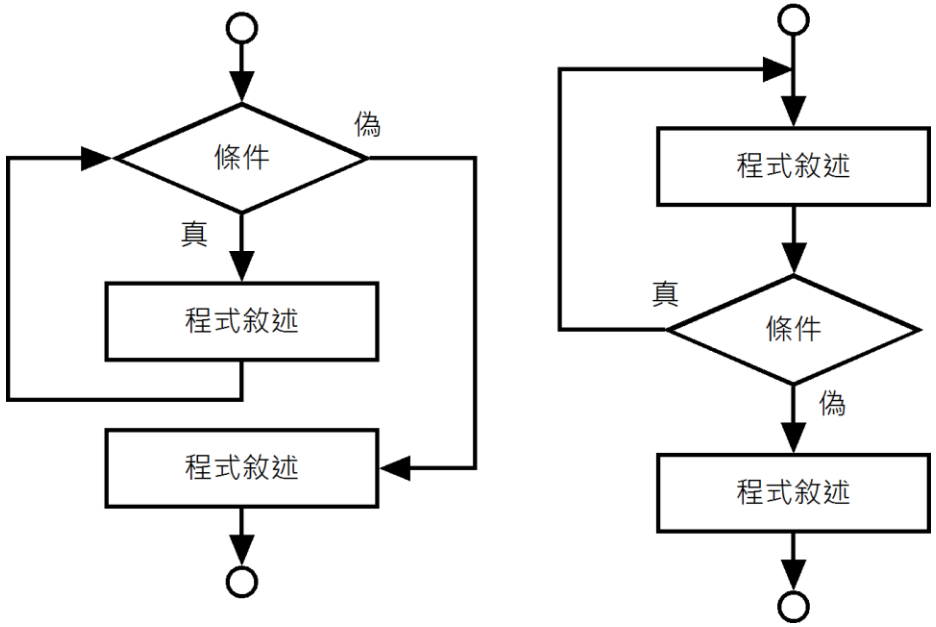
4-1-2 選擇結構

- 選擇結構（Selection）是一種條件判斷，這是一個選擇題，分為單選、二選一或多選一共三種。程式執行順序是依照關係或比較運算式的條件，決定執行哪一個區塊的程式碼（在流程圖上方和下方的連接符號是控制結構的單一進入點和離開點，從左至右依序為單選、二選一或多選一共三種積木），如下圖所示：



4-1-3 重複結構

- 重複結構（Iteration）是迴圈控制，可以重複執行一個程式區塊的程式碼，提供結束條件結束迴圈的執行，依結束條件測試的位置不同分為兩種：前測式重複結構（左圖）和後測式重複結構（右圖），Python語言並不支援後測式重複結構，如下圖所示：



4-2 關係運算子與條件運算式

- 4-2-1 認識條件運算式
- 4-2-2 關係運算子的種類
- 4-2-3 使用關係運算子



4-2-1 認識條件運算式

- 如同回家的路不會只有一條路；回家的方式也不會只有一種方式，在日常生活中，我們常常需要面臨一些抉擇，決定作什麼；或是不作什麼，例如：
  - 如果天氣有些涼的話，出門需要加件衣服。
  - 如果下雨的話，出門需要拿把傘。
  - 如果下雨的話，就搭公車上學。
  - 如果成績及格的話，就和家人去旅行。
  - 如果成績不及格的話，就在家K書。

4-2-1 認識條件運算式

- 我們人類會因不同狀況的發生，需要使用條件（Conditions）判斷來決定如何解決這些問題，不同情況，就會有不同解決方式。同理，Python可以將決策符號轉換成條件，以便程式依據條件是否成立來決定走哪一條路。
- 例如：當我們使用「如果」開頭說話時，隱含的就是一個條件，如下所示：

  「如果成績及格的話...」
- 上述描述是人類的思考邏輯，轉換到程式語言，就是使用條件運算式（conditional expressions）來描述條件和執行運算，不同於第4章算術運算式是運算出常數值的結果，條件運算式的運算結果只有2個值，如下所示：

  條件成立 → 真（True）

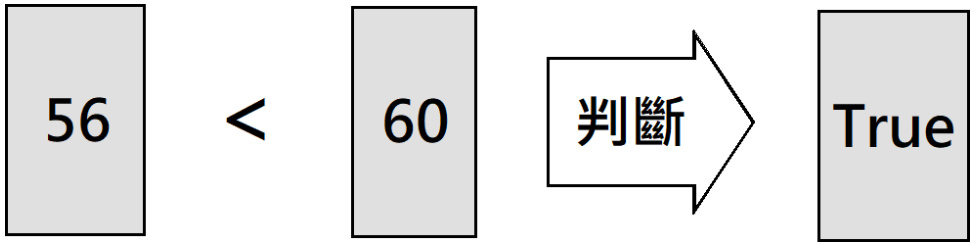
  條件不成立 → 假（False）

4-2-2 關係運算子的種類 – 說明

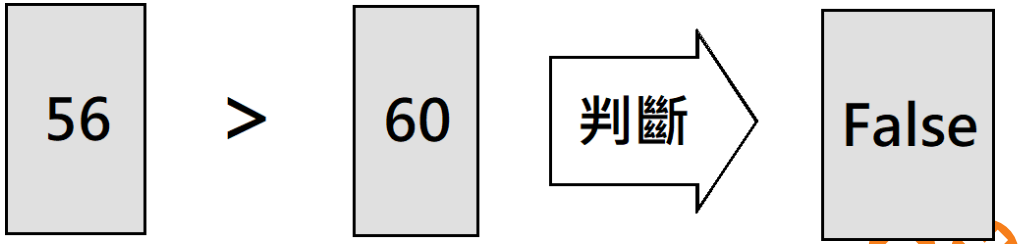
- Python是使用關係運算子來建立條件運算式，也就是我們熟知的大於、小於和等於條件的不等式，例如：成績56分是否不及格，需要和60分進行比較，如下所示：

  56 < 60

- 上述不等式的值56真的小於60分，所以條件成立（True），如下圖所示：



- 反過來，56 > 60的不等式就不成立（False），如下圖所示：



4-2-2 關係運算子的種類 – Python關係運算子

- Python關係運算子是2個運算元的二元運算子，其說明如下表所示：

運算子	說明
Opd1 == Opd2	右邊運算元Opd1「等於」左邊運算元Opd2
Opd1 != Opd2	右邊運算元Opd1「不等於」左邊運算元Opd2
Opd1 < Opd2	右邊運算元Opd1「小於」左邊運算元Opd2
Opd1 > Opd2	右邊運算元Opd1「大於」左邊運算元Opd2
Opd1 <= Opd2	右邊運算元Opd1「小於等於」左邊運算元Opd2
Opd1 >= Opd2	右邊運算元Opd1「大於等於」左邊運算元Opd2

4-2-2 關係運算子的種類 – Python布林資料型態

- Python支援布林資料型態，其值是True和False關鍵字（字首是大寫），條件運算式的運算結果就是布林資料型態的True或False。除了True和False關鍵字外，當下列變數值使用在條件或迴圈作為判斷條件時，這些變數值也視為False，如下所示：
  - 0、0.0：整數值0或浮點數值0.0。
  - []、()、{}：容器型態的空串列、空元組和空字典。
  - None：關鍵字None。

4-2-3 使用關係運算子 – 範例

- 我們可以使用第4-2-2節的關係運算子來建立條件運算式，一些條件運算式的範例和說明（Python程式：ch4-2-3.py），如下表所示：

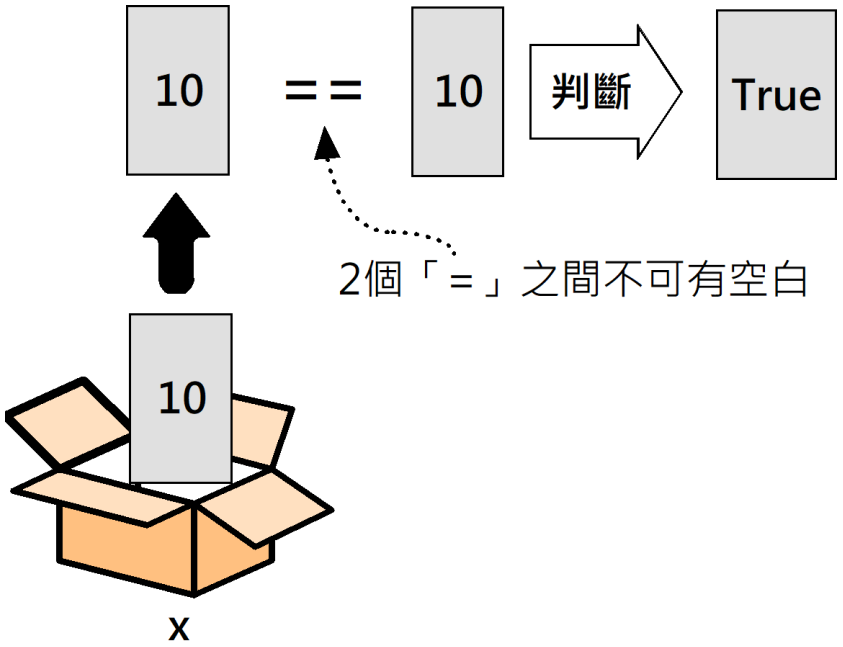
條件運算式	運算結果	說明
3 == 4	False	等於，條件不成立
3 != 4	True	不等於，條件成立
3 < 4	True	小於，條件成立
3 > 4	False	大於，條件不成立
3 <= 4	True	小於等於，條件成立
3 >= 4	False	大於等於，條件不成立

4-2-3 使用關係運算子 – 範例說明

- 上述條件運算式的運算元是常數值，如果其中有一個是變數，運算結果需視變數儲存的值而定，如下所示：

  x == 10

- 上述變數x的值如果是10，條件運算式就成立是True；如果變數x是其他值，就不成立為False，如下圖所示：



4-3 if單選條件敘述

- 4-3-1 if條件只執行單一程式敘述
- 4-3-2 if條件執行多行程式敘述：程式區塊



4-3-1 if條件只執行單一程式敘述 – 語法

- 在日常生活中，單選的情況十分常見，我們常常需要判斷氣溫是否有些涼，需要加件衣服；如果下雨需要拿把傘。
- if條件敘述是一種是否執行的單選題，只是決定是否執行程式敘述，如果條件運算式的結果為True，就執行程式敘述；否則就跳過程式敘述，這是一條額外的路徑，其語法如下所示：

```
if 條件運算式:
    程式敘述    # 條件成立執行此程式敘述
```

4-3-1 if條件只執行單一程式敘述 – 範例

- 例如：在第4-2-1節的成績條件：「如果成績及格的話，就和家人去旅行。」，改寫成的if條件，如下所示：

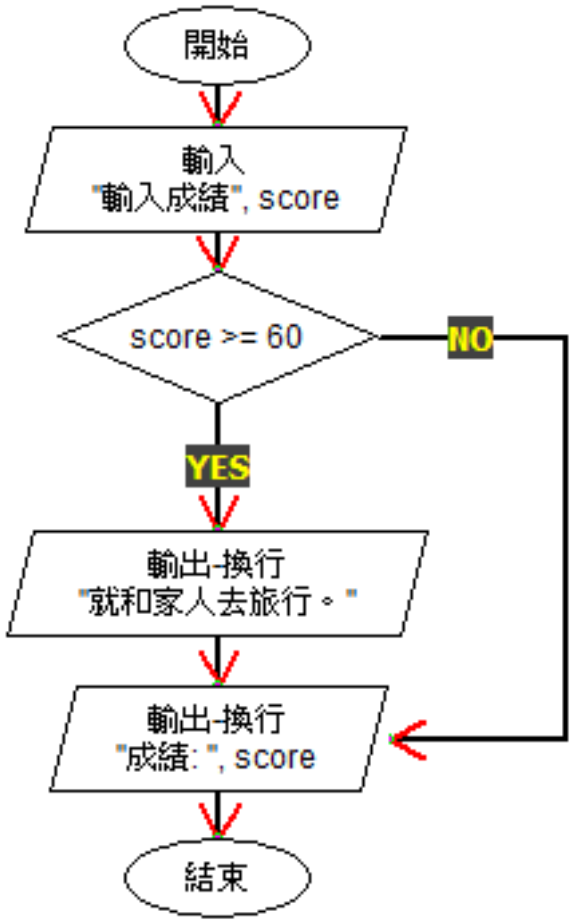
```
if 成績及格:
    顯示就和家人去旅行。
```

- 然後，我們可以量化成績及格分數是60分，顯示是使用print()函數，轉換成Python程式碼，如下所示：

```
if score >= 60:
    print("就和家人去旅行。")
```

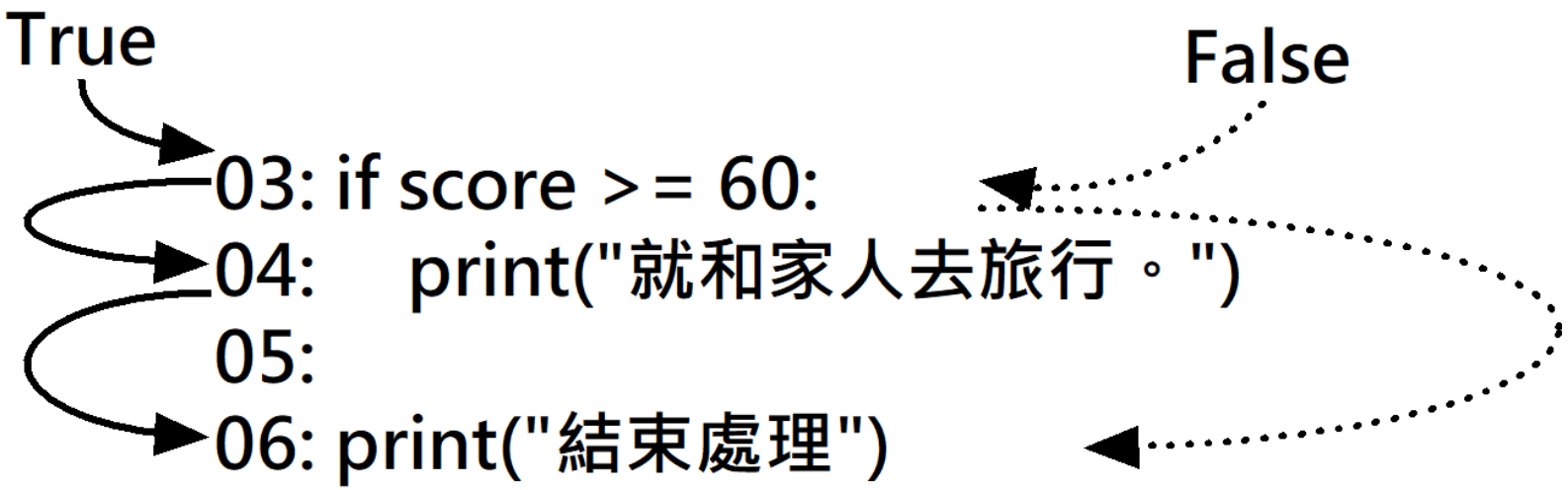
4-3-1 if條件只執行單一程式敘述 – 流程圖

- if條件敘述判斷變數score值是否大於等於60分，條件成立，就執行print()函數顯示訊息（額外多走的一條路）；反之，如果成績低於60分，就跳過此行程式敘述，直接執行下一行程式敘述（當作沒有發生），其流程圖（ch4-3-1.fpp，在主功能表執行【fChart流程圖直譯器】，可以開啟和執行此流程圖）如右圖所示：



4-3-1 if條件只執行單一程式敘述 – 執行結果

- Python程式if單選條件判斷的執行過程，如下圖所示：



4-3-2 if條件執行多行程式敘述：程式區塊語法

- 在第4-3-1節的if條件敘述，當條件成立時，只會執行一行程式敘述，如果需要執行2行或多行程式敘述時，在Python程式需要建立相同縮排的多個程式敘述，即程式區塊，其語法如下所示：

```
if 條件運算式:
    程式敘述1    # 條件成立執行的程式碼
    程式敘述2
    ……
```

4-3-2 if條件執行多行程式敘述：程式區塊範例

- 例如：當成績及格時，顯示2行訊息文字，如下所示：

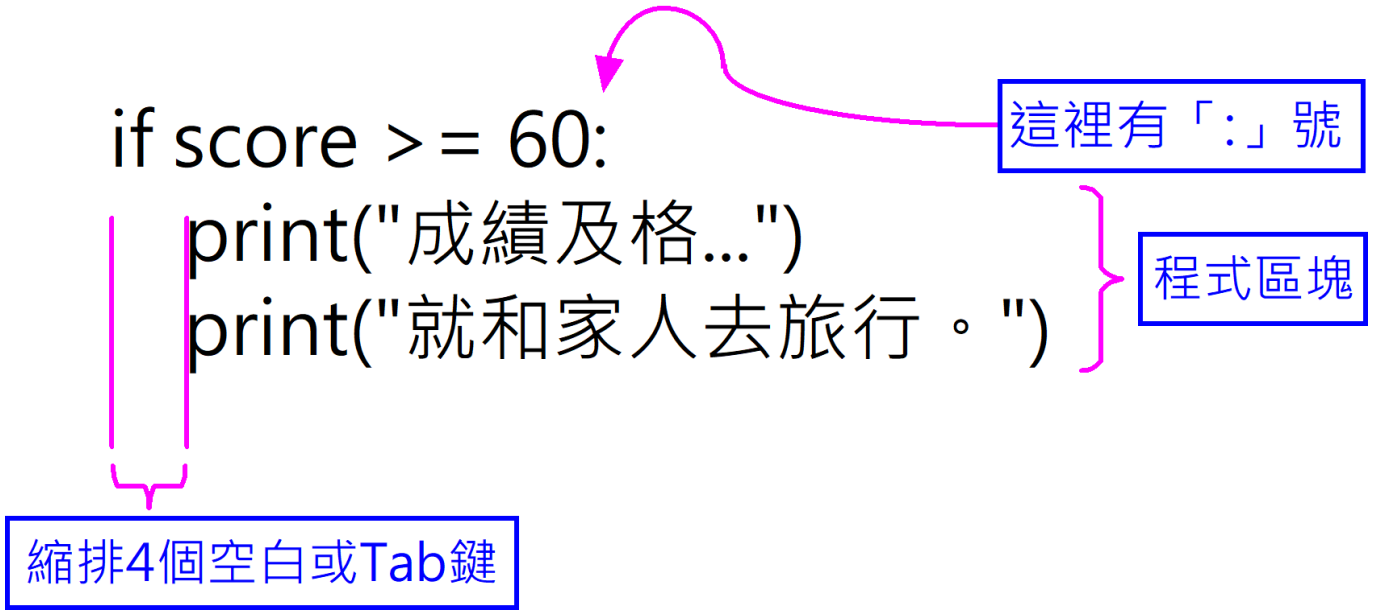
```
if 成績及格:
    顯示成績及格...
    顯示就和家人去旅行。
```

- 然後，我們可以轉換成Python程式碼，如下所示：

```
if score >= 60:
    print("成績及格...")
    print("就和家人去旅行。")
```

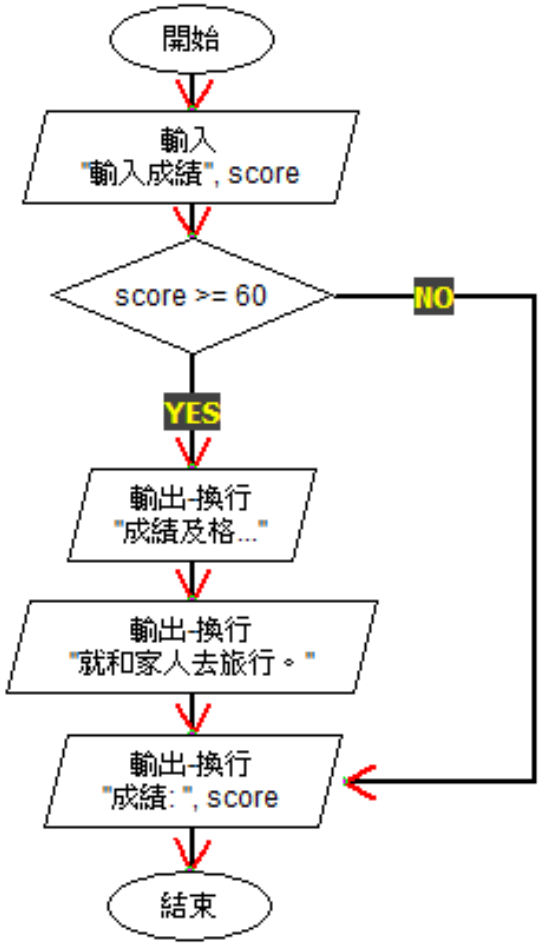
4-3-2 if條件執行多行程式敘述：程式區塊圖例

- Python程式區塊（Blocks）是從「:」號的下一行開始，整個之後相同縮排的多行程式敘述就是程式區塊，通常是縮排4個空白字元或1個Tab鍵，如下圖所示：



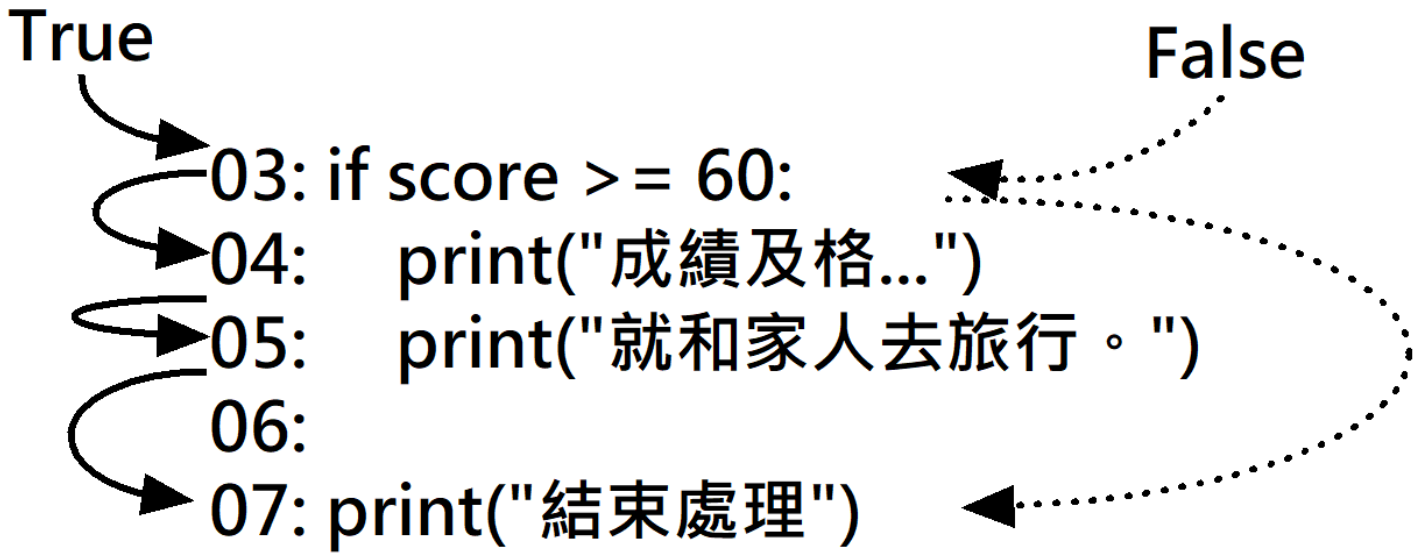
4-3-2 if條件執行多行程式敘述：程式區塊流程圖

- if條件執行多行程式敘述的流程圖（ch4-3-2.fpp），如右圖所示：



4-3-2 if條件執行多行程式敘述：程式區塊執行結果

- Python程式if條件執行多行程式敘述的執行過程，如下圖所示：



4-4 if/else二選一條件敘述

- 4-4-1 if/else二選一條件敘述
- 4-4-2 單行if/else條件敘述



4-4-1 if/else二選一條件敘述 – 說明與語法

- 日常生活的二選一條件敘述是一種二分法，可以將一個集合分成二種互斥的群組；超過60分屬於成績及格群組；反之為不及格群組，身高超過120公分是購買全票的群組；反之是購買半票的群組。
- 在第4-3節的if條件敘述是選擇執行或不執行的單選，進一步，如果是排它情況的兩個程式敘述，只能二選一，我們可以加上else敘述建立二條不同的路徑，其語法如下所示：

```
if 條件運算式:
    程式敘述1   # 條件成立執行的程式碼
else:
    程式敘述2   # 條件不成立執行的程式碼
```

4-4-1 if/else二選一條件敘述 – 語法

- 同樣的，如果條件成立或不成立時，執行多行程式敘述，我們一樣是使用相同縮排的程式區塊，其語法如下所示：

```
if 條件運算式:
    程式敘述1   # 條件成立執行的程式區塊
    程式敘述2
    ……
else:
    程式敘述1   # 條件不成立執行的程式區塊
    程式敘述2
    ……
```

4-4-1 if/else二選一條件敘述 – 範例

- 例如：學生成績以60分區分為是否及格的if/else條件敘述，如下所示：

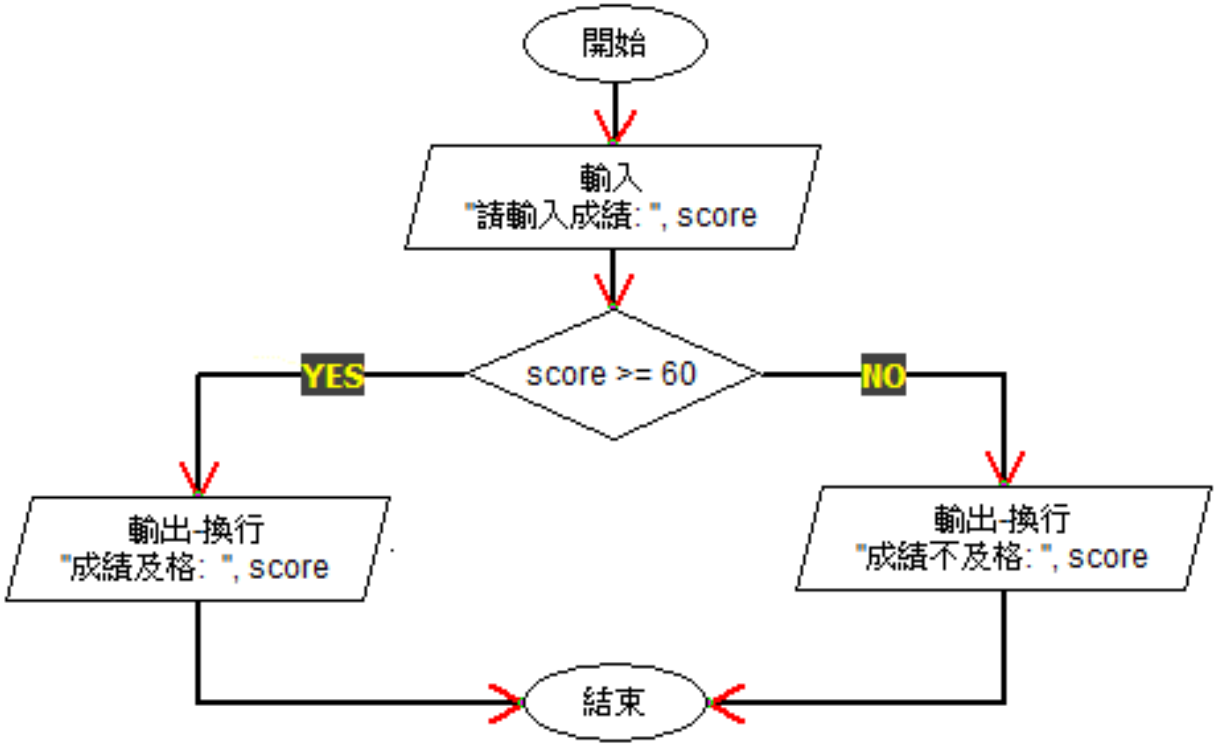
```
if 成績及格:
    顯示成績及格!
else:
    顯示成績不及格!
```

- 然後，我們可以轉換成Python程式碼，如下所示：

```
if score >= 60:
    print("成績及格:", score)
else:
    print("成績不及格:", score)
```

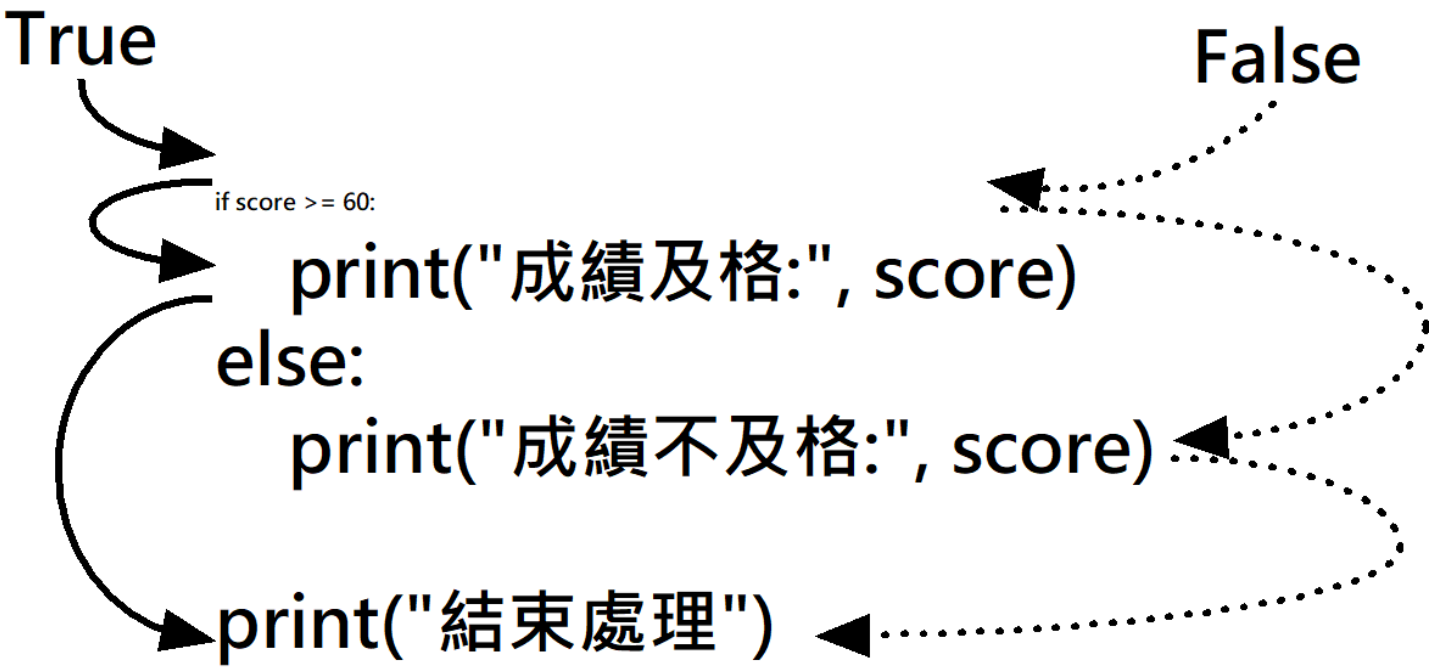
4-4-1 if/else二選一條件敘述 – 流程圖

- 因為成績有排它性，60分以上是及格分數，60分以下是不及格，所以只會執行其中一個程式區塊，走二條路徑中的其中一條，其流程圖（ch4-4-1.fpp）如下圖所示：



4-4-1 if/else二選一條件敘述 –執行結果

- Python程式if/else二選一條件敘述的執行過程，如下圖所示：



4-4-2 單行if/else條件敘述

- Python沒有C/C++、Java和C#語言的條件運算式（Conditional Expressions），不過，我們可以使用單行if/else條件敘述來代替，其語法如下所示：

  變數 = 變數1 if 條件運算式 else 變數2

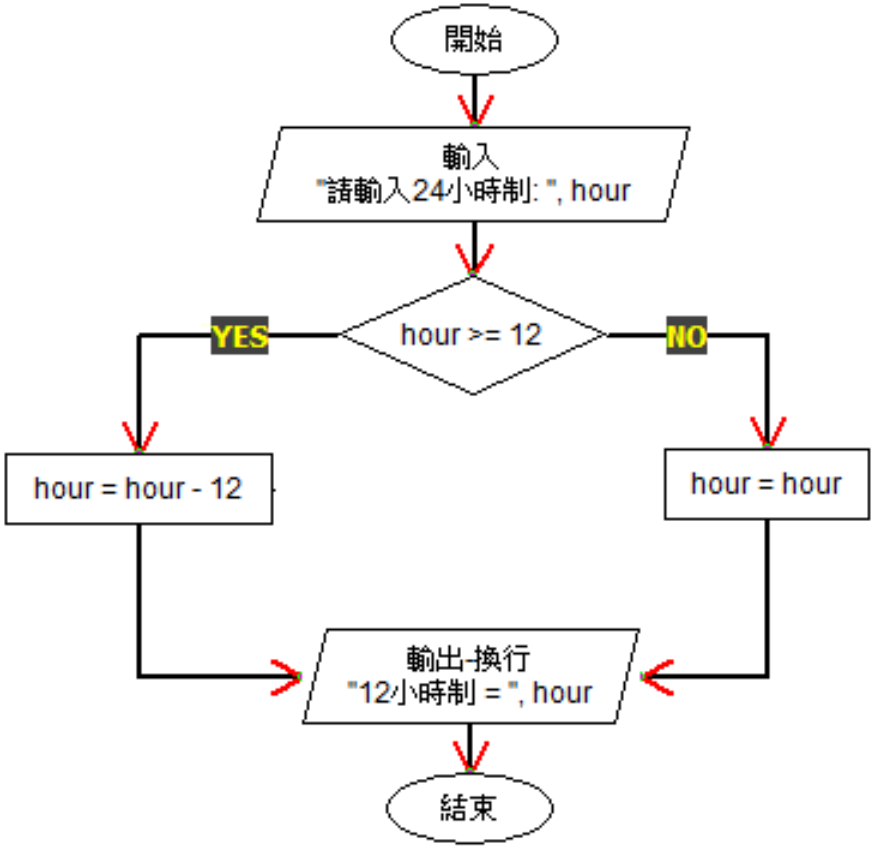
- 上述指定敘述的「=」號右邊是單行if/else條件敘述，如果條件成立，就將【變數】指定成【變數1】的值；否則就指定成【變數2】的值。例如：12/24制的時間轉換運算式，如下所示：

  hour = hour-12 if hour >= 12 else hour

- 上述程式碼開始是條件成立指定的變數值或運算式，接著是if加上條件運算式，最後else之後是不成立，所以，當條件為True，hour變數值為hour-12；False是hour。

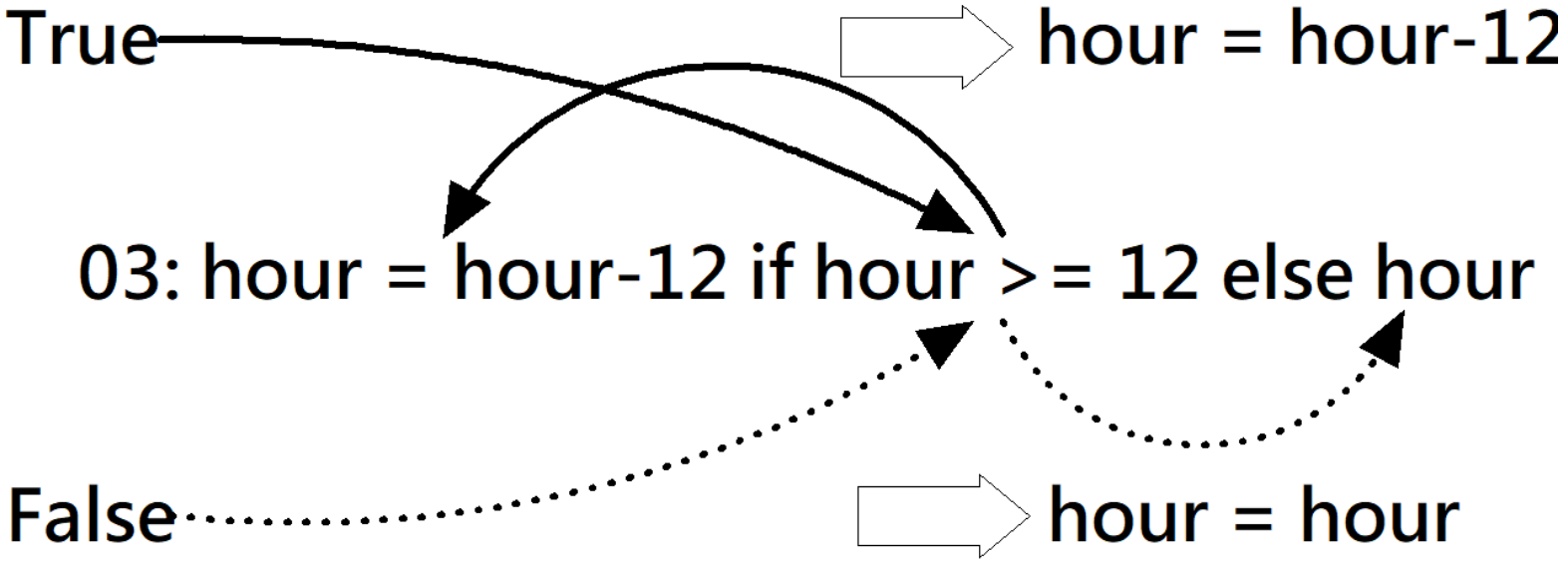
4-4-2 單行if/else條件敘述 – 流程圖

- 條件運算式的流程圖與上一節if/else相似，其流程圖（ch4-4-2.fpp）如下圖所示：



4-4-2 單行if/else條件敘述 – 執行結果

- Python程式單行if/else二選一條件敘述的執行過程，如下圖所示：



4-5 if/elif/else多選一條件敘述 – 說明

- 如果回家的路有多種選擇，不是二選一兩種，而是多種，因為條件是多種情況，我們需要使用多選一條件敘述。Python多選一條件敘述是if/else條件的擴充，使用elif關鍵字再新增一個條件判斷，來建立多選一條件敘述。

4-5 if/elif/else多選一條件敘述 – 語法

```
if 條件運算式1:
    程式敘述1   # 條件運算式1成立執行的程式碼
    程式敘述2   #，否則執行elif程式敘述
    ……
elif 條件運算式2:
    程式敘述3   # 條件運算式1不成立
    程式敘述4   # 且條件運算式2成立執行的程式碼
    ……
elif 條件運算式3:
    程式敘述5   # 條件運算式1和2不成立
    ……          # 且條件運算式3成立執行的程式碼
else:
    程式敘述6   # 所有條件運算式都不成立執行的程式碼
    ……
```

4-5 if/elif/else多選一條件敘述 – 範例1

- 例如：功能表選項值是1~3，我們可以使用if/elif/else條件敘述判斷輸入選項值是1、2或3，如下所示：

```
if 選項值是1:
    顯示輸入選項值是1
elif 選項值是2:
    顯示輸入選項值是2
elif 選項值是3:
    顯示輸入選項值是3
else:
    顯示請輸入1~3選項值
```

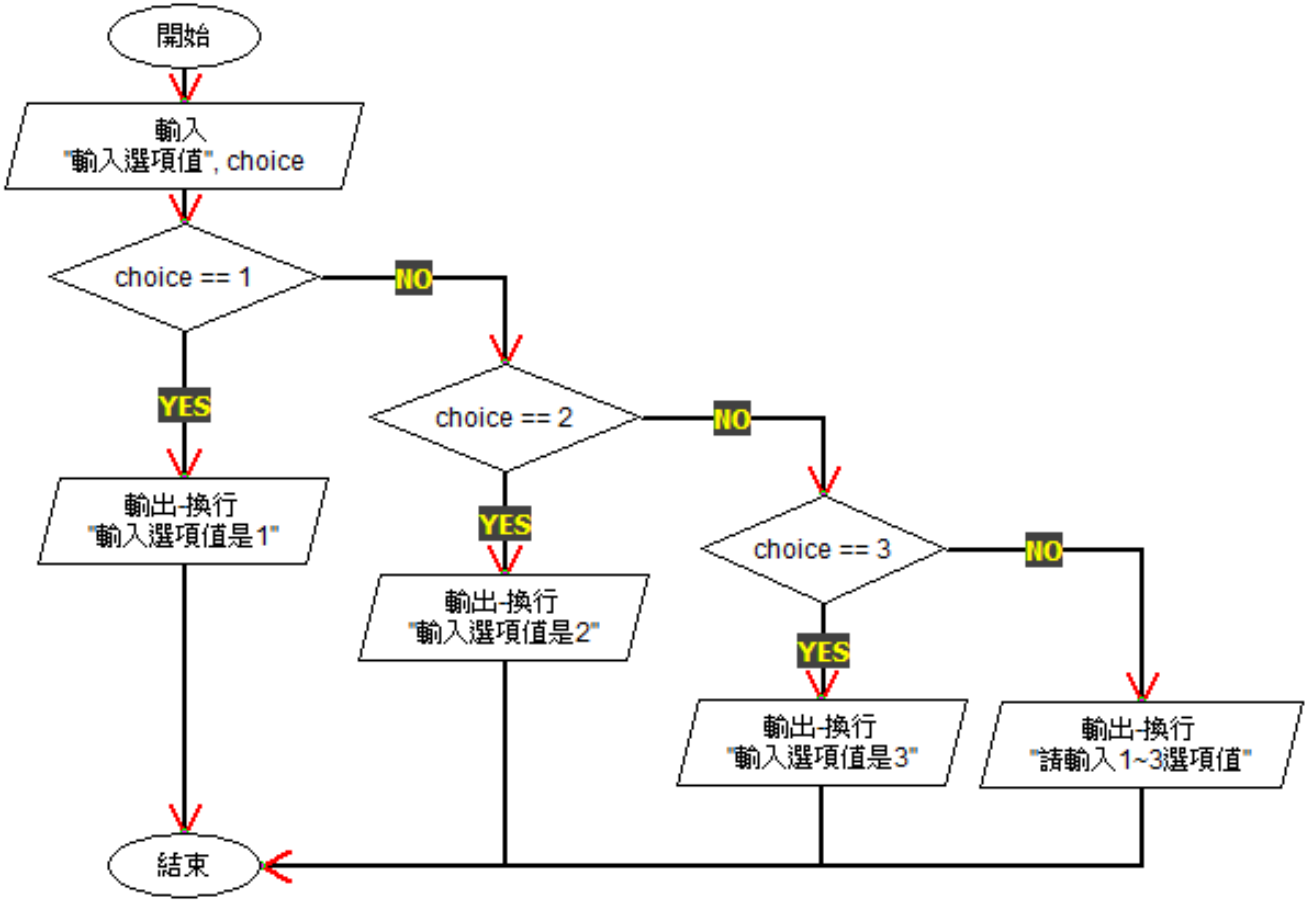
4-5 if/elif/else多選一條件敘述 – 範例2

- 然後，我們可以轉換成Python程式碼，如下所示：

```
if choice == 1:
    print("輸入選項值是1")
elif choice == 2:
    print("輸入選項值是2")
elif choice == 3:
    print("輸入選項值是3")
else:
    print("請輸入1~3選項值")
```

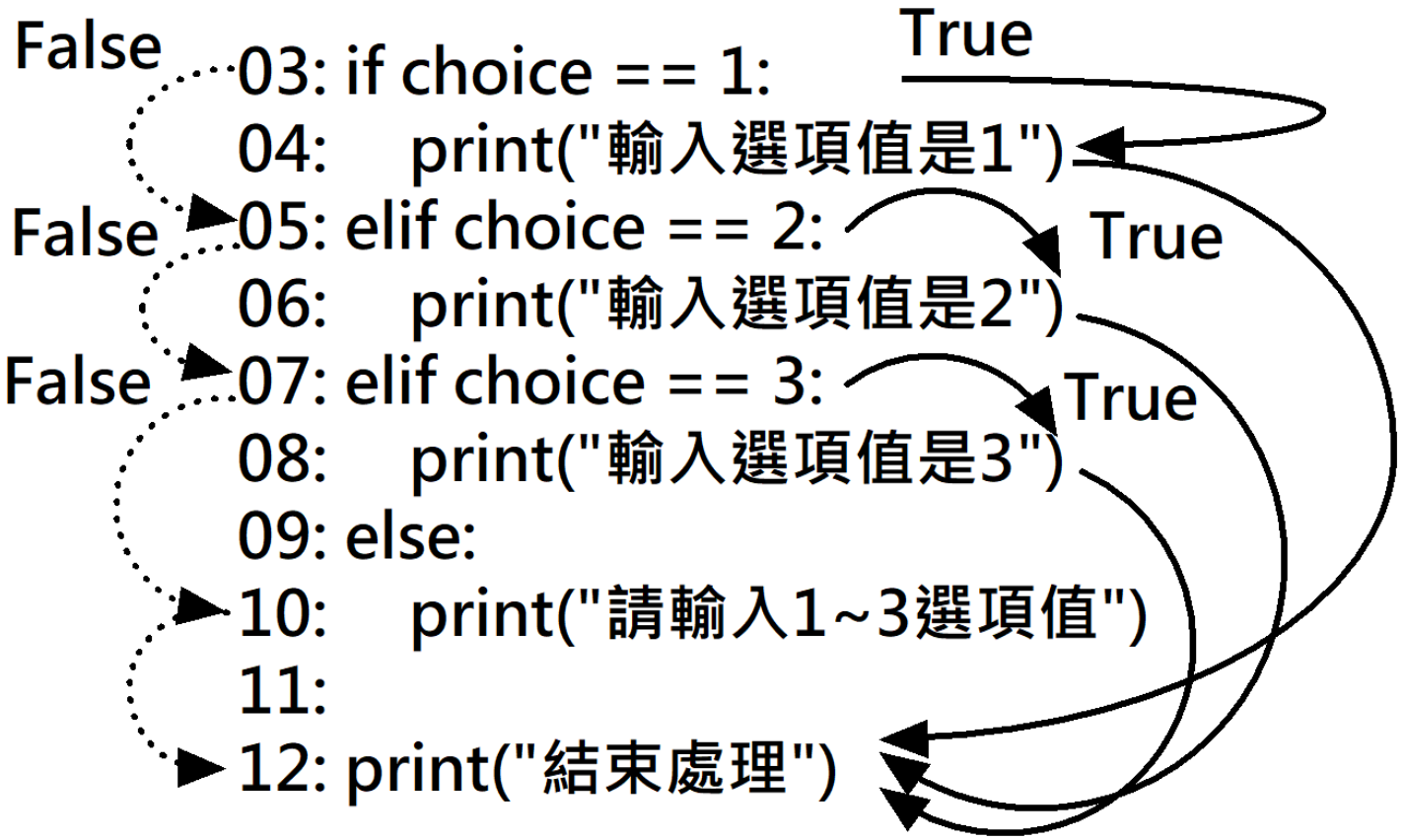
4-5 if/elif/else多選一條件敘述 – 流程圖

- 其流程圖（ch4-5.fpp）如下圖所示：



4-5 if/elif/else多選一條件敘述 – 執行結果

- Python程式if/elif/else多選一條件敘述的執行過程，如下圖所示：



4-6 在條件敘述使用邏輯運算子

- 4-6-1 認識邏輯運算子
- 4-6-2 Python邏輯運算子
- 4-6-3 使用邏輯運算子建立複雜條件



4-6-1 認識邏輯運算子

- 邏輯運算子（logical operators）可以連接多個第4-2節的條件運算式來建立複雜的條件運算式，如下所示：

  身高大於50「且」身高小於200 →「符合身高條件」

- 上述描述的條件比第4-2節複雜，共有2個條件運算式，如下所示：

  身高大於50

  身高小於200

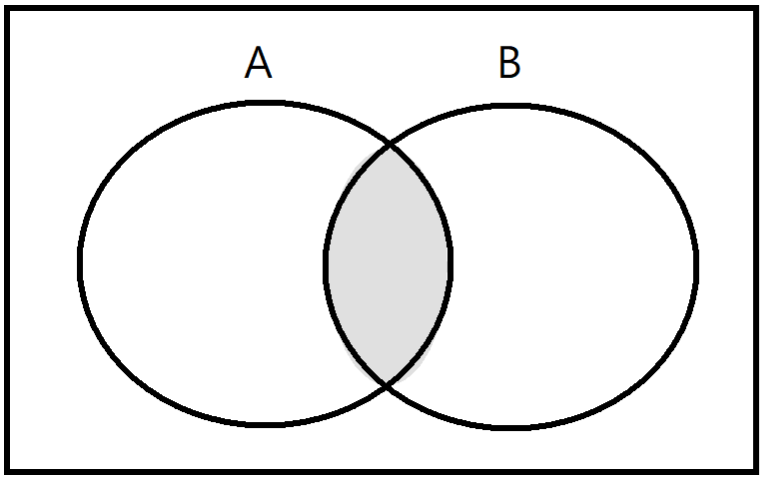
4-6-1 認識邏輯運算子

- 上述2個條件運算式是使用「且」連接，這就是邏輯運算子，其目的是進一步判斷2個條件運算式的條件組合，可以得到最後的True或False。以此例的複雜條件可以寫成Python的「and」且邏輯運算式，如下所示：

  身高大於50 and 身高小於200

4-6-2 Python邏輯運算子
「and」運算子的「且」運算

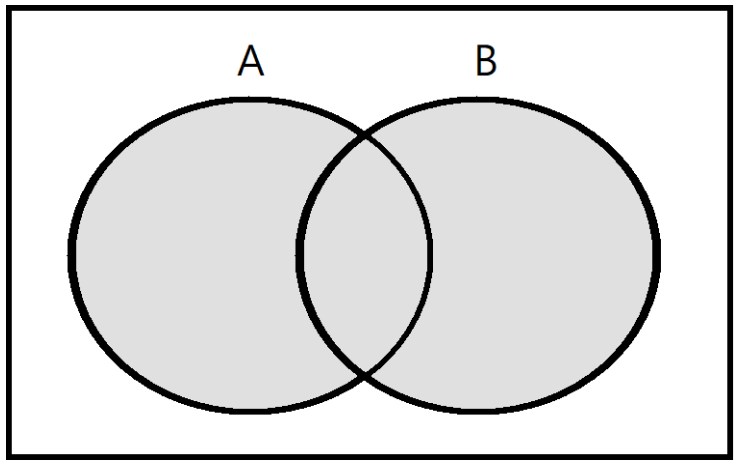
- 「and」運算子的「且」運算是指連接的左右2個運算元都為True，運算式才為True，其圖例和真假值表，如下所示：



A	B	A and B
False	False	False
False	True	False
True	False	False
True	True	True

4-6-2 Python邏輯運算子 「or」運算子的「或」運算

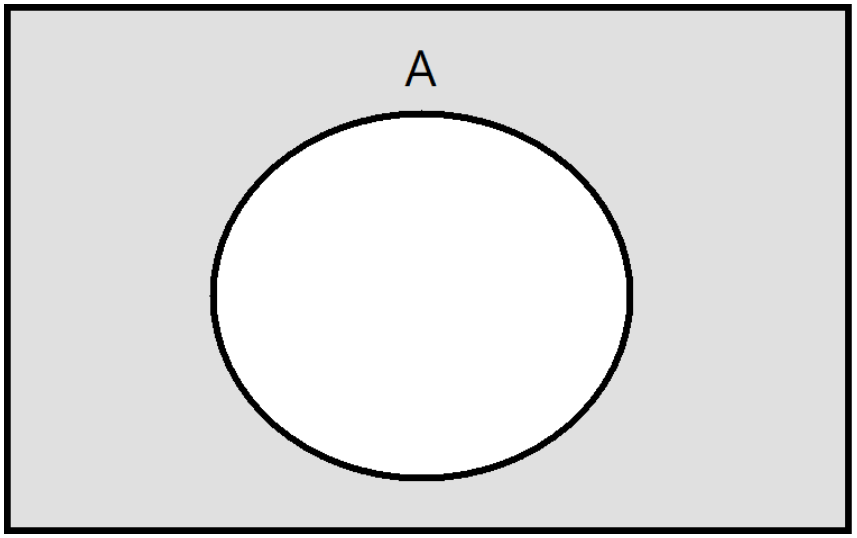
- 「or」運算子的「或」運算是連接的2個運算元，任一個為True，運算式就為True，其圖例和真假值表，如下所示：



A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

4-6-2 Python邏輯運算子 「not」運算子的「非」運算

- 「not」運算子的「非」運算是傳回運算元相反的值，True成為False；False成為True，其圖例和真假值表，如下所示：



A	not A
False	True
True	False

4-6-3 使用邏輯運算子建立複雜條件 – 範例

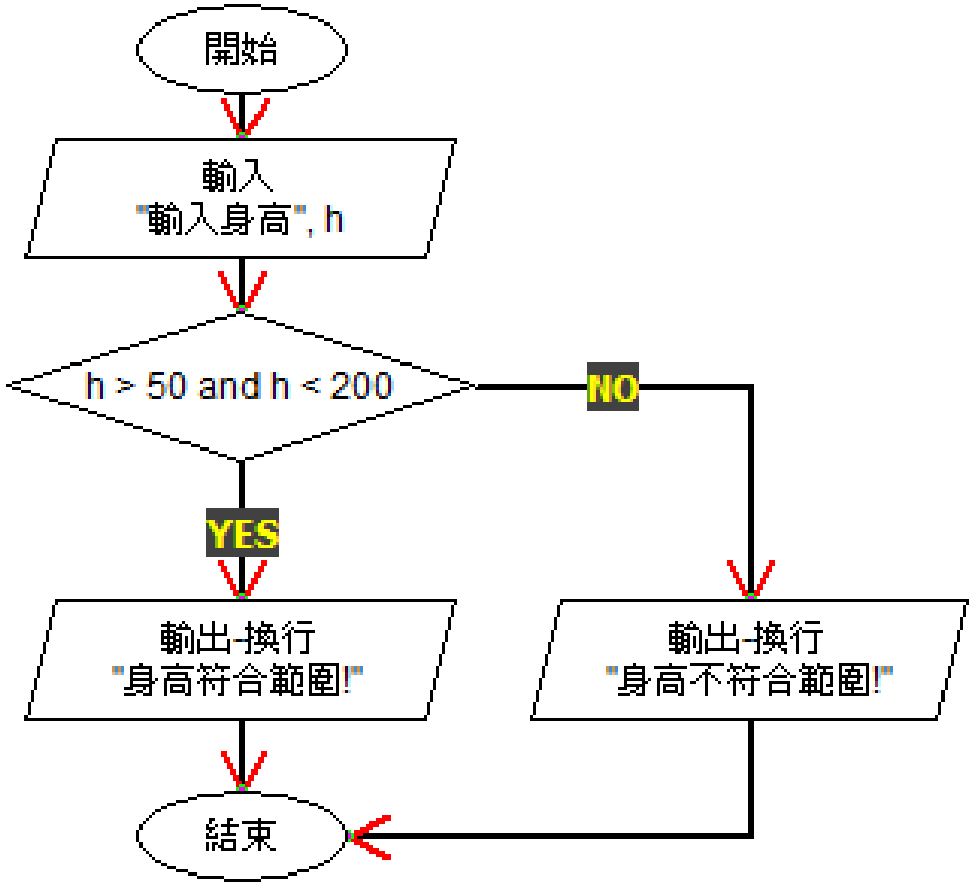
- 當if條件敘述的條件運算式有多個，我們可以使用邏輯運算子連接多個條件來建立複雜條件。例如：身高大於50（公分）「且」身高小於200（公分）就符合身高條件；否則不符合，我們可以使用「and」運算子建立邏輯運算式來判斷輸入的身高是否符合，如下所示：

```
if h > 50 and h < 200:
    print("身高符合範圍!")
else:
    print("身高不符合範圍!")
```

- 上述if/else條件敘述的判斷條件是一個邏輯運算式，條件成立，就顯示身高符合範圍，反之，不符合範圍。

4-6-3 使用邏輯運算子建立複雜條件 –流程圖

- 其流程圖（ch4-6-3.fpp）如下圖所示：



第5章 重複執行程式碼

- 5-1 認識迴圈敘述
- 5-2 for計數迴圈
- 5-3 while條件迴圈
- 5-4 改變迴圈的執行流程
- 5-5 巢狀迴圈與無窮迴圈
- 5-6 在迴圈中使用條件敘述



5-1 認識迴圈敘述

- 在第4章的條件判斷是讓程式走不同的路，但是，我們回家的路還有另一種情況是繞圈圈，例如：為了今天的運動量，在圓環繞了3圈才回家；為了看帥哥、正妹或偶像，不知不覺繞了幾圈來多看幾次。在日常生活中，我們常常需要重複執行相同工作，如下所示：
  - **在畢業前 → 不停的寫作業**
  - **在學期結束前 → 不停的寫Python程式**
  - **重複說5次"大家好!"**
  - **從1加到100的總和**

5-1 認識迴圈敘述

- 上述重複執行工作的4個描述中，前2個描述的執行次數未定，因為畢業或學期結束前，到底會有幾個作業，或需寫幾個Python程式，可能真的要到畢業後，或學期結束才會知道，我們並沒有辦法明確知道迴圈會執行多少次。

5-1 認識迴圈敘述

- 因為，這種情況的重複工作是由條件來決定迴圈是否繼續執行，稱為條件迴圈，重複執行寫作業或寫Python程式工作，需視是否畢業，或學期結束的條件而定，在Python是使用while條件迴圈來處理這種情況的重複執行程式碼。
- 後2個描述很明確可以知道需執行5次來說"大家好!"，從1加到100，就是重複執行100次加法運算，這些已經明確知道執行次數的工作，我們是直接使用Python的for計數迴圈來處理重複執行程式碼。

「迴圈的主要目的是簡化程式碼，可以將重複的複雜工作簡化成迴圈敘述，讓我們不用再寫出冗長的重複程式碼或運算式，就可以完成所需的工作。」

5-2 for計數迴圈

- 5-2-1 使用for計數迴圈
- 5-2-2 在for迴圈的程式區塊使用計數器變數
- 5-2-3 for迴圈的應用：計算總和
- 5-2-4 range()範圍函數



5-2-1 使用for計數迴圈 – 語法

- Python的for計數迴圈是一種執行固定次數的迴圈，其語法是使用range()函數來產生計數，如下所示：

```
for 計數器變數 in range(起始值, 終止值+1):
    程式敘述1
    程式敘述2
    …
```

- 上述for迴圈的計數器變數是for關鍵字之後的變數，迴圈的執行次數是從range()括號的起始值開始，執行到終止值為止，因為不包含終止值，所以第2個參數值是【終止值+1】。請注意！在range()函數的右括號後需加上「:」冒號，因為下一行是縮排程式敘述的程式區塊。

5-2-1 使用for計數迴圈 – 範例

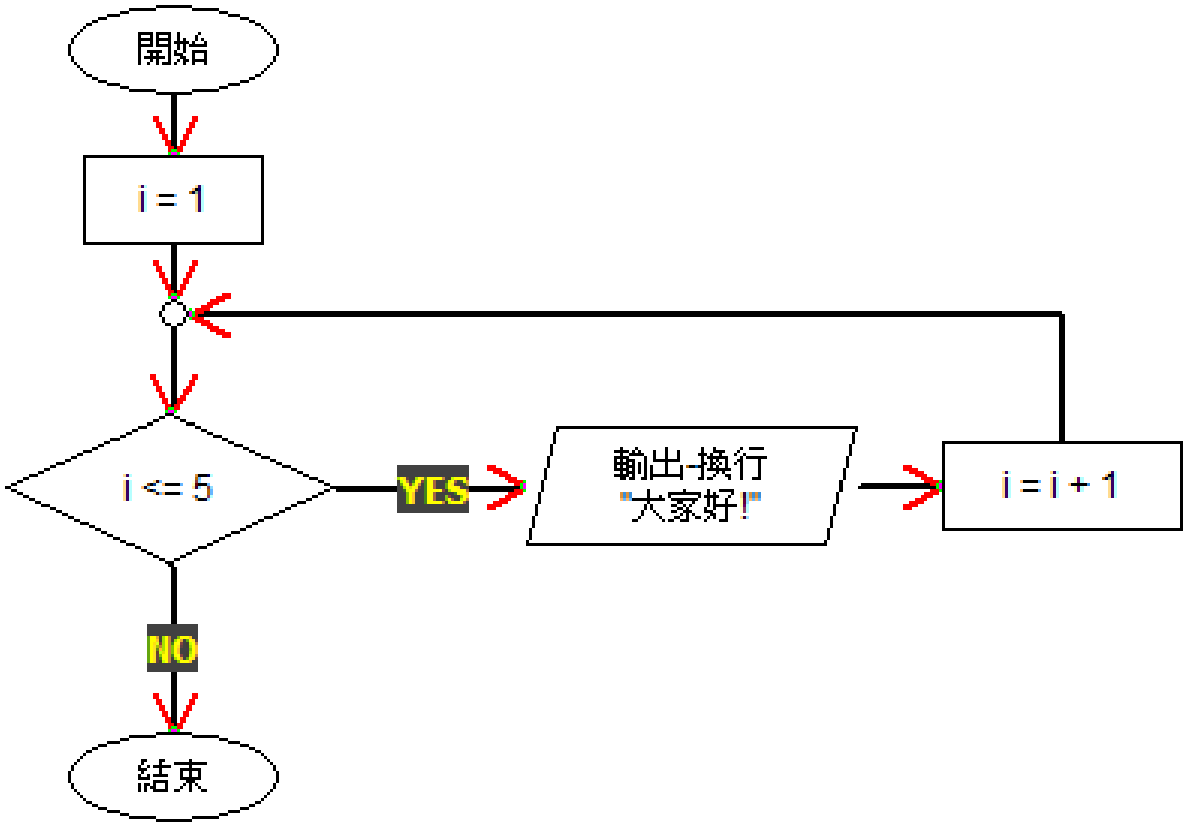
- 例如：我們準備將第5-1節的「重複說5次"大家好!"」使用for迴圈來實作，如下所示：

```
for i in range(1, 6):
    print("大家好!")
```

- 上述for迴圈的執行次數是從1執行到6-1 = 5，共5次，可以顯示5次"大家好!"。

5-2-1 使用for計數迴圈 – 流程圖

- 其流程圖（ch5-2-1.fpp）如下圖所示：



5-2-1 使用for計數迴圈 – 更多for迴圈範例

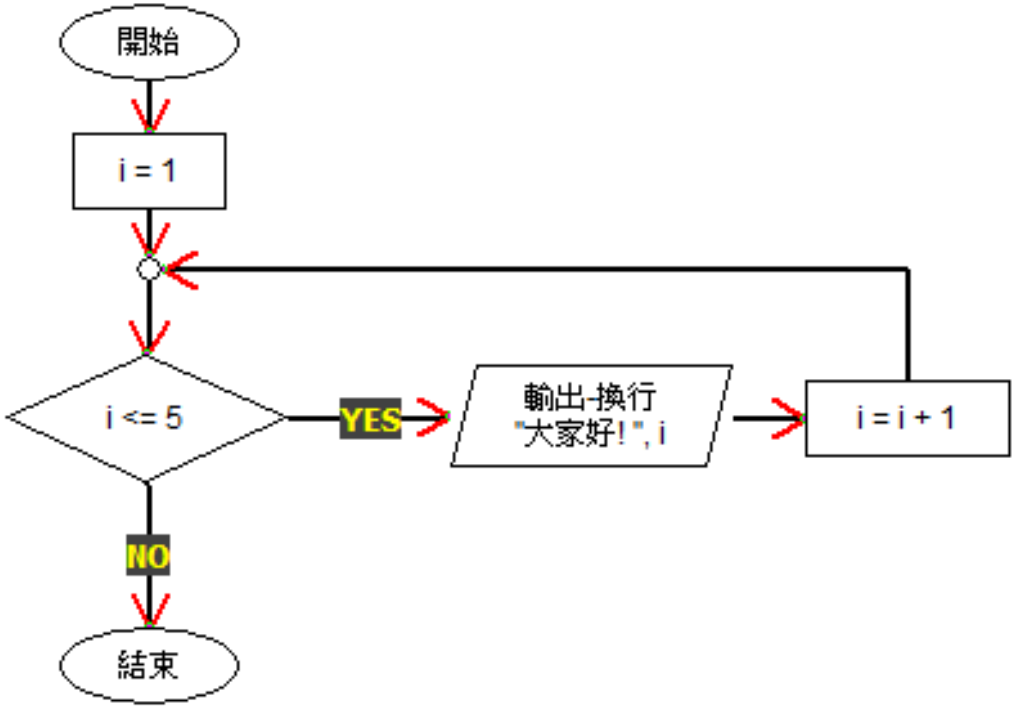
- 同樣技巧，我們可以使用for迴圈來重複輸出其他內容的訊息文字，如下所示：

```
for i in range(1, 6):
    print("參加社團活動!")
```

- 上述for迴圈執行從1至5共5次，共輸出5次"參加社區活動!"訊息文字。

5-2-2 在for迴圈的程式區塊使用計數器變數

- 在第5-2-1節的for迴圈共執行5次，輸出5次"大家好!"訊息文字，讀者有注意到嗎？計數器變數值是從1~5，就是輸出訊息文字的次數，我們可以在for迴圈的程式區塊使用計數器變數來顯示執行次數，其流程圖（ch5-2-2.fpp）如下圖所示：



5-2-2 在for迴圈的程式區塊使用計數器變數

```
01: for i in range(1, 6):    # for計數迴圈
02:     print("第", i, "次大家好!")
03:
04: print("結束迴圈處理")
```

5-2-3 for迴圈的應用：計算總和(說明)

- 在for迴圈的程式區塊可以使用變數進行所需的數學運算，例如：第5-2-2節or迴圈可以顯示執行次數，從執行次數值可以清楚看出，的f如果將每一次顯示的計數器變數值相加，就相當於是在執行1加到5的總和運算，如下所示：

  1 + 2 + 3 + 4 + 5

5-2-3 for迴圈的應用：計算總和(程式碼)

- 運算式可以宣告total變數，改建立for迴圈來計算總和，如下所示：

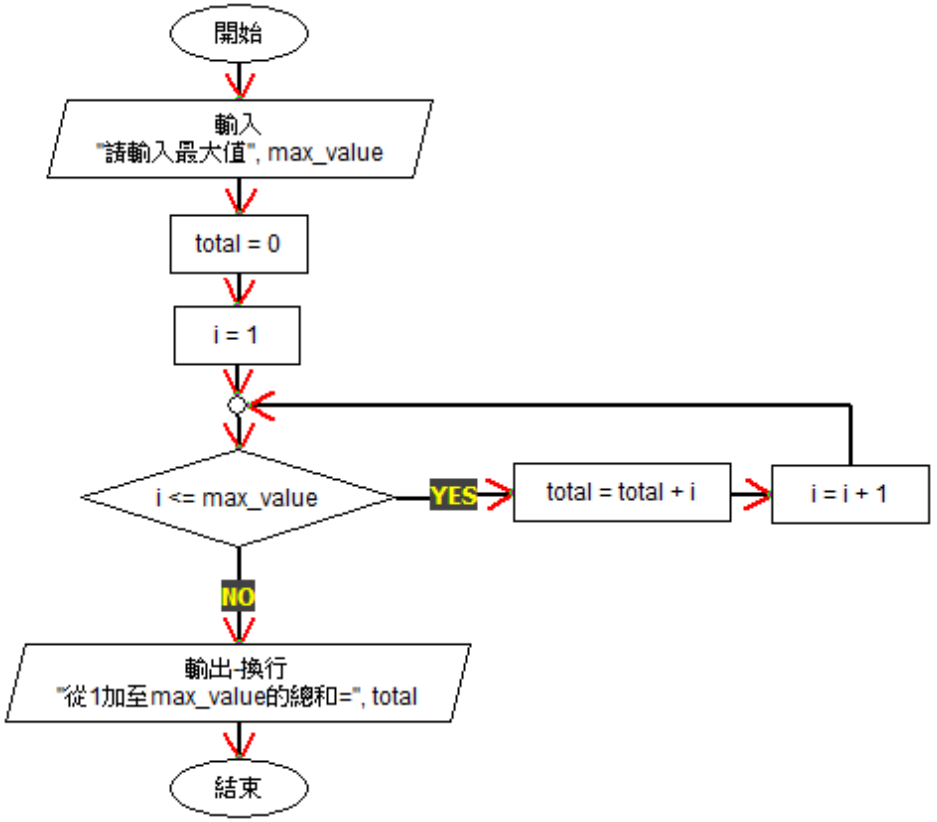
```
total = 0
for i in range(1, 6):
    total = total + i
```

- 上述for迴圈每執行一次迴圈，就會將計數器變數i的值加入變數total，執行完5次迴圈，可以計算出1加至5的總和。

5-2-3 for迴圈的應用：計算總和(流程圖)

- 更進一步，for迴圈的range()函數，可以在第2個參數使用變數，如下所示：

```
for i in range(1, max_value+1):
    total = total + i
```



5-2-4 range()範圍函數

- Python的for迴圈事實上是一種迭代（Iteration）操作，也就是依序從in關鍵字之後的集合取出其值，每次取一個，可以使用在Python容器資料型態，一一取出容器中的元素，特別適用在不知道有多少元素的情況。
- 事實上，for迴圈之所以成為計數迴圈，就是因為range()函數，此函數可以依序產生for迴圈所需一序列的整數值。range()函數是Python內建函數，可以分別有1、2和3個參數，如下所示：

5-2-4 range()範圍函數

1個參數的range()函數：ch5-2-4.py

- Python的range()函數如果只有1個參數，參數是【終止值+1】，預設的起始值是0，如下表所示：

range()函數	整數值範圍
range(5)	0~4
range(10)	0~9
range(11)	0~10

5-2-4 range()範圍函數

2個參數的range()函數：ch5-2-4a.py

- Python的range()函數如果有2個參數，第1參數是起始值，第2個參數是【終止值+1】，如下表所示：

range()函數	整數值範圍
range(1, 5)	1~4
range(2, 10)	2~9
range(1, 11)	1~10

5-2-4 range()範圍函數

3個參數的range()函數：ch5-2-4b.py

- Python的range()函數如果有3個參數，第1參數是起始值，第2個參數是【終止值+1】，第3個參數是增量值，如下表所示：

range()函數	整數值範圍
range(1, 11, 2)	1、3、5、7、9
range(1, 11, 3)	1、4、7、10
range(1, 11, 4)	1、5、9
range(0, -10, -1)	0、-1、-2、-3、-4...-7、-8、-9
range(0, -10, -2)	0、-2、-4、-6、-8

5-3 while條件迴圈

- 5-3-1 使用while迴圈
- 5-3-2 將for迴圈改成while迴圈



5-3-1 使用while迴圈 – 語法

- while迴圈是在程式區塊的開頭檢查條件，如果條件為True才允許進入迴圈執行，如果一直為True，就持續重複執行迴圈，直到條件成為False為止，其語法如下所示：

```
while 條件運算式:
    程式敘述1
    程式敘述2
    ...
```

- 上述while迴圈是在程式區塊開頭檢查條件，如果條件為True就進入迴圈執行；False結束執行，所以迴圈執行次數是直到條件False為止（別忘了在條件運算式後需加上「:」冒號）。

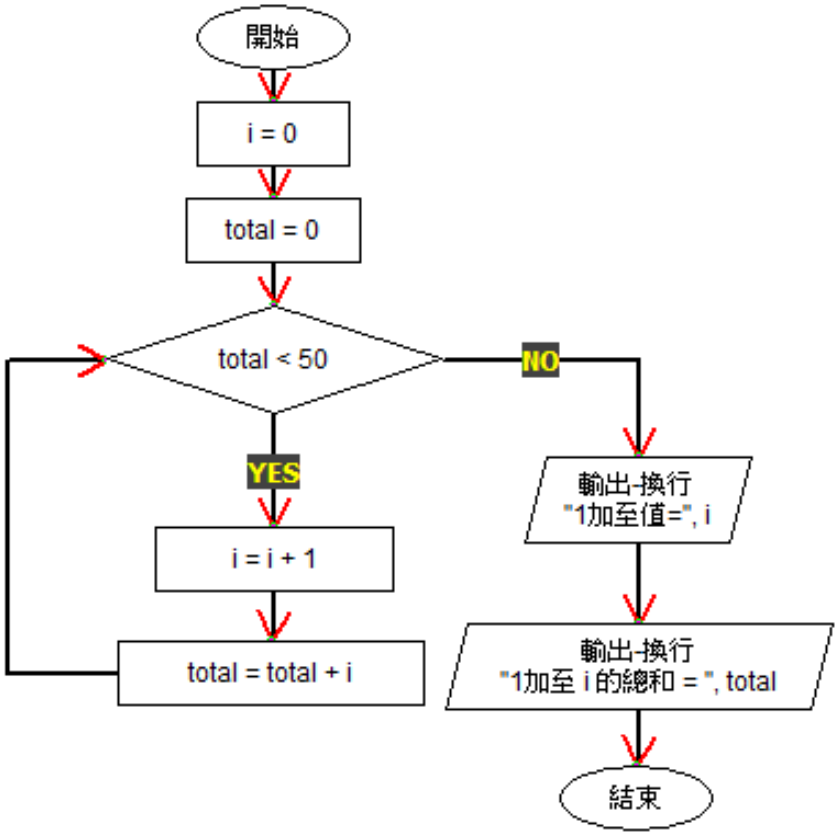
5-3-1 使用while迴圈 – 範例

- 例如：計算1加到多少時的總和會大於等於50，因為迴圈執行次數需視運算結果而定，迴圈執行次數未定，我們可以使用while條件迴圈來執行總和計算，如下所示：

```
while total < 50:
    i = i + 1
    total = total + i
```

5-3-1 使用while迴圈 – 流程圖

- 變數i和total的初值都是0，while迴圈的變數i值從1、2、3、4....相加計算總和是否大於等於50，等到條件「total < 50」不成立結束迴圈，就可以計算出(1+2+3+4+..+i) >= 50的i值，其流程圖（ch5-3-1.fpp）如右圖所示：



5-3-2 將for迴圈改成while迴圈 - 原始for迴圈

- 在ch5-2-3.py是使用for迴圈計算1加至max_value的總和，我們準備將此for迴圈改為while迴圈，range()函數首先改寫成完整的3個參數，如下所示：

```
total = 0
...
for i in range(1, max_value+1, 1):
    total = total + i
```

5-3-3 將for迴圈改成while迴圈 - 改為while迴圈

- 在for迴圈的range()函數，第1個參數是計數器變數的初值，第2個參數是結束條件「i <= max_value」條件，這就是while迴圈的條件，for迴圈的計數器變數i是while迴圈的計數器變數，如下所示：

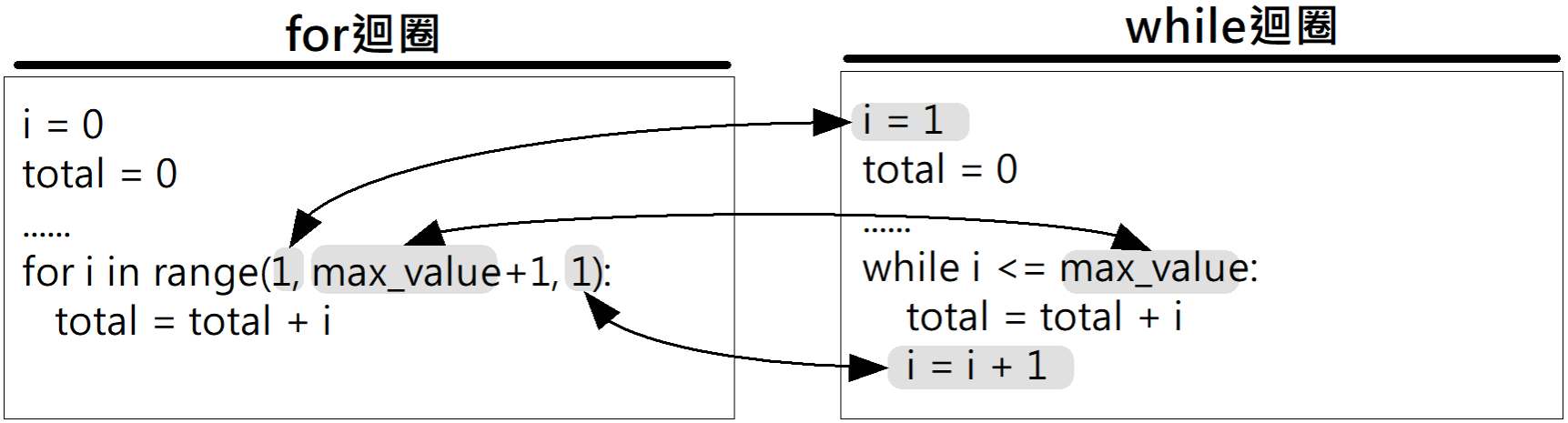
```
i = 1
total = 0
...
while i <= max_value:
    total = total + i
    i = i + 1
```

5-3-3 將for迴圈改成while迴圈 - for迴圈轉換成while迴圈的基本步驟

Step 1：在進入while迴圈之前需要指定計數器變數的初值。

Step 2：在while迴圈判斷條件是否成立，如為True，就繼續執行迴圈的程式區塊；不成立False時，結束迴圈的執行。

Step 3：在迴圈程式區塊需要自行使用程式碼增減計數器變數值，然後回到Step 2測試是否繼續執行迴圈。



5-4 改變迴圈的執行流程

- 5-4-1 break敘述跳出迴圈
- 5-4-2 continue敘述繼續迴圈
- 5-4-3 在迴圈使用else程式區塊



5-4-1 break敘述跳出迴圈 – 說明

- Python可以使用break和continue敘述來改變迴圈的執行流程，break敘述跳出迴圈；continue敘述能夠馬上繼續執行下一次迴圈。
- Python的break敘述可以強迫終止for和while迴圈的執行。雖然迴圈敘述可以在開頭測試結束條件，但有時需要在迴圈的程式區塊中來測試結束條件，break敘述可以在迴圈中搭配if條件敘述來進行條件判斷，成立就使用break敘述跳出迴圈的程式區塊。

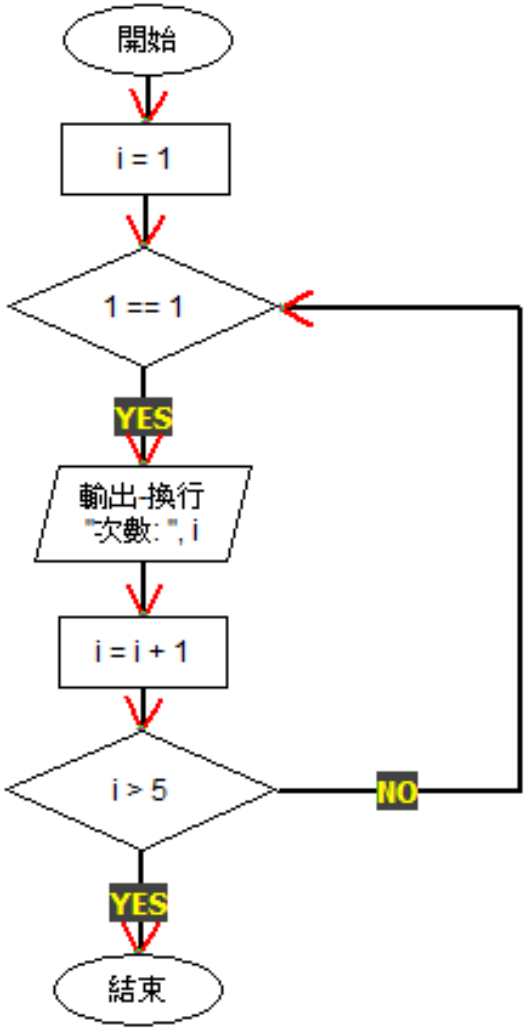
5-4-1 break敘述跳出迴圈 – 範例

- while迴圈是無窮迴圈，在迴圈中使用if條件敘述進行判斷，當「i > 5」條件成立，就執行break敘述跳出迴圈，可以跳至while之後的程式敘述，顯示次數1到5，因為if條件是在程式區塊的最後，事實上，這就是後測式迴圈，如下所示：

```
while True:
    print("第", i, "次")
    i = i + 1;
    if i > 5:
        break
```

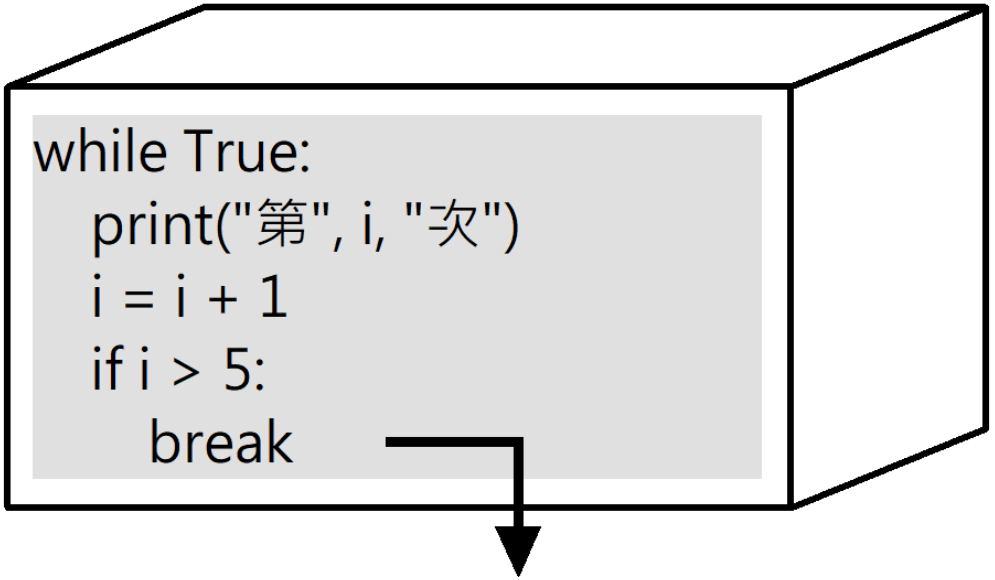
5-4-1 break敘述跳出迴圈 – 流程圖

- 其流程圖（ch5-4-1.fpp）如右圖所示：



5-4-1 break敘述跳出迴圈 – 執行結果

- Python程式使用break敘述跳出while迴圈的過程，如下圖所示：



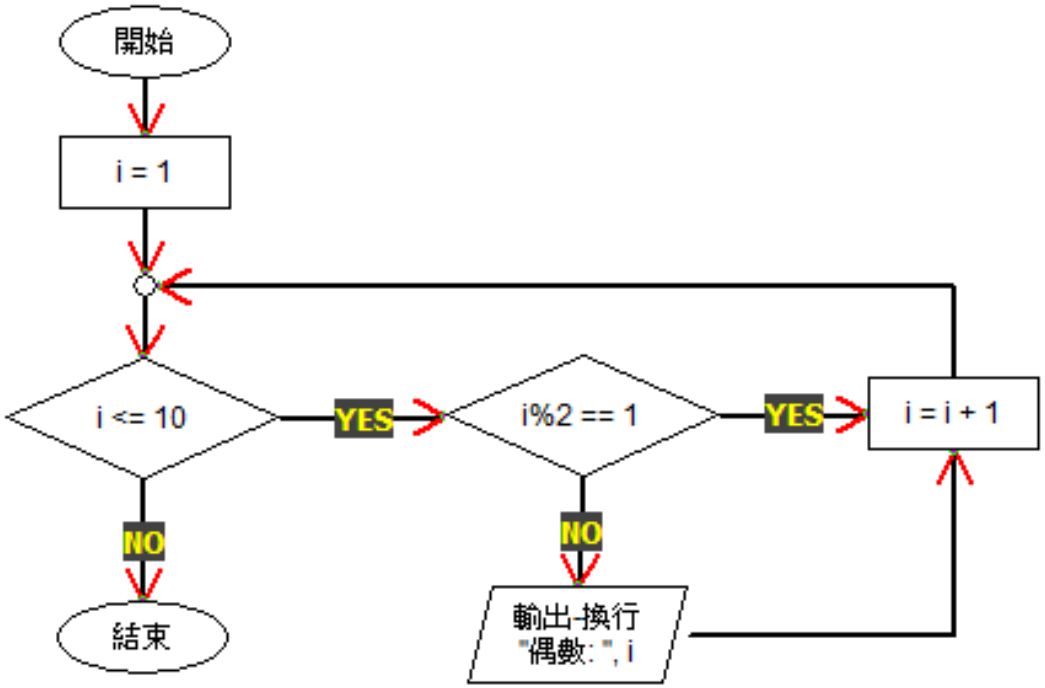
5-4-2 continue敘述繼續迴圈 – 說明與範例

- 在迴圈的執行過程中，相對第5-4-1節使用break敘述跳出迴圈，Python的continue敘述可以馬上繼續執行下一次迴圈，而不執行程式區塊中位在continue敘述之後的程式碼，如果使用在for迴圈，一樣會更新計數器變數來取得下一個值，如下所示：

```
for i in range(1, 11):
    if i % 2 == 1:
        continue
    print("偶數:", i)
```

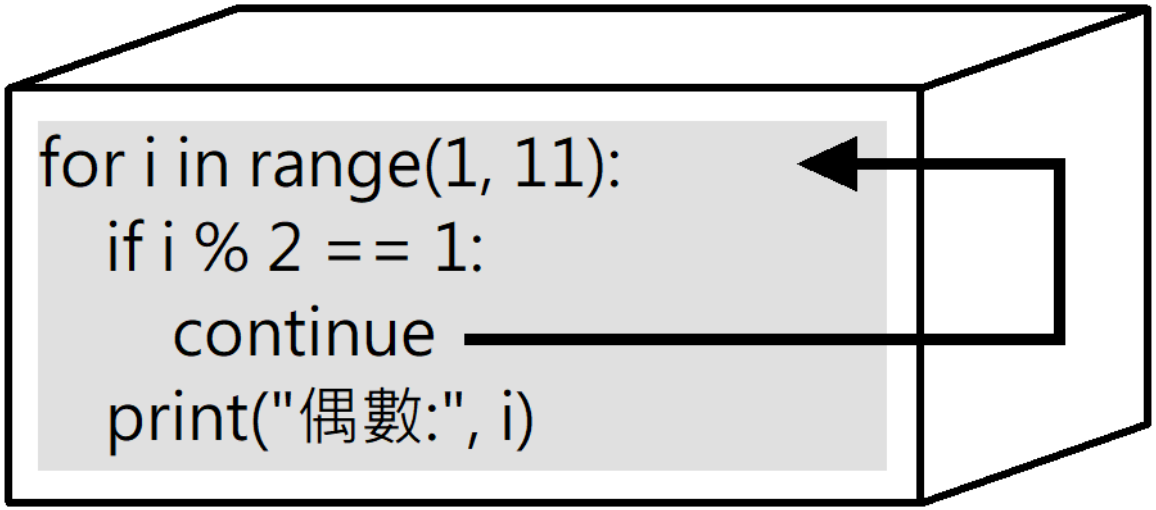
5-4-2 continue敘述繼續迴圈 – 流程圖

- if條件敘述是當計數器變數i為奇數時，就使用continue敘述馬上繼續執行下一次迴圈，而不執行之後的print()函數，可以馬上從頭開始執行下一次for迴圈，所以迴圈只顯示1到10之間的偶數，其流程圖（ch5-4-2.fpp）如下圖所示：



5-4-2 continue敘述繼續迴圈 – 執行結果

- Python程式使用continue敘述繼續for迴圈的過程，如下圖所示：



5-4-3 在迴圈使用else程式區塊

在for迴圈使用else程式區塊：ch5-4-3.py

•在for迴圈使用else程式區塊，可以在else程式區塊顯示計算結果，例如：計算1加至5的總和，如下所示：

```
s = 0
for i in range(1, 6):
    s = s + i
else:
    print("for迴圈結束!")
    print("總和 =", s)
```

5-4-3 在迴圈使用else程式區塊

在while迴圈使用else程式區塊：ch5-4-3a.py

- 同樣的，在while迴圈一樣可以使用else程式區塊，讓我們在else程式區塊顯示計算結果，例如：計算5!的階層值，如下所示：

```
r = n = 1
while n <= 5:
    r = r * n
    n = n + 1
else:
    print("while迴圈結束!")
    print("5!階層值 =", r)
```

5-5 巢狀迴圈與無窮迴圈

- 5-5-1 for敘述的巢狀迴圈
- 5-5-2 for與while敘述的巢狀迴圈
- 5-5-3 while無窮迴圈



5-5-1 for敘述的巢狀迴圈 – 範例

- Python巢狀迴圈可以有二或二層以上，例如：在for迴圈中擁有另一個for迴圈，如下所示：

```
for i in range(1, 4):      # for外層迴圈
    for j in range(1, 6):  # for內層迴圈
        print("i =", i, "j =", j)
```

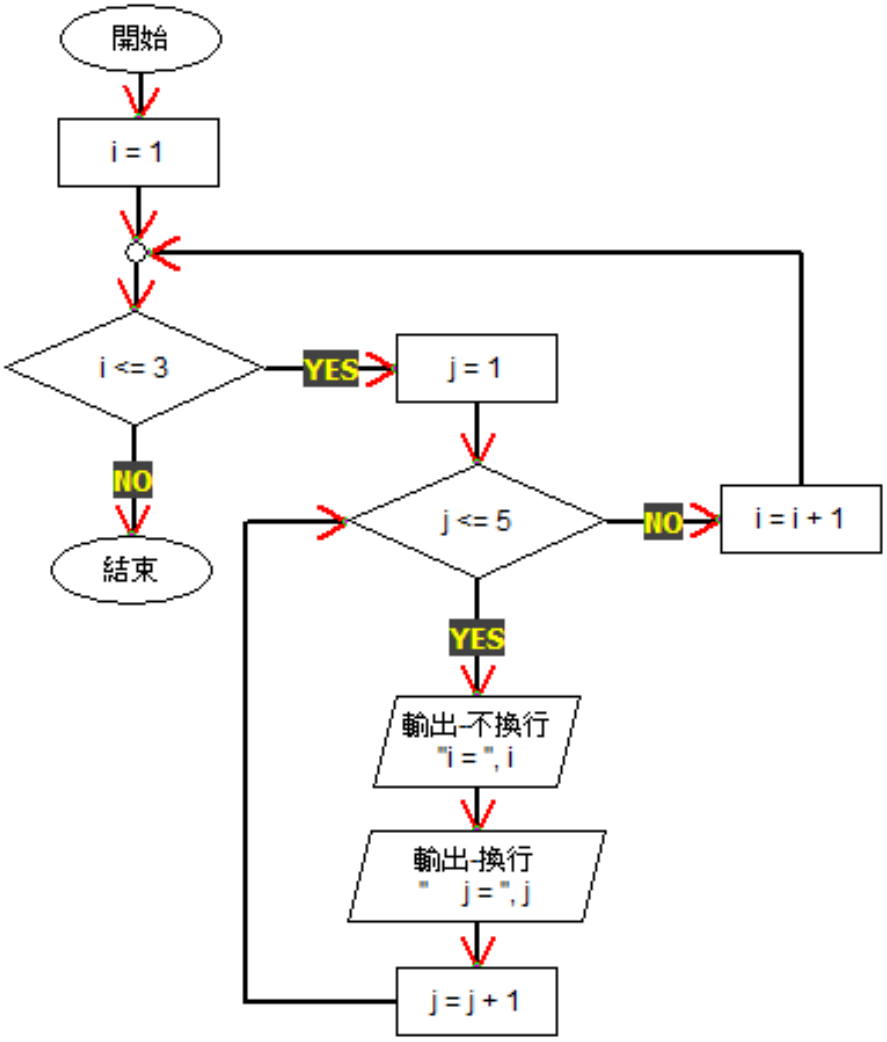
5-5-1 for敘述的巢狀迴圈 – 執行過程的變數值

- 迴圈共有兩層，第一層for迴圈執行1~3共3次，第二層for迴圈執行1~5共5次，兩層迴圈共可執行3 * 5 = 15次。其執行過程的變數值，如下表所示：

第一層迴圈的i值	第二層迴圈的j值					離開迴圈的i值
1	1	2	3	4	5	1
2	1	2	3	4	5	2
3	1	2	3	4	5	3

5-5-1 for敘述的巢狀迴圈 – 執行過程的變數值

- 其流程圖（ch5-5-1.fpp）如右圖所示：



5-5-2 for與while敘述的巢狀迴圈

- Python巢狀迴圈也可以搭配不同種類的迴圈，例如：在for迴圈之中擁有while迴圈，如下所示：

```
for i in range(1, 4):    # for外層迴圈
    j = 1
    while j <= 5:        # while內層迴圈
        print("i =", i, "j =", j)
        j = j + 1
```

5-5-3 while無窮迴圈

- 無窮迴圈（Endless Loops）是指迴圈不會結束，它會無止境的一直重複執行迴圈的程式區塊。while無窮迴圈可以使用True關鍵字的條件來建立無窮迴圈，如下所示：

```
while True:
    pass
```

- 上述while迴圈因為條件必為True，所以是無窮迴圈，並且使用pass關鍵字代表這是空程式區塊。

5-5-3 while無窮迴圈

- while無窮迴圈大都是因為計數器變數或條件出了問題。例如：修改第5-3-1節ch5-3-1.py的while迴圈（Python程式：ch5-5-3.py），如下所示：

```
i = 0
total = 0

while total < 50:
    total = total + i
```

- 上述while迴圈的程式區塊因為少了「i = i + 1」，所以i值永遠是0，total計算結果也是0，永遠不可能大於50，所以造成無窮迴圈，請按Ctrl+C鍵來中斷無窮迴圈的執行。

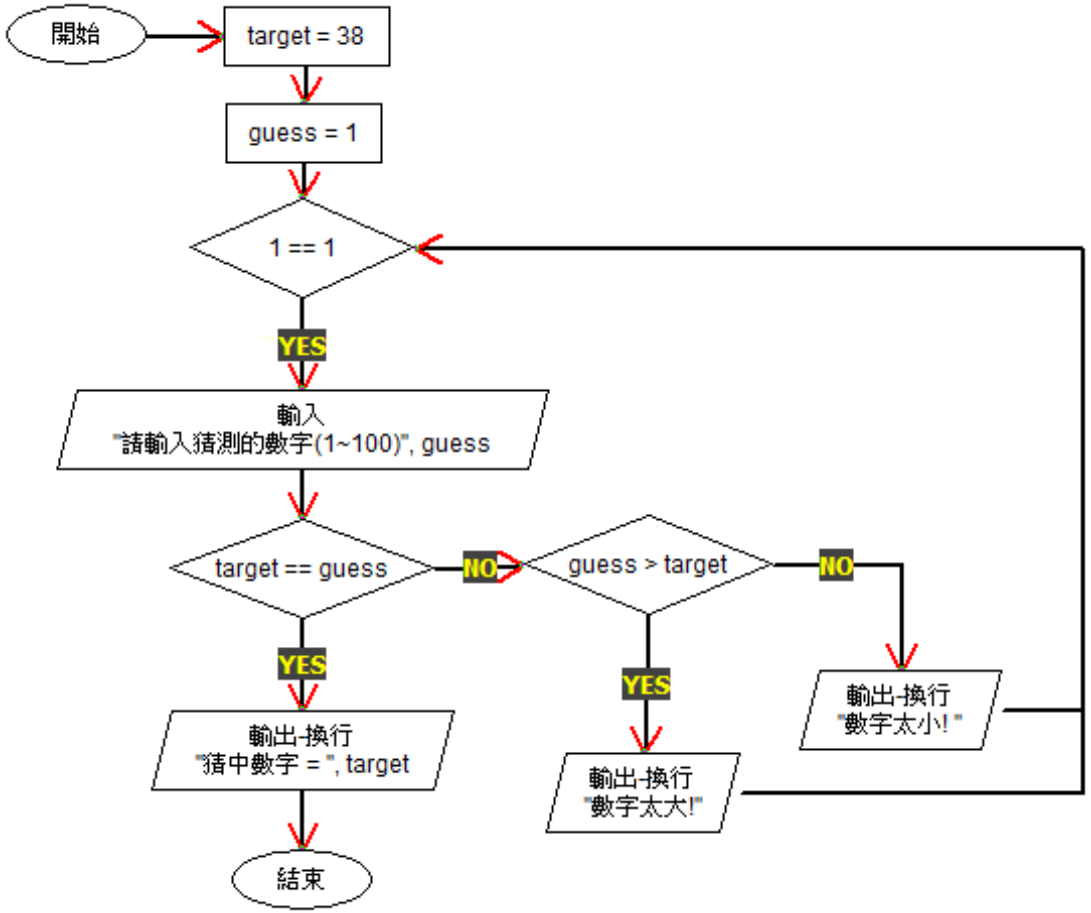
5-6 在巢狀迴圈使用if條件敘述

- 在Python的for和while迴圈之中，一樣可以搭配使用if/else條件敘述來執行條件判斷。例如：使用while迴圈建立猜數字遊戲，在迴圈中使用if/else條件判斷是否猜中數字，如下所示：

```
while True:
    guess = int(input("請輸入猜測的數字(1~100) => "))
    if target == guess:   # if條件敘述
        break             # 跳出迴圈
    if guess > target:    # if/else條件敘述
        print("數字太大!")
    else:
        print("數字太小!")
```

5-6 在巢狀迴圈使用if條件敘述

- Python程式碼的流程圖（ch5-6.fpp），如下圖所示：



第6章 函數

- 6-1 認識函數
- 6-2 使用者自訂函數
- 6-3 函數的參數
- 6-4 函數的回傳值
- 6-5 函數的實際應用
- 6-6 變數範圍和內建函數



6-1 認識函數

- 6-1-1 函數的結構
- 6-1-2 函數是一個黑盒子



6-1-1 函數的結構 – 重複步驟

- 不論是日常生活，或實際撰寫程式碼時，有些工作可能會重複出現，而且這些工作不是單一程式敘述，而是完整的工作單元，例如：我們常常在自動販賣機購買茶飲，此工作的完整步驟，如下所示：

  Step 1：將硬幣投入投幣口

  Step 2：按下按鈕，選擇購買的茶飲

  Step 3：在下方取出購買的茶飲

6-1-1 函數的結構 – 多次的重複步驟

- 上述步驟如果只有一次到無所謂，如果幫3位同學購買果汁、茶飲和汽水三種飲料，這些步驟就需重複3次，如下所示：

Step 1：將硬幣投入投幣口
Step 2：按下按鈕，選擇購買的果汁
Step 3：在下方取出購買的果汁
} 購買果汁

Step 1：將硬幣投入投幣口
Step 2：按下按鈕，選擇購買的茶飲
Step 3：在下方取出購買的茶飲
} 購買茶飲

Step 1：將硬幣投入投幣口
Step 2：按下按鈕，選擇購買的汽水
Step 3：在下方取出購買的汽水
} 購買汽水

6-1-1 函數的結構 – 簡化成3個工作

- 相信沒有同學請你幫忙買飲料時，每一次都說出左邊3個步驟，而會很自然的簡化成3個工作，直接說：
  - 購買果汁
  - 購買茶飲
  - 購買汽水
- 上述簡化的工作描述就是函數（Functions）的原型，因為我們會很自然的將一些工作整合成更明確且簡單的描述「購買??」。

6-1-1 函數的結構 – 建立函數

- 程式語言也是使用相同觀念，可以將整個自動販賣機購買飲料的步驟使用一個整合名稱來代表，即【購買()】函數，如下所示：

  購買(果汁)

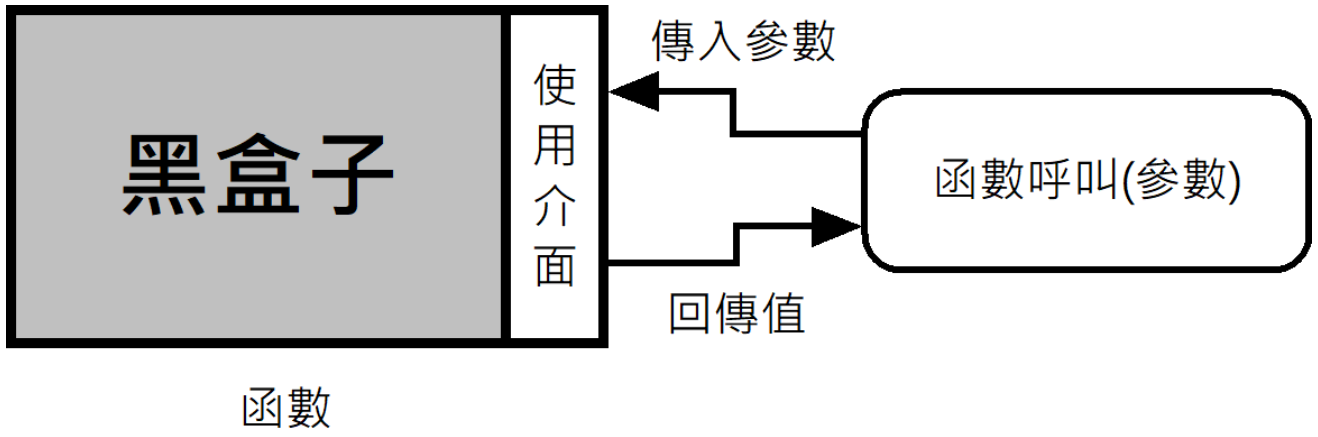
  購買(茶飲)

  購買(汽水)

- 上述程式碼是函數呼叫，在括號中是傳入購買函數的資料，即引數（Arguments），以便3個操作步驟知道購買哪一種飲料，執行此函數的結果是拿到飲料，這就是函數的傳回值。

6-1-2 函數是一個黑盒子

- 函數是一個獨立功能的程式區塊，如同是一個黑盒子（Black Box），我們不需要了解函數定義的程式碼內容是什麼，只要告訴我們使用黑盒子的介面（Interface），就可以呼叫函數來使用函數的功能，如下圖所示：



6-2 使用者自訂函數

- 6-2-1 建立和呼叫函數
- 6-2-2 多次呼叫同一個函數
- 6-2-3 Python主程式函數



6-2-1 建立和呼叫函數

定義函數

- Python建立函數就是在撰寫函數定義（Function Definition）的函數標頭和程式區塊，其內容是需要重複執行的程式碼。Python是函數標頭和程式區塊組成函數定義，其語法如下所示：

```
def 函數名稱():
    程式敘述1
    程式敘述2
    ...
```

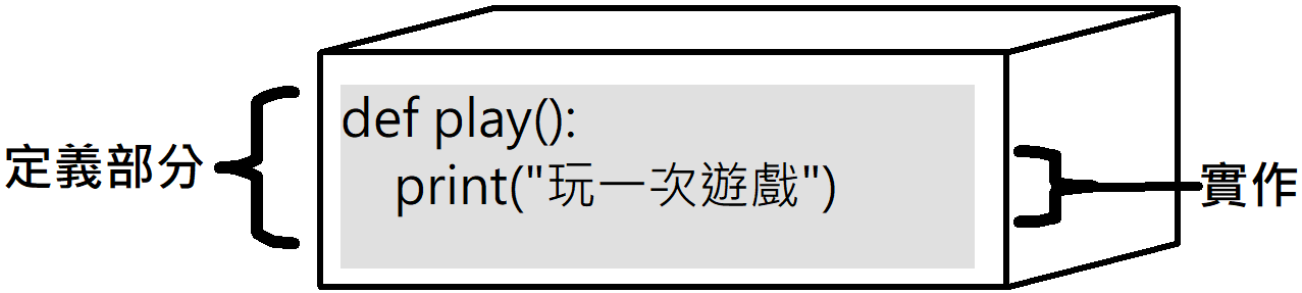
- 上述語法使用def關鍵字定義函數，第1行是函數標頭（Function Header），函數名稱如同變數是識別字，其命名方式和變數相同，在函數名稱後的括號是參數列，沒有參數，就是空括號，最後是「:」冒號。

6-2-1 建立和呼叫函數

定義函數

- 例如：我們準備建立可以顯示「玩一次遊戲」訊息文字的函數，如下所示：

```
# play()函數的定義
def play():
    print("玩一次遊戲")
```



6-2-1 建立和呼叫函數

呼叫函數

- 在定義函數後，就可以使用函數呼叫的介面，在程式碼呼叫此函數，其語法如下所示：

  函數名稱()

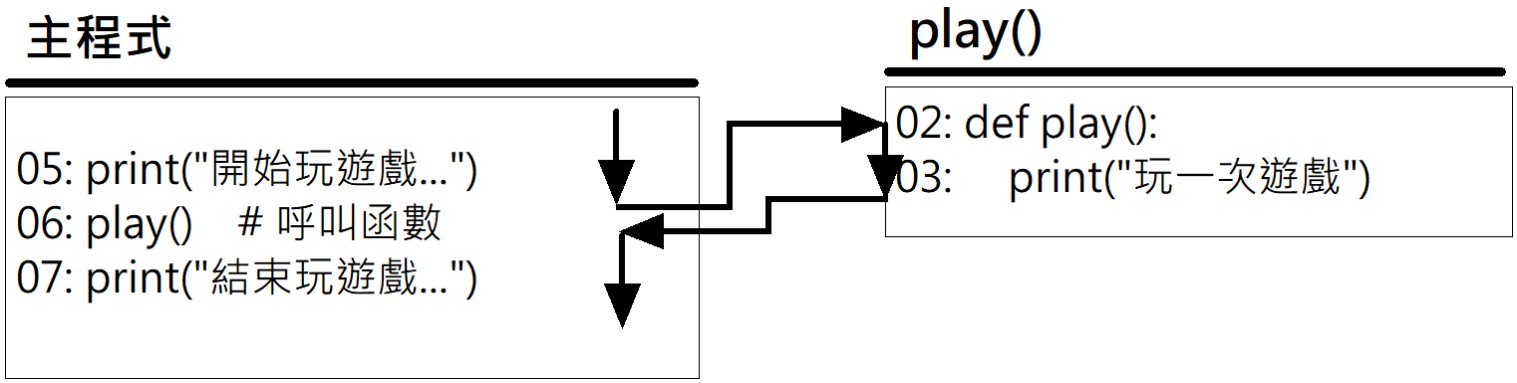
- 上述語法使用函數名稱來呼叫函數，因為沒有參數，所以之後是空括號。例如：呼叫play()函數，如下所示：

```
play()   # 呼叫函數
```

6-2-1 建立和呼叫函數

函數的執行過程

- 現在，讓我們看一看ch6-2-1.py函數呼叫的執行過程，首先在沒有縮排的第5行顯示一行訊息文字後，第6行呼叫play()函數，此時程式執行順序就會轉移至play()函數，即跳到執行第2~3行play()函數的程式區塊，如下圖所示：



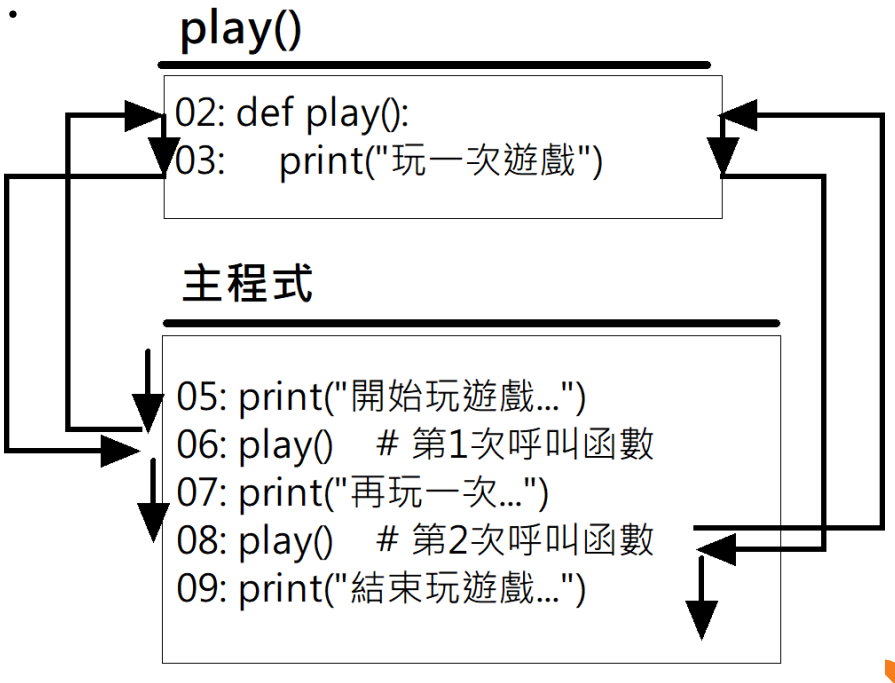
6-2-2 多次呼叫同一個函數

- 函數的目的是為了之後可以重複呼叫此函數，如同工具箱的各種工具，如果需要時，就拿出來重複使用，同理，函數是程式工具箱中擁有特定功能的工具，如果程式需要此功能，就直接呼叫函數來進行處理，而不用每次都重複撰寫相同功能的程式碼。
- 例如：重複呼叫2次play()函數，顯示2次相同的訊息文字，如下所示：

```
print("開始玩遊戲...")
play()   # 第1次呼叫函數
print("再玩一次...")
play()   # 第2次呼叫函數
print("結束玩遊戲...")
```

6-2-2 多次呼叫同一個函數

- 現在，讓我們看一看ch6-2-2.py函數呼叫的執行過程，首先在第5行顯示一行訊息文字後，第6行第1次呼叫play()函數，跳到執行第2~3行play()函數的程式區塊，顯示第3行的訊息文字後，返回繼續執行第7行的程式碼，顯示一行訊息文字，如下圖所示：



6-2-3 Python主程式函數

- Python程式預設沒有主程式函數，沒有縮排的程式碼就是其他程式語言的主程式。當然，我們也可以指定Python函數作為主程式，例如：main()函數（Python程式：ch6-2-3.py），if條件判斷__name__特殊變數的值是否是"__main__"，如下所示：

```
if __name__ == "__main__":
    main()   # 呼叫主程式函數
```

- 上述if條件如果成立，表示是執行此Python程式（而不是被其他Python程式所匯入），所以呼叫main()主程式函數，如果if條件不成立，此Python程式是當成其他Python程式工具箱的模組（詳見第9-1節說明），因為是當成模組，不是執行，所以不會呼叫main()函數。

6-3 函數的參數

- 6-3-1 使用參數傳遞資料
- 6-3-2 使用鍵盤輸入參數值
- 6-3-3 建立擁有多參數的函數
- 6-3-4 參數預設值、位置與關鍵字參數



6-3-1 使用參數傳遞資料 – 說明

- 函數的參數是函數的資訊傳遞機制，可以從函數呼叫，將資料送入函數的黑盒子，簡單的說，參數是函數傳遞資料的使用介面，即呼叫函數和函數之間的溝通管道。
- 在第6-2節建立的函數單純只能執行固定工作，每一次的執行結果都完全相同。當函數擁有參數列時，我們可以使用參數來傳遞資料，依據收到的資料進行運算，或執行對應的處理，讓函數擁有更大的彈性，換句話說，函數可以依據傳入不同的參數值，而得到不同的執行結果。

6-3-1 使用參數傳遞資料 – 建立擁有參數的函數(語法)

- Python函數可以在函數名稱後的括號中加上參數列，其語法如下所示：

```
def 函數名稱( 參數列 ):
    程式敘述1
    程式敘述2
    ...
```

- 上述函數定義位在括號中的就是參數列，如果有多個參數，請使用「,」逗號分隔。

6-3-1 使用參數傳遞資料 – 建立擁有參數的函數(範例)

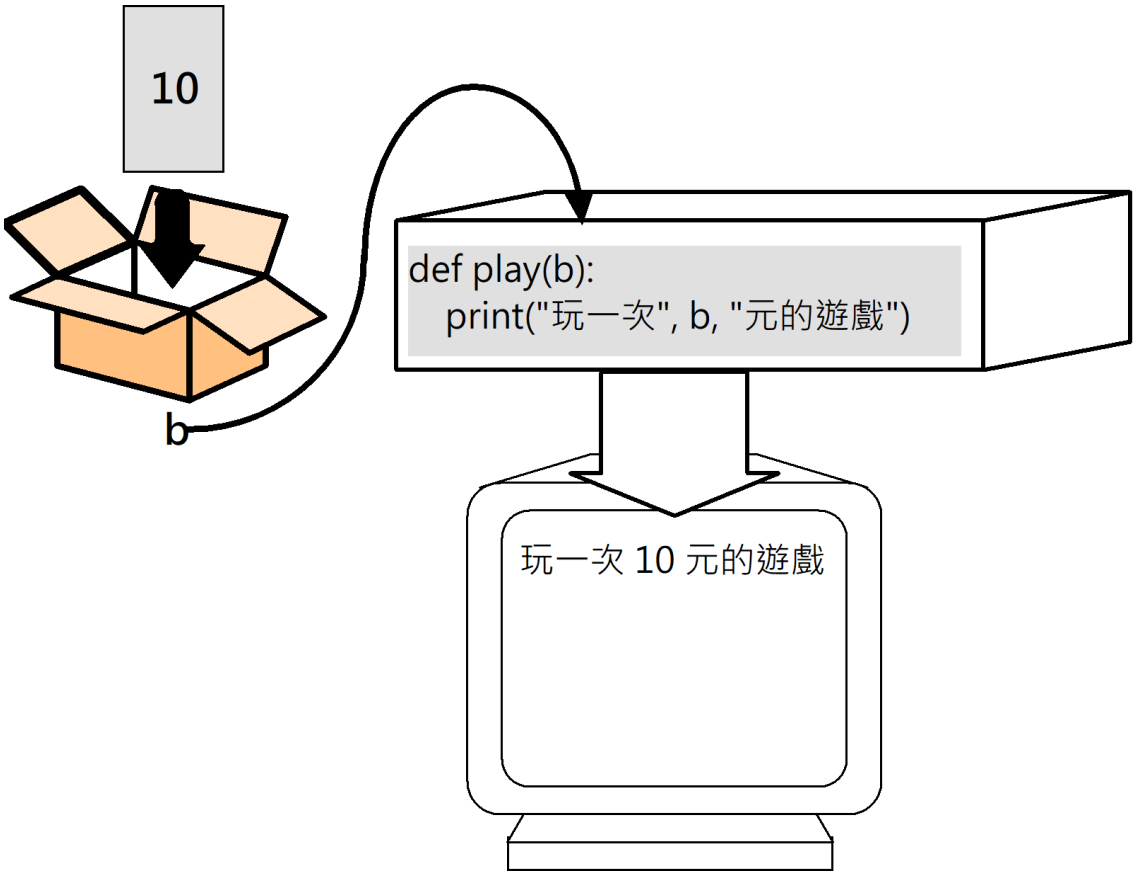
- 例如：我們準備擴充第6-2-1節的play()函數，新增1個名為b的參數，如下所示：

```
# play()函數的定義
def play(b):
    print("玩一次", b, "元的遊戲")
```

- 上述play()函數擁有1個名為b的參數，可以讓我們在呼叫play()函數時，將資料傳入函數。

6-3-1 使用參數傳遞資料 – 建立擁有參數的函數(圖例)

- 圖例參數b的值是10，所以函數可以使用參數b的值來建立輸出結果，可以看到呼叫play()函數顯示傳入的參數值10。



6-3-1 使用參數傳遞資料 – 呼叫擁有參數的函數

- 函數如果擁有參數，在Python程式呼叫函數時，就需要在括號加入參數值，其語法如下所示：

  函數名稱(參數值列)

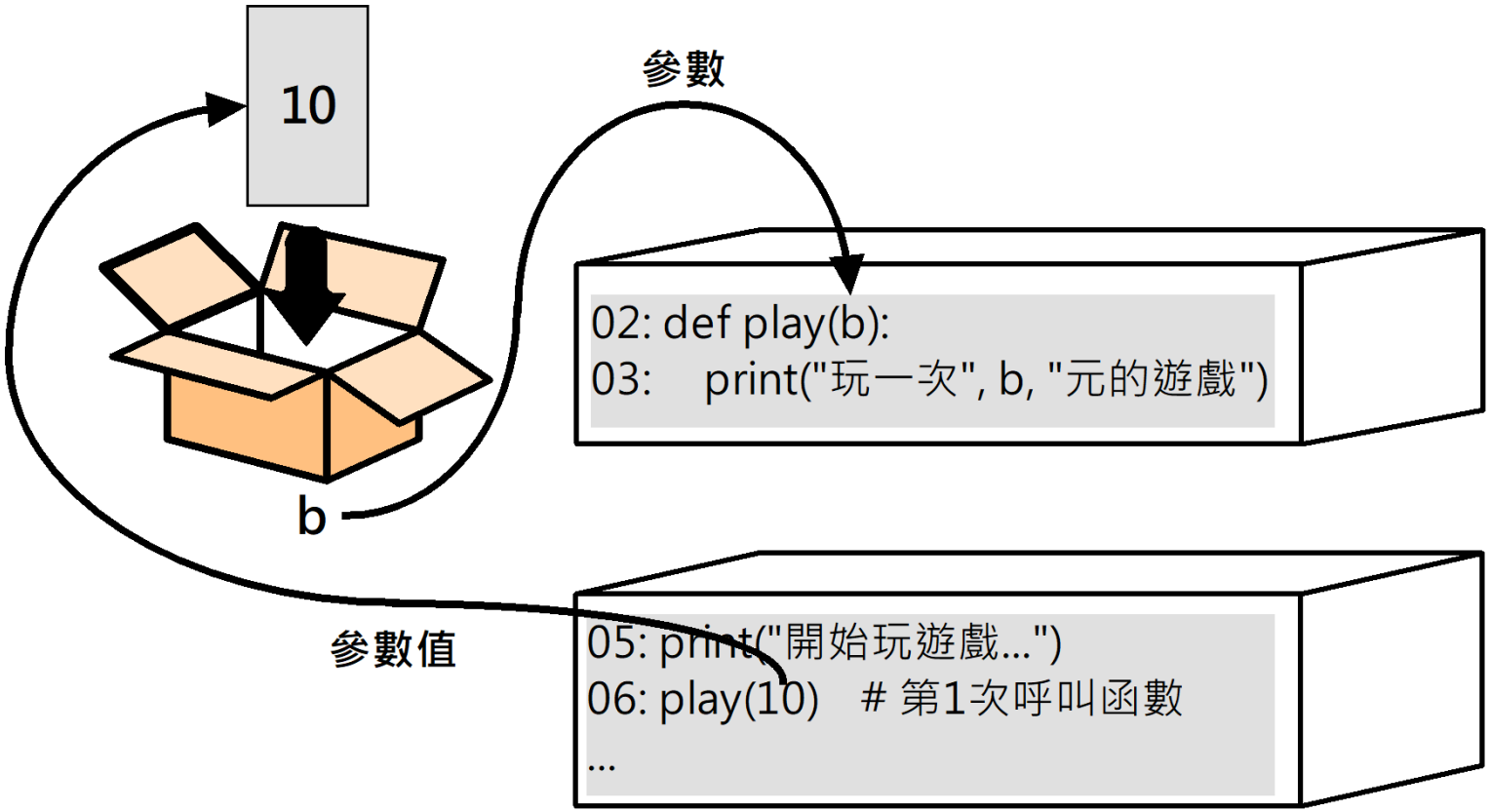
- 上述語法的函數如果有參數，在呼叫時需要加上傳入的參數值列，如果有多個，請使用「,」逗號分隔。例如：play()函數擁有1個參數b，在呼叫play()函數時需要使用1個參數值來傳遞至函數，如下所示：

```
play(10)   # 呼叫函數
```

- 上述程式碼傳遞值10至play()函數，此時參數b的值就是10。

6-3-1 使用參數傳遞資料 – 執行結果

- Python使用參數傳遞資料10至play(b)函數的過程，如下圖所示：



6-3-2 使用鍵盤輸入引數值 – 說明

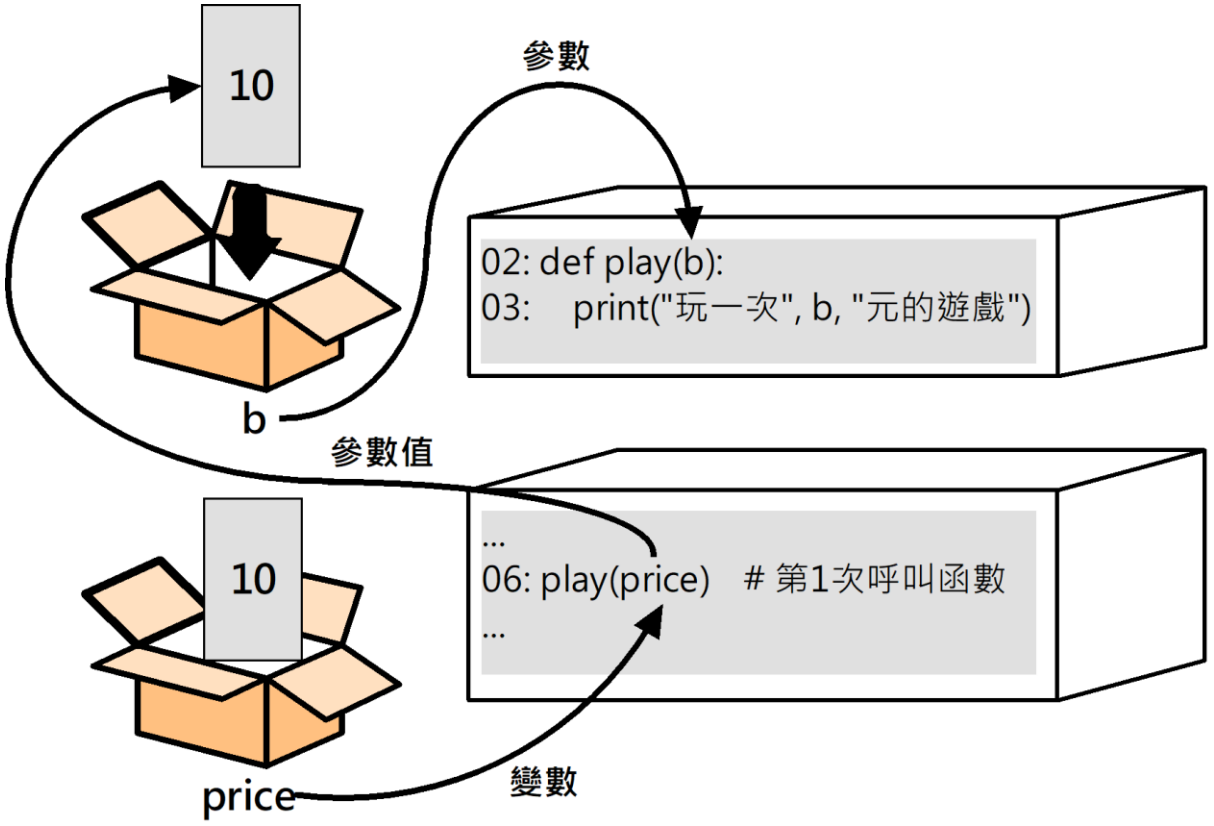
- Python程式呼叫函數的參數值除了使用文字值外，也可以使用變數，在這一節我們準備讓使用者輸入變數price值後，使用變數作為函數的參數值，如下所示：

```
price = int(input("第1次玩多少錢的遊戲==> "))
play(price)   # 呼叫函數
```

- 上述程式碼使用變數price的值作為參數值來呼叫play()函數。

6-3-2 使用鍵盤輸入引數值 – 傳值呼叫

- 因為Python呼叫函數傳遞的並不是變數，而是變數儲存的文字值10和50，這種參數傳遞方式稱為傳值呼叫（Call by Value），如下圖所示：



6-3-3 建立擁有多參數的函數

- Python函數可以擁有「,」逗號分隔的多個參數，例如：play()函數擁有2個參數，如下所示：

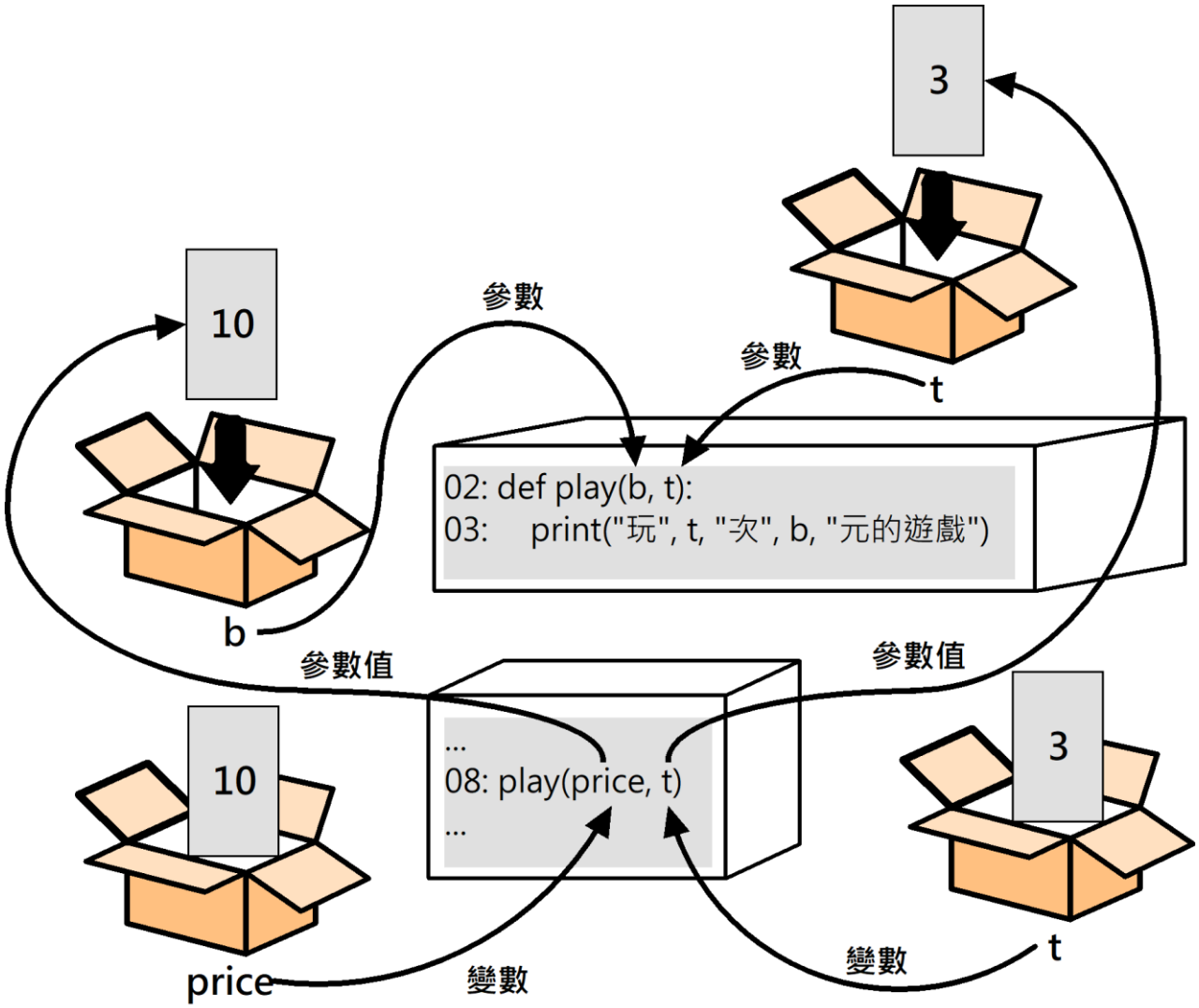
```
# play()函數的定義
def play(b, t):
    print("玩", t, "次", b, "元的遊戲")
```

- 因為play()函數擁有2個參數，呼叫play()函數也需要使用2個參數值，如下所示：

```
play(price, t)    # 呼叫函數
```

- 上述呼叫函數的參數值可以是文字值、變數或運算式的運算結果。

6-3-3 建立擁有多參數的函數 – 圖例



6-3-4 參數預設值、位置與關鍵字參數

函數參數的預設值：ch6-3-4.py

- Python函數的參數可以有預設值，當函數呼叫時沒有指定參數值，就使用參數的預設值（預設值參數在參數列的順序是在沒有預設值的參數之後，以此例是位在length參數之後）。例如：計算盒子體積的volume()函數，如下所示：

```
def volume(length, width = 2, height = 3):
    return length * width * height
```

- volume()函數呼叫如下所示：

```
print("盒子體積: ", volume(l, w, h))
print("盒子體積: ", volume(l, w))
print("盒子體積: ", volume(l))
```

- 上述函數呼叫分別指定長、寬和高，只有長和寬、最後只有長的參數，其他沒有指定參數值就是使用預設參數值。

6-3-4 參數預設值、位置與關鍵字參數

關鍵字參數：ch6-3-4a.py

- Python函數也可以使用關鍵字參數，直接使用參數名稱來指定參數值，例如：將3個參數加總的total()函數，如下所示：

```
def total(a, b, c):
    return a + b + c
```

- 上述函數擁有3個參數，如果使用關鍵字參數來呼叫，我們可以先傳b，再傳c，最後傳入a，如下所示：

```
r1 = total(1, 2, 3)
r2 = total(b=2, c=3, a=1)
```

6-3-4 參數預設值、位置與關鍵字參數

混合使用位置和關鍵字參數：ch6-4-4b.py

- Python可以混合位置和關鍵字參數來呼叫函數，請注意！位置順序的參數一定在關鍵字參數之前，如下所示：

```
r3 = total(1, c=3, b=2)
r4 = total(1, 2, c=3)
```

6-4 函數的回傳值

- 6-4-1 使用函數的回傳值
- 6-4-2 建立回傳多個值的函數

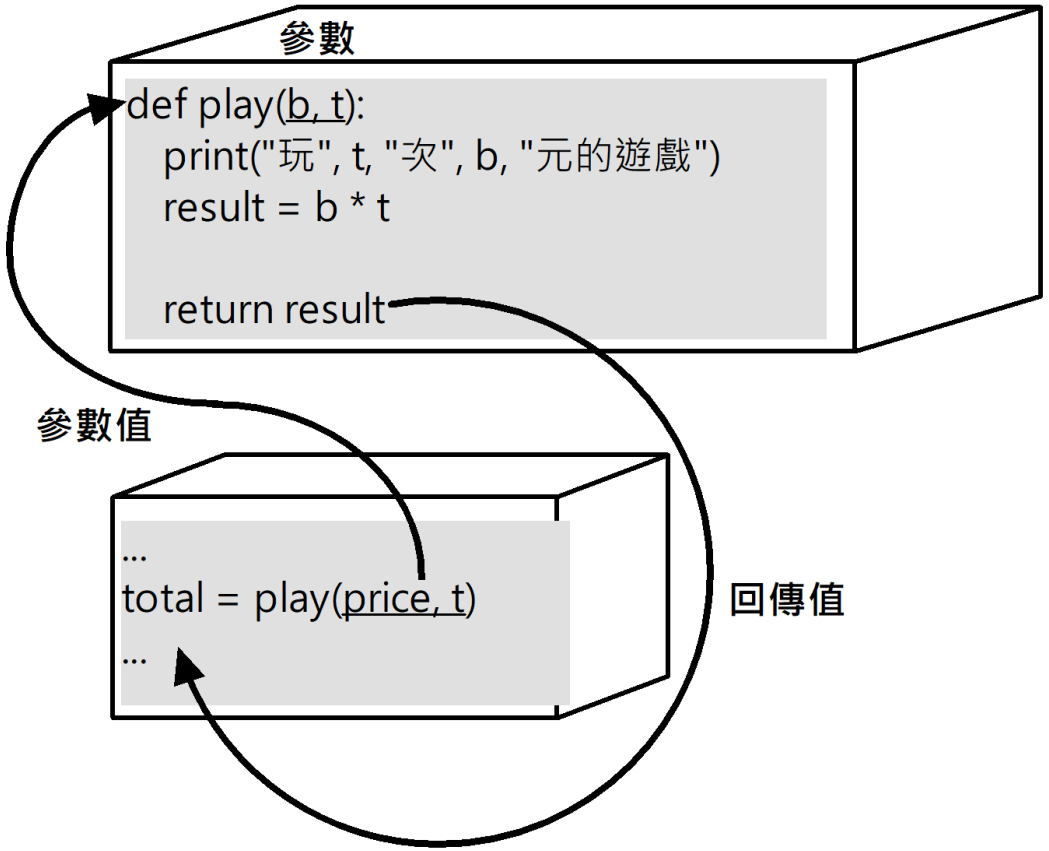


6-4 函數的回傳值 – 說明

- 函數的參數可以從呼叫的函數傳遞資料至函數，反過來，函數的回傳值就是從函數傳遞資料回到呼叫函數的程式碼，例如：在Python程式呼叫play()函數，如下所示：
  - 函數的參數：將資料從呼叫play()函數的參數值，透過函數參數傳遞至play()函數中。
  - 函數的回傳值：將資料從play()函數回傳至呼叫play()函數的程式碼，可以將回傳值使用指定敘述指定給其他變數。

6-4 函數的回傳值 –圖例

- 當函數有回傳值，函數和呼叫函數之間就擁有雙向的資料傳遞機制，如右圖所示：



6-4-1 使用函數的回傳值

建立擁有回傳值的函數 (語法)

- Python函數是在函數程式區塊使用return敘述來回傳值，我們可以在呼叫函數的程式碼取得函數的回傳值。
- 函數如果有回傳值，在函數程式區塊需要使用return敘述來回傳值，其語法如下所示：

```
def 函數名稱( 參數列 ):
    程式敘述1~n
    ...
    return 運算式
```

- 上述函數程式區塊使用return敘述回傳運算式值。

6-4-1 使用函數的回傳值
建立擁有回傳值的函數 (範例)

- 例如：play()函數可以回傳共花了多少錢來玩這幾次遊戲，如下所示：

```
# play()函數的定義
def play(b, t):
    print("玩", t, "次", b, "元的遊戲")
    result = b * t

    return result
```

- 上述play()函數的程式區塊可以計算參數相乘的總花費，即「b * t」，然後使用return敘述回傳金額的result變數值。

6-4-1 使用函數的回傳值

呼叫擁有回傳值的函數

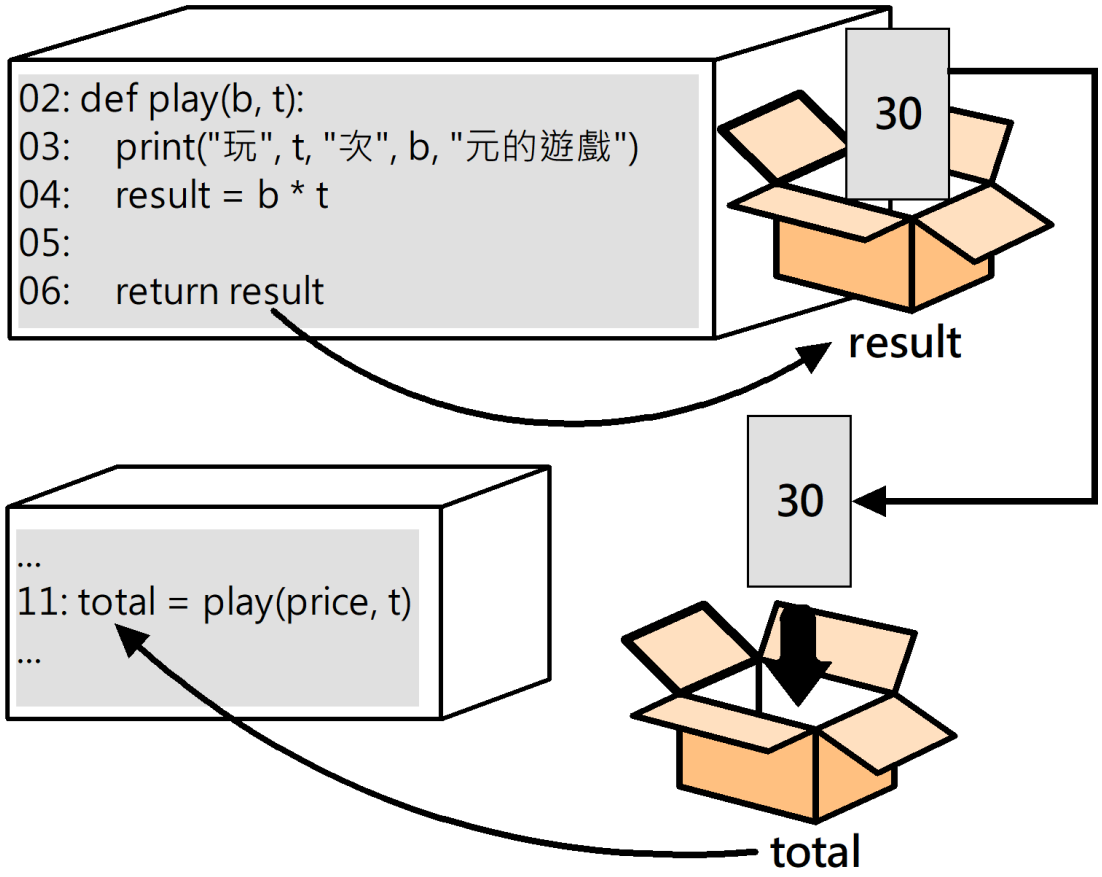
- 函數如果擁有回傳值，在呼叫時可以使用指定敘述來取得回傳值，如下所示：

```
total = play(price, t)    # 呼叫函數
```

- 上述程式碼的變數total可以取得play()函數的回傳值。

6-4-1 使用函數的回傳值 – 執行結果

- 因為play()函數有回傳值，我們需要使用指定敘述來取得回傳值，變數total存入的值就是play()函數的回傳值，如右圖所示：



6-4-2 建立回傳多個值的函數

- Python函數可以使用return敘述同時回傳多個值，這就是回傳第7章元組（Tuple）的容器型態，例如：bigger()函數可以同時回傳2個參數值建立的元組，其中第1個元素是最大值，如下所示：

```
def bigger(a, b):
    if a > b:
        return a, b
    else:
        return b, a
```

- 上述return敘述回傳「,」逗號分隔的多個值，如果參數a比較小，就回傳a, b；反之是回傳b, a。

6-5 函數的實際應用

- 6-5-1 計算參數的總和
- 6-5-2 找出最大值



6-5-1 計算參數的總和

- 我們可以修改第6-4-1節的play()函數成為add()加法函數，可以計算和回傳2個參數的總和。

```
def add(a, b):
    result = a + b

    return result
```

6-5-2 找出最大值

- 我們只需活用第4章的if/else條件敘述，就可以建立函數來回傳2個參數中的最大值，如下所示：

```
if a > b:
    return a
else:
    return b
```

- 上述if/else條件敘述判斷2個參數的大小，如果a比較大，就回傳參數a；反之，回傳參數b。

6-5-2 找出最大值 – maxValue()函數

- maxValue()函數可以找出2個參數的最大值，如下所示：

```
def maxValue(a, b):
    if a > b:
        return a
    else:
        return b
```

6-6 變數範圍和內建函數

- 6-6-1 區域與全域變數
- 6-6-2 Python內建函數



6-6-1 區域與全域變數
使用全域變數：ch6-6-1.py

- 在函數外宣告的變數是全域變數（Global Variables），變數沒有屬於哪一個函數，可以在函數之中和之外存取此變數值。在函數可以使用global關鍵字來指明變數是使用全域變數，如下所示：

```
t = 1
def increment():
    global t  # 全域變數t
    t += 1
    print("increment()中 : t = ", str(t))
```

6-6-1 區域與全域變數

使用全域變數：ch6-6-1.py

- 事實上，在Python函數之中可以直接「取得」全域變數x（不需global關鍵字來宣告，當更新全域變數值就需要宣告），如下所示：

```
x = 50
def print_x():
    print("print_x()中 : x = ", x)

print("全域變數初值: x = ", x)
print_x()
print("呼叫print_x()後 : x = ", x)
```

6-6-1 區域與全域變數

使用區域變數：ch6-6-1a.py

- 在函數程式區塊建立變數（使用指定敘述指定變數值）是一種區域變數（Local Variables），區域變數只能在建立變數的函數之中使用，在函數之外的程式碼並無法存取此變數，如下所示：

```
x = 50
def print_x():
    x = 100
    print("print_x()中 : x = ", x)

print("全域變數初值: x = ", x)
print_x()
print("呼叫print_x()後 : x = ", x)
```

6-6-2 Python內建函數

- 在本章前已經說明過type()、int()、str()、float()、range()、input()和print()等函數，這些函數是Python內建函數（Built-in Functions）。這一節是一些Python內建數學函數（Python程式：ch6-6-2.py），如下表所示：

函數	說明
abs(x)	回傳參數x的絕對值
max(x1, x2, ..., xn)	回傳函數參數之中的最大值
min(x1, x2, ..., xn)	回傳函數參數之中的最小值
pow(a, b)	回傳第1個參數a為底，第2個參數b的次方值
round(number [, ndigits])	如果沒有第2個參數，回傳參數number最接近的整數值（即四捨五入值），如果有第2個參數的精確度，回傳指定位數的四捨五入值

第7章 字串與容器型態

- 7-1 字串型態
- 7-2 串列型態
- 7-3 元組型態
- 7-4 字典型態
- 7-5 字串與容器型態運算子



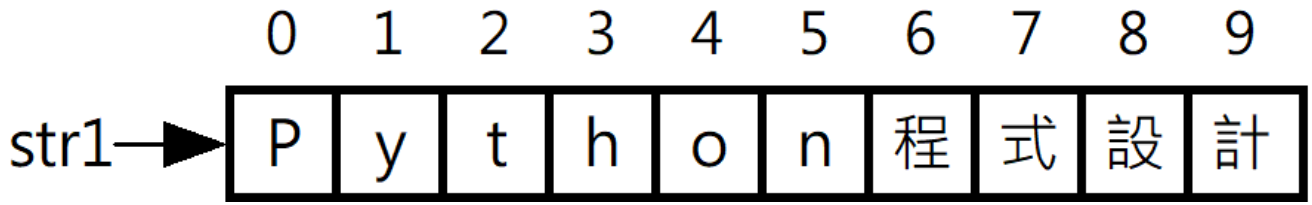
7-1 字串型態

- 7-1-1 建立和輸出字串
- 7-1-2 取出字元和走訪字串
- 7-1-3 字串函數與方法



7-1-1 建立和輸出字串

- 字串是使用「'」單引號或「"」雙引號括起的一序列Unicode字元，可以是英文、數字、符號和中文字等字元。Python字串如同社區大樓的一排信箱，一個信箱儲存的一個英文字元或中文字，門牌號碼就是索引值（從0開始），如下圖所示：



7-1-1 建立和輸出字串

- Python可以使用指定敘述或str()物件方式建立字串，然後使用print()函數輸出字串內容（Python程式：ch7-1-1.py），如下所示：

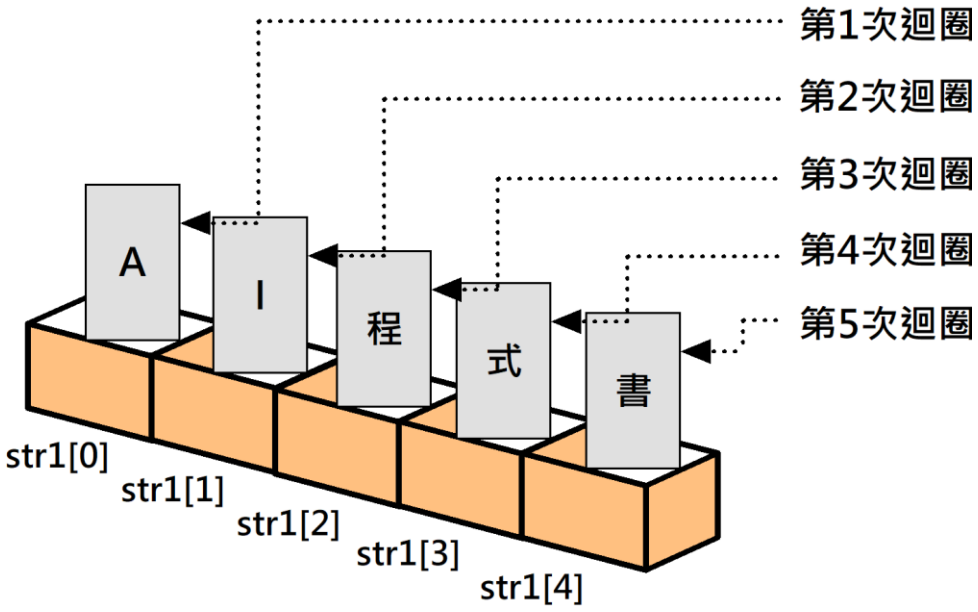
```
str1 = "Python程式設計"
name1 = str("陳會安")
print("str1 = " + str1)
print(name1)
```

7-1-2 取出字元和走訪字串

走訪字串的每一個字元：ch7-1-2.py

•字串是一序列Unicode字元，可以使用for迴圈走訪顯示每一個字元，正式的說法是迭代（Iteration），如下所示：

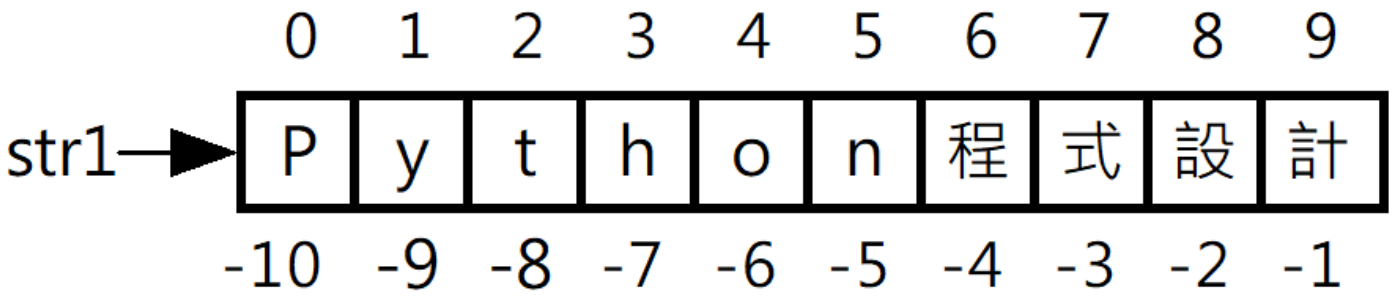
```
str1 = "AI程式書"
for ch in str1:
    print(ch, end=" ")
```



7-1-2 取出字元和走訪字串

使用索引運算子取得指定字元：ch7-1-2a.py

•Python字串、串列和元組都可以使用索引方式來存取元素（索引值從0開始），如同社區大樓的信箱，使用門牌號碼來取出信件，字串就是取出每一個英文字元或中文字（元素值並不能更改），索引值可以是正的從0~9，或負的從-1~-10，如下圖所示：



7-1-2 取出字元和走訪字串

使用索引運算子取得指定字元：ch7-1-2a.py

- Python字串可以使用「[]」索引運算子取出指定索引值的字元，如下所示：

```
str1 = str("Python程式設計")
print(str1[0])
print(str1[1])
print(str1[-1])
print(str1[-2])
```

7-1-3 字串函數與方法

- Python內建字元和字串處理的相關函數，字串方法需要使用物件變數加上「.」句號來呼叫，如下所示：

```
str1 = "Python"
print(str1.islower())
```

- 上述程式碼建立字串str1後，呼叫islower()方法檢查內容是否都是小寫英文字母，請注意！字串方法不只可以使用在字串變數，也可以使用在字串文字值（因為Python都是物件，文字值也是），如下所示：

```
print("2022".isdigit())
```

7-1-3 字串函數與方法

字元函數

•Python字元函數可以處理ASCII碼，其說明如下表所示：

字元函數	說明
ord()	回傳字元的ASCII碼
chr()	回傳參數ASCII碼的字元

7-1-3 字串函數與方法

字串函數

•Python字串函數可以取得字串長度、字串的最大和最小字元，其說明如下表所示：

字串函數	說明
len()	回傳參數字串的長度
max()	回傳參數字串的最大字元
min()	回傳參數字串的最小字元

7-1-3 字串函數與方法

檢查字串內容的方法

•字串物件提供檢查字串內容的相關方法，如下表所示：

字串方法	說明
isalnum()	如果字串內容是英文字母或數字，回傳True；否則為False
isalpha()	如果字串內容只有英文字母，回傳True；否則為False
isdigit()	如果字串內容只有數字，回傳True；否則為False
isidentifier()	如果字串內容是合法識別字，回傳True；否則為False
islower()	如果字串內容是小寫英文字母，回傳True；否則為False
isupper()	如果字串內容是大寫英文字母，回傳True；否則為False
isspace()	如果字串內容是空白字元，回傳True；否則為False

7-1-3 字串函數與方法

搜尋子字串方法

•字串物件關於搜尋子字串的相關方法說明，如下表所示：

字串方法	說明
endswith(str1)	字串是以參數字串str1結尾，回傳True；否則為False
startswith(str1)	字串是以參數字串str1開頭，回傳True；否則為False
count(str1)	回傳字串出現多少次參數字串str1的整數值
find(str1)	回傳字串出現參數字串str1的最小索引位置值，沒有找到回傳-1
rfind(str1)	回傳字串出現參數字串str1的最大索引位置值，沒有找到回傳-1

7-1-3 字串函數與方法

轉換字串內容的方法

•字串物件的轉換字串內容方法可以輸出英文大小寫轉換的字串，或是取代參數的字串內容，其說明如下表所示：

字串方法	說明
capitalize()	回傳只有第1個英文字母大寫；其他小寫的字串
lower()	回傳小寫英文字母的字串
upper()	回傳大寫英文字母的字串
title()	回傳字串中每1個英文字的第1個英文字母大寫的字串
swapcase()	回傳英文字母大寫變小寫；小寫變大寫的字串
replace(old, new)	將字串中參數old子字串取代成參數new子字串
split(str1)	字串使用參數str1來切割成串列，例如：str4.split(",")、str5.split("\n")是分別使用","和"\n"來分割字串
splitlines()	即split("\n")，使用"\n"將字串切割成串列

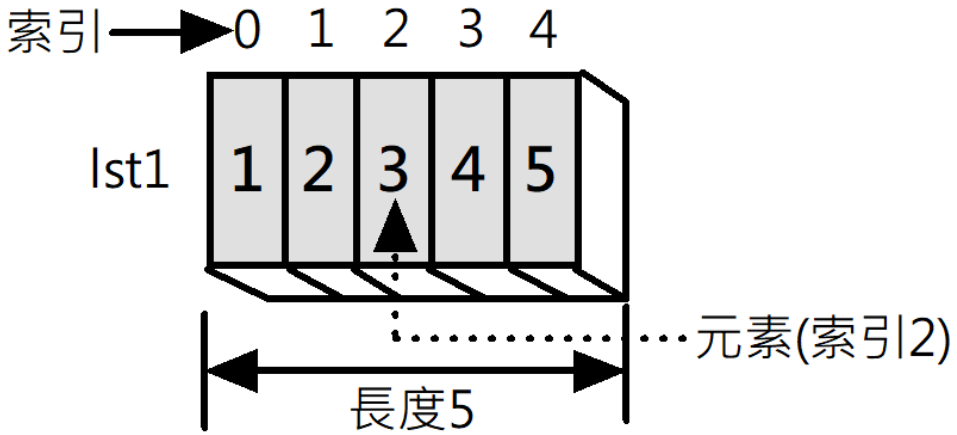
7-2 串列型態

- 7-2-1 建立與輸出串列
- 7-2-2 存取與走訪串列項目
- 7-2-3 插入、新增與刪除串列項目
- 7-2-4 串列函數與方法



7-2 串列型態

- Python串列（Lists）就是其他程式語言的陣列（Array），中文譯名還有清單和列表等，陣列是一種儲存大量循序資料的結構，可以將多個變數集合起來，使用一個名稱lst1代表，如下圖所示：



7-2-1 建立與輸出串列

- 串列是使用「[]」方括號括起的多個項目，每一個項目（Items）使用「,」逗號分隔。

建立串列和輸出串列項目：ch7-2-1.py

- Python可以使用指定敘述指定變數值是串列，串列項目可以是相同資料型態，也可以是不同資料型態，如下所示：

```
lst1 = [1, 2, 3, 4, 5]
lst2 = [1, 'Python', 5.5]
lst3 = list(["tom", "mary", "joe"])
lst4 = list("python")
print(lst1)
print(lst2, lst3, lst4)
```

7-2-1 建立與輸出串列

建立巢狀串列：ch7-2-1a.py

•因為串列項目可以是另一個串列，在串列中的串列可以建立其他程式語言的多維陣列，即巢狀串列，如下所示：

```
lst1 = [1, ["tom", "mary", "joe"], [3, 4, 5]]
print(lst1)
print("lst1:" + str(lst1))
```

7-2-2 存取與走訪串列項目

使用索引運算子取出串列項目：ch7-2-2.py

•Python串列和字串一樣可以使用「[]」索引運算子存取指定索引值的項目，索引值是從0開始，也可以是負值（即從最後至第1個倒數值），如下所示：

```
lst1 = [1, 2, 3, 4, 5, 6]
print(lst1[0])
print(lst1[1])
print(lst1[-1])
print(lst1[-2])
```

7-2-2 存取與走訪串列項目

使用索引運算子更改串列項目：ch7-2-2a.py

- 當使用索引運算子取出項目後，可以使用指定敘述更改此項目，例如：更改第2個項目成為10（索引值是1），如下所示：

```
lst1 = [1, 2, 3, 4, 5, 6]
lst1[1] = 10
lst1[2] = "Python"
print(lst1)
```

7-2-2 存取與走訪串列項目

走訪串列項目：ch7-2-2b.py

•如同字串，Python一樣可以使用for迴圈走訪串列的每一個項目，如下所示：

```
lst1 = [1, 2, 3, 4, 5, 6]
for e in lst1:
    print(e, end=" ")
```

7-2-2 存取與走訪串列項目

走訪顯示串列項目的索引值：ch7-2-2c.py

•如果需要顯示串列項目的索引值，請使用enumerate()函數，如下所示：

```
animals = ['cat', 'dog', 'bat']
for index, animal in enumerate(animals):
    print(index, animal)
```

7-2-2 存取與走訪串列項目

存取巢狀串列：ch7-2-2d.py

•因為Python巢狀串列有很多層，所以需要使用多個索引值來存取指定項目，例如：2層巢狀串列的第1層有3個項目，每一個項目是另一個串列，所以需要使用2個索引值來存取，如下所示：

```
lst2 = [[2, 4], ['cat', 'dog', 'bat'], [1, 3, 5]]
print(lst2[1][0])
lst2[2][1] = 7
print(lst2)
```

7-2-2 存取與走訪串列項目

使用巢狀迴圈走訪巢狀串列：ch7-2-2e.py

- 當Python巢狀串列有兩層時，我們需要使用2層for迴圈走訪每一個項目（3層是使用3層for迴圈），如下所示：

```
lst2 = [[2, 4], ['cat', 'dog', 'bat'], [1, 3, 5]]
for e1 in lst2:
    for e2 in e1:
        print(e2, end=" ")
```

7-2-3 插入、新增與刪除串列項目

在串列新增項目：ch7-2-3.py

•Python可以呼叫append()方法新增參數的單一項目，新增就是新增在串列的最後，如下所示：

```
lst1 = [1, 5]
lst1.append(7)
print(lst1)
lst1.append(9)
print(lst1)
```

7-2-3 插入、新增與刪除串列項目

在串列同時新增多個項目：ch7-2-3a.py

•如果需要同時新增多個項目，請使用extend()方法，如下所示：

```
lst1 = [1, 5]
lst1.extend([7, 9, 11, 13])
print(lst1)
```

7-2-3 插入、新增與刪除串列項目

在串列插入項目：ch7-2-3b.py

•Python串列可以使用insert()方法在參數的指定索引值插入1個項目，如下所示：

```
lst1 = [1, 5]
lst1.insert(1, 3)
print(lst1)
```

7-2-3 插入、新增與刪除串列項目

刪除串列項目：ch7-2-3c.py

- Python可以使用del關鍵字刪除指定索引值的串列項目，如下所示：

```
lst1 = [1, 3, 5, 7, 9, 11, 13]
del lst1[2]
print(lst1)
del lst1[4]
print(lst1)
```

7-2-3 插入、新增與刪除串列項目

刪除和回傳最後1個項目：ch7-2-3d.py

- Python可以使用pop()方法刪除和回傳最後1個項目，如下所示：

```
lst1 = [1, 3, 5, 7, 9, 11, 13]
e1 = lst1.pop()
print(e1, lst1)
e2 = lst1.pop(1)
print(e2, lst1)
```

7-2-3 插入、新增與刪除串列項目

刪除指定項目值的項目：ch7-2-3e.py

•如果準備刪除指定項目值（不是索引值），我們可以使用remove()方法刪除參數的項目值，如下所示：

```
lst1 = [1, 3, 5, 7, 9, 11, 13]
lst1.remove(9)
print(lst1)
lst1.remove(4)
print(lst1)
```

7-2-4 串列函數與方法

串列函數

•Python串列函數可以取得項目數、排序串列、加總串列項目、取得串列中的最大和最小項目等。常用串列函數說明，如下表所示：

串列函數	說明
len()	回傳參數串列的長度，即項目數
max()	回傳參數串列的最大項目
min()	回傳參數串列的最小項目
list()	回傳參數字串、元組、字典和集合轉換成的串列
enumerate()	回傳enumerate物件，其內容是串列索引和項目的元組
sum()	回傳參數串列項目的總和
sorted()	回傳參數串列的排序結果串列

7-2-4 串列函數與方法

串列方法

•Python串列的append()、extend()、insert()、pop()和remove()方法已經說明過。其他常用串列方法的說明，如下表所示：

串列方法	說明
count(item)	回傳串列中等於參數item項目的個數
index(item)	回傳串列第1個找到參數item項目的索引，項目不存在，就會產生ValueError錯誤
sort()	排序串列項目
reverse()	反轉串列項目，第1個是最後1個；最後1個是第1個

7-3 元組型態

- 7-3-1 建立與輸出元組
- 7-3-2 取出與走訪元組項目
- 7-3-3 元組函數與元組方法



7-3-1 建立與輸出元組

- Python「元組」（Tuple）是唯讀版的串列，一旦指定元組的項目，就不能再更改元組的項目。Python元組是使用「()」括號建立，每一個項目使用「,」逗號分隔。在Python使用元組的優點，如下所示：
  - 因為元組項目不允許更改，走訪元組比起走訪串列更有效率，可以輕微增加程式的執行效能。
  - 元組因為項目不允許更改，可以作為字典的鍵（Keys）來使用，但串列不可以。
  - 如果程式需要使用不允許更改的唯讀串列，可以使用元組來實作，而且保證項目不會被更改。

7-3-1 建立與輸出元組

- Python可以使用指定敘述指定變數值是一個元組，元組的項目可以是相同資料型態，也可以是不同資料型態（Python程式：ch7-3-1.py），如下所示：

```
t1 = (1, 2, 3, 4, 5)
t2 = (1, 'Joe', 5.5)
t3 = tuple(["tom", "mary", "joe"])
t4 = tuple("python")
print(t1)
print(t2, t3)
print("t4 = " + str(t4))
```

7-3-2 取出與走訪元組項目

使用索引運算子取出元組項目：ch7-3-2.py

•Python元組因為是唯讀串列，可以使用「[]」索引運算子取出指定索引值的項目，索引值是從0開始，也可以是負值，如下所示：

```
t1 = (1, 2, 3, 4, 5, 6)
print(t1[0])
print(t1[1])
print(t1[-1])
print(t1[-2])
```

7-3-2 取出與走訪元組項目

走訪元組的每一個項目：ch7-3-2a.py

•Python的for迴圈一樣可以走訪元組的每一個項目，如下所示：

```
t1 = (1, 2, 3, 4, 5, 6)
for e in t1:
    print(e, end=" ")
```

7-3-3 元組函數與元組方法

元組函數

•Python元組函數和和串列函數幾乎相同，只有list()換成了tuple()，如下表所示：

元組函數	說明
tuple()	回傳參數字串、串列和字典轉換成的元組

7-3-3 元組函數與元組方法

元組方法

•Python元組方法可以搜尋項目和計算出現次數。常用元組方法的說明，如下表所示：

元組方法	說明
count(item)	回傳元組中等於參數item項目的個數
index(item)	回傳元組第1個找到參數item項目的索引，項目不存在，就會產生ValueError錯誤

7-4 字典型態

- 7-4-1 建立與輸出字典
- 7-4-2 取出、更改、新增與走訪字典項目
- 7-4-3 刪除字典項目
- 7-4-4 字典函數與字典方法



7-4-1 建立與輸出字典

- Python「字典」（Dictionaries）是一種儲存鍵值資料的容器型別，可以使用鍵（Key）取出和更改值（Value），或使用鍵新增和刪除項目。Python字典是使用大括號「{}」定義成對的鍵和值（Key-value Pairs），每一對使用「,」逗號分隔，鍵和值是使用「:」冒號分隔，如下所示：

```
{
  "key1": "value1",
  "key2": "value2",
  "key3": "value3",
  ...
}
```

7-4-1 建立與輸出字典

建立字典和輸出字典內容：ch7-4-1.py

•Python可以使用指定敘述指定變數值是一個字典，字典的鍵和值可以是相同資料型態，也可以是不同資料型態，如下所示：

```
d1 = {1: 'apple', 2: 'ball'}
d2 = dict([(1, "tom"), (2, "mary"), (3, "john")])
print(d1)
print("d2 = " + str(d2))
```

7-4-1 建立與輸出字典

建立複雜結構的字典：ch7-4-1a.py

- Python字典的值可以是整數、字串外，還可以是串列或其他的字典，如下所示：

```
d1 = {
   "name": "joe",
   1: [2, 4, 6],
   "grade": {
          "english":80,
          "math":78
        }
   }
print(d1)
```

7-4-2 取出、更改、新增與走訪字典項目

取出字典值：ch7-4-2.py

•Python字典也是使用「[]」索引運算子存取指定鍵的值，如下所示：

```
d1 = {"chicken": 2, "dog": 4, "cat":3}
print(d1["cat"])
print(d1["dog"])
print(d1["chicken"])
```

7-4-2 取出、更改、新增與走訪字典項目

更改字典值：ch7-4-2a.py

•更改字典值是使用指定敘述「＝」等號，例如：更改鍵"cat"的值成為4，如下所示：

```
d1 = {"chicken": 2, "dog": 4, "cat":3}
d1["cat"] = 4
print(d1)
```

7-4-2 取出、更改、新增與走訪字典項目

新增字典的鍵值對：ch7-4-2b.py

- 當指定敘述更改的字典鍵不存在，就是新增字典的鍵值對，如下所示：

```
d1 = {"chicken": 2, "dog": 4, "cat":3}
d1["spider"] = 8
print(d1)
```

7-4-2 取出、更改、新增與走訪字典項目

走訪字典的鍵來取出值：ch7-4-2c.py

- Python可以使用for迴圈走訪字典的鍵來取出值，如下所示：

```
d1 = {"chicken": 2, "dog": 4, "cat":3}
for animal in d1:
    legs = d1[animal]
    print(animal, legs, end=" ")
```

7-4-2 取出、更改、新增與走訪字典項目

同時走訪字典的鍵和值：ch7-4-2d.py

•如果需要同時走訪字典的鍵和值，請使用items()方法，如下所示：

```
d1 = {"chicken": 2, "dog": 4, "cat":3}
for animal, legs in d1.items():
    print("動物:", animal, "/腳:", legs, "隻")
```

7-4-3 刪除字典項目

使用del關鍵字刪除字典值：ch7-4-3.py

- Python可以使用del關鍵字刪除指定鍵的值，如下所示：

```
d1 = {1:1, 2:4, "name":"joe", "age":20, 5:22}
del d1[2]
print(d1)
del d1["age"]
print(d1)
```

7-4-3 刪除字典項目

刪除和回傳字典值：ch7-4-3a.py

•Python可以使用pop()方法刪除參數的鍵，和回傳值，如下所示：

```
d1 = {1:1, 2:4, "name":"joe", "age":20, 5:22}
e1 = d1.pop(5)
print(e1, d1)
```

7-4-3 刪除字典項目

刪除字典的所有鍵值對：ch7-4-3b.py

- Python可以使用clear()方法刪除字典的所有鍵值對，即清空成一個空字典：{}，如下所示：

```
d1 = {1:1, 2:4, "name":"joe", "age":20, 5:22}
d1.clear()
print(d1)
```

7-4-4 字典函數與字典方法

字典函數

•Python字典函數可以取得字典長度的項目數、建立字典和排序字典的鍵等。常用字典函數說明，如下表所示：

字典函數	說明
len()	回傳參數字典的長度，即項目數
dict()	回傳參數轉換成的字典
sorted()	回傳字典中鍵排序結果的串列

7-4-4 字典函數與字典方法

字典方法

•Python字典物件的pop()、popitem()和clear()方法已經說明過，其他常用字典方法的說明，如下表所示：

字典方法	說明
get(key, default)	回傳字典中參數key鍵的項目值，如果key鍵不存在，回傳None，也可以指定第2個參數default是當沒有key鍵時，回傳的預設值
keys()	回傳字典中所有鍵的dict_keys物件
values()	回傳字典中所有值的dict_values物件

7-5 字串與容器型態的運算子

- 7-5-1 連接運算子
- 7-5-2 重複運算子
- 7-5-3 成員運算子
- 7-5-4 關係運算子
- 7-5-5 切割運算子



7-5-1 連接運算子

- 連接2個字串，如下所示：

```
str1, str2 = "Hello ", "World!"
str3 = str1 + str2
print(str3)
```

- 連接2個串列，即合併串列，如下所示：

```
lst1, lst2 = [2, 4], [6, 8, 10]
lst3 = lst1 + lst2
print(lst3)
```

- 連接2個元組，即合併元組，如下所示：

```
t1, t2 = (2, 4), (6, 8, 10)
t3 = t1 + t2
print(t3)
```

7-5-2 重複運算子

- 重複3次str1字串內容，如下所示：

```
str1 = "Hello"
str2 = str1 * 3
print(str2)
```

- 重複3次lst1串列的項目，如下所示：

```
lst1 = [1, 2]
lst2 = lst1 * 3
print(lst2)
```

- 重複3次t1元組的項目，如下所示：

```
t1 = (1, 2)
t2 = t1 * 3
print(t2)
```

7-5-3 成員運算子

- 檢查字串"come"是否存在str字串中，如下所示：

```
str = "Welcome!"
print("come" in str)      # True
print("come" not in str)  # False
```

- 檢查項目8是否存在lst1串列，項目2是否不存在lst1串列，如下所示：

```
lst1 = [2, 4, 6, 8]
print(8 in lst1)          # True
print(2 not in lst1)      # False
```

7-5-3 成員運算子

- 檢查項目8是否存在t1元組，項目2是否不存在t1元組，如下所示：

```
t1 = (2, 4, 6, 8)
print(8 in t1)          # True
print(2 not in t1)      # False
```

- 檢查鍵"tom"是否存在字典d1，是否不存在字典d1，如下所示：

```
d1 = {"tom": 2, "joe": 3}
print("tom" in d1)      # True
print("tom" not in d1)  # False
```

7-5-4 關係運算子

- 字串是一個字元和一個字元進行比較，直到分出大小，如下所示：

```
print("green" == "glow")    # False
print("green" != "glow")    # True
print("green" > "glow")     # True
print("green" >= "glow")    # True
print("green" < "glow")     # False
print("green" <= "glow")    # False
```

7-5-4 關係運算子

- 串列和元組的關係運算子是一個項目和一個項目依序的比較，如果是相同型別，就比較其值，不同型別，就使用型別名稱來比較。
- 字典只支援關係運算子「==」和「!=」，可以判斷2個字典是否相等，或不相等（字典不支援其他關係運算子），如下所示：

```
d1 = {"tom":30, "bobe":3}
d2 = {"bobe":3, "tom":30}
print(d1 == d2)          # True
print(d1 != d2)          # False
```

7-5-5 切割運算子

- Python的「[]」索引運算子也是「切割運算子」（Slicing Operator），可以從原始字串、串列和元組切割出所需的部分內容，其基本語法如下所示：

  字串、串列或元組[start:end]

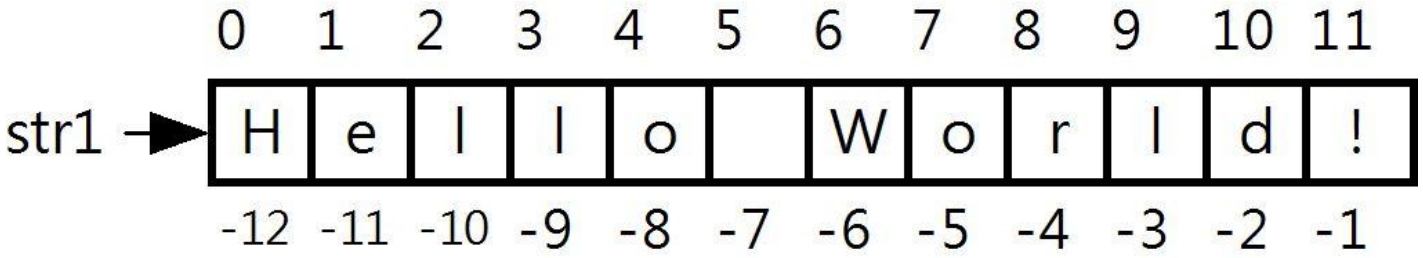
- 上述[]語法中使用「:」冒號分隔成2個索引位置，可以取回字串、串列和元組從索引位置start開始到end-1之間的部分內容，如果沒有start，就是從0開始；沒有end就是到最後1個字元或項目。

7-5-5 切割運算子

- 例如：本節範例str1字串和lst1串列和t1元組都是相同內容，如下所示：

```
str1 = 'Hello World!'
lst1 = list('Hello World!')
t1 = tuple('Hello World!')
```

- 上述程式碼建立串列和元組項目都是：['H', 'e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd', '!']。以字串為例的索引位置值可以是正，也可以是負值，如下圖所示：



看圖

切割內容	索引值範圍	取出的子字串、子串列和子元組
T[1:3]	1~2	"el" ['e', 'l'] ('e', 'l')
T[1:5]	1~4	"ello" ['e', 'l', 'l', 'o'] ('e', 'l', 'l', 'o')
T[:7]	0~6	"Hello W" ['H', 'e', 'l', 'l', 'o', ' ', 'W'] ('H', 'e', 'l', 'l', 'o', ' ', 'W')
T[4:]	4~11	"o World!" ['o', ' ', 'W', 'o', 'r', 'l', 'd', '!'] ('o', ' ', 'W', 'o', 'r', 'l', 'd', '!')
T[1:-1]	1~(-2)	"ello World" ['e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd'] ('e', 'l', 'l', 'o', ' ', 'W', 'o', 'r', 'l', 'd')
T[6:-2]	6~(-3)	"Worl" ['W', 'o', 'r', 'l'] ('W', 'o', 'r', 'l')

第8章 檔案、類別與例外處理

- 8-1 檔案處理
- 8-2 二進位檔案讀寫
- 8-3 類別與物件
- 8-4 建立例外處理



8-1 檔案處理

- 8-1-1 開啟與關閉檔案
- 8-1-2 寫入資料到檔案
- 8-1-3 在檔案新增資料
- 8-1-4 讀取檔案的全部內容
- 8-1-5 讀取檔案的部分內容
- 8-1-6 with/as程式區塊和走訪檔案物件



8-1-1 開啟與關閉檔案

開啟與關閉檔案：ch8-1-1.py

- 在Python程式可以使用open()函數開啟檔案；close()方法關閉檔案，如下所示：

```
fp = open("note.txt", "w")
if fp != None:
    print("檔案開啟成功!")
fp.close()
```

8-1-1 開啟與關閉檔案

開啟檔案的檔案路徑：ch8-1-1a.py

- Python開啟檔案的路徑如果使用「\」符號，在Windows作業系統需要使用逸出字元「\\」，如下所示：

```
fp = open("temp\\note.txt", "w")
```

- 上述參數"temp\\note.txt"就是路徑「temp\note.txt」。

8-1-2 寫入資料到檔案

寫入換行資料至檔案：ch8-1-2.py

- 請注意！不同於print()函數預設加上換行的「\n」新行字元，write()方法如需換行，請自行在字串後加上新行字元，如下所示：

```
"陳會安\n"
```

- Python程式開啟寫入檔案note.txt後，寫入2行姓名資料，如下所示：

```
fp = open("note.txt", "w")
fp.write("陳會安\n")
fp.write("江小魚\n")
print("已經寫入2個姓名到檔案note.txt!")
fp.close()
```

8-1-2 寫入資料到檔案

寫入沒有換行的資料至檔案：ch8-1-2a.py

•Python程式準備寫入資料至「temp/note.txt」檔案，此時呼叫的write()方法並沒有使用新行字元，如下所示：

```
fp = open("temp/note.txt", "w")
fp.write("陳會安")
fp.write("江小魚")
print("已經寫入2個姓名到檔案note.txt!")
fp.close()
```

8-1-3 在檔案新增資料

- 在第8-1-2節寫入資料到檔案前會清除檔案內容，如同在全新檔案寫入資料，如果想在檔案現有資料最後新增資料，例如：在note.txt檔案最後再新增姓名資料，請使用"a"模式字串開啟新增檔案，如下所示：

```
fp = open("note.txt", "a")
```

8-1-3 在檔案新增資料

新增換行資料至檔案：ch8-1-3.py

•在Python程式開啟新增檔案note.txt後，再新增1行姓名資料至檔尾，如下所示：

```
fp = open("note.txt", "a")
fp.write("陳允傑\n")
print("已經新增1個姓名到檔案note.txt!")
fp.close()
```

8-1-3 在檔案新增資料

新增沒有換行資料至檔案：ch8-1-3a.py

•Python程式準備新增資料至「temp/note.txt」檔案，此時呼叫的write()方法並沒有使用新行字元，如下所示：

```
fp = open("temp/note.txt", "a")
fp.write("陳允傑")
print("已經新增1個姓名到檔案note.txt!")
fp.close()
```

8-1-4 讀取檔案的全部內容

- 檔案物件提供多種方法來讀取檔案內容，在這一節是讀取檔案的全部內容，下一節只讀取檔案的部分內容。因為是讀取檔案，open()函數是使用"r"模式字串來開啟檔案，如下所示：

```
fp = open("note.txt", "r")
```

8-1-4 讀取檔案的全部內容

使用read()方法讀取檔案全部內容：ch8-1-4.py

- 當檔案物件的read()方法沒有參數時，就是讀取檔案的全部內容，如下所示：

```
fp = open("note.txt", "r")
str1 = fp.read()
print("檔案內容:")
print(str1)
fp.close()
```

8-1-4 讀取檔案的全部內容

使用readlines()方法讀取檔案全部內容：ch8-1-4a.py

•Python程式也可以使用檔案物件的readlines()方法，讀取檔案內容成為一個串列，每一行是一個項目，如下所示：

```
fp = open("note.txt", "r")
list1 = fp.readlines()
print("檔案內容:")
print(list1)
for line in list1:
    print(line, end="")
fp.close()
```

8-1-5 讀取檔案的部分內容

使用read()方法讀取檔案的部分內容：ch8-1-5.py

•在檔案物件的read()方法可以加上參數值來讀取所需的字元數，如下所示：

```
fp = open("note.txt", "r")
str1 = fp.read(1)
str2 = fp.read(2)
print("檔案內容:")
print(str1)
print(str2)
fp.close()
```

8-1-5 讀取檔案的部分內容

使用readline()方法讀取檔案的部分內容：ch8-1-5a.py

•Python程式可以使用readline()方法只讀取檔案的1行文字內容，如下所示：

```
fp = open("note.txt", "r")
str1 = fp.readline()
str2 = fp.readline()
print("檔案內容:")
print(str1)
print(str2)
fp.close()
```

8-1-6 with/as程式區塊和走訪檔案物件

使用with/as程式區塊讀取檔案全部內容：ch8-1-6.py

•Python程式改用with/as程式區塊讀取檔案全部內容，如下所示：

```
with open("note.txt", "r") as fp:
    str1 = fp.read()
    print("檔案內容:")
    print(str1)
```

8-1-6 with/as程式區塊和走訪檔案物件

走訪檔案物件來讀取資料：ch8-1-6a.py

•Python程式開啟2次note.txt檔案，然後分別使用for迴圈走訪檔案物件來讀取每一行的資料，如下所示：

```
fp = open("note.txt", "r")
print("檔案內容(有換行):")
for line in fp:
    print(line)
fp.close()
fp = open("note.txt", "r")
print("檔案內容(沒換行):")
for line in fp:
    print(line, end="")
fp.close()
```

8-2 二進位檔案讀寫

- 8-2-1 將資料寫入二進位檔案
- 8-2-2 從二進位檔案讀取資料



8-2-1 將資料寫入二進位檔案

- Python二進位檔案處理需要使用pickle模組。在Python程式首先需要匯入模組（關於模組和套件的說明，請參閱第9章），如下所示：

```
import pickle
```

- 上述程式碼使用import關鍵字匯入名為pickle的模組後，就可以使用此模組的函數或方法來執行二進位檔案處理。

8-2-1 將資料寫入二進位檔案

開啟和關閉二進位檔案：ch8-2-1.py

•Python一樣是使用open()函數開啟二進位檔案，只是開啟模式字串不同，如下所示：

```
fp = open("note.dat", "wb")
if fp != None:
    print("二進位檔案開啟成功!")
fp.close()
```

8-2-1 將資料寫入二進位檔案

將資料寫入二進位檔案：ch8-2-1a.py

•Python程式是呼叫pickle模組的dump()方法將資料寫入二進位檔案，在開啟note.dat二進位檔案後，呼叫3次dump()方法依序寫入整數、字串和串列，如下所示：

```
fp = open("note.dat", "wb")
print("寫入整數: 11")
pickle.dump(11, fp)
print("寫入字串: '陳會安'")
pickle.dump("陳會安", fp)
print("寫入串列: [1, 2, 3, 4]")
pickle.dump([1, 2, 3, 4], fp)
fp.close()
```

8-2-2 從二進位檔案讀取資料

- Python程式是使用pickle模組的load()方法從二進位檔案讀取資料，首先開啟讀取的二進位檔案，如下所示：

```
fp = open("note.dat", "rb")
```

- 上述程式碼使用open()函數開啟檔案，第2個參數"rb"是開啟讀取的二進位檔案。

8-2-2 從二進位檔案讀取資料

從二進位檔案讀取資料：ch8-2-2.py

- 在Python程式開啟note.dat二進位檔案後，呼叫3次load()方法依序讀取整數、字串和串列，如下所示：

```
fp = open("note.dat", "rb")
i = pickle.load(fp)
print("讀取整數 = ", str(i))
str1 = pickle.load(fp)
print("讀取姓名 = ", str1)
list1 = pickle.load(fp)
print("讀取串列 = ", str(list1))
fp.close()
```

8-2-2 從二進位檔案讀取資料

使用二進位檔案存取字典資料：ch8-2-2a.py

- Python程式的pickle模組一樣可以處理字典資料，我們可以將字典變數存入二進位檔案後，原封不動的再從二進位檔案讀取字典資料，如下所示：

```
data = {
    "name": "Joe Chen",
    "age": 22,
    "score": 95,
}
with open("dic.dat", "wb") as f:
    pickle.dump(data, f)
with open("dic.dat", "rb") as f:
    new_data = pickle.load(f)
print(new_data)
```

8-3 類別與物件

- 8-3-1 定義類別和建立物件
- 8-3-2 類別的繼承



8-3-1 定義類別和建立物件

使用class定義類別

•類別（Class）是物件的模子和藍圖，我們需要先定義類別，才能依據類別的模子來建立物件。例如：定義Student類別，如下所示：

```
class Student:
    def __init__(self, name, grade):
        self.name = name
        self.grade = grade

    def displayStudent(self):
        print("姓名 = " + self.name)
        print("成績 = " + str(self.grade))
```

8-3-1 定義類別和建立物件

類別建構子__init__

- 類別建構子是每一次使用類別建立新物件時，就會自動呼叫的方法，Python類別的建構子名稱是__init__，不允許更名，請注意！在init前後是2個「_」底線，如下所示：

```
def __init__(self, name, grade):
    self.name = name
    self.grade = grade
```

- 上述建構子寫法和Python函數相同，在建立新物件時，可以使用參數來指定資料欄位name和grade的初值。

8-3-1 定義類別和建立物件

建構子和方法的self變數

- 在Python類別建構子和方法的第1個參數是self變數，這是一個特殊變數，絕對不可以忘記此參數。不過，self不是Python語言的關鍵字，只是約定俗成的變數名稱，self變數的值是參考呼叫建構子或方法的物件，以建構子__init__()方法來說，參數self的值就是參考新建立的物件，如下所示：

```
self.name = name
self.grade = grade
```

- 上述程式碼self.name和self.grade就是指定新物件資料欄位name和grade的值。

8-3-1 定義類別和建立物件

資料欄位：name和grade

- 類別的資料欄位，或稱為成員變數（Member Variables），在Python類別定義資料欄位並不需要特別語法，只要是使用self開頭存取的變數，就是資料欄位，在Student類別的資料欄位有name和grade，如下所示：

```
self.name = name
self.grade = grade
```

- 上述程式碼是在建構子指定資料欄位的初值，沒有特別語法，name和grade就是類別的資料欄位。

8-3-1 定義類別和建立物件

方法：displayStudent()

•類別的方法就是Python函數，只是第1個參數一定是self變數，而且在存取資料欄位時，不要忘了使用self變數來存取（因為有self才是存取資料欄位），如下所示：

```
def displayStudent(self):
    print("姓名 = " + self.name)
    print("成績 = " + str(self.grade))
```

8-3-1 定義類別和建立物件

使用類別建立物件

- 當定義類別後，就可以使用類別建立物件，也稱為實例（Instances），同一類別可以如同工廠生產般的建立多個物件，如下所示：

```
s1 = Student("陳會安", 85)
s1.displayStudent()
print("s1.name = " + s1.name)
print("s1.grade = " + str(s1.grade))
```

8-3-2 類別的繼承

- 繼承是物件的再利用，當定義好類別後，其他類別可以繼承此類別的資料和方法，新增或取代繼承類別的資料和方法，而不用修改其繼承類別的程式碼。

8-3-2 類別的繼承

父類別Vehicle

- 在Python實作類別繼承的父類別Vehicle，如下所示：

```
class Vehicle:
    def __init__(self, name):
        self.name = name

    def setName(self, name):
        self.name = name

    def getName(self):
        return self.name
```

8-3-2 類別的繼承

子類別Car

- 在子類別Car定義是繼承父類別Vehicle，如下所示：

```
class Car(Vehicle):
    def __init__(self, name, model):
        super().__init__(name)
        self.model = model

    def displayCar(self):
        print("名稱 = " + self.getName())
        print("車型 = " + self.model)
```

8-4 建立例外處理

- 8-4-1 例外處理程式敘述
- 8-4-2 同時處理多種例外
- 8-4-3 else和finally程式區塊



8-4 建立例外處理

- 當Python程式執行時偵測出錯誤就會產生例外（Exception），例外處理（Exception Handling）就是建立try/except程式區塊，以便Python程式碼在執行時產生例外時，能夠撰寫程式碼來進行補救處理。
- 簡單的說，例外處理是希望程式碼產生錯誤時可以讓我們進行補救，而不是讓直譯器顯示錯誤訊息且中止程式的執行，我們可以在Python程式使用例外處理程式敘述來處理這些錯誤。

8-4-1 例外處理程式敘述

- Python例外處理程式敘述主要分為try和except二個程式區塊，其基本語法，如下所示：

```
try:
    # 產生例外的程式碼
except <Exception Type>:
    # 例外處理
```

8-4-1 例外處理程式敘述

建立檔案不存在的例外處理：ch8-4-1.py

•如果Python程式開啟的檔案不存在，就會產生FileNotFoundError例外，我們可以使用try/except處理檔案不存在的例外，如下所示：

```
try:
    fp = open("myfile.txt", "r")
    print(fp.read())
    fp.close()
except FileNotFoundError:
    print("錯誤: myfile.txt檔案不存在!")
```

8-4-1 例外處理程式敘述

串列索引值不存在的例外處理：：ch8-4-1b.py

•如果Python程式存取的串列索引值不存在，就會產生IndexError例外，我們可以使用try/except處理串列索引值不存在的例外，如下所示：

```
lst1 = [1, 2, 3, 4, 5]
try:
    print(lst1[6])
except IndexError:
    print("錯誤: 串列的索引值錯誤!")
```

8-4-2 同時處理多種例外

使用eval()內建函數：ch8-4-2.py

•Python的eval()內建函數可以在執行期執行參數的Python程式片段，如下所示：

```
m = 10
eval("print('Python')")
eval("print(33 + 22)")
eval("print(55 / 9)")
eval("print('m' * 6)")
eval("print(m+10)")
```

8-4-2 同時處理多種例外

同時處理多種例外的try/except程式敘述：ch8-4-2a.py

•例外處理程式敘述有3個except程式區塊，try程式區塊使用eval()函數輸入2個「,」號分隔n1和n2值，如下所示：

```
try:
    n1, n2 = eval(input("輸入2個整數(n1,n2) => "))
    r = n1 / n2
    print("變數r的值 = " + str(r))
except ZeroDivisionError:
    print("錯誤: 除以0的錯誤!")
except SyntaxError:
    print("錯誤: 輸入數字需以逗號分隔!")
except:
    print("錯誤: 輸入錯誤!")
```

8-4-3 else和finally程式區塊

- Python的try/except例外處理程式敘述還可以加上else和finally兩個選項的程式區塊，其語法如下所示：

```
try:
    # 產生例外的程式碼
except <Exception Type>:
    # 例外處理
else:
    # 如果沒有例外，就會執行
finally:
    # 不論是否有產生例外，都會執行
```

8-4-3 else和finally程式區塊

使用else程式區塊：ch8-4-3.py

•Python程式是修改ch8-4-1b.py，保留except程式區塊和新增else程式區塊，當輸入不同的索引值後，可以顯示不同的錯誤訊息文字，如下所示：

```
lst1 = [1, 2, 3, 4, 5]
try:
    idx = int(input("輸入索引值 => "))
    print(lst1[idx])
except IndexError:
    print("錯誤: 串列的索引值錯誤!")
else:
    print("Else: 輸入的索引沒有錯誤!")
```

8-4-3 else和finally程式區塊

使用else和finally程式區塊：ch8-4-3a.py

•修改ch8-4-3.py，再新增finally程式區塊，當輸入不同的索引值後，可以顯示不同的訊息文字，如下所示：

```
lst1 = [1, 2, 3, 4, 5]
try:
    idx = int(input("輸入索引值 => "))
    print(lst1[idx])
except IndexError:
    print("錯誤: 串列的索引值錯誤!")
else:
    print("Else: 輸入的索引沒有錯誤!")
finally:
    print("Finally: 你有輸入資料!")
```

第9章 Python模組與套件

- 9-1 Python模組與套件
- 9-2 os模組：檔案操作與路徑處理
- 9-3 math模組：數學函數
- 9-4 turtle模組：海龜繪圖
- 9-5 pywin32套件：Office軟體自動化



9-1 Python模組與套件

- 9-1-1 建立與匯入自訂模組
- 9-1-2 使用模組擴充Python程式功能



9-1 Python模組與套件

- Python模組（Modules）就是副檔名.py的Python程式檔案，套件是一個內含多個模組集合的目錄，而且在根目錄有一個名為__init__.py的Python檔案（在名稱前後是2個「_」底線）。
- 當撰寫的程式碼愈來愈多時，就可以將相關Python程式檔案的模組群組成套件，以方便其他Python程式重複使用這些Python程式碼。

9-1-1 建立與匯入自訂模組

建立自訂模組：mybmi.py

- 請建立名為mybmi.py的Python程式檔案，如下所示：

```
name = None

def bmi(h, w):
    r = w/h/h
    return r
```

9-1-1 建立與匯入自訂模組

匯入和使用自訂模組：ch9-1-1.py

- 當建立自訂模組mybmi.py後，其他Python程式檔案如果需要使用bmi()函數，可以直接匯入此模組來使用，其基本語法如下所示：

```
import 模組名稱1[, 模組名稱2...]
```

- 上述語法使用import關鍵字匯入之後的模組名稱，如果不只一個，請使用「,」分隔，模組名稱是Python程式檔案名稱（不需副檔名.py），例如：匯入自訂模組mybmi.py，如下所示：

```
import mybmi
```

- 上述程式碼匯入mybmi模組，即mybmi.py程式檔案（位在相同目錄）。

9-1-1 建立與匯入自訂模組

匯入和使用自訂模組：ch9-1-1.py

- 我們可以存取模組變數和呼叫模組函數，其語法如下所示：

  模組名稱.變數或函數

- 上述語法使用「.」運算子存取模組的變數和呼叫函數，在「.」運算子之前是模組名稱；之後是模組的變數或函數名稱，例如：Python程式準備使用自訂模組mybmi來指定姓名name變數值，和呼叫bmi()函數，如下所示：

```
mybmi.name = "陳會安"
print("姓名=", mybmi.name)
r = mybmi.bmi(1.75, 75)
print("BMI值=", r)
```

9-1-2 使用模組擴充Python程式功能

匯入和使用random模組：ch9-1-2.py

•Python程式一樣是使用import關鍵字來匯入內建模組或第三方開發的套件，例如：匯入名為random的內建模組，然後呼叫此模組的randint()方法來產生1~100之間的整數亂數值，如下所示：

```
import random

value = random.randint(1, 100)
print(value)
```

9-1-2 使用模組擴充Python程式功能

模組的別名：ch9-1-2a.py

•在Python程式檔匯入模組，除了使用模組名稱來呼叫函數，如果模組名稱太長，我們可以使用as關鍵字替模組取一個別名，然後改用別名來呼叫函數，如下所示：

```
import random as R

value = R.randint(1, 100)
print(value)
```

9-1-2 使用模組擴充Python程式功能

匯入模組的部分名稱：ch9-1-2b.py

- 在實務上，如果模組十分龐大，但只使用到模組的1或2個函數，此時請改用form/import程式敘述只匯入模組的部分名稱，其語法如下所示：

```
from 模組名稱 import 名稱1[,名稱2..]
```

- 例如：匯入第9-1-1節mybmi模組的bmi()函數，如下所示：

```
from mybmi import bmi

r = bmi(1.75, 75)
print("BMI值=", r)
```

9-1-2 使用模組擴充Python程式功能

將模組所有名稱匯入成為目前範圍：ch9-1-2c.py

•我們在Python程式使用from/import程式敘述匯入的名稱，如同是在此程式檔案建立的識別字，如果想將模組所有名稱都匯入成為目前範圍，以便使用時不用指明模組名稱，請使用「*」萬用字元代替匯入的名稱清單，如下所示：

```
from mybmi import *

name = "陳會安"
print("姓名 = " + name)
r = bmi(1.75, 75)
print("BMI值 = " + str(r))
```

9-1-2 使用模組擴充Python程式功能

顯示模組的所有名稱：ch9-1-2d.py

•對於Python程式匯入的模組，我們可以呼叫dir()函數顯示此模組的所有名稱，如下所示：

```
import random

print(dir(random))
```

9-2 os模組：檔案操作與路徑處理

- 9-2-1 os模組
- 9-2-2 os.path模組處理路徑字串
- 9-2-3 os.path模組檢查檔案是否存在



9-2-1 os模組

- Python的os模組提供目錄處理的相關方法，可以刪除檔案、建立目錄和更名/刪除目錄/檔案。在Python程式使用os模組需要先匯入此模組，如下所示：

```
import os
```

9-2-1 os模組

取得目前工作目錄和顯示檔案/目錄清單：ch9-2-1.py

•Python程式可以呼叫os模組的getcwd()方法回傳目前工作目錄，listdir(path)方法回傳參數path路徑下的檔案和目錄清單（儲存在串列），如下所示：

```
import os

path = os.getcwd() + "\\temp"
os.chdir(path)
print(path)
print(os.listdir(path))
```

9-2-1 os模組

建立與切換目錄：ch9-2-1a.py

•在os模組可以使用chdir(path)方法切換至參數路徑的目錄，和呼叫mkdir(path)方法建立參數路徑的目錄，如下所示：

```
path = os.getcwd() + "\\temp"
print("目前工作路徑: ", os.getcwd())
print(path)
os.chdir(path)
print("chdir(): ", os.getcwd())
os.mkdir('newDir')
print("mkdir(): ", os.listdir(path))
```

9-2-1 os模組

目錄和檔案更名：ch9-2-1b.py

- 在os模組的rename(old, new)方法可以更名參數old的檔案或目錄成為新名稱new的檔案或目錄名稱，如下所示：

```
path = os.getcwd() + "\\temp"
os.chdir(path)
os.rename('newDir','newDir2')
print("rename(): ", os.listdir(path))
```

9-2-1 os模組

刪除目錄和檔案：ch9-2-1c.py

•在os模組可以使用rmdir(path)方法刪除參數路徑的目錄，和remove(path)方法刪除參數路徑的檔案，如下所示：

```
path = os.getcwd() + "\\temp"
os.chdir(path)
os.rmdir('newDir2')
fp = open("aa.txt", "w")
fp.close()
print("rmdir(): ", os.listdir(path))
os.remove("aa.txt")
print("remove(): ", os.listdir(path))
```

9-2-2 os.path模組處理路徑字串

- os.path模組提供方法取得指定檔案的完整路徑，和路徑字串處理的相關方法，可以取得路徑字串中的檔名和路徑，或合併建立存取檔案的完整路徑字串。
- 在Python程式使用os.path模組需要先匯入此模組（取別名path），如下所示：

```
import os.path as path
```

9-2-2 os.path模組處理路徑字串

取得檔案完整路徑、檔名和副檔名：ch9-2-2.py

•在os.path模組可以使用realpath(fname)方法回傳參數檔名的完整路徑字串，split(fname)方法將參數分割成路徑和檔案字串的元組，splittext(fname)方法將參數分割成路徑（僅含檔名）和副檔名字串的元組，如下所示：

```
import os.path as path

fname = path.realpath("ch9-2-2.py")
print(fname)
r = path.split(fname)
print("os.path.split() =", r)
r = path.splitext(fname)
print("os.path.splitext() =", r)
```

9-2-2 os.path模組處理路徑字串

分割檔案路徑成為路徑和檔名：ch9-2-2a.py

•在os.path模組可以使用dirname(fname)方法回傳參數fname的路徑字串；basename(fname)方法回傳參數fname的檔名字串，如下所示：

```
fname = path.realpath("ch9-2-2.py")
print(fname)
p = path.dirname(fname)
print("p = os.path.dirname() =", p)
f = path.basename(fname)
print("f = os.path.basename() =", f)
```

9-2-2 os.path模組處理路徑字串

合併路徑和檔名：ch9-2-2b.py

•如果已經取得路徑和檔名，我們可以使用os.path模組的join(path, fname)方法合併路徑和檔名，其回傳值是合併參數path路徑和fname檔名的完整檔案路徑字串，如下所示：

```
p = "C:\Python\ch09"
f = "ch9-2-2.py"
print(p, f)
r = path.join(p, f)
print("os.path.join(p,f) =", r)
```

9-2-3 os.path模組檢查檔案是否存在

- os.path模組提供檢查檔案是否存在，路徑字串是檔案，或目錄的方法。相關方法的說明，如下表所示：

方法	說明
exists(fname)	檢查參數fname的檔案是否存在，如果存在，回傳True；否則為False
isdir(fname)	檢查參數fname是否是目錄，如果是，回傳True；否則為False
isfile(fname)	檢查參數fname是否是檔案，如果是，回傳True；否則為False

9-3 math模組：數學函數

- Python除了內建數學函數外，還可以使用math內建模組的數學、三角和對數函數。在Python程式需要匯入此模組，如下所示：

```
import math
```

- 在匯入math模組後，就可以取得常數值和呼叫相關的數學方法。

math模組的數學常數：ch9-3.py

- math模組提供2個常用的數學常數，其說明如下表所示：

常數	說明
e	自然數e=2.718281828459045
pi	圓周率π=3.141592653589793

9-3 math模組：數學函數

math模組的數學方法：ch9-3a.py

方法	說明
fabs(x)	回傳參數x的絕對值
acos(x)	反餘弦函數
asin(x)	反正弦函數
atan(x)	反正切函數
atan2(y, x)	參數y/x的反正切函數值
ceil(x)	回傳x值大於或等於參數x的最小整數
cos(x)	餘弦函數
exp(x)	自然數的指數e^x
floor(x)	回傳x值大於或等於參數x的最大整數
log(x)	自然對數
pow(x, y)	回傳第1個參數x為底，第2個參數y的次方值
sin(x)	正弦函數
sqrt(x)	回傳參數的平方根
tan(x)	正切函數
degrees(x)	將參數x的徑度轉換成角度
radians(x)	將參數x的角度轉換成徑度

9-4 turtle模組：海龜繪圖

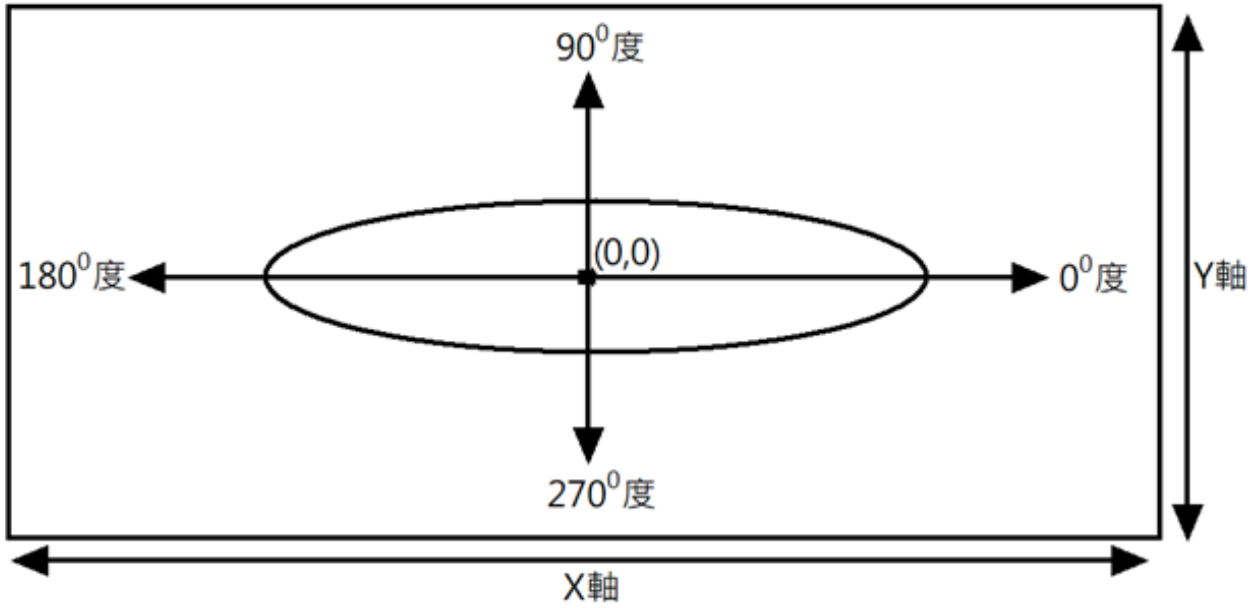
- 9-4-1 認識Python海龜繪圖
- 9-4-2 Python海龜繪圖的基本使用
- 9-4-3 使用海龜繪圖繪出幾何圖形



9-4-1 認識Python海龜繪圖

- 海龜繪圖是使用電腦程式來模擬這隻在沙灘上爬行的海龜，海龜使用相對位置的前進和旋轉指令來移動位置和更改方向，我們只需重複執行這些操作，就可以透過海龜行走經過的足跡來繪出幾何圖形。
- 基本上，海龜繪圖的這隻海龜擁有三種屬性：目前位置、方向和畫筆（即足跡），畫筆可以指定色彩和寬度，下筆繪圖或提筆不繪圖。Python海龜圖示在沙灘行走的座標系統說明，如下所示：
  - 海龜本身是使用圖示來標示。
  - 初始座標是視窗中心點(0, 0)，方向是面向東方0度。
  - 線條的色彩預設是黑色；線寬是1像素且下筆繪圖。

9-4-1 認識Python海龜繪圖



Python的turtle模組

•Python的turtle模組是內建模組，並不需要額外安裝，我們只需匯入turtle模組，就可以使用海龜繪圖，如下所示：

```
import turtle
```

9-4-2 Python海龜繪圖的基本使用

- Python程式在匯入turtle模組後，就可以使用turtle模組的4種基本行走和轉向方法來繪圖，如下表所示：

方法	說明
turtle.forward(x)	從目前方向向前走x步
turtle.back(x)	從目前方向後退走x步
turtle.left(x)	從目前方向反時鐘向左轉x度
turtle.right(x)	從目前方向順時鐘向右轉x度

9-4-2 Python海龜繪圖的基本使用

控制海龜的行走和轉向：ch9-4-2.py

•Python程式首先匯入turtle模組和取得螢幕screen物件的Windows視窗後，呼叫setup()方法指定螢幕尺寸是(500, 400)，如下所示：

```
import turtle


screen = turtle.Screen()
screen.setup(500, 400)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
screen.exitonclick()
```

9-4-2 Python海龜繪圖的基本使用

更改海龜的圖示形狀與色彩：ch9-4-2a.py

- 在turtle模組可以使用color()方法更改海龜圖示的色彩（參數是色彩名稱）；shape()方法是更改海龜圖示的形狀，如下所示：

```
turtle.color("blue")
turtle.shape("turtle")
turtle.forward(100)
```

9-4-2 Python海龜繪圖的基本使用

提筆/下筆、畫筆色彩與尺寸：ch9-4-2b.py

•在turtle模組可以使用pensize()方法更改畫筆尺寸；pencolor()方法更改畫筆色彩，penup()方法是提筆不繪圖；pendown()方法是下筆繪圖，如下所示：

```
turtle.pensize(5)
turtle.pencolor("blue")
turtle.forward(100)
turtle.penup()
turtle.left(90)
turtle.forward(50)
turtle.pendown()
turtle.left(90)
turtle.forward(100)
```

9-4-2 Python海龜繪圖的基本使用

設定沙灘視窗的位置：ch9-4-2c.py

- 在screen.setup()方法除了指定螢幕尺寸，還可以使用startx和starty參數指定螢幕的顯示位置(20, 50)，如下所示：

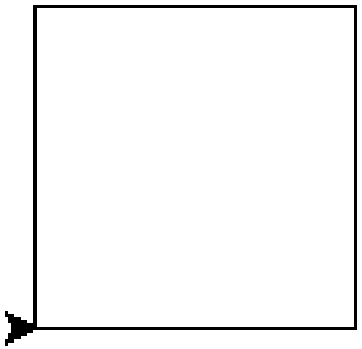
```
screen.setup(500, 400, startx=20, starty=50)
```

9-4-3 使用海龜繪圖繪出幾何圖形

繪出正方形：ch9-4-3.py

•在Python程式的for迴圈共執行4次，每次轉90度來繪出4個邊的正方形，如下所示：

```
for i in range(1, 5):
    turtle.forward(100)
    turtle.left(90)
```

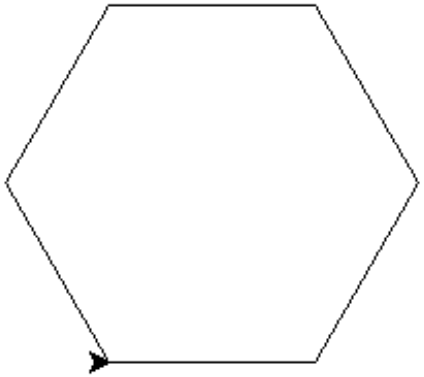


9-4-3 使用海龜繪圖繪出幾何圖形

繪出六角形：ch9-4-3a.py

•在Python程式的for迴圈共執行6次，每次轉60度來繪出6個邊的六角形，如下所示：

```
for i in range(1, 7):
    turtle.forward(100)
    turtle.left(60)
```

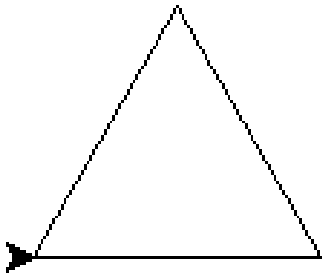


9-4-3 使用海龜繪圖繪出幾何圖形

繪出三角形：ch9-4-3b.py

•在Python程式的for迴圈共執行3次，每次轉120度來繪出3個邊的三角形，如下所示：

```
for i in range(1, 4):
    turtle.forward(100)
    turtle.left(120)
```

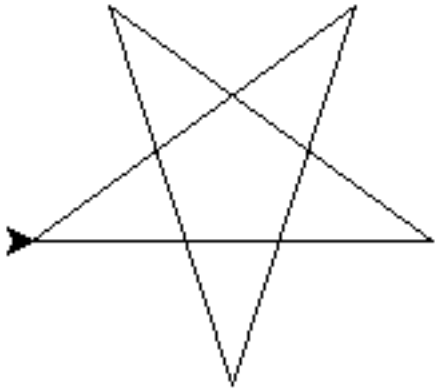


9-4-3 使用海龜繪圖繪出幾何圖形

繪出星形：ch9-4-3c.py

•在Python程式的for迴圈共執行5次，每次轉144度來繪出5個邊的星形，如下所示：

```
for i in range(1, 6):
    turtle.forward(150)
    turtle.left(144)
```

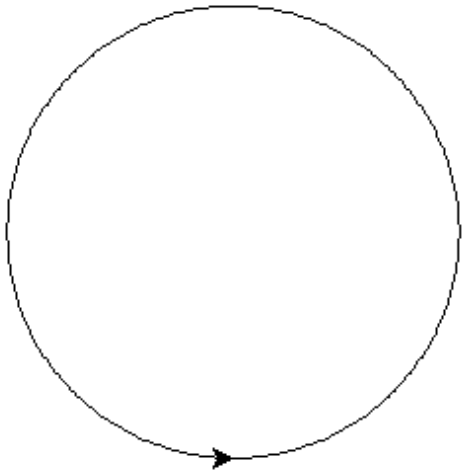


9-4-3 使用海龜繪圖繪出幾何圖形

繪出圓形：ch9-4-3d.py

•在Python程式的for迴圈共執行360次，每次轉1度來繪出360個邊的圓形，如下所示：

```
for i in range(360):
    turtle.forward(2)
    turtle.left(1)
```



9-5 pywin32套件：Office軟體自動化

- 9-5-1 Python套件管理：安裝pywin32套件
- 9-5-2 Word軟體自動化
- 9-5-3 Excel軟體自動化
- 9-5-4 PowerPoint軟體自動化



9-5-1 Python套件管理：安裝pywin32套件

- 套件管理（Package Manager）就是管理Python程式開發所需的套件，可以安裝新套件、檢視安裝的套件清單或移除不需要的套件，Python預設套件管理工具是pip，這是一個命令列工具來管理套件。

使用pip安裝Python套件

- Python開發環境尚未安裝的Python套件，例如：pywin32套件，可以輸入命令列指令進行安裝，如下所示：

  pip install pywin32 Enter

- 上述install參數是安裝，可以安裝之後名為pywin32的套件。如果需要指定安裝的版本，請使用「==」指定安裝的版本號碼，如下所示：

  pip install pywin32==303 Enter

9-5-1 Python套件管理：安裝pywin32套件

使用pip檢視已經安裝的Python套件清單

- 我們可以使用pip list指令來檢視已安裝的Python套件清單，如下所示：

pip list Enter

9-5-2 Word軟體自動化

- 當成功安裝pywin32套件後，就可以在Python程式匯入pywin32套件來控制Office軟體。首先是Word軟體自動化，如下所示：

```
import win32com
from win32com.client import Dispatch
import os
```

- 上述程式碼的前2行是匯入pywin32套件，第3行匯入os模組是為了取得工作目錄來建立檔案的絕對路徑。

9-5-2 Word軟體自動化

啟動Word開啟現存文件：ch9-5-2.py

•Python程式在匯入pywin32套件後，就可以建立COM物件來啟動Word軟體，和開啟存在的Word文件，如下所示：

```
...
app = win32com.client.Dispatch("Word.Application")
app.Visible = 1
app.DisplayAlerts = 0
docx =
app.Documents.Open(os.getcwd()+"\\test.docx")
```

9-5-2 Word軟體自動化

取得Word文件的段落數和段落內容：ch9-5-2a.py

•當成功使用pywin32套件開啟test.docx文件後，我們可以計算文件的段落數，和走訪段落來顯示各段落文字內容，如下所示：

```
...
docx =
app.Documents.Open(os.getcwd()+"\\test.docx")
print ("段落數: ", docx.Paragraphs.count)
for i in range(len(docx.Paragraphs)):
    para = docx.Paragraphs[i]
    print(para.Range.text)
docx.Close()
app.Quit()
```

9-5-2 Word軟體自動化

新增Word文件插入文字後儲存檔案：ch9-5-2b.py

- Python程式除了使用pywin32套件開啟存在文件外，也可以新增文件，即呼叫app.Documents.Add()方法建立文件物件，然後在文件插入文字內容，如下所示：

```
...
docx = app.Documents.Add()
pos = docx.Range(0, 0)
pos.InsertBefore("Python程式設計")
docx.SaveAs(os.getcwd()+"\\test2.docx")
```

- ...

9-5-3 Excel軟體自動化

啟動Excel開啟試算表取得儲存格值：ch9-5-3.py

•Python程式只需指定"Excel.Application"軟體名稱字串，就可以啟動Excel軟體來開啟存在的試算表檔案test.xlsx，如下所示：

```
app = win32com.client.Dispatch("Excel.Application")
app.Visible = 1
app.DisplayAlerts = 0
xlsx =
app.Workbooks.Open(os.getcwd()+"\\test.xlsx")
sheet = xlsx.Worksheets(1)
row = sheet.UsedRange.Rows.Count
col = sheet.UsedRange.Columns.Count
print(row, col)
```

9-5-3 Excel軟體自動化

啟動Excel開啟試算表取得儲存格值：ch9-5-3.py

•程式碼開啟試算表檔案後，使用Worksheets(1)取得第1個工作表，然後顯示已使用的儲存格範圍。在下方取得指定儲存格的值，或取得儲存格範圍的內容，如下所示：

```
print("Cells(2, 1)=", sheet.Cells(2, 1).Value)
print("Cells(2, 2)=", sheet.Cells(2, 2).Value)
value = sheet.Range("A1:B3").Value
print(value)
xlsx.Close(False)
app.Quit()
```

9-5-3 Excel軟體自動化

新增Excel試算表指定儲存格值後儲存檔案：ch9-5-3a.py

- Python程式呼叫app.Workbooks.Add()方法建立試算表物件，即可取得第1個工作表，如下所示：

```
...
xlsx = app.Workbooks.Add()
sheet = xlsx.Worksheets(1)
sheet.Cells(1, 1).Value = 1
sheet.Cells(1, 2).Value = 2
sheet.Cells(2, 1).Value = 3
sheet.Cells(2, 2).Value = 4
sheet.Cells(3, 1).Value = 5
sheet.Cells(3, 2).Value = 6
xlsx.SaveAs(os.getcwd()+"\\test2.xlsx")
...
```

9-5-4 PowerPoint軟體自動化

- Python程式：ch9-5-4.py，如下所示：

```
...
app =
win32com.client.Dispatch("PowerPoint.Application")
app.Visible = 1
app.DisplayAlerts = 0
pptx =
app.Presentations.Open(os.getcwd()+"\\test.pptx")
pptx.SlideShowSettings.Run()
time.sleep(1)
pptx.SlideShowWindow.View.Next()
```

9-5-4 PowerPoint軟體自動化

```
time.sleep(1)
pptx.SlideShowWindow.View.Next()
time.sleep(1)
pptx.SlideShowWindow.View.Previous()
time.sleep(1)
pptx.SlideShowWindow.View.Exit()
os.system('taskkill /F /IM POWERPNT.EXE')
#app.Quit() not work
```

第10章 OpenCV影像處理與Webcam

- 10-1 OpenCV安裝與基本使用
- 10-2 OpenCV影像處理
- 10-3 OpenCV視訊處理與Webcam
- 10-4 OpenCV影像資料：NumPy陣列



10-1 OpenCV安裝與基本使用

- 10-1-1 安裝和使用OpenCV讀取和顯示圖檔
- 10-1-2 取得影像資訊和調整影像尺寸
- 10-1-3 讀取灰階影像和寫入圖檔



10-1-1 安裝和使用OpenCV讀取和顯示圖檔

- 在Python開發環境安裝OpenCV套件的命令列指令，如下所示：

  pip install opencv-python==4.5.4.60 Enter

- 當成功安裝OpenCV後（同時會安裝第10-4節的NumPy套件），就可以在Python程式使用OpenCV進行影像處理和Webcam網路攝影機。在Python程式使用OpenCV影像處理需要匯入cv2，如下所示：

  import cv2

10-1-1 安裝和使用OpenCV讀取和顯示圖檔

- Python程式：ch10-1-1.py 是使用OpenCV讀取和顯示圖檔，首先呼叫imread()方法讀取圖檔"penguins.jpg"後，再呼叫imshow()方法顯示圖檔內容的影像，如下所示：

```
import cv2

img = cv2.imread("penguins.jpg")
cv2.imshow("Penguins", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



10-1-2 取得影像資訊和調整影像尺寸

取得影像資訊：ch10-1-2.py

- 在OpenCV可以使用shape屬性取得影像尺寸和色彩數，如下所示：

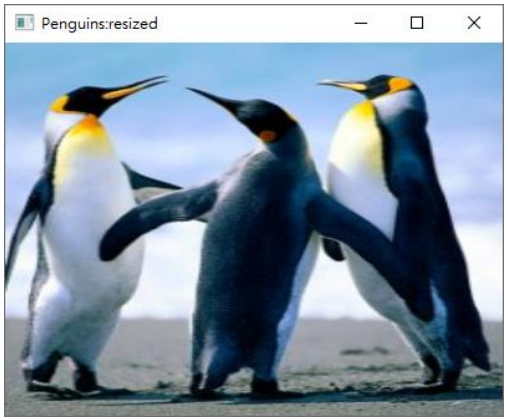
```
img = cv2.imread("penguins.jpg")
print(img.shape)
h, w, c = img.shape
print("影像高:", h)
print("影像寬:", w)
```

10-1-2 取得影像資訊和調整影像尺寸

調整影像尺寸：ch10-1-2a.py

- OpenCV是使用resize()方法來調整影像尺寸，如下所示：

```
img = cv2.imread("penguins.jpg")
print(img.shape)
resized_img = cv2.resize(img, (400, 300))
print(resized_img.shape)
cv2.imshow("Penguins:resized", resized_img)
```



10-1-3 讀取灰階影像和寫入圖檔

讀取圖檔成灰階影像：ch10-1-3.py

- 在imread()方法的第2個參數可以使用常數來指定3種讀取格式，如下表所示：

常數	說明
cv2.IMREAD_COLOR	讀取彩色影像，此為預設值。
cv2.IMREAD_GRAYSCALE	讀取灰階影像
cv2.IMREAD_UNCHANGED	影像沒有改變，包含透明度的影像內容讀取

- 例如：讀取圖檔成為灰階影像，如下所示：

```
gray_img = cv2.imread("koala.jpg", cv2.IMREAD_GRAYSCALE)
cv2.imshow("Koala:gray", gray_img)
```



10-1-3 讀取灰階影像和寫入圖檔

將影像內容寫入圖檔：ch10-1-3a.py

•OpenCV可以呼叫imwrite()方法將影像寫入圖檔，如下所示：

```
img = cv2.imread("koala.jpg")
cv2.imwrite("result.png", img)
```

10-2 OpenCV影像處理

- 10-2-1 影像幾何轉換
- 10-2-2 影像色彩空間轉換
- 10-2-3 在影像上繪圖和加上文字



10-2-1 影像幾何轉換

- 影像幾何轉換是針對影像內容的影像進行縮放、翻轉、剪裁、旋轉和位移等操作。除了使用OpenCV內建功能外，imutils套件可以讓我們更容易執行相關的影像處理。在Python開發環境安裝imutils套件的命令列指令，如下所示：

  pip install imutils==0.5.4 Enter

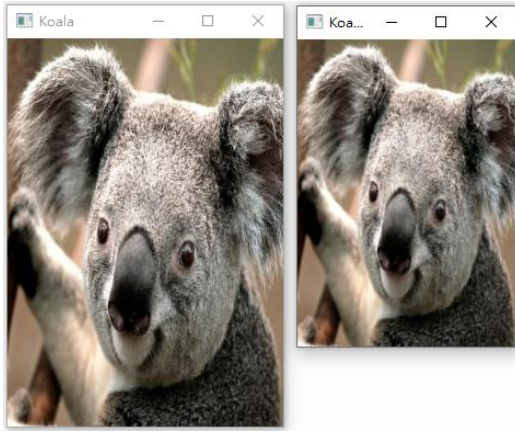
10-2-1 影像幾何轉換

調整影像尺寸：ch10-2-1.py

•在第10-1-2節是使用resize()方法調整影像尺寸，此方法會改變影像的長寬比例，如需保持比例，請改用imutils模組的方法來調整影像尺寸，首先匯入imutils，如下所示：

```
import imutils

img = cv2.imread("koala.jpg")
resized_img=imutils.resize(img,width=200)
```



10-2-1 影像幾何轉換

改變影像比例：ch10-2-1a.py

•OpenCV的resize()方法在調整影像尺寸時會改變長寬比例，我們就可以使用此方法來改變影像比例，如下所示：

```
img = cv2.imread("koala.jpg")
width = int(img.shape[1] * 0.8)
height = int(img.shape[0] * 0.5)
resized_img = cv2.resize(img, (width, height))
```



10-2-1 影像幾何轉換

剪裁影像：ch10-2-1b.py

•OpenCV讀取的影像是NumPy陣列（詳見第10-4節的說明），剪裁影像就是在切割NumPy陣列（如同第7-5-5節使用切割運算子來切割巢狀串列），如下所示：

```
img = cv2.imread("koala.jpg")
x = 10; y = 10
w = 150; h= 200
crop_img = img[y:y+h, x:x+w]
```



10-2-1 影像幾何轉換

旋轉和位移影像：ch10-2-1c.py

•OpenCV並沒有旋轉和位移影像的方法，我們可以改用imutils模組來旋轉和位移影像。首先使用imutils套件來旋轉影像，如下所示：

```
img = cv2.imread("koala.jpg")
rotated_img = imutils.rotate(img, angle=90)
```



10-2-1 影像幾何轉換

旋轉和位移影像：ch10-2-1c.py

- 位移影像是使用imutils.translate()方法，如下所示：

```
img = cv2.imread("koala.jpg")
translated_img = imutils.translate(img, 25, -75)
```

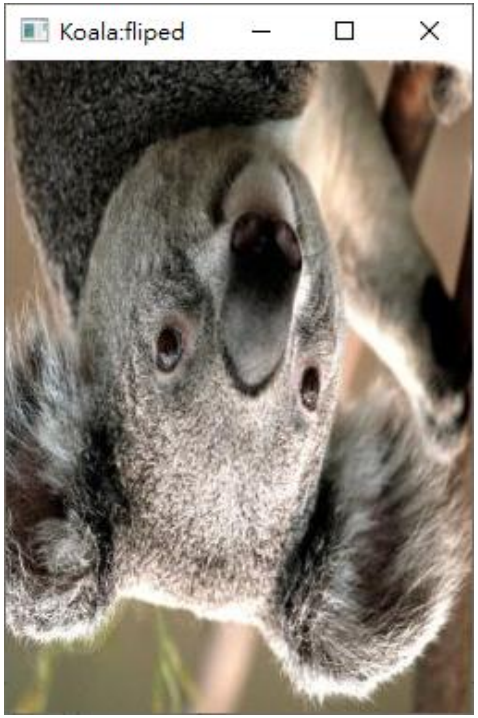


10-2-1 影像幾何轉換

翻轉影像：ch10-2-1d.py

- OpenCV可以使用flip()方法來翻轉影像，如下所示：

```
img = cv2.imread("koala.jpg")
fliped_img = cv2.flip(img, -1)
```



10-2-2 影像色彩空間轉換

- 色彩空間是數位影像描述色彩的方法，我們可以使用色彩三原色紅、綠和藍色來描述，即RGB格式，除此之外，還有BGR、HSV、HIS和HSL等多種色彩格式。

轉換成灰階色彩：ch10-2-2.py

- OpenCV可以呼叫cvtColor()方法轉換彩色成灰階影像，方法的第2個參數值是cv2.COLOR_BGR2GRAY常數，如下所示：

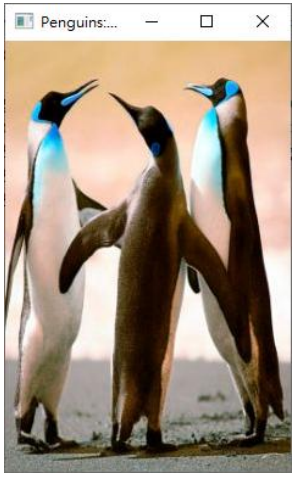
```
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

10-2-2 影像色彩空間轉換

轉換成RGB色彩：ch10-2-2a.py

•如果影像需要RGB格式，第2個參數值是cv2.COLOR_BGR2RGB常數，可以將BGR轉換成RGB（如果需要將RGB轉換成BGR格式，請使用cv2.COLOR_RGB2BGR常數），如下所示：

```
img = cv2.imread("penguins.jpg")
rgb_img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
cv2.imshow("Penguins:rgb", rgb_img)
```



10-2-3 在影像上繪圖和加上文字

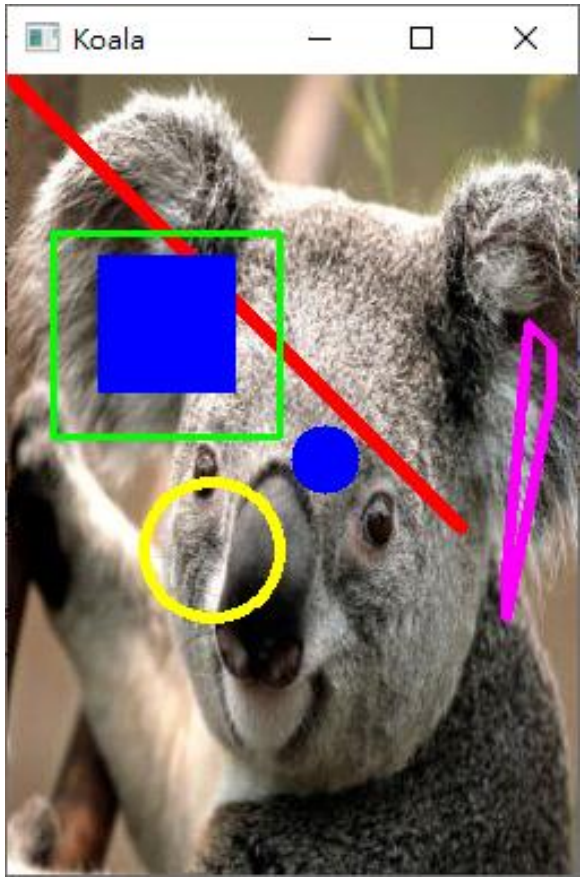
使用OpenCV在影像上繪圖：ch10-2-3.py

•Python程式讀取影像後，可以在影像上呼叫上述方法來繪出圖形，如下所示：

```
img = cv2.imread("koala.jpg")
cv2.line(img, (0,0), (200,200), (0,0,255), 5)
cv2.rectangle(img, (20,70), (120,160), (0,255,0), 2)
cv2.rectangle(img, (40,80), (100,140), (255,0,0), -1)
cv2.circle(img,(90,210), 30, (0,255,255), 3)
cv2.circle(img,(140,170), 15, (255,0,0), -1)
points = np.array([[220,220],[230,110],[240,120],
                   [240,140],[220,240]], np.int32)
cv2.polylines(img, [points], True, (255, 0, 255), 3)
cv2.imshow("Koala", img)
```

10-2-3 在影像上繪圖和加上文字

使用OpenCV在影像上繪圖：ch10-2-3.py



10-2-3 在影像上繪圖和加上文字

使用OpenCV在影像上加上文字內容：ch10-2-3a.py

•Python程式讀取影像後，可以呼叫上述方法在影像上加上文字內容，如下所示：

```
img = cv2.imread("penguins.jpg")
cv2.putText(img, 'OpenCV', (10, 40),
        cv2.FONT_HERSHEY_SIMPLEX,
        1, (0,255,255), 5, cv2.LINE_AA)
cv2.putText(img, 'Hello!', (10, 100),
        cv2.FONT_HERSHEY_SIMPLEX,
        1, (255,0,255), 5, cv2.LINE_AA)
cv2.imshow("Penguins", img)
```



10-3 OpenCV視訊處理與Webcam

- 10-3-1 OpenCV視訊處理
- 10-3-2 OpenCV網路攝影機操作

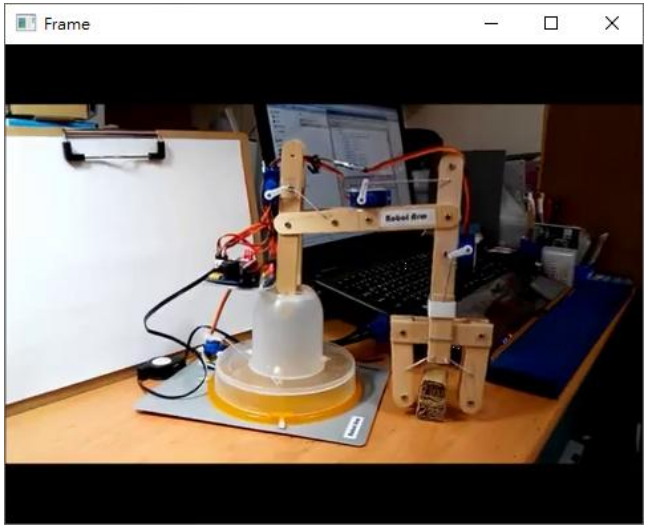


10-3-1 OpenCV視訊處理

播放視訊檔：ch10-3-1.py

•Python程式是建立OpenCV的VideoCapture物件來播放視訊檔，如下所示：

```
cap = cv2.VideoCapture("YouTube.mp4")
while(cap.isOpened()):
  ret, frame = cap.read()
  if ret:
      cv2.imshow("Frame", frame)
  if cv2.waitKey(1) & 0xFF == ord("q"):
      break
cap.release()
cv2.destroyAllWindows()
```



10-3-1 OpenCV視訊處理

取得視訊的相關資訊：ch10-3-1a.py

- 在Python程式建立VideoCapture物件後，可以取得視訊相關資訊的尺寸和編碼，如下所示：

```
cap = cv2.VideoCapture("YouTube.mp4")
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
print("影格尺寸:", width, "x", height)
fourcc = int(cap.get(cv2.CAP_PROP_FOURCC))
codec = (chr(fourcc&0xFF)+chr((fourcc>>8)&0xFF)+

chr((fourcc>>16)&0xFF)+chr((fourcc>>24)&0xFF))
print("Codec編碼:", codec)
```

10-3-1 OpenCV視訊處理

計算視訊的總影格數：ch10-3-1b.py

- Python程式只需使用計數變數frame_count，就可以計算出整個視訊檔的總影格數，如下所示：

```
cap = cv2.VideoCapture("YouTube.mp4")
frame_count = 0
while True:
  ret, frame = cap.read()
  if not ret:
      break
  frame_count = frame_count + 1
print("總影格數 =", frame_count)
```

10-3-1 OpenCV視訊處理

取得視訊的影格率（FPS）：ch10-3-1c.py

•VideoCapture物件的get()方法，可以使用cv2.CAP_PROP_FPS參數值取得視訊的影格率（FPS），如下所示：

```
fps = cap.get(cv2.CAP_PROP_FPS)
print("FPS =", fps)
```

10-3-2 OpenCV網路攝影機操作

取得網路攝影機的影像：ch10-3-2.py

- 在OpenCV的VideoCapture物件除了開啟視訊檔案，也可以開啟攝影機，參數0或-1是第一台攝影機，1是第2台...，以此類推，如下所示：

```
cap = cv2.VideoCapture(0)
while(cap.isOpened()):
  ret, frame = cap.read()
  cv2.imshow("Frame", frame)
  if cv2.waitKey(1) & 0xFF == ord("q"):
      break
cap.release()
cv2.destroyAllWindows()
```



10-3-2 OpenCV網路攝影機操作

更改視訊的解析度：ch10-3-2a.py

- 在Python程式建立VideoCapture物件後，可以呼叫set()方法更改影格的寬、高和影格率，如下所示：

```
cap = cv2.VideoCapture(0)
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 180)
cap.set(cv2.CAP_PROP_FPS, 25)
while(cap.isOpened()):
  ret, frame = cap.read()
  cv2.imshow("Frame", frame)
  if cv2.waitKey(1) & 0xFF == ord("q"):
      break
...
```

10-3-2 OpenCV網路攝影機操作

將影格寫入視訊檔案：ch10-3-2b.py

•OpenCV是建立VideoWrite物件寫入視訊檔案，如下：

```
cap = cv2.VideoCapture(0)
fourcc = cv2.VideoWriter_fourcc(*"XVID")
out = cv2.VideoWriter("output.avi", fourcc, 20, (640,480))
while(cap.isOpened()):
  ret, frame = cap.read()
  if ret == True:
    out.write(frame)
    cv2.imshow("Frame", frame)
    if cv2.waitKey(1) & 0xFF == ord("q"):
      break
  else:
    break
```

10-4 OpenCV影像資料：NumPy陣列

- 10-4-1 建立陣列
- 10-4-2 陣列屬性
- 10-4-3 陣列內容與形狀操作



10-4-1 建立陣列

- NumPy陣列（Arrays）類似Python串列（Lists），不過陣列元素的資料型態必須是相同的。NumPy套件的核心是ndarray物件，這是一序列的整數int或浮點數float值，每一個值的陣列元素都是相同的資料型態。在Python程式需要匯入NumPy套件的別名np，如下所示：

```
import numpy as np
```

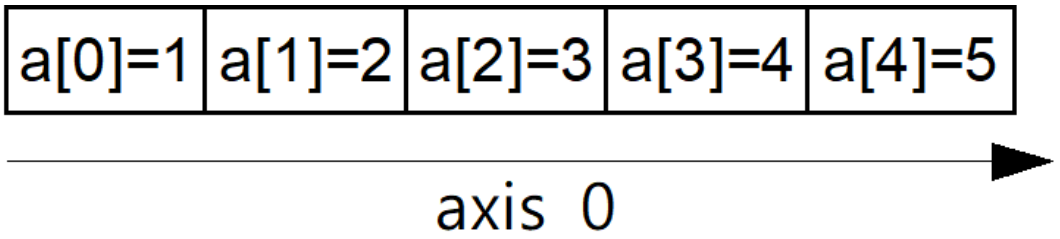
10-4-1 建立陣列

使用串列與元組建立一維陣列：ch10-4-1.py

•在匯入NumPy套件後，使用array()方法建立NumPy陣列，如下所示：

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
print(type(a))
print(a[0], a[1], a[2], a[3], a[4])
```



10-4-1 建立陣列

更改一維陣列的元素值：ch10-4-1a.py

- NumPy陣列可以使用索引來更改陣列元素值，如下所示：

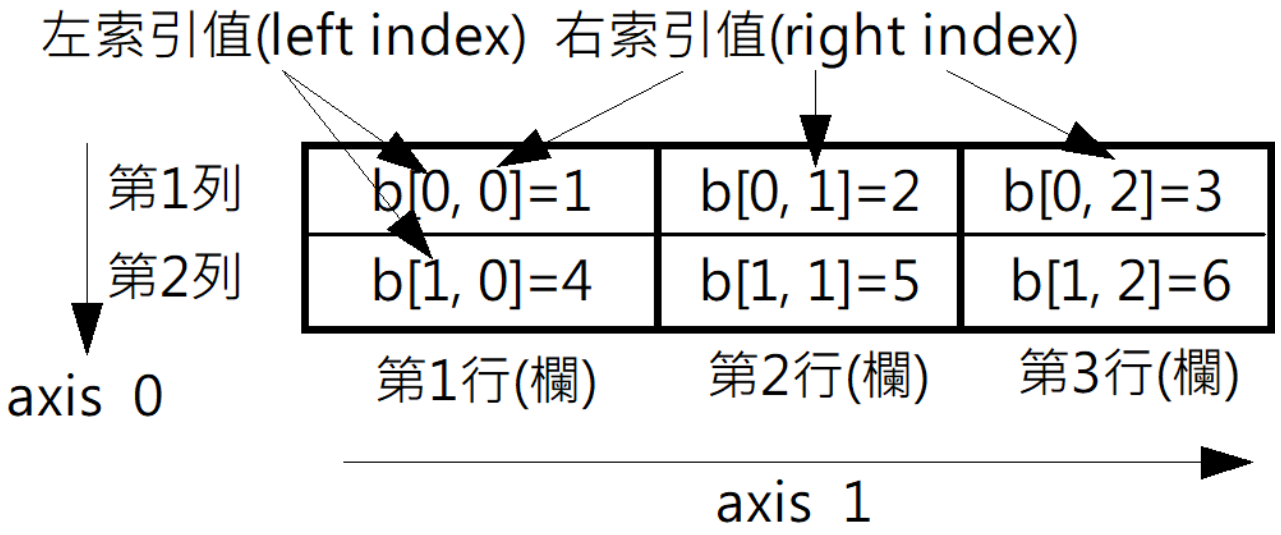
```
a = np.array([1, 2, 3, 4, 5])
a[0] = 5
print(a)
a[3] = 0
print(a)
```

10-4-1 建立陣列

使用巢狀串列建立二維陣列：ch10-4-1b.py

- NumPy二維陣列是使用巢狀串列來建立，如下所示：

```
b = np.array([[1,2,3],[4,5,6]])
print(b[0, 0], b[0, 1], b[0, 2])
print(b[1, 0], b[1, 1], b[1, 2])
```



10-4-1 建立陣列

更改二維陣列的元素值：ch10-4-1c.py

•NumPy二維陣列需要使用2個索引值來存取二維陣列的元素值，例如：更改左上角和右下角的2個元素值，如下所示：

```
b = np.array([[1,2,3],[4,5,6]])
b[0, 0] = 6
b[1, 2] = 1
print(b)
```

10-4-1 建立陣列

建立指定元素型態的陣列：ch10-4-1d.py

- Python程式在建立NumPy陣列時，可以指定陣列元素是哪一種資料型態，如下所示：

```
a = np.array([1, 2, 3, 4, 5], int)
print(a)
b = np.array((1, 2, 3, 4, 5), dtype=float)
print(b)
```

10-4-2 陣列屬性

- NumPy陣列是物件，提供相關屬性來顯示陣列資訊。相關屬性的說明如下表所示：

屬性	說明
dtype	陣列元素的資料型態，整數int32/64或浮點數float32/64等
size	陣列的元素總數
shape	N X M陣列的形狀（Shape）
itemsize	陣列元素佔用的位元組數
ndim	幾維陣列，一維是1；二維是2
nbytes	整個陣列佔用的位元組數

10-4-3 陣列內容與形狀操作

將一維陣列轉換成二維陣列：ch10-4-3.py

- Python程式可以使用reshape()方法將一維陣列轉換成3 X 2的二維陣列，如下所示：

```
a = np.array([1,2,3,4,5,6])
print(a)
b = a.reshape((3, 2))
print(b)
```

10-4-3 陣列內容與形狀操作

切割出部分陣列元素：ch10-4-3a.py

- NumPy陣列一樣可以使用第7-5-5節的切割運算子來切割出部分陣列元素，如下所示：

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
print(a)
b, c, d = a[1:3], a[:4], a[3:]
print(b, c, d)
```

10-4-3 陣列內容與形狀操作

平坦化陣列：ch10-4-3b.py

- 平坦化陣列可以將二維陣列平坦化成一維陣列，在NumPy是使用flatten()方法來執行平坦化陣列，如下所示：

```
a = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
b = a.reshape((3, 3))
print(b)
c = b.flatten()
print(c)
```

10-4-3 陣列內容與形狀操作

合併多個二維陣列：ch10-4-3c.py

- NumPy的np.concatenate()方法可以合併多個二維陣列，參數axis軸指定合併方向，值0是直向（預設值），可以合併在目前二維陣列的下方；值1是橫向，陣列是合併在目前陣列的右方，如下所示：

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])
c = np.concatenate((a, b), axis=0)
print(c)
d = np.concatenate((a, b), axis=1)
print(d)
```

10-4-3 陣列內容與形狀操作

擴充與刪除陣列的維度：ch10-4-3d.py

•因為機器學習的輸入資料和回傳結果都是NumPy多維陣列，有時我們需要擴充陣列維度來符合輸入資料的形狀，或刪除陣列維度來方便存取資料。在NumPy擴充維度是使用np.expand_dims()方法，如下所示：

```
a = np.array([[1,2,3,4,5,6,7,8]])
b = a.reshape(2, 4)
print(b.shape)
c = np.expand_dims(b, axis=0)
d = np.expand_dims(b, axis=1)
print(c.shape, d.shape)
```

10-4-3 陣列內容與形狀操作

擴充與刪除陣列的維度：ch10-4-3d.py

- 刪除陣列維度是使用np.squeeze()方法，可以刪除陣列shape屬性值是1的維度，如下所示：

```
e = np.squeeze(c)
f = np.squeeze(d)
print(e.shape, f.shape)
```

10-4-3 陣列內容與形狀操作

取得陣列最大/最小值和索引：ch10-4-3e.py

•NumPy陣列可以使用np.max()方法取得陣列最大元素值；np.min()方法取得最小值，如果欲取得陣列哪一個索引值是最大值或最小值，請使用np.argmax()和np.argmin()方法，如下所示：

```
a = np.array([[11,22,13,74,35,6,27,18]])
min_value = np.min(a)
max_value = np.max(a)
print(min_value, max_value)
min_idx = np.argmin(a)
max_idx = np.argmax(a)
print(min_idx, max_idx)
```

第11章 認識人工智慧、機器學習與深度學習

- 11-1 人工智慧概論
- 11-2 認識機器學習
- 11-3 什麼是深度學習



11-1 人工智慧概論

- 11-1-1 人工智慧簡介
- 11-1-2 人工智慧的應用領域



11-1-1 人工智慧簡介

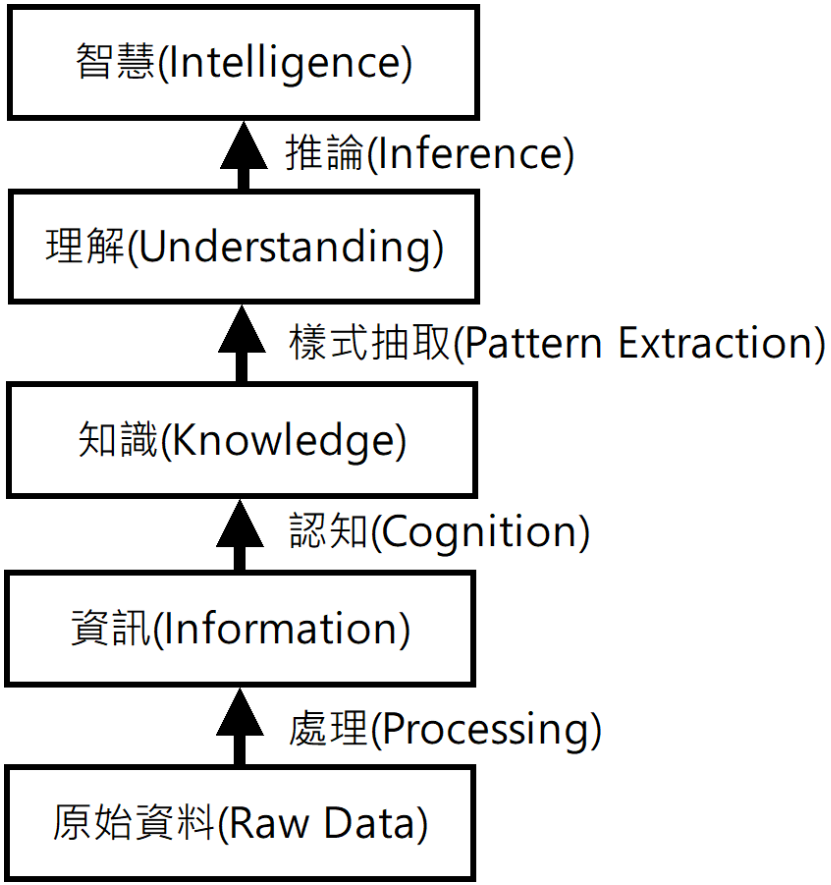
認識人工智慧

- 人工智慧（Artificial Intelligence，AI）也稱為人工智能，這是讓機器變得更聰明的一種科技，也就是讓機器具備和人類一樣的思考邏輯與行為模式。簡單的說，人工智慧就是讓機器展現出人類的智慧，像人類一樣的思考，基本上，人工智慧是一個讓電腦執行人類工作的廣義名詞術語，其衍生的應用和變化至今仍然沒有定論。
- 人工智慧屬於計算機科學領域的範疇，其發展過程包括學習（大量讀取資訊和判斷何時與如何使用該資訊）、感知、推理（使用已知資訊來做出結論）、自我校正和操縱或移動物品等。

11-1-1 人工智慧簡介

從原始資料轉換成智慧的過程

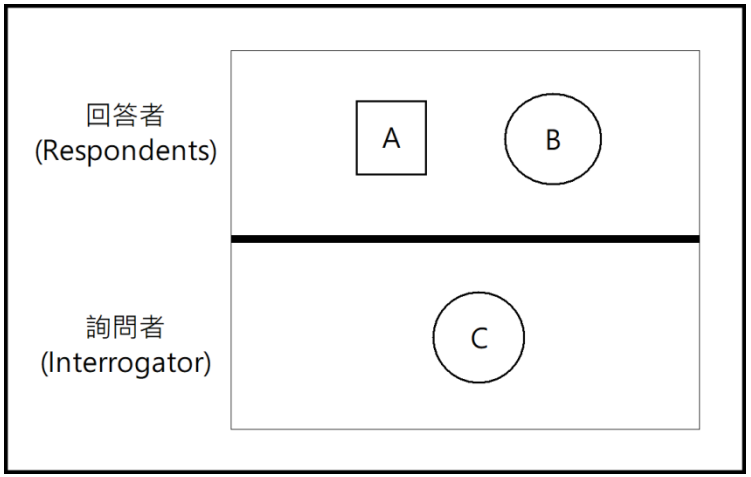
- 人工智慧是在研究如何從原始資料轉換成智慧的過程，這是需要經過多個不同層次的處理步驟，如下圖所示：



11-1-1 人工智慧簡介

圖靈測試

- 圖靈測試（Turing Test）是計算機科學和人工智慧之父-艾倫圖靈（Alan Turing）在1950年提出，一個定義機器是否擁有智慧的測試，能夠判斷機器是否能夠思考的著名試驗。
- 圖靈測試提出了人工智慧的概念，讓我們相信機器是有可能具備智慧的能力，簡單的說，圖靈測試是在測試機器是否能夠表現出與人類相同或無法區分的智慧表現，如下圖所示：



11-1-1 人工智慧簡介
圖靈測試

- 上述正方形A代表一台機器，圓形B代表人類，這兩位是回答者（Respondents），人類C是一位詢問者（Interrogator），展開與A和B的對話，對話是透過文字模式的鍵盤輸入和螢幕輸出來進行，如果A不會被辨別出是一台機器的身份，就表示這台機器A具有智慧。

11-1-2 人工智慧的應用領域

- 手寫辨識（Handwriting Recognition）：這是大家常常使用的人工智慧應用領域，想想看智慧型手機或平板電腦的手寫輸入法，這就是手寫辨識，系統可以辨識寫在紙上、或觸控螢幕上的筆跡，依據外形和筆劃等特徵來轉換成可編輯的文字內容。
- 語音識別（Speech Recognition）：這是能夠聽懂和了解語音說話內容的系統，還能分辨出人類口語的不同音調、口音、背景雜訊或感冒鼻音等，例如：Apple公司智慧語音助理系統Siri等。
- 電腦視覺（Computer Vision）：處理多媒體影像或視訊的人工智慧系統，能夠依需求抽取特徵來了解這些影像或視訊的內容是什麼，例如：Google搜尋相似影像、人臉識別的犯罪預防或公司門禁管理等。

11-1-2 人工智慧的應用領域

- 專家系統（Expert Systems）：使用人工智慧技術提供建議和做決策的系統，通常是使用資料庫儲存大量財務、行銷、醫療等不同領域的專業知識，以便依據這些資料來提供專業的建議。
- 自然語言處理（Natural Language Processing）：能夠了解自然語言（即人類語言）的文字內容，我們可以輸入自然語言的句子和系統直接對談，例如：Google搜尋引擎。
- 電腦遊戲（Game）：人工智慧早已應用在電腦遊戲，只需是擁有電腦代理人（Agents）的各種棋類遊戲，都屬於人工智慧的應用，最著名的當然是AlphaGo人工智慧圍棋程式。

11-1-2 人工智慧的應用領域

- 智慧機器人（Intelligent Robotics）：機器人基本上涉及多種領域的人工智慧，才足以完成不同任務，這是依賴安裝在機器人上的多種感測器來偵測外部環境，可以讓機器人模擬人類的行為或表情等。

11-2 認識機器學習

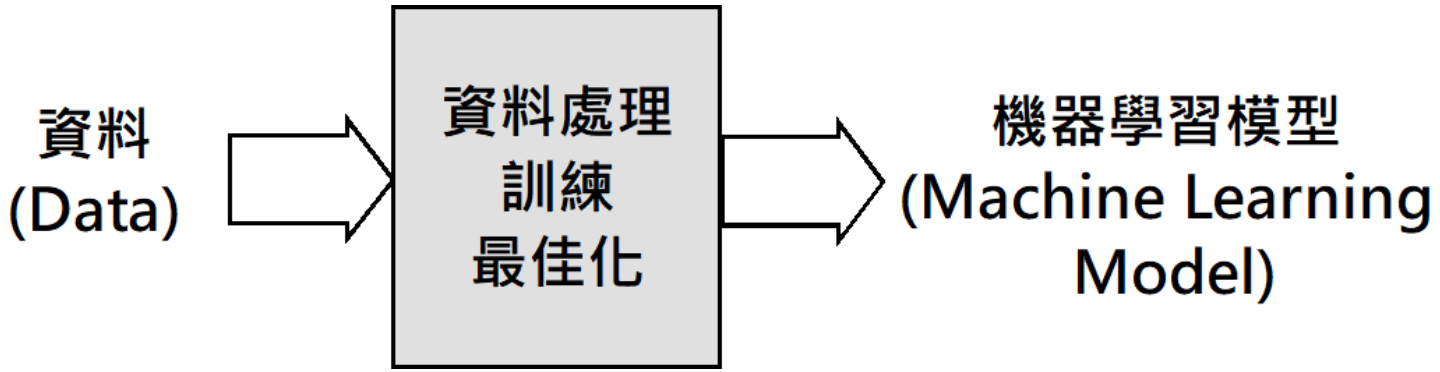
- 11-2-1 機器學習簡介
- 11-2-2 機器學習可以解決的問題
- 11-2-3 機器學習的種類



11-2-1 機器學習簡介

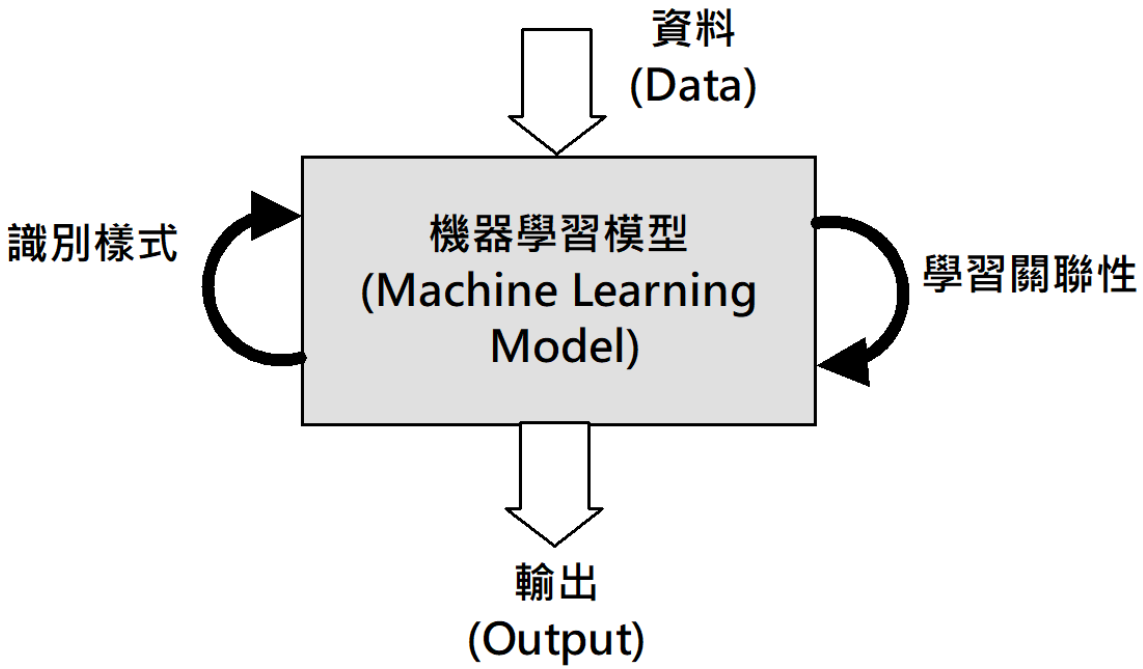
什麼是機器學習

- 機器學習的定義是：「從過往資料和經驗中自我學習並找出其運行的規則，以達到人工智慧的方法。」，機器學習就是目前人工智慧發展的核心研究領域。
- 機器學習是一種人工智慧，可以讓電腦使用現有資料來進行訓練和學習，以便建立預測模型，當成功建立模型後，就可以使用此模型來預測未來的行為、結果和趨勢，如下圖所示：



11-2-1 機器學習簡介
從資料中自我訓練學習

- 機器學習主要目的是預測資料，其厲害之處在於可以自主學習，和找出資料之間的關係和規則，如下圖所示：



11-2-1 機器學習簡介
從資料中自我訓練學習

- 當資料送入機器學習模型後，就會自行找出資料之間的關聯性（Relationships）和識別出樣式，其輸出結果是已經學會的模型。機器學習主要是透過下列方式來進行訓練，如下所示：
  - 需要大量資料訓練模型。
  - 從資料中自行學習來找出關聯性和識別出樣式（Pattern）。
  - 根據自行學習和識別出樣式獲得的經驗，可以幫助我們將未來的新資料做分類，並且推測其行為、結果和趨勢。

11-2-2 機器學習可以解決的問題

分類

- 分類演算法是用來解決只有二種或多種結果的問題。二元分類（Two-class Classification）演算法是區分成A或B類、是或否、開或關、抽煙或不抽煙等二種結果。一些常見範例，如下所示：
  - 客戶是否會續約？
  - 圖片是貓，還是狗？
  - 回饋10元或打75折，哪一種促銷方法更能提昇業積？
- 多元分類（Multi-class Classification）是二元分類的擴充，可以用來解決有多種結果的問題，例如：哪種口味、哪間公司或哪一位參選人等。一些常見範例，如下所示：
  - 哪種動物的圖片？哪種植物的圖片？
  - 雷達訊號來自哪一種飛機？
  - 錄音裡的說話者是誰？

11-2-2 機器學習可以解決的問題
異常值判斷

- 異常值判斷演算法是用來偵測異常情況（Anomaly Detection），簡單的說，就是辨認出不正常的資料，或找出奇怪的地方。基本上，異常值判斷和二元分類看起來好像十分相似，不過，二元分類一定有兩種結果，異常值判斷就不一定，可以只有一種結果。一些常見範例，如下所示：
  - 偵測信用卡盜刷？
  - 網路訊息是否正常？
  - 這些消費和之前消費行為是否落差很大？
  - 管路壓力大小是否有異常？

11-2-2 機器學習可以解決的問題
預測性分析

- 預測性分析演算法解決的問題是數值而非分類，也就是預測量有多少？需要多少錢？未來是漲價；還是跌價等，此類演算法稱為迴歸（Regression）。一些常見範例，如下所示：
  - 下星期四的氣溫是幾度？
  - 在台北市第二季的銷售量有多少？
  - 下周Facebook臉書會新增幾位追蹤者？
  - 下周日可以賣出多少個產品？

11-2-2 機器學習可以解決的問題
分群

- 分群演算法是在解決資料是如何組成的問題，屬於第11-2-3節的非監督式學習，其基本作法是測量資料之間的距離或相似度，即距離度量（Distance Metric），例如：智商的差距、相同基因組的數量、兩點之間的最短距離，然後據此來分成均等的群組。一些常見範例，如下所示：
  - 哪些消費者對水果有相似的喜好？
  - 哪些觀眾喜歡同一類型的電影？
  - 哪些型號的手機有相似的故障？
  - 部落格訪客可以分成哪些不同類別的群組？

11-2-2 機器學習可以解決的問題
協助決策

- 協助決策演算法是在決定下一步是什麼？其基本原理是源於大腦對懲罰和獎勵的反應機制，可以決定獎勵最高的下一步，和避開懲罰的選擇。一些常見範例，如下所示：
  - 網頁廣告置於哪一個位置，才能讓訪客最容易點選？
  - 看到黃燈時，應該保持目前速度、煞車還時加速通過？
  - 溫度是調高、調低，還是不動？
  - 下圍棋時決定下一步棋的落子位置？

11-2-3 機器學習的種類

監督式學習

- 監督式學習（Supervised Learning）是一種機器學習方法，可以從訓練資料（Training Data）建立學習模型（Learning Model），並且依據此模型來推測新資料是什麼？基本上，監督式學習的訓練過程中，需要告訴機器答案，稱為「有標籤資料」（Labeled Data），因為仍然需要老師提供答案，所以稱為監督式學習。
- 例如：垃圾郵件過濾的機器學習，在輸入1000封電子郵件且告知每一封是垃圾郵件（Y）；或不是（N）後，即可從這些訓練資料建立出學習模型，然後模型即可判斷出一封新郵件是否是垃圾郵件。

11-2-3 機器學習的種類
非監督式學習

- 非監督式學習（Unsupervised Learning）和監督式學習的最大差異是訓練資料不需有答案，即標籤，所以，機器是在沒有老師告知答案的情況下進行學習。簡單來說，如果訓練資料有標籤，需要老師提供答案，就是監督式學習；訓練資料沒標籤，不需要老師，機器能夠自行摸索出資料的規則和關聯性，稱為非監督式學習。

11-2-3 機器學習的種類
半監督式學習

- 半監督式學習（Semisupervised Learning）是介於監督式學習與非監督式學習之間的一種機器學習方法，此方法使用的訓練資料大部分是沒有標籤的資料，只有少量資料有標籤。
- 例如：當上傳家庭全部成員的照片至Google相簿後，Google相簿就會學習分辨出相片1、2、5、11擁有成員A，相片4、5、11有成員B，相片3、6、9有成員C等，等到輸入成員A的姓名後（有標籤資料），Google相簿馬上在有此成員的照片上標示姓名，這就是一種半監督式學習。

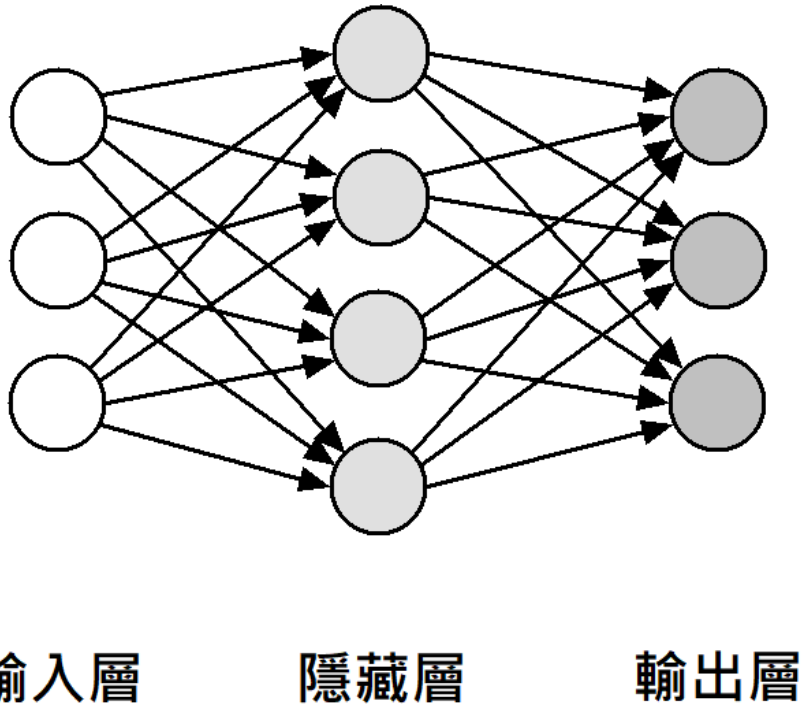
11-3 什麼是深度學習

- 深度學習（Deep Learning）是機器學習的分支，其使用的演算法是模仿人類大腦功能的類神經網路（Artificial Neural Networks，ANNs），或稱為人工神經網路。
- 深度學習（Deep Learning）的定義很簡單：「一種實現機器學習的技術。」所以，深度學習就是一種機器學習。記得長輩常常說過的一句話：「我吃過的鹽比你吃過的米還多」，這句話的意思是指老人家的經驗比你豐富，因為經驗豐富，看的東西多，他的直覺比你準確，並不表示長輩真的比你聰明，或更加有學問。
- 深度學習是在訓練機器的直覺，例如：訓練深度學習辨識一張狗照片，我們是訓練機器知道這張照片是狗，並不是訓練機器學習到狗有4隻腳、會叫或是一種哺乳類動物等關於狗的相關知識。

11-3 什麼是深度學習

深度學習是一種神經網路

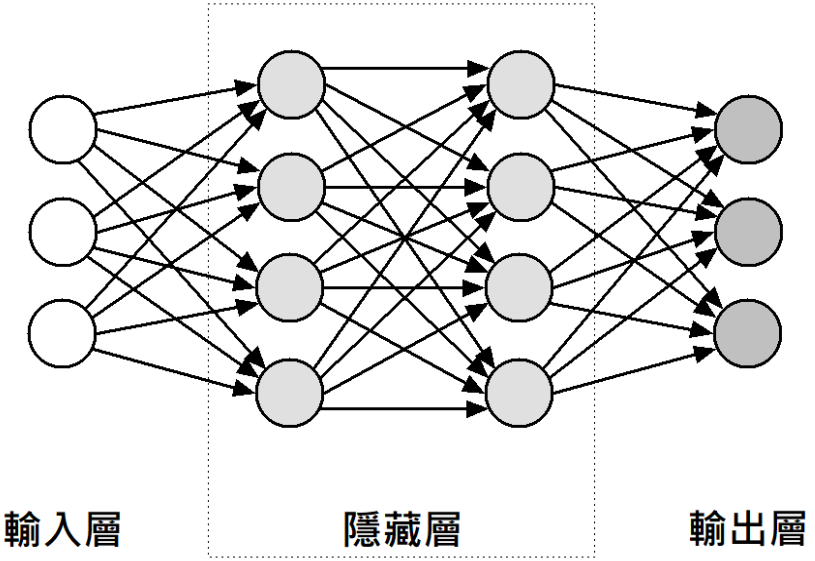
- 深度學習就是模仿人類大腦神經元（Neuron）傳輸的一種神經網路架構（Neural Network Architectures），如下圖所示：



11-3 什麼是深度學習

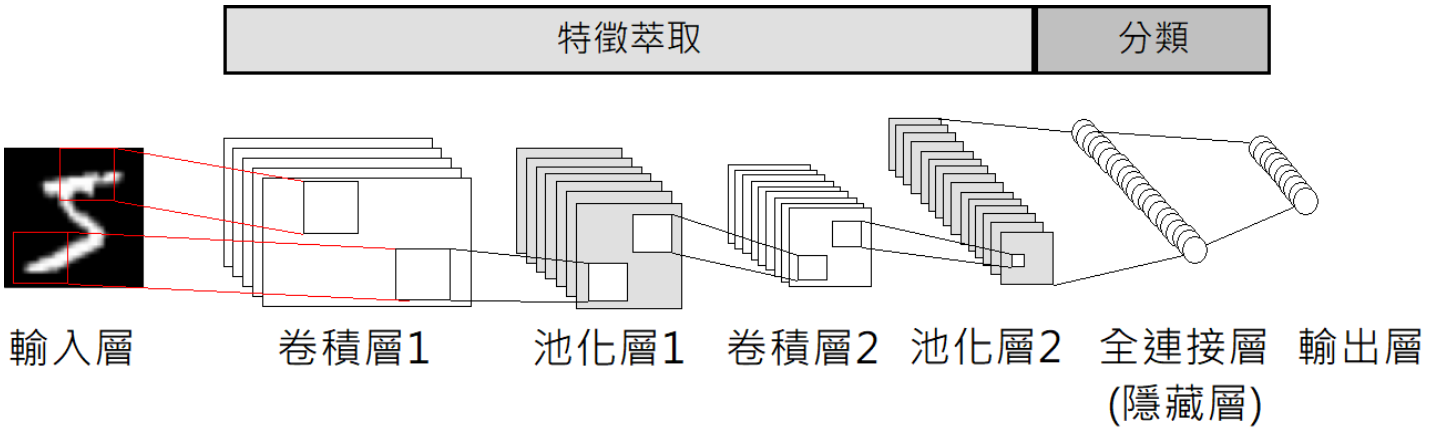
深度學習是一種神經網路

- 深度學習使用的神經網路稱為深度神經網路（Deep Neural Networks，DNNs），其中間的隱藏層有很多層，意味著整個神經網路十分的深（Deep），可能高達150層隱藏層。基本上，神經網路只需擁有2層隱藏層，加上輸入層和輸出層共四層之上，就可以稱為深度神經網路，即所謂的深度學習，如下圖所示：



11-3 什麼是深度學習
卷積神經網路CNN

- 卷積神經網路（Convolutional Neural Network，CNN）簡稱CNNs或ConvNets，其基礎是1998年Yann LeCun提出名為LeNet-5的卷積神經網路架構，模仿人腦視覺處理區域的神經迴路，針對影像處理的神經網路，例如：影像分類、人臉/手寫辨識等。
- 卷積神經網路的基本結構是卷積層（Convolution Layers）和池化層（Pooling Layers），使用多種不同的神經層來依序連接成神經網路，如下圖所示：



11-3 什麼是深度學習

深度學習能作什麼

- 深度學習可以處理所有感知問題（Perceptual Problems），例如：聽覺和視覺問題，這些技能對於人類來說，只不過是一些直覺和與生俱來的能力，但是這些看似簡單的技能，早已困擾傳統機器學習多年且無法解決。
- 事實上，深度學習已經成功解決傳統機器學習的一些困難領域，如下所示：
  - 模仿人類的影像分類、物體識別、語音識別、手寫辨識和自動駕駛等。
  - 大幅改進機器翻譯和文字轉語音的正確率。
  - 大幅改進數位助理、搜尋引擎和網頁廣告投放效果。
  - 自然語言對話的問答系統，例如：聊天機器人。
  - 持續增加中...

第12章 人工智慧應用（一）：人臉偵測與人臉識別

- 12-1 MediaPipe框架與CVZone電腦視覺套件
- 12-2 使用CVZone人臉偵測
- 12-3 使用CVZone臉部網格
- 12-4 CVZone辨識臉部表情：張嘴/閉嘴與開眼/閉眼
- 12-5 face-recognition人臉識別



12-1 MediaPipe框架與CVZone電腦視覺套件

- MediaPipe是 Google公司開發的機器學習框架，這是跨平台的機器學習解決方案，可以讓AI研究者和開發者建立世界等級，針對手機、PC、雲端、Web和IoT裝置的機器學習應用程式和解決方案。
- MediaPipe跨平台支援Android、iOS、Web和邊緣運算裝置，C++、JavaScript和Python程式語言，隨著平台釋出的相關應用範例，可以讓我們馬上執行人工智慧應用，包含：人臉偵測、多手勢追蹤和人體姿態評估等。
- 在Python開發環境安裝MediaPipe套件的命令列指令，如下所示：
  - pip install mediapipe==0.8.9.1 Enter

12-2 使用CVZone人臉偵測

- 12-2-1 CVZone人臉偵測的基本使用
- 12-2-2 顯示臉部關鍵點和剪裁出人臉
- 12-2-3 CVZone即時人臉偵測



12-2 使用CVZone人臉偵測

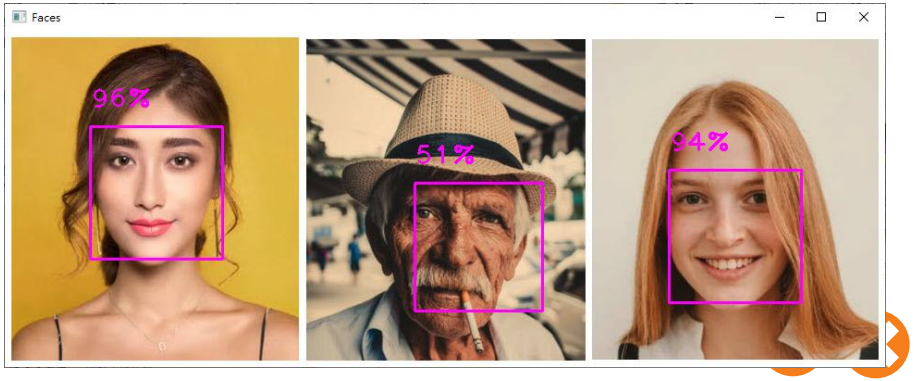
- CVZone人臉偵測是基於MediaPipe人臉偵測（Face Detection），這是使用Blazeface模型的一種超快速人臉偵測，可以在影像中偵測出多張人臉，和標示臉部的6個關鍵點（Key Points）：左眼、右眼、鼻尖、嘴巴、左耳和右耳。

12-2-1 CVZone人臉偵測的基本使用

影像的人臉偵測：ch12-2-1.py

```
from cvzone.FaceDetectionModule import FaceDetector
import cv2

img = cv2.imread("images/faces.jpg")
detector = FaceDetector(minDetectionCon=0.5)
img, faces = detector.findFaces(img)
if faces:
    print("偵測到人臉數:", len(faces))
cv2.imshow("Faces", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

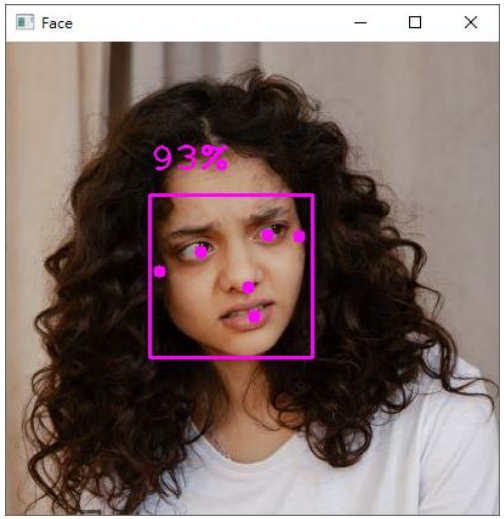


12-2-1 CVZone人臉偵測的基本使用

取得人臉的相關資訊：ch12-2-1b.py

```
img = cv2.imread("images/faces.jpg")
detector = FaceDetector()
img, faces = detector.findFaces(img)
if faces:
    print("偵測到人臉數:", len(faces))
    face = faces[0]
    print("id:", face["id"])
    print("bbox:", face["bbox"])
    print("score:", face["score"])
    print("center:", face["center"])
```

12-2-2 顯示臉部關鍵點和剪裁出人臉



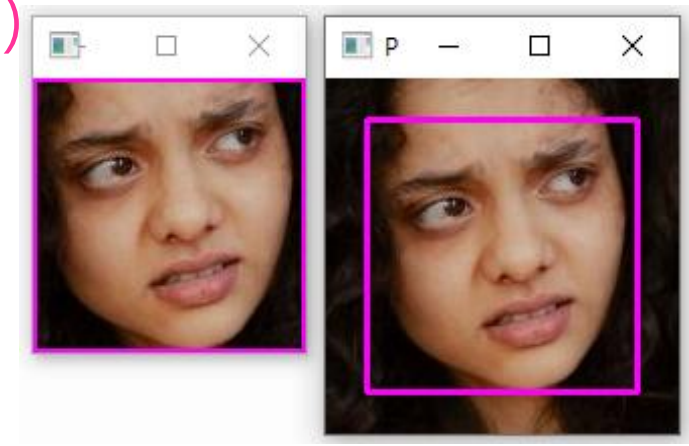
顯示臉部6個關鍵點：ch12-2-2.py

```
img = cv2.imread("images/face.jpg")
detector = FaceDetector()
img, faces = detector.findFaces(img)
if faces:
    keypoints = detector.getFaceKeypoints(img, face_idx=0)
    print(keypoints)
    for keypoint in keypoints:
        cv2.circle(img, keypoint["keypoint"],5,(255,0,255), cv2.FILLED)
```

12-2-2 顯示臉部關鍵點和剪裁出人臉

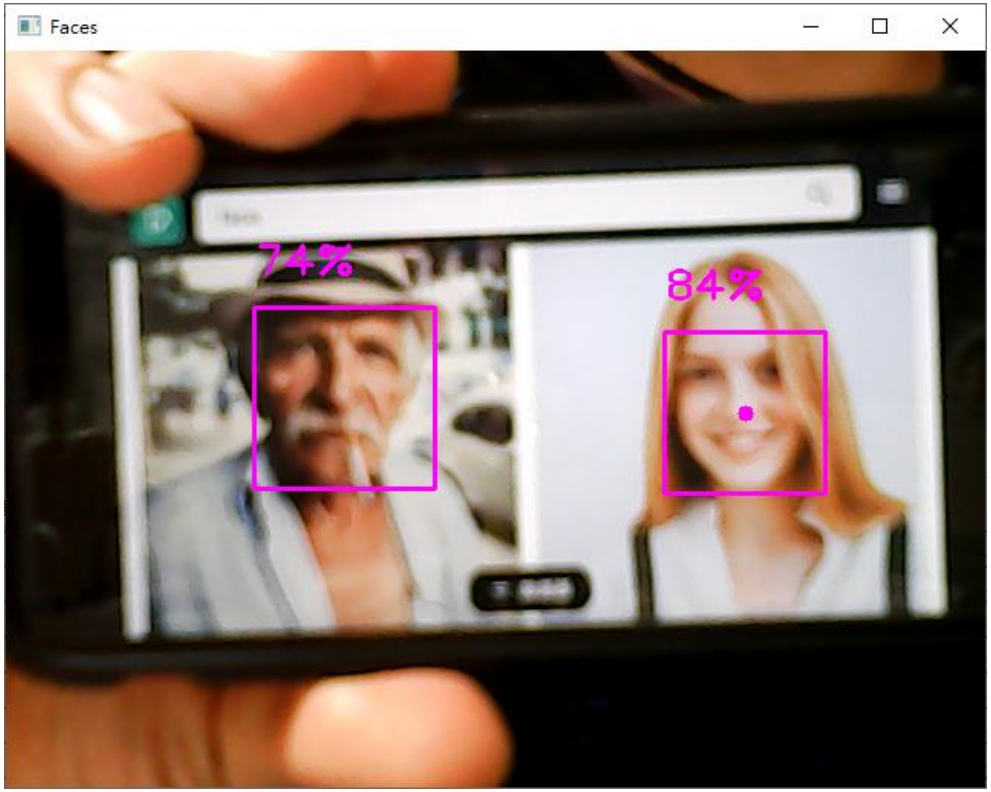
剪裁出臉部的部分影像：ch12-2-2a.py

```
img, faces = detector.findFaces(img)
if faces:
    for face in faces:
        x, y, w, h = face["bbox"]
        face = img[y:y+h, x:x+w]
        cv2.imshow("Non Padded", face)
        padding = 20
        padded_face = img[y-padding:y+h+padding,x-
padding:x+w+padding]
        cv2.imshow("Padded", padded_face)
        cv2.waitKey(0)
```



12-2-3 CVZone即時人臉偵測

- Python程式：ch12-2-3.py整合第10-3-2節的Webcam，可以建立CVZone即時人臉偵測，如下圖所示：



12-3 使用CVZone臉部網格

- 12-3-1 CVZone臉部網格的基本使用
- 12-3-2 取出臉部特徵資訊
- 12-3-3 CVZone即時顯示臉部網格



12-3 使用CVZone臉部網格

- CVZone臉部網格是基於MediaPipe臉部網格（MediaPipe Face Mesh），這是使用Blazeface模型預測出468個關鍵點，和使用網格方式來繪出3D臉部模型。

12-3-1 CVZone臉部網格的基本使用

人臉偵測和繪出3D臉部網格：ch12-3-1.py

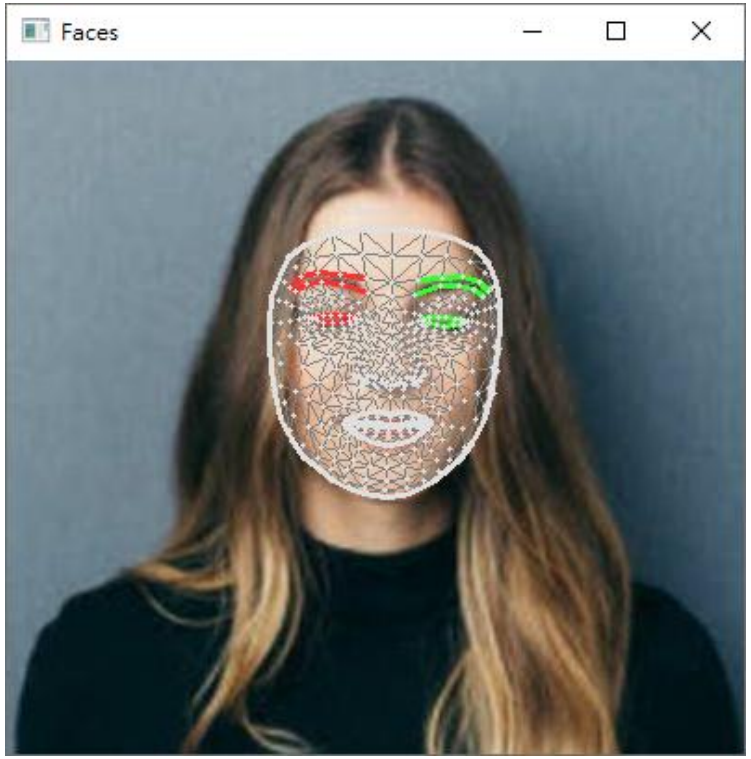
```
from cvzone.FaceMeshModule import
FaceMeshDetector
import cv2


img = cv2.imread("images/face4.jpg")
detector = FaceMeshDetector(maxFaces=2,
minDetectionCon=0.5,
                            minTrackCon=0.5)
img, faces = detector.findFaceMesh(img)
```

12-3-1 CVZone臉部網格的基本使用

人臉偵測和繪出3D臉部網格：ch12-3-1.py

```
if faces:
   print(len(faces[0]))
cv2.imshow("Faces", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



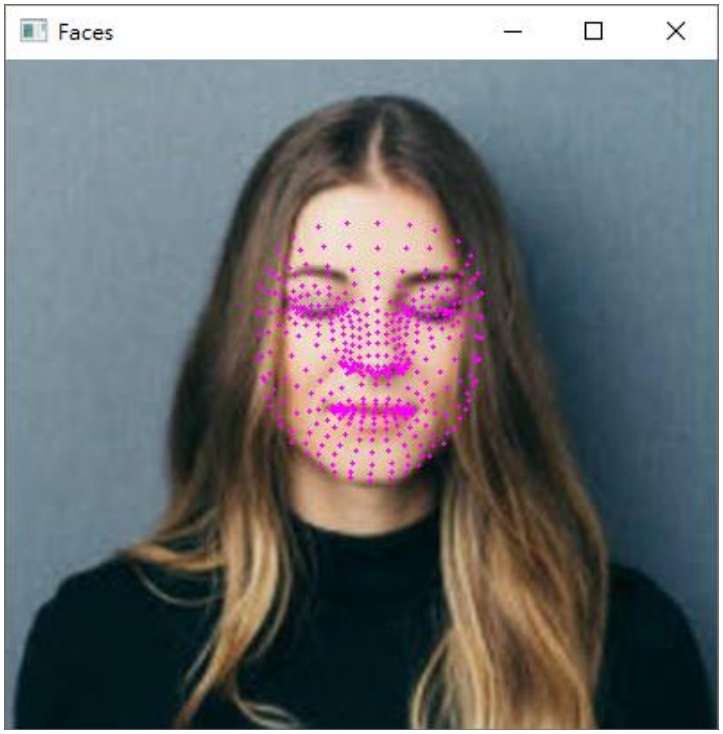
12-3-1 CVZone臉部網格的基本使用

取得臉部網格的點座標：ch12-3-1a.py

```
img = cv2.imread("images/face5.jpg")
detector = FaceMeshDetector(maxFaces=2)
img, faces = detector.findFaceMesh(img, draw=False)
if faces:
   print(len(faces[0]))
   for point in faces[0]:
      cv2.circle(img, point, 1, (255, 0, 255), cv2.FILLED)
```

12-3-1 CVZone臉部網格的基本使用

取得臉部網格的點座標：ch12-3-1a.py



12-3-2 取出臉部特徵資訊

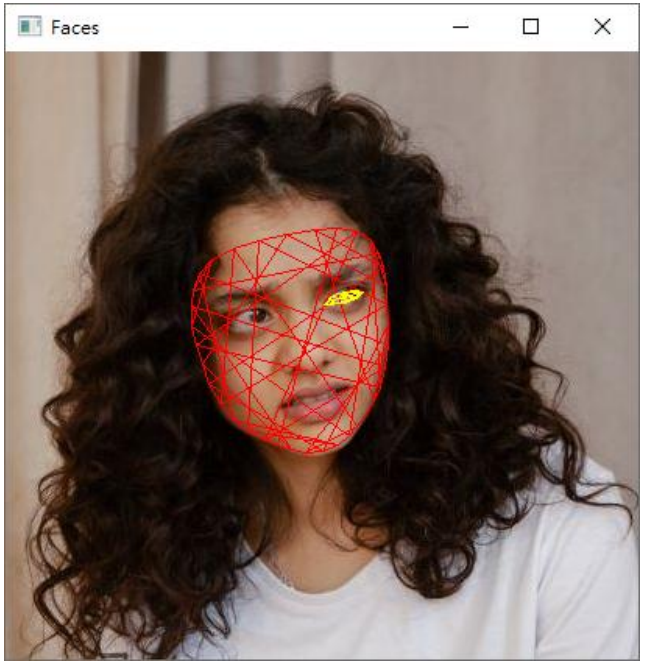
取出臉部特徵資訊：ch12-3-2.py

```
img = cv2.imread("images/face.jpg")
detector = FaceMeshDetector(maxFaces=8)
img, faces = detector.findFaceMesh(img, draw=False)
if faces:
    face_oval = detector.getFacePart(img, 0,
"FACE_OVAL")
    points = np.array(face_oval, np.int32)
    cv2.polylines(img, [points], True, (0, 0, 255), 1)
```

12-3-2 取出臉部特徵資訊

取出臉部特徵資訊：ch12-3-2.py

```
face_leye = detector.getFacePart(img, 0, "LEFT_EYE")
points = np.array(face_leye, np.int32)
cv2.polylines(img, [points], True, (0, 255, 255), 1)
```



12-3-2 取出臉部特徵資訊

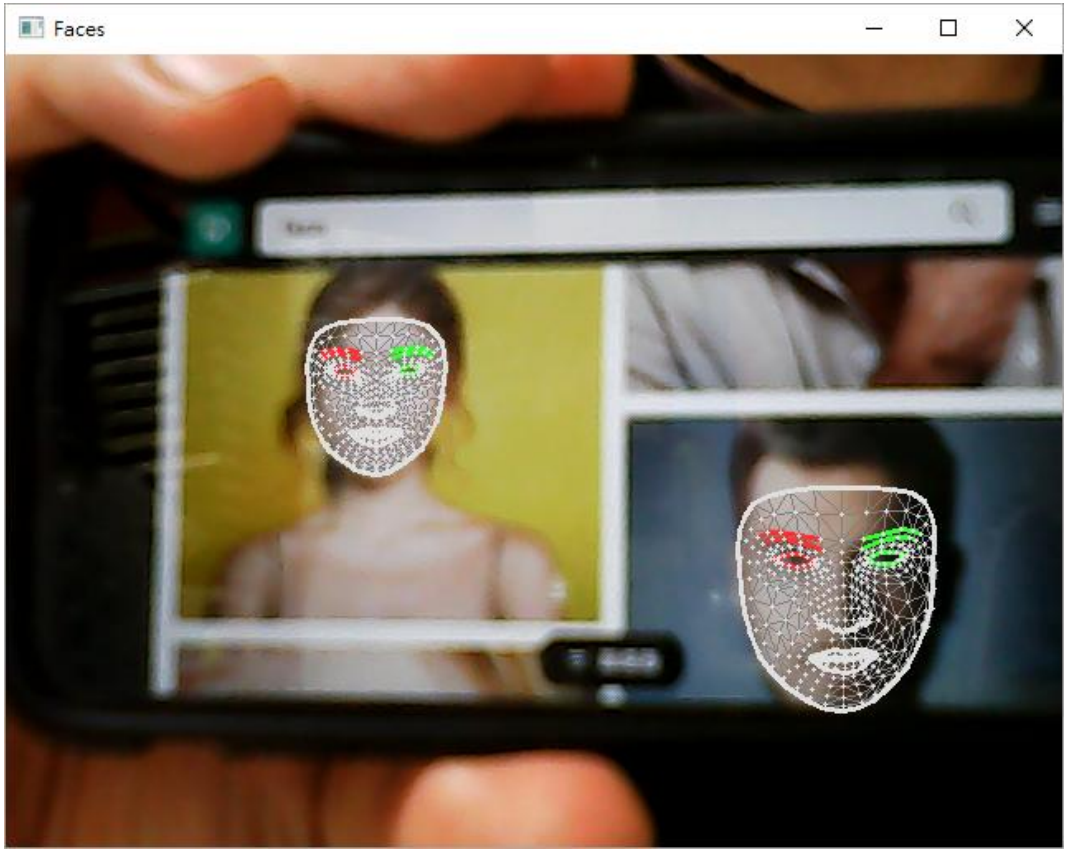
計算臉部特徵的尺寸：ch12-3-2a.py

- 在Python程式取得指定臉部特徵的座標串列後，就可以呼叫getFacePartSize()方法計算出臉部特徵的尺寸，即寬和高，如下所示：

```
face_leye = detector.getFacePart(img, 0, "LEFT_EYE")
height_leye, width_leye =
detector.getFacePartSize(face_leye)
print("LEFT_EYE:", width_leye, height_leye)
```

12-3-3 CVZone即時顯示臉部網格

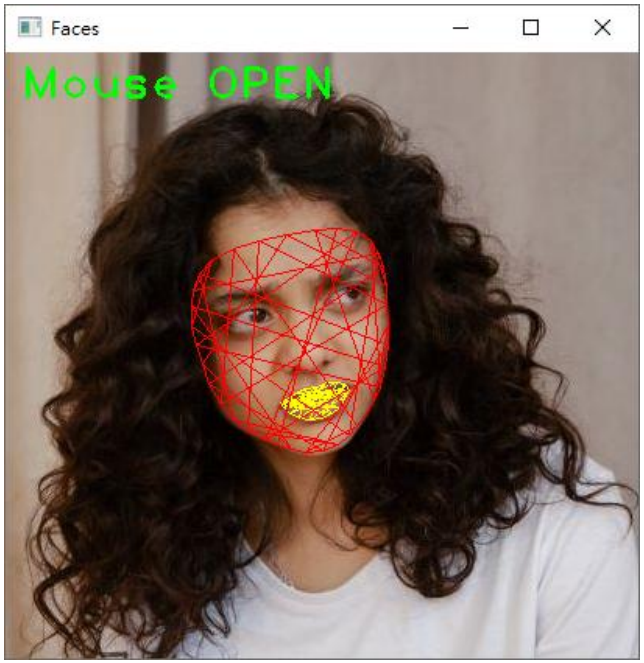
- Python程式：ch12-3-3.py整合第10-3-2節的Webcam，可以建立CVZone即時顯示3D臉部網格，如下圖所示：

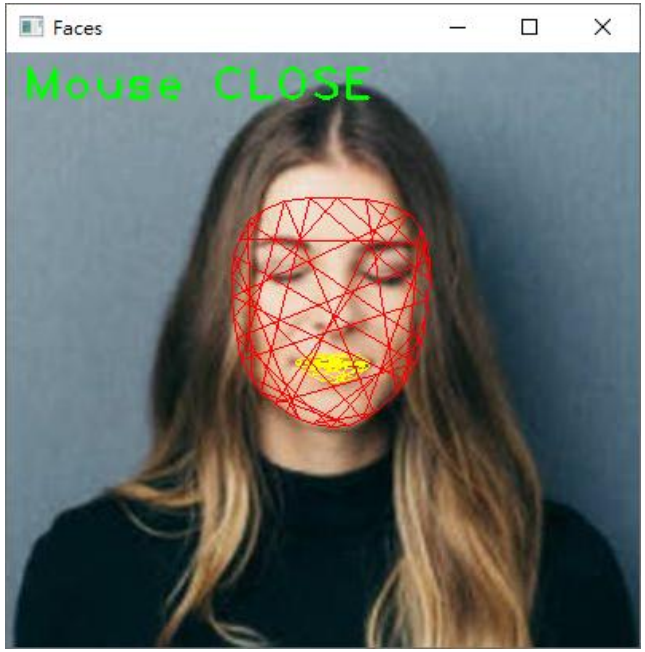


12-4 CVZone辨識臉部表情：張嘴/閉嘴與開眼/閉眼

辨識臉部表情是張嘴/閉嘴：ch12-4.py

- Python程式可以呼叫getFacePart()方法取得“FACE_OVAL”（臉形）和“LIPS”（嘴唇）臉部特徵，即可判斷臉部表情是張嘴或閉嘴，如下圖所示：

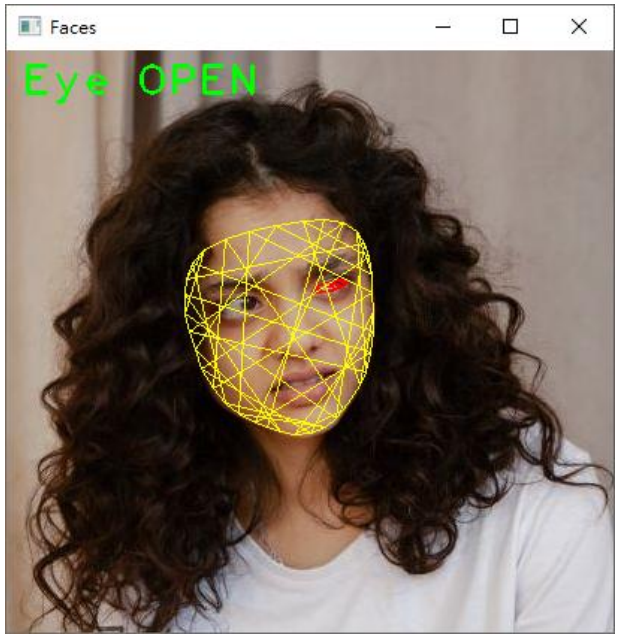


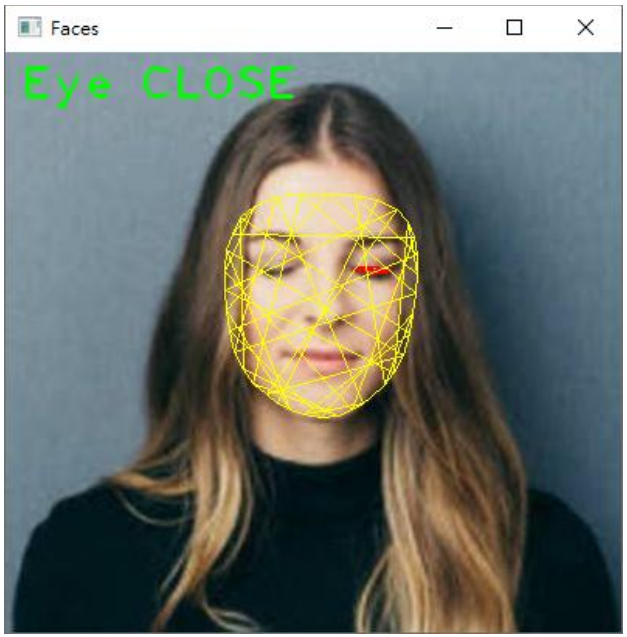


12-4 CVZone辨識臉部表情：張嘴/閉嘴與開眼/閉眼

辨識臉部表情是開眼/閉眼：ch12-4a.py

- Python程式可以呼叫getFacePart()方法取得“FACE_OVAL”（臉形）和“”LEFT_EYE“”（左眼）臉部特徵，即可判斷臉部表情的左眼是張開或閉著，如下圖所示：





12-5 face-recognition人臉識別

- 12-5-1 face-recognition安裝
- 12-5-2 face-recognition基本使用



12-5 face-recognition人臉識別

- Python的face_recognition套件是基於Dlib函式庫的一套目前已知最簡單的人臉識別套件。Dlib是一個包含機器學習、電腦視覺、圖像處理等功能強大的C++函式庫，已經廣泛使用在工業界、學術界、機器人、嵌入式系統、手機和大型運算架構。
- 基本上，人臉偵測和人臉識別的差異，如下所示：
  - 人臉偵測（Face Detection）：偵測影像或影格中是否有人臉。
  - 人臉識別（Face Recognition）：不只偵測出人臉，還需判斷這張臉是誰？

12-5-1 face-recognition安裝

- 在Windows作業系統的Python開發環境安裝face-recognition套件，首先需要安裝Dlib，我們是使用已編譯好的.whl檔進行安裝，其URL網址如下所示：
  - https://github.com/fchart/test
- 下載檔名是【dlib-19.22.99-cp39-cp39-win_amd64.whl】，請將檔案複製至D:\根目錄後，使用下列命令列指令安裝Dlib，如下所示：

pip install D:/dlib-19.22.99-cp39-cp39-win_amd64.whl Enter

- 當成功安裝Dlib後，就可以安裝face-recognition人臉識別套件，其命令列指令如下所示：

pip install face-recognition==1.3.0 Enter

12-5-2 face-recognition基本使用

- 人臉識別（Face Recognition）需要認出這張臉是誰，所以需要預先建立好人臉編碼，如此才能判斷其他人臉是否是同一張人臉。在Python程式使用face-recognition需要先匯入此套件，如下所示：

```
import face_recognition
```

12-5-2 face-recognition基本使用

人臉偵測：ch12-5-2.py

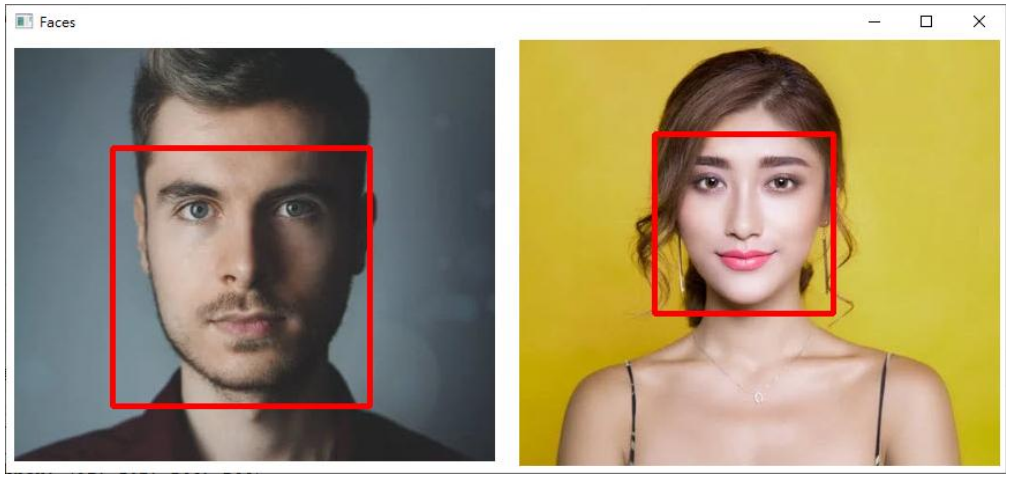
```
import face_recognition
import cv2

img = cv2.imread("images/faces2.jpg")
faces = face_recognition.face_locations(img,
                    number_of_times_to_upsample=1,
                    model="hog")
print("臉數=", len(faces))
```

12-5-2 face-recognition基本使用

人臉偵測：ch12-5-2.py

```
for face in faces:
    top, right, bottom, left = face
    cv2.rectangle(img, (left, top), (right, bottom),
                  (0, 0, 255), 3)
cv2.imshow("Faces", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



12-5-2 face-recognition基本使用

臉部資料編碼：ch12-5-2a.py

- 為了能夠識別出人臉，我們需要將人臉的臉部資料進行128維度的編碼，然後使用此編碼資料來比對與其他人臉的相似度，如下所示：

```
img = cv2.imread("images/faces2.jpg")
faces = face_recognition.face_locations(img)
print("臉數=", len(faces))
img_encodings = face_recognition.face_encodings(img,
            known_face_locations=faces, num_jitters=10)
print("128維度的編碼=", len(img_encodings))
print(img_encodings[0])
```

12-5-2 face-recognition基本使用

人臉識別：ch12-5-2b.py

```
img = cv2.imread("images/mary.jpg")
known_encoding =
face_recognition.face_encodings(img)[0]
new_img = cv2.imread("images/mary2.jpg")
new_encoding =
face_recognition.face_encodings(new_img)[0]
result =
face_recognition.compare_faces([known_encoding],
                               new_encoding, tolerance=0.6)
print(result)
```

12-5-2 face-recognition基本使用

計算兩張人臉之間差異的距離：ch12-5-2c.py

```
img = cv2.imread("images/mary.jpg")
known1_encoding =
face_recognition.face_encodings(img)[0]
img = cv2.imread("images/jane.jpg")
known2_encoding =
face_recognition.face_encodings(img)[0]
new_img = cv2.imread("images/mary2.jpg")
new_encoding =
face_recognition.face_encodings(new_img)[0]
```

12-5-2 face-recognition基本使用

計算兩張人臉之間差異的距離：ch12-5-2c.py

```
know_encodings = [known1_encoding,
known2_encoding]
distance =
face_recognition.face_distance(know_encodings,
                               new_encoding)
print(distance)
```

```
>>> %Run ch12-5-2c.py

  [0.25766463 0.81334325]
```

12-5-2 face-recognition基本使用

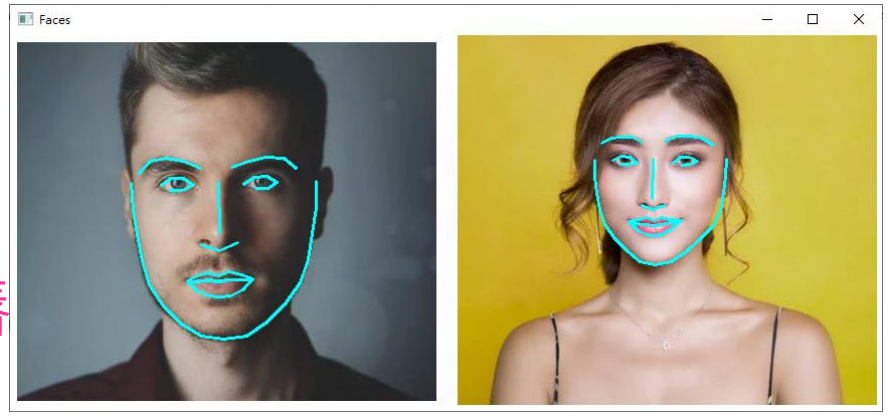
識別和繪出臉部特徵：ch12-5-2d.py

```
img = cv2.imread("images/faces2.jpg")
faces = face_recognition.face_locations(img)
landmarks = face_recognition.face_landmarks(img,
face_locations=faces)
features = ["chin",          # 下巴
           "left_eyebrow",   # 左眉
           "right_eyebrow",  # 右眉
           "nose_bridge",    # 鼻樑
           "nose_tip",       # 鼻尖
```

12-5-2 face-recognition基本使用

識別和繪出臉部特徵：ch12-5-2d.py

```
            "left_eye",      # 左眼
            "right_eye",     # 右眼
            "top_lip",       # 上嘴唇
            "bottom_lip"]   # 下嘴唇
for landmark in landmarks:
    for feature in features:
        points = np.array(landmark[feature], np.int32)
        points = points.reshape((-1, 1, 2))
        cv2.polylines(img, [points], False, (255, 255, 0), 2)
```



第13章 人工智慧應用（二）：多手勢追蹤與人體姿態評估

- 13-1 CVZone多手勢追蹤
- 13-2 CVZone辨識手勢（一）：剪刀、石頭與布
- 13-3 CVZone辨識手勢（二）：OK手勢
- 13-4 CVZone人體姿態評估
- 13-5 CVZone辨識人體姿勢（一）：仰臥起坐
- 13-6 CVZone辨識人體姿勢（二）：伏地挺身



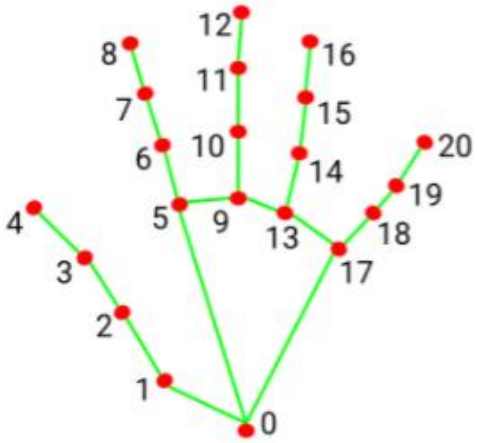
13-1 CVZone多手勢追蹤

- 13-1-1 CVZone多手勢追蹤的基本使用
- 13-1-2 偵測伸出幾根手指、取得距離和角度
- 13-1-3 CVZone即時多手勢追蹤



13-1 CVZone多手勢追蹤

- CVZone多手勢追蹤是基於MediaPipe手勢（MediaPipe Hands）的多手勢追蹤，這是使用手掌偵測模型（Palm Detection Model）進行多手勢追蹤，在第一步偵測出手掌和拳頭後，使用手部地標模型（Hand Landmark Model）偵測出手部21個關鍵點（下圖的數字就是關鍵點索引值），如下圖所示：



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

圖片來源:https://google.github.io/mediapipe/solutions/hands.html

13-1-1 CVZone多手勢追蹤的基本使用

影像的單手偵測：ch13-1-1.py

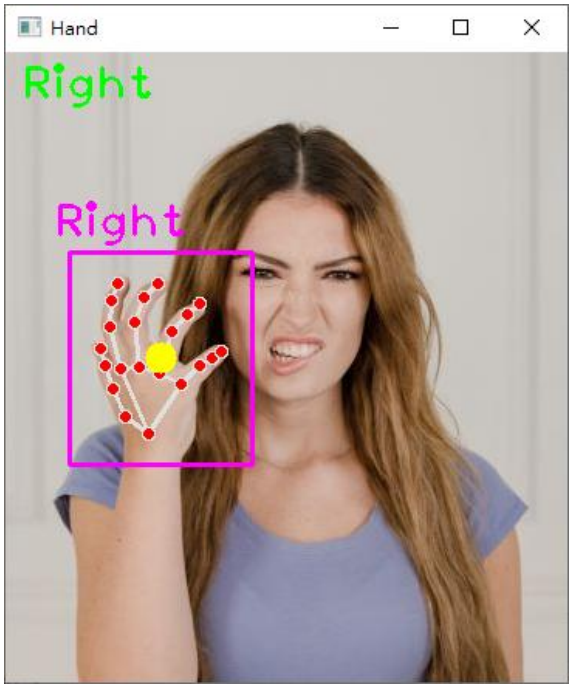
```
from cvzone.HandTrackingModule import
HandDetector
import cv2

img = cv2.imread("images/hand.jpg")
detector = HandDetector(detectionCon=0.5,
maxHands=1)
hands, img = detector.findHands(img)
if hands:
    hand1 = hands[0]   # 第1隻手
    centerPoint1 = hand1["center"]
    print(centerPoint1)
```

13-1-1 CVZone多手勢追蹤的基本使用

影像的單手偵測：ch13-1-1.py

```
    cv2.circle(img, centerPoint1, 10,
            (0, 255, 255), cv2.FILLED)
    handType1 = hand1["type"]
    print(handType1)
    cv2.putText(img, handType1, (10, 30),
            cv2.FONT_HERSHEY_PLAIN,
            2, (0, 255, 0), 2)
cv2.imshow("Hand", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



13-1-1 CVZone多手勢追蹤的基本使用

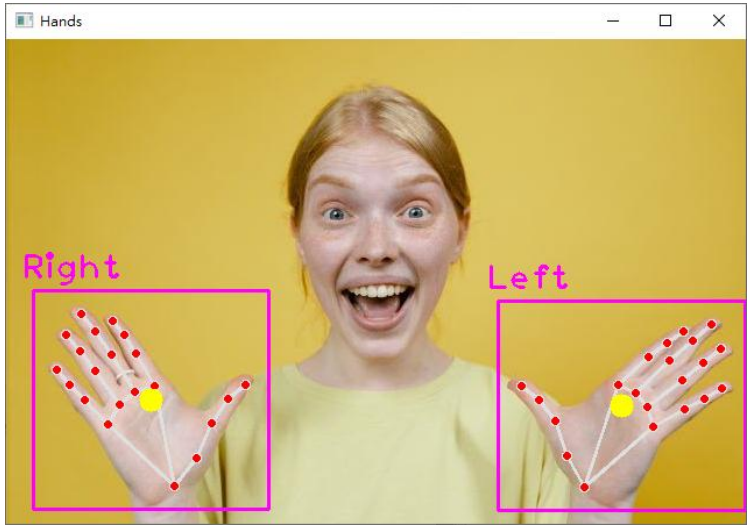
影像的雙手偵測：ch13-1-1a.py

```
img = cv2.imread("images/hands.jpg")
detector = HandDetector(detectionCon=0.5, maxHands=2)
hands, img = detector.findHands(img)
if hands:
    hand1 = hands[0]   # 第1隻手
    centerPoint1 = hand1["center"]
    cv2.circle(img, centerPoint1, 10, (0, 255, 255), cv2.FILLED)
```

13-1-1 CVZone多手勢追蹤的基本使用

影像的雙手偵測：ch13-1-1a.py

```
if len(hands) == 2:
    hand2 = hands[1]  # 第2隻手
    centerPoint2 = hand2["center"]
    print(centerPoint2)
    cv2.circle(img, centerPoint2, 10, (0, 255, 255),
cv2.FILLED)
```



13-1-1 CVZone多手勢追蹤的基本使用

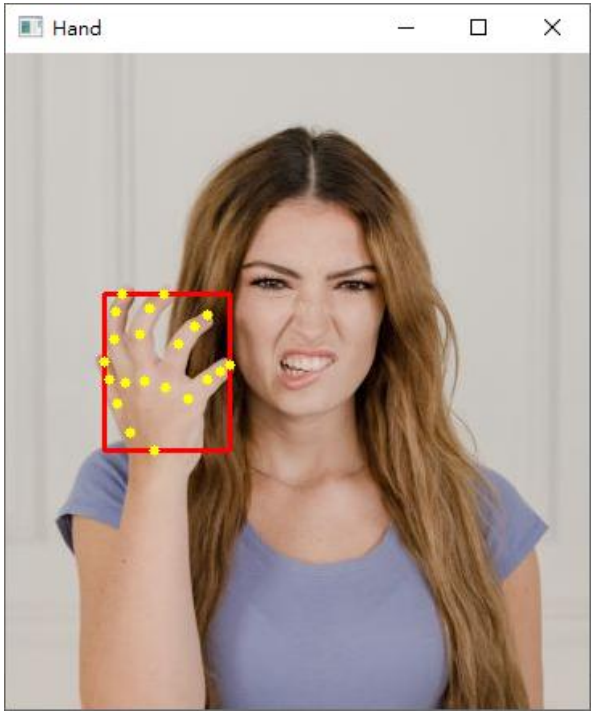
取得單手方框和21個關鍵點座標：ch13-1-1b.py

```
hands = detector.findHands(img, draw=False)
if hands:
    hand1 = hands[0]   # 第1隻手
    bbox1 = hand1["bbox"]
    x, y, w, h = bbox1
    cv2.rectangle(img, (x, y), (x+w, y+h),
                        (0, 0, 255), 2)
    lmList1 = hand1["lmList"]
```

13-1-1 CVZone多手勢追蹤的基本使用

取得單手方框和21個關鍵點座標：ch13-1-1b.py

```
for point in lmList1:
    x, y = point
    cv2.circle(img, (x, y), 3,
            (0, 255, 255), cv2.FILLED)
```



13-1-1 CVZone多手勢追蹤的基本使用

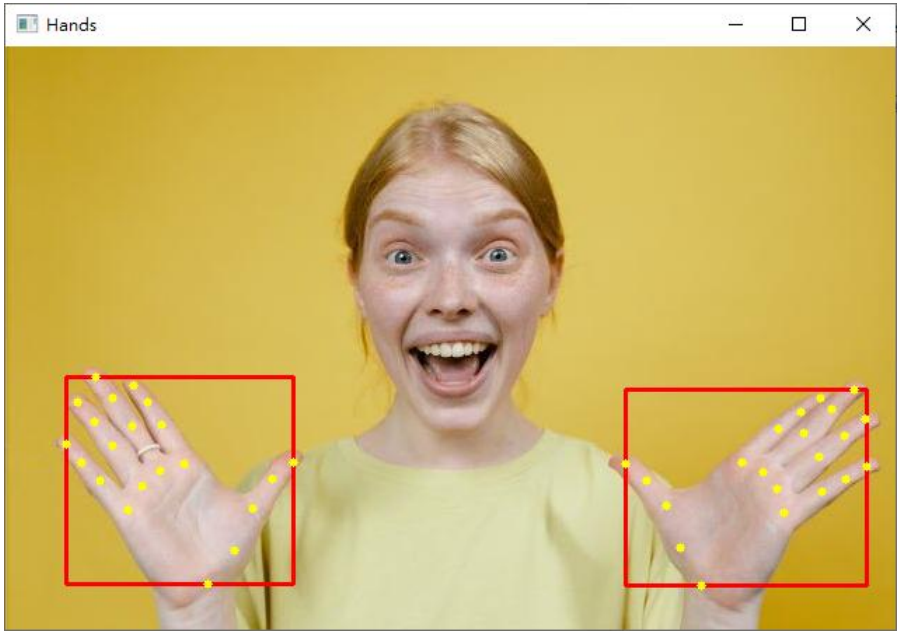
取得雙手方框和21個關鍵點座標：ch13-1-1c.py

```
if hands:
    hand1 = hands[0]   # 第1隻手
    x, y, w, h = hand1["bbox"]
    cv2.rectangle(img, (x, y), (x+w, y+h),
                        (0, 0, 255), 2)
    for point in hand1["lmList"]:
        cv2.circle(img, point, 3, (0, 255, 255), cv2.FILLED)
    if len(hands) == 2:
        hand2 = hands[1]   # 第2隻手
        x, y, w, h = hand2["bbox"]
```

13-1-1 CVZone多手勢追蹤的基本使用

取得雙手方框和21個關鍵點座標：ch13-1-1c.py

```
cv2.rectangle(img, (x, y), (x+w, y+h),
                    (0, 0, 255), 2)
for point in hand2["lmList"]:
    cv2.circle(img, point, 3, (0, 255, 255), cv2.FILLED)
```



13-1-2 偵測伸出幾根手指、取得距離和角度

偵測共伸出幾根手指：ch13-1-2.py



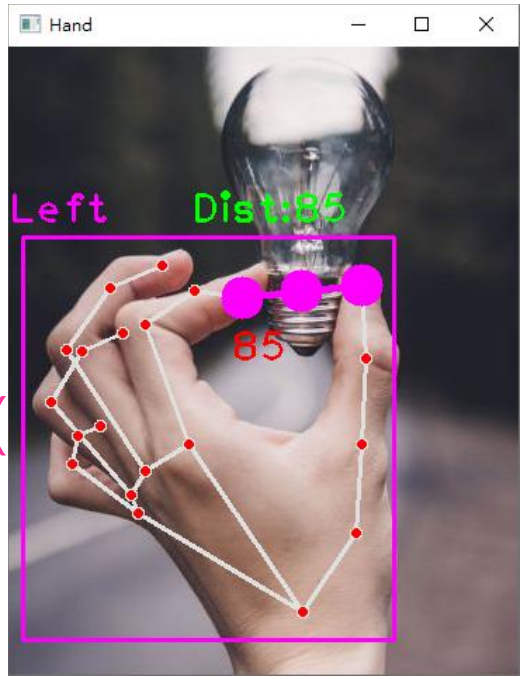

```
hand = hands[0]
bbox = hand["bbox"]
fingers = detector.fingersUp(hand)
print(fingers)
totalFingers = fingers.count(1)
msg = "Fingers:" + str(totalFingers)
cv2.putText(img, msg, (bbox[0]+100,bbox[1]-30),
        cv2.FONT_HERSHEY_PLAIN, 2, (0, 255, 0), 2)
```

```
>>> %Run ch13-1-2.py

  INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
  [1, 1, 1, 1, 0]
```

13-1-2 偵測伸出幾根手指、取得距離和角度

計算單手關鍵點之間的距離：ch13-1-2a.py




```
hand1 = hands[0]
lmList1 = hand1["lmList"]
bbox1 = hand1["bbox"]
length, info, img = detector.findDistance(
                    lmList1[4],lmList1[8],img)
print(info)
msg = "Dist:" + str(int(length))
cv2.putText(img, msg, (bbox1[0]+100,bbox1[1]-30),
        cv2.FONT_HERSHEY_PLAIN, 2, (0, 255, 0), 2)
```

13-1-2 偵測伸出幾根手指、取得距離和角度

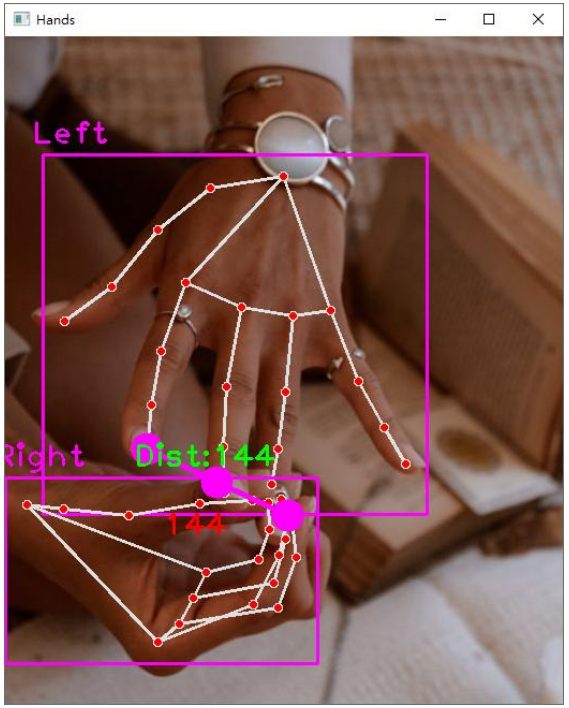
計算雙手關鍵點之間的距離：ch13-1-2b.py

```
hand1 = hands[0]
lmList1 = hand1["lmList"]
if len(hands) == 2:
    hand2 = hands[1]
    lmList2 = hand2["lmList"]
    bbox2 = hand2["bbox"]
    length, info, img = detector.findDistance(lmList1[8],
                                              lmList2[8], img)
    print(info)
```

13-1-2 偵測伸出幾根手指、取得距離和角度

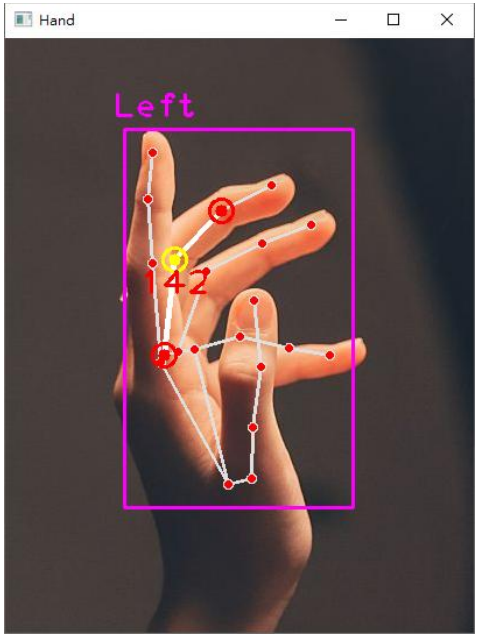
計算雙手關鍵點之間的距離：ch13-1-2b.py

```
msg = "Dist:" + str(int(length))
cv2.putText(img, msg,(bbox2[0]+100,bbox2[1]-30),
            cv2.FONT_HERSHEY_PLAIN, 2, (0, 255, 0), 2)
```



13-1-2 偵測伸出幾根手指、取得距離和角度

計算手指關鍵點之間的角度：ch13-1-2c.py




```
hand1 = hands[0]
lmList1 = hand1["lmList"]
angle, img = detector.findAngle(lmList1[9],
                lmList1[10], lmList1[11], img)

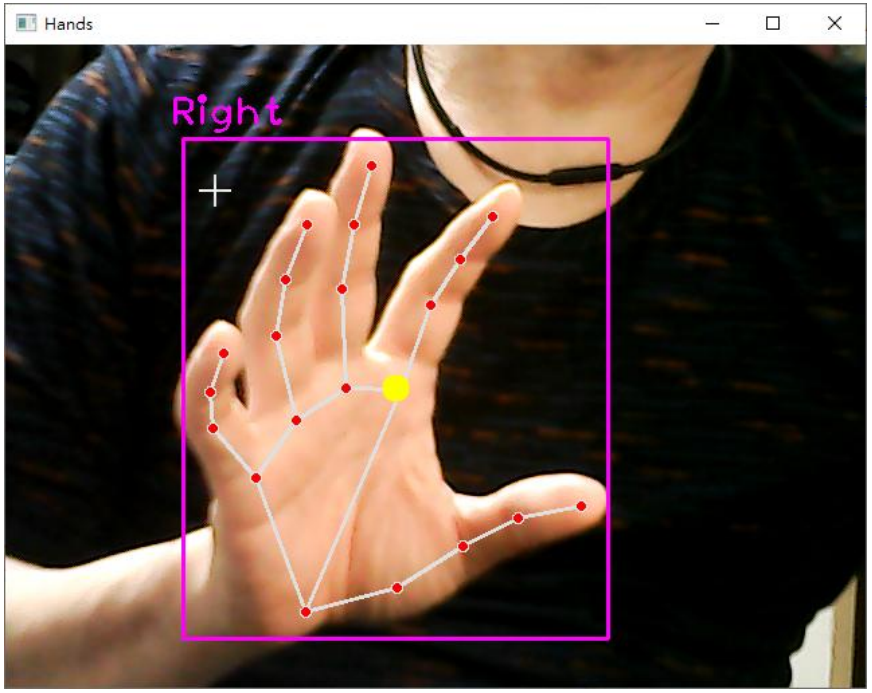
print(angle)
print(detector.angleCheck(angle, 150, addOn=20))
print(detector.angleCheck(angle, 100, addOn=20))
```

```
>>> %Run ch13-1-2c.py

  INFO: Created TensorFlow Lite XNNPACK delegate for CPU.
  142.66067164163093
  True
  False
```

13-1-3 CVZone即時多手勢追蹤

- Python程式：ch13-1-3.py整合第10-3-2節的Webcam，可以建立CVZone即時多手勢追蹤，在使用OpenCV讀取影格後，使用CVZone進行多手勢追蹤，如下圖所示：



13-2 CVZone辨識手勢（一）：剪刀、石頭與布

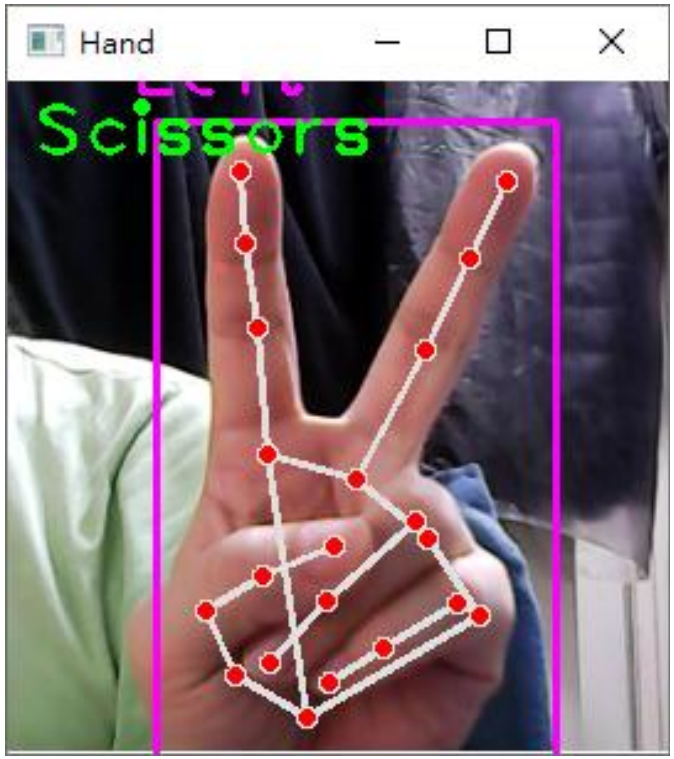
- 在Python程式可以使用HandDetector物件的fingersUp()方法，偵測出手掌伸出了哪幾根手指，如下所示：

```
fingers = detector.fingersUp(hand)
```

- 上述方法的回傳值是5根手指（從姆指開始至小指）的串列，例如：[0, 1, 1, 0, 0]，值1是伸出；0沒有，以此例是伸出食指和中指，所以，我們可以使用此串列來辨識手勢是：剪刀、石頭或布。

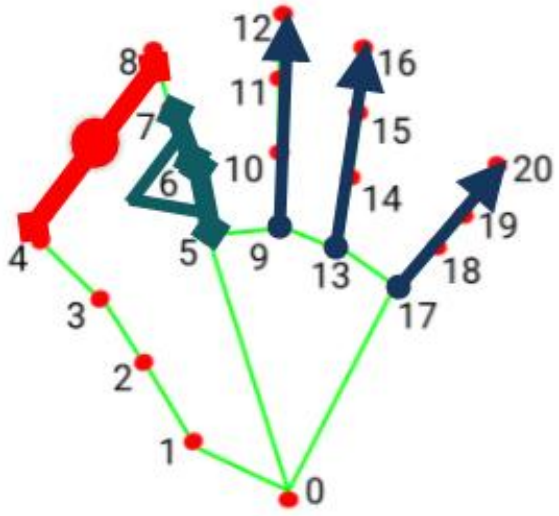
13-2 CVZone辨識手勢（一）：剪刀、石頭與布

- Python程式：ch13-2.py在讀取影像後，使用CVZone進行手勢追蹤，可以偵測出手掌伸出哪幾根手指來辨識出手勢是剪刀、石頭或布，其執行結果如下圖所示：



13-3 CVZone辨識手勢（二）：OK手勢

- CVZone多手勢追蹤可以判斷伸出幾隻手指、計算指關節角度和指尖距離，我們可以使用這些資訊來判斷各種手勢，例如：OK手勢可以使用下列特徵來進行判斷，如下圖所示：

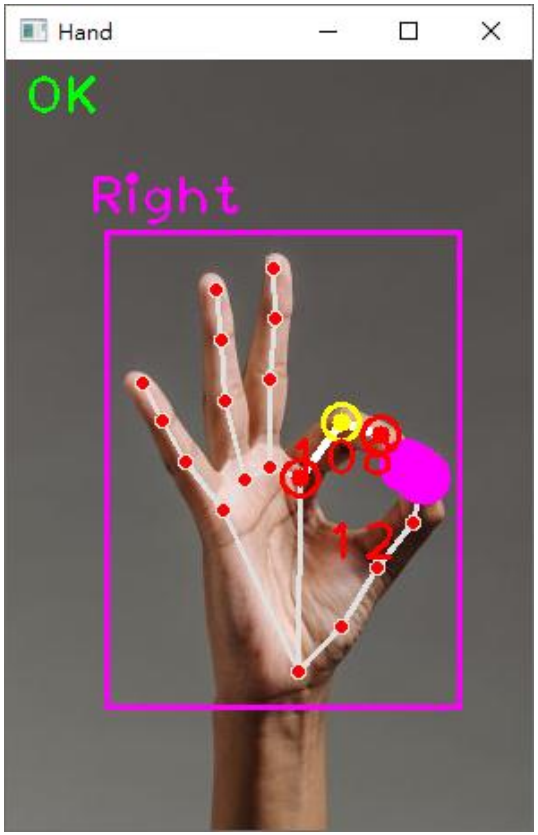


0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

圖片來源:https://google.github.io/mediapipe/solutions/hands.html

13-3 CVZone辨識手勢（二）：OK手勢

- Python程式：ch13-3.py在讀取影像後，使用CVZone進行手勢追蹤，可以辨識手勢是否是OK手勢，其執行結果如下圖所示：



13-4 CVZone人體姿態評估

- 13-4-1 CVZone人體姿態評估的基本使用
- 13-4-2 計算關鍵點之間的距離和角度
- 13-4-3 CVZone即時人體姿態評估



13-4-1 CVZone人體姿態評估的基本使用

影像的人體姿態評估：ch13-4-1.py

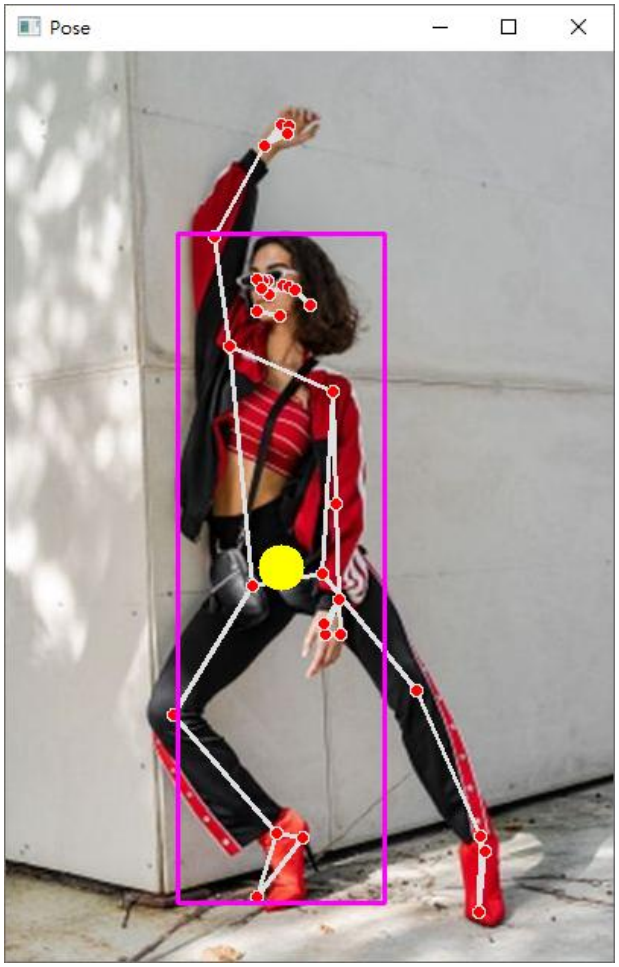
```
from cvzone.PoseModule import PoseDetector
import cv2

img = cv2.imread("images/woman.jpg")
detector = PoseDetector(detectionCon=0.5,
trackCon=0.5)
pose, img = detector.findPose(img,
bboxWithHands=False)
if pose:
    x1, y1, w, h = pose["bbox"]
    cv2.rectangle(img, (x1, y1),
                  (x1 + w, y1 + h),
```

13-4-1 CVZone人體姿態評估的基本使用

影像的人體姿態評估：ch13-4-1.py

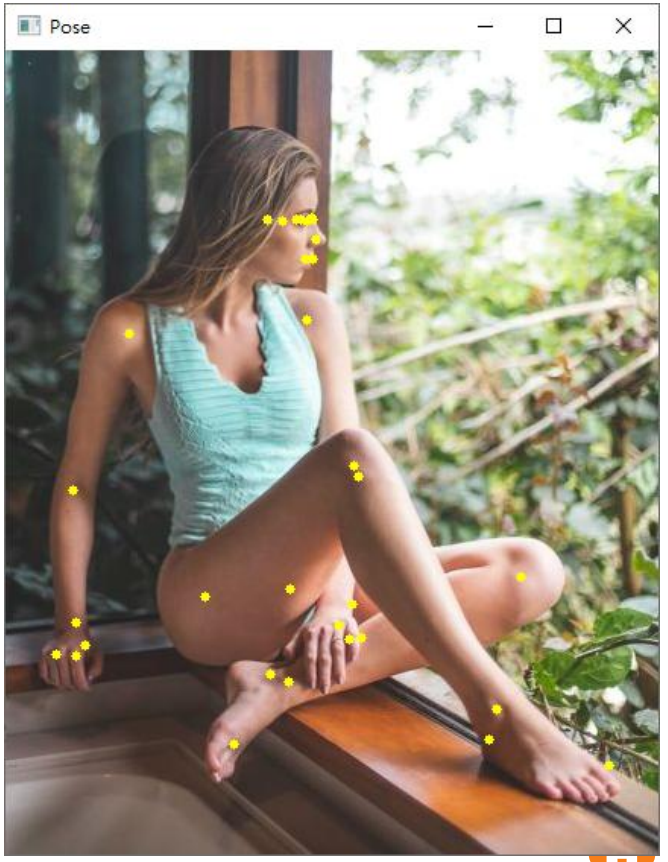
```
                (255, 0, 255), 2)
    center = pose["center"]
    cv2.circle(img, center, 15,
               (0, 255, 255), cv2.FILLED)
cv2.imshow("Pose", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



13-4-1 CVZone人體姿態評估的基本使用

取得33個關鍵點座標：ch13-4-1a.py

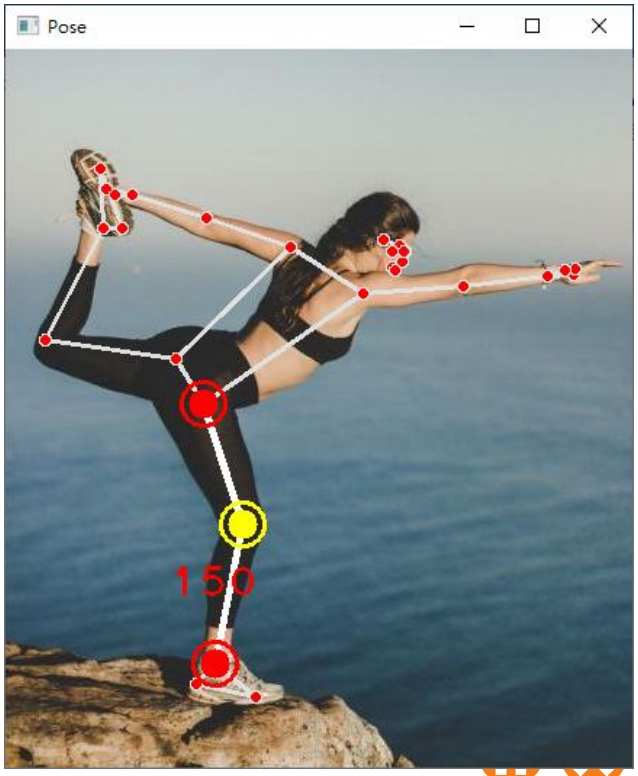
```
pose = detector.findPose(img, draw=False)
if pose:
    lmList = pose["lmList"]
    for point in lmList:
        cv2.circle(img, point, 3,
            (0, 255, 255), cv2.FILLED)
```



13-4-2 計算關鍵點之間的距離和角度

計算3個關鍵點之間的角度：ch13-4-2.py

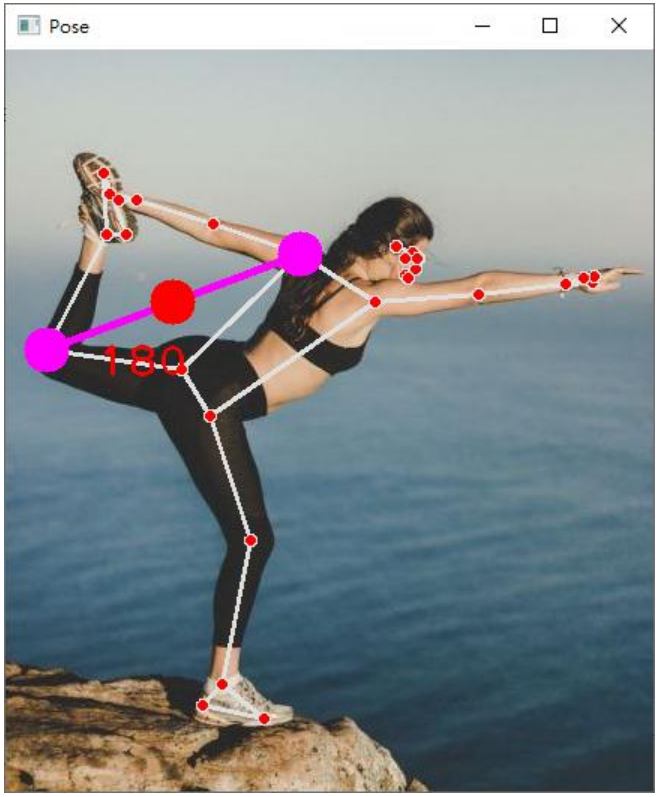
```
lmList = pose["lmList"]
angle, img = detector.findAngle(lmList[24], lmList[26],
lmList[28], img)
print(angle, info)
print(detector.angleCheck(angle, 140,
                          addOn=20))
print(detector.angleCheck(angle, 90,
                          addOn=20))
```



13-4-2 計算關鍵點之間的距離和角度

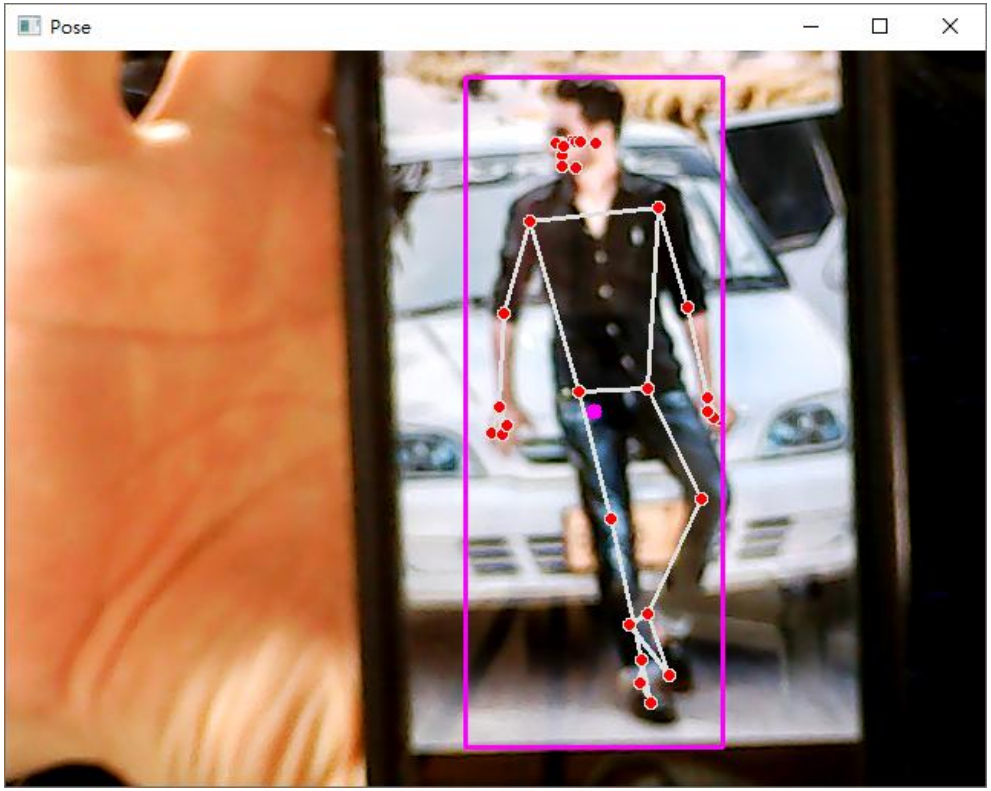
計算2個關鍵點之間的距離：ch13-4-2b.py

```
lmList = pose["lmList"]
length, distInfo, img = detector.findDistance(lmList[11], lmList[25], img)
print(length, distInfo)
```



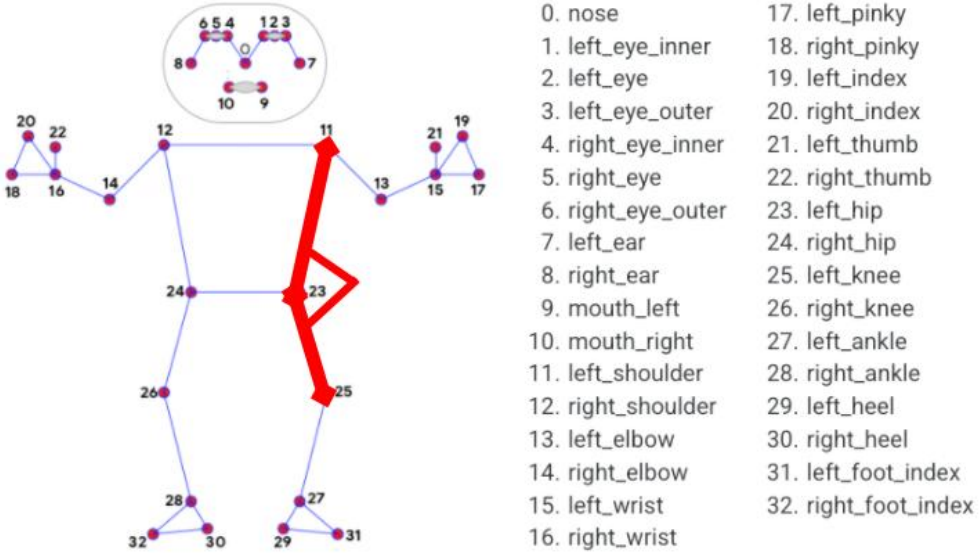
13-4-3 CVZone即時人體姿態評估

- Python程式：ch13-4-3.py整合第10-3-2節的Webcam，可以建立CVZone即時人體姿態評估，在使用OpenCV讀取影格後，使用CVZone進行人體姿態評估，如下圖所示：



13-5 CVZone辨識人體姿勢（一）：仰臥起坐

- CVZone人體姿態評估可以辨識人體姿勢，主要是使用關鍵點角度來進行判斷，例如：伏地挺身的姿勢有2個狀態：仰臥和起坐，完整一次循環才是做一次仰臥起坐，很明顯！我們可以使用腰部角度來判斷目前的狀態，即關鍵點（11, 23, 25），如下圖所示：



圖片來源:https://google.github.io/mediapipe/solutions/pose.html

13-5 CVZone辨識人體姿勢（一）：仰臥起坐

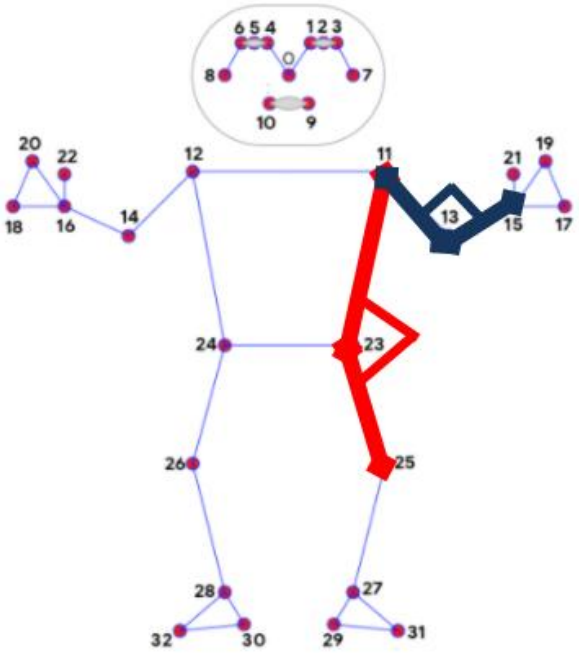
- Python程式：ch13-5.py在讀取【仰臥】影像site_up.jpg（Youtube視訊截圖）後，使用CVZone辨識仰臥起坐的姿勢，可以計算出腰部角度是116，透過更多視訊截圖的測試，我們可以找出腰部角度範圍來判斷姿勢是【仰臥】，其執行結果如下圖所示：





13-6 CVZone辨識人體姿勢（二）：伏地挺身

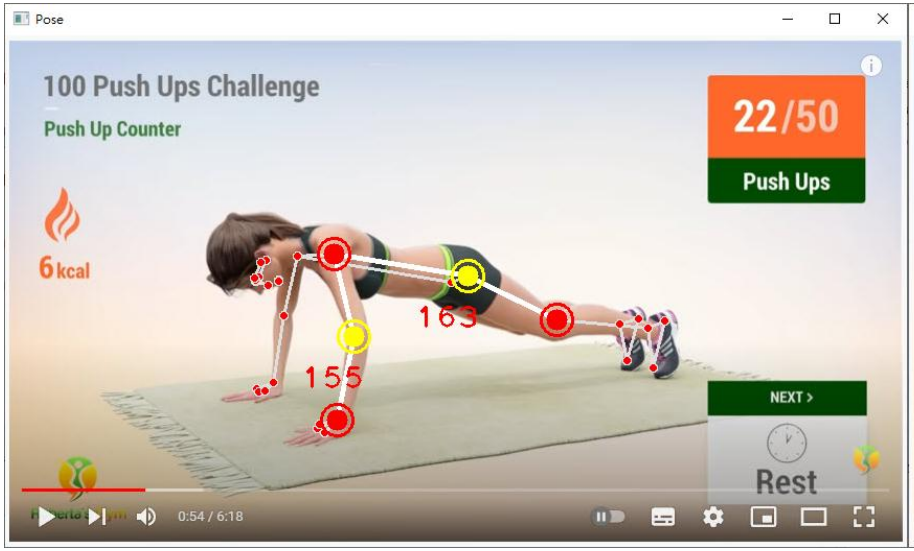
- 當使用CVZone辨識伏地挺身的人體姿勢時，需要使用2個關鍵點角度，第1個是腰部角度，即關鍵點（11, 23, 25），第2個是手臂關節的角度，即關鍵點（11, 13, 15），如下圖所示：

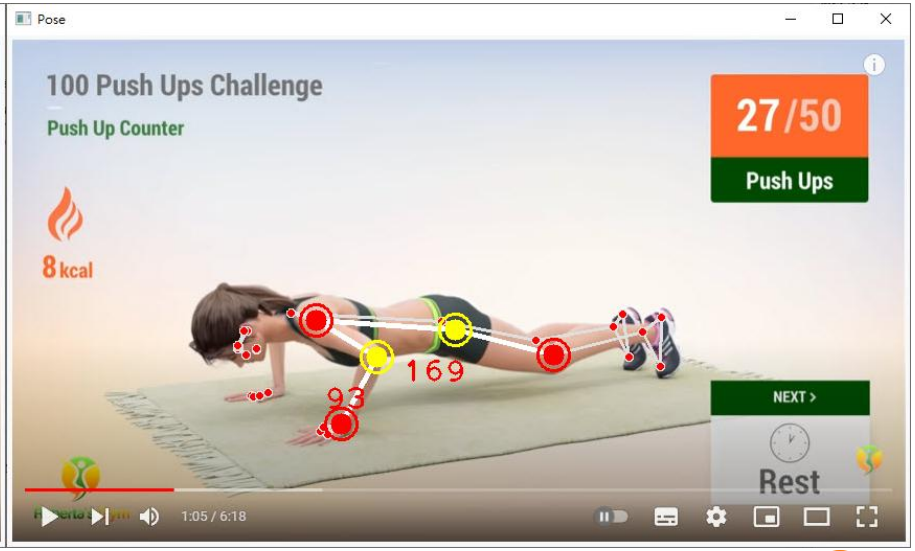


圖片來源:https://google.github.io/mediapipe/solutions/pose.html

13-6 CVZone辨識人體姿勢（二）：伏地挺身

- Python程式：ch13-6.py在讀取【挺身】影像push_up.jpg（Youtube視訊截圖）後，使用CVZone辨識伏地挺身的姿勢，可以計算出腰部角度是163；手臂關節角度是155，透過更多測試，我們可以找出2個角度的範圍來判斷姿勢是【挺身】，其執行結果如下圖所示：





第14章 AI整合實戰（一）：手勢操控、健身教練與刷臉簽到

- 14-1 AI整合實戰：手勢操控
- 14-2 AI整合實戰：健身教練
- 14-3 AI整合實戰：刷臉簽到



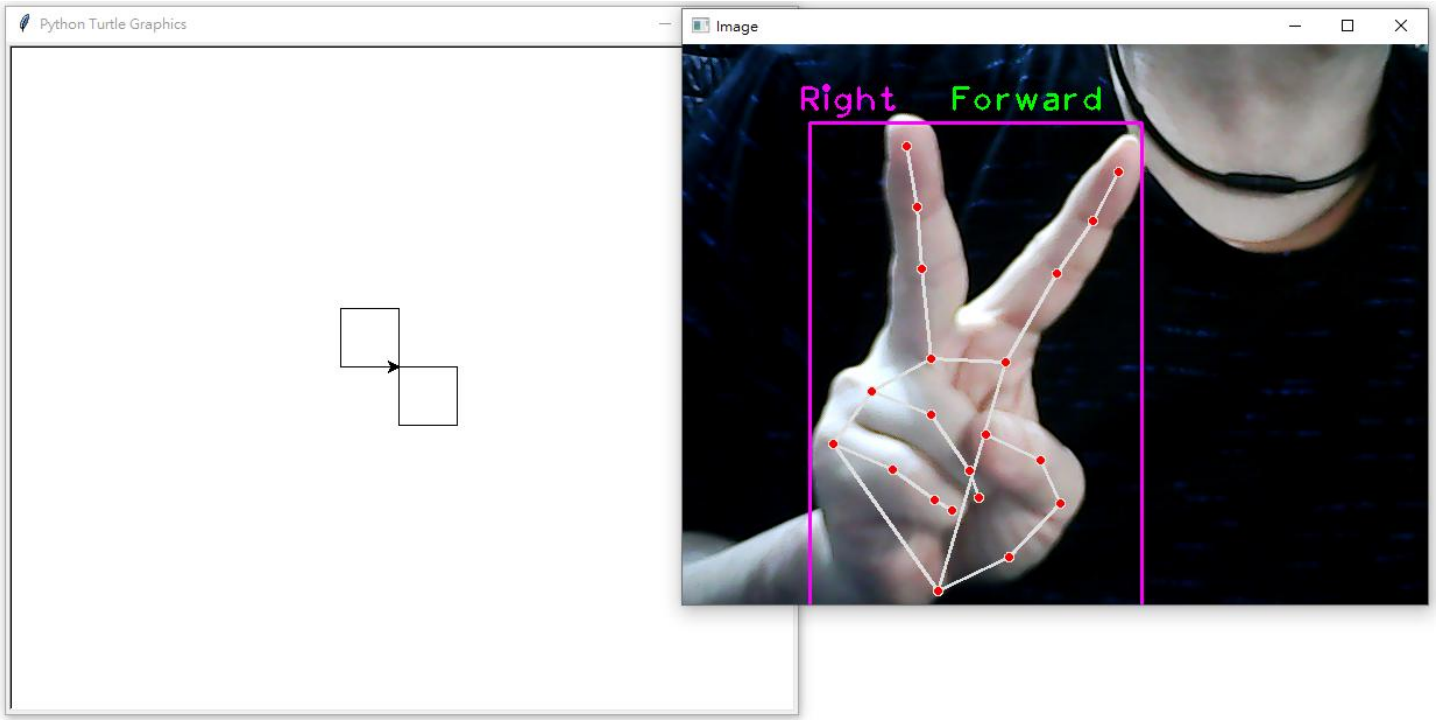
14-1 AI整合實戰：手勢操控

- 14-1-1 手勢偵測操控海龜繪圖
- 14-1-2 手勢偵測操控PowerPoint



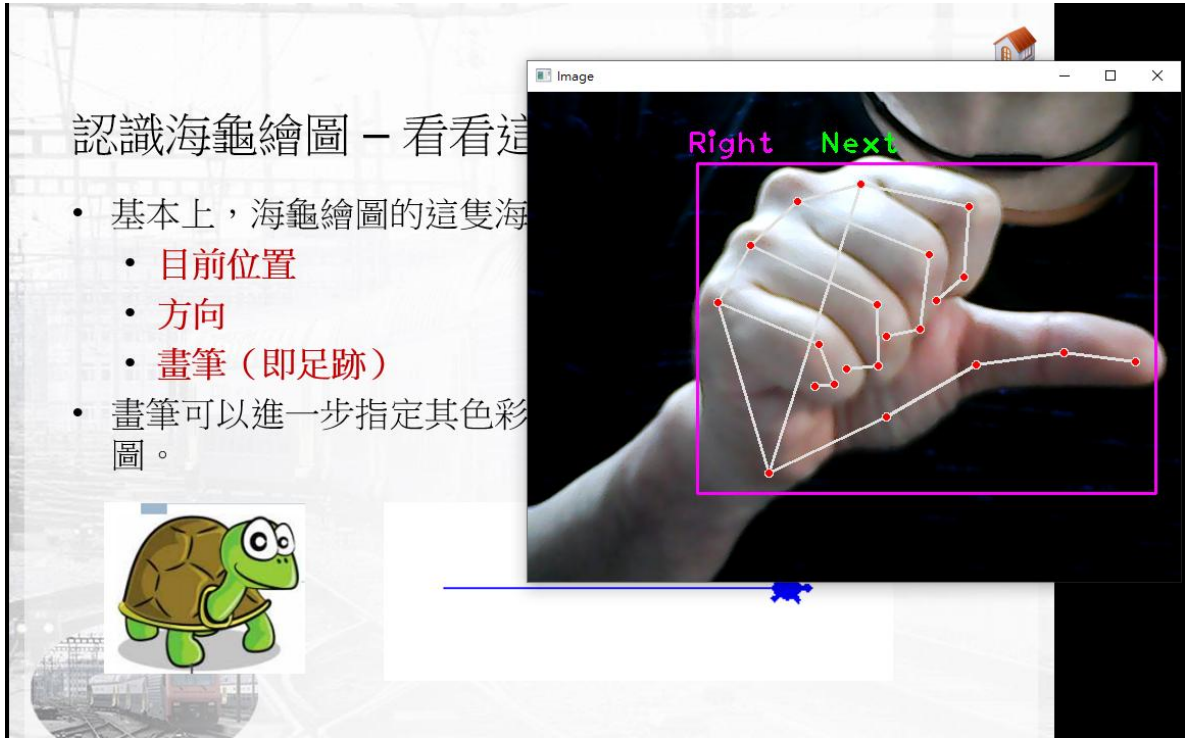
14-1-1 手勢偵測操控海龜繪圖

- Python程式：ch14-1-1.py在使用OpenCV讀取影像後，就可以使用CVZone進行手勢追蹤，可以偵測手掌的手勢來操控海龜繪圖是前進或右轉，即可繪出所需的圖形，其執行結果如下圖所示：



14-1-2 手勢偵測操控PowerPoint

- Python程式：ch14-1-2.py在使用OpenCV讀取影像後，可以使用CVZone進行手勢追蹤，透過偵測手勢來操控PowerPoint播放簡報，使用手勢來切換前一頁或下一頁，其執行結果如下圖所示：



14-2 AI整合實戰：健身教練

- 14-2-1 仰臥起坐AI健身教練
- 14-2-2 伏地挺身AI健身教練
- 14-2-3 深蹲訓練AI健身教練



14-2-1 仰臥起坐AI健身教練

- 在第13-5節已經說明如何判斷仰臥起坐的2個姿勢，問題是如何判斷依序完成這2個姿勢來做完一次完整的仰臥起坐。我們可以觀察到仰臥起坐就是依序上/下兩個方向，可以使用變數dir切換值0（仰臥）和值 1（起坐），判斷是否是正確的順序來完成2個姿勢。
- Python程式是使用2個if巢狀條件判斷是否依序完成仰臥起坐的2個姿勢。第1個巢狀if條件判斷目前狀態是否是起坐姿勢，如下所示：

```
if angle <= 50:   # 目前狀態:起坐
    if dir == 0:  # 之前狀態:仰臥
        count = count + 0.5
        dir = 1   # 更新狀態:起坐
```

14-2-1 仰臥起坐AI健身教練

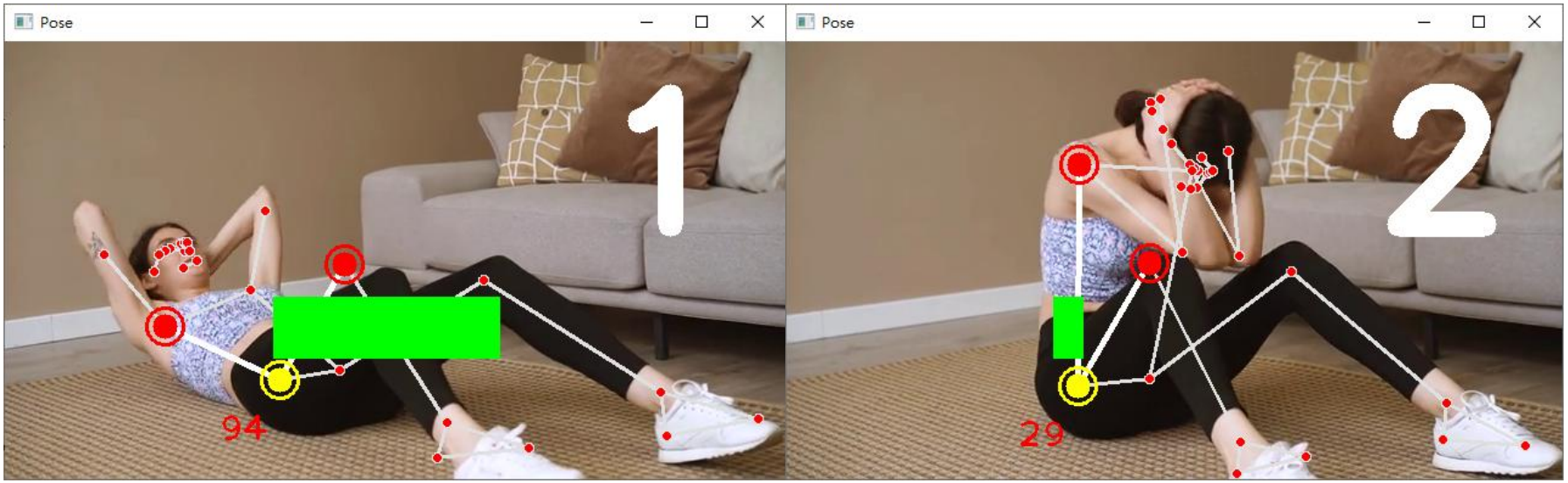
- 當上述變數angle值小於等於50時，表示目前狀態是起坐，然後判斷dir變數值的之前狀態是否是仰臥，如果是，順序正確，即可將計次變數count的值加0.5（完成一半姿勢），和更新狀態成起坐。同理，第2個巢狀if條件判斷目前是否是仰臥姿勢，如下所示：

```
if angle >= 90:   # 目前狀態:仰臥
    if dir == 1:  # 之前狀態:起坐
        count = count + 0.5
        dir = 0   # 更新狀態:仰臥
```

- 上述變數angle值大於等於90時，表示目前狀態是仰臥，然後判斷dir變數值的之前狀態是否是起坐，如果是，順序正確，即可將計次變數count的值加0.5（完成另一半姿勢），和更新狀態成仰臥。

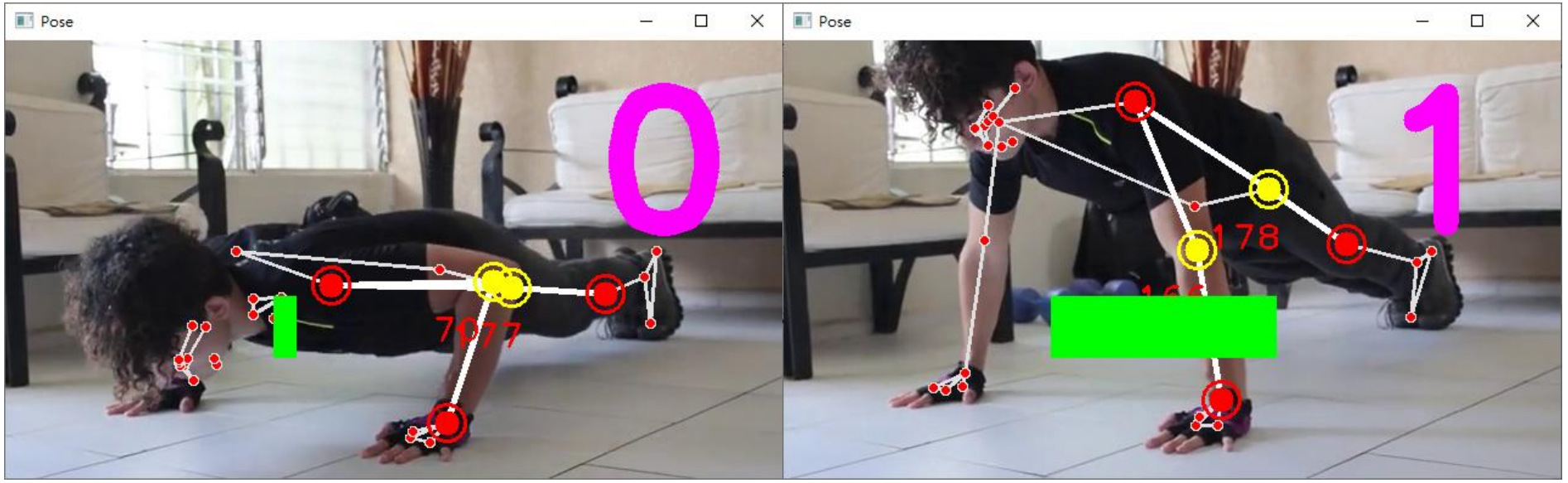
14-2-1 仰臥起坐AI健身教練

- Python程式：ch14-2-1.py在使用OpenCV讀取視訊的影格後，使用CVZone進行人體姿態評估，可以計算共做了幾次仰臥起坐，其執行結果如下圖所示：



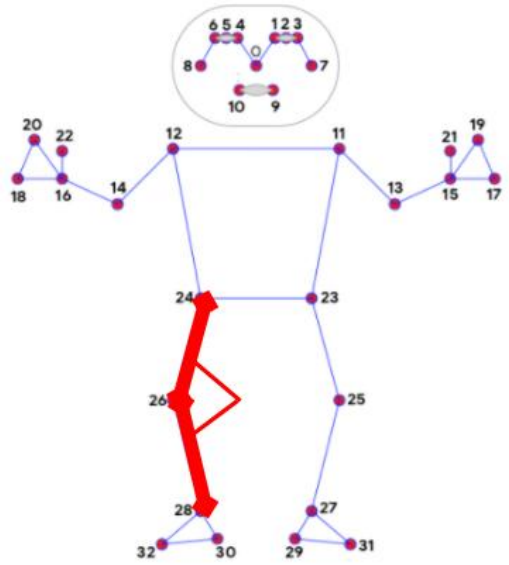
14-2-2 伏地挺身AI健身教練

- 在第13-6節已經說明如何判斷伏地挺身的二個姿勢，Python程式：ch14-2-2.py使用OpenCV讀取視訊的影格後，可以使用CVZone進行人體姿態評估，計算共做了幾次伏地挺身，其執行結果如下圖所示：



14-2-3 深蹲訓練AI健身教練

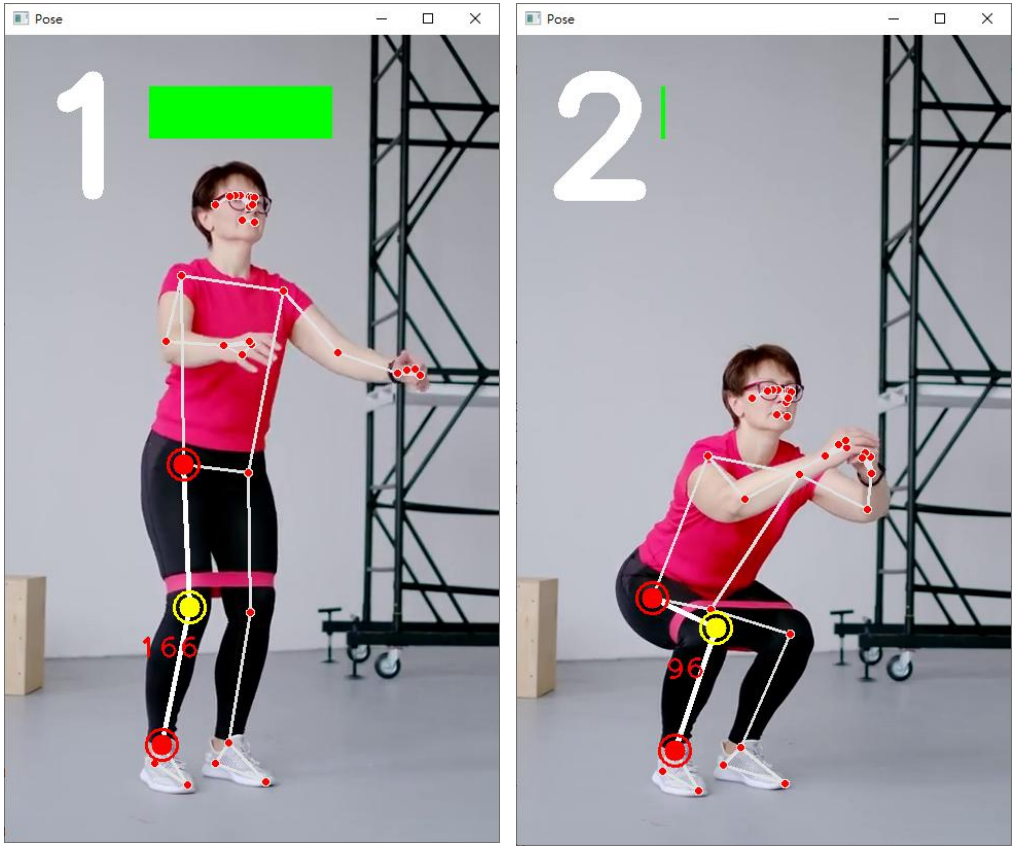
- CVZone人體姿態評估主要是使用關鍵點角度來辨識人體姿態，例如：深蹲訓練的姿勢有2個狀態：站起和蹲下，完整一次循環才是做了一次深蹲，很明顯！我們可以使用腳部的膝蓋角度來判斷目前的姿勢，即關鍵點（24, 26, 28），如下圖所示：



圖片來源:https://google.github.io/mediapipe/solutions/pose.html

14-2-3 深蹲訓練AI健身教練

- Python程式：ch14-2-3.py在使用OpenCV讀取視訊的影格後，可以使用CVZone進行人體姿態評估，計算共做了幾次深蹲，其執行結果如下圖所示：



14-3 AI整合實戰：刷臉簽到

- Python程式可以使用第12-5節face-recognition套件的人臉識別（Face Recognition）功能，輕鬆建立出刷臉簽到應用，即判斷Webcam影格的人臉是哪一位先生。
- 因為人臉識別不只需要偵測出人臉，還需要判斷這張臉是誰？所以，我們需要建立2個Python程式來執行二項工作，如下所示：
  - ch14-3.py：預先建立人臉圖檔的編碼，雖然一張圖檔允許有多張人臉，但請使用一張圖檔一張臉來進行臉部資料編碼。
  - ch14-3a.py：使用Webcam取得人臉影像，即可和預建立的臉部資料編碼進行比較，即可判斷人臉是哪一位先生。

14-3 AI整合實戰：刷臉簽到

臉部資料編碼：ch14-3.py

•Python程式是使用字典儲存face-recognition套件建立的臉部資料編碼，並且使用pickle模組成儲存字典串列成為faces_encoding.dat檔案，其執行結果如下所示：

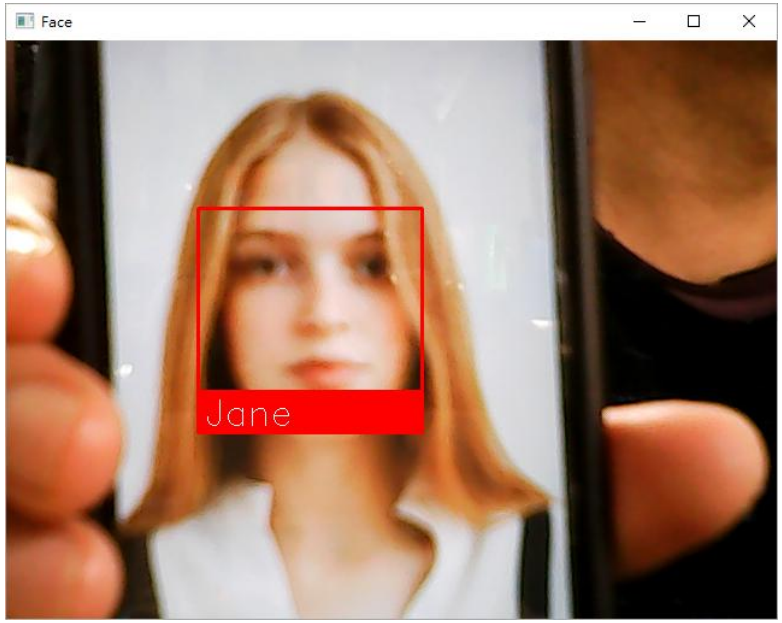
```
>>> %Run ch14-3.py

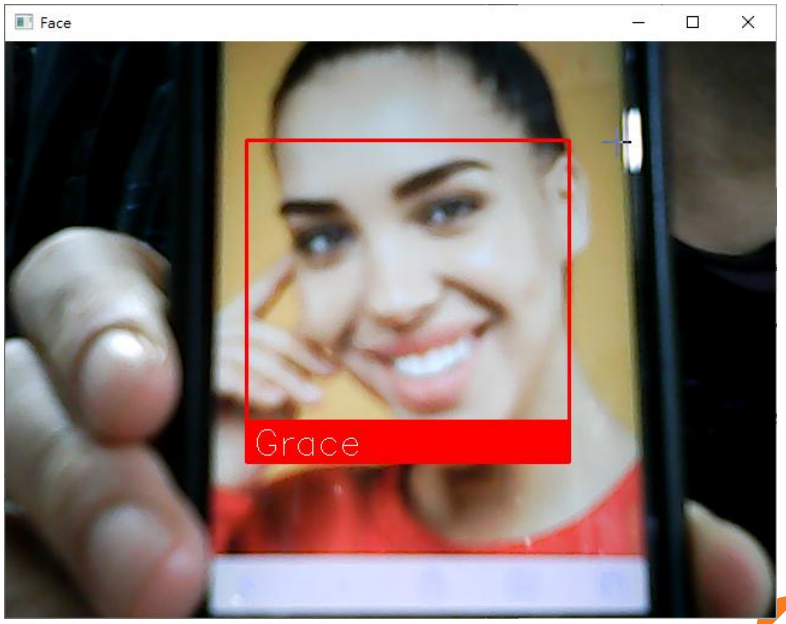
  人臉:[ images/mary.jpg ]編碼中...
  人臉:[ images/mary.jpg ]編碼完成...
  人臉:[ images/jane.jpg ]編碼中...
  人臉:[ images/jane.jpg ]編碼完成...
  人臉:[ images/grace.jpg ]編碼中...
  人臉:[ images/grace.jpg ]編碼完成...
  人臉編碼已經成功寫入faces_encoding.dat...
```

14-3 AI整合實戰：刷臉簽到

刷臉簽到：ch14-3a.py

•當成功執行ch14-3.py建立臉部資料編碼faces_encoding.dat二進位檔案後，就可以使用此檔案的臉部資料編碼來執行人臉識別，可以識別出Webcam的影格是Jane和Grace，如下圖所示：





第15章 人工智慧應用（三）：影像分類、物體識別與OCR文字識別

- 15-1 OpenCV DNN模組和TensorFlow Lite
- 15-2 影像分類
- 15-3 物體識別
- 15-4 文字偵測
- 15-5 Tesseract-OCR文字識別



15-1 OpenCV DNN模組和TensorFlow Lite

- 15-1-1 OpenCV DNN模組
- 15-1-2 TensorFlow和TensorFlow Lite
- 15-1-3 下載本書使用的預訓練模型



15-1-1 OpenCV DNN模組

認識OpenCV DNN模組

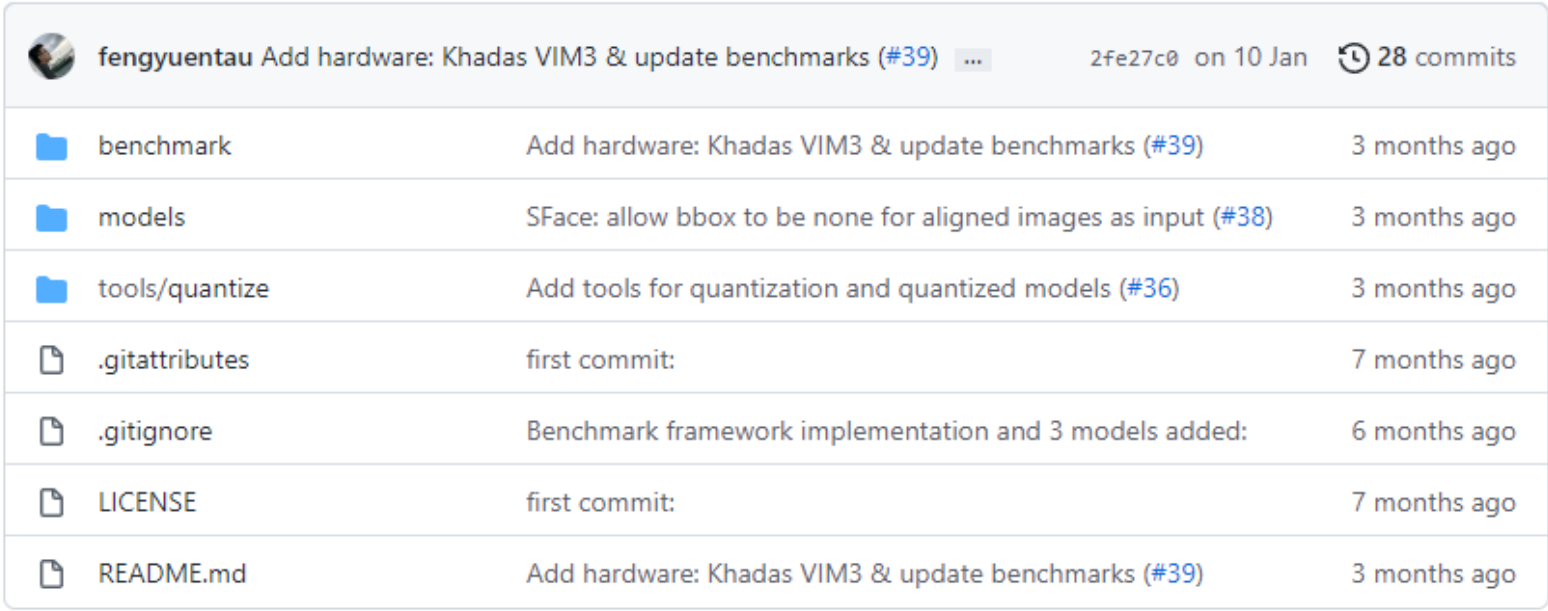
- OpenCV是在3.3版加入 DNN模組，全名Deep Neural Networks Module深度神經網路模組，目前只支援影像和視訊的深度學習推論，可以使用Caffe、TensorFlow、Torch/Pytorch和Darknet等多種深度學習框架所訓練的深度學習模型。
- 目前OpenCV DNN模組已經支援常見的預訓練模型：AlexNet、GoogLeNet V1、ResNet-34/50/...、SqueezeNet V1.1、VGG-based FCN、ENet、VGG-based SSD 和MobileNet-based SSD等。換句話說，我們只需下載這些預訓練模型檔，就可以馬上建立Python程式來實作人工智慧的相關應用。

15-1-1 OpenCV DNN模組

OpenCV DNN模組的預訓練模型

•在OpenCV官方GitHub網站提供一些現成的預訓練模型和使用範例，其URL網址如下所示：

https://github.com/opencv/opencv_zoo



fengyuentau Add hardware: Khadas VIM3 & update benchmarks (#39) ... 2fe27c0 on 10 Jan 28 commits

benchmark	Add hardware: Khadas VIM3 & update benchmarks (#39)	3 months ago
models	SFace: allow bbox to be none for aligned images as input (#38)	3 months ago
tools/quantize	Add tools for quantization and quantized models (#36)	3 months ago
.gitattributes	first commit:	7 months ago
.gitignore	Benchmark framework implementation and 3 models added:	6 months ago
LICENSE	first commit:	7 months ago
README.md	Add hardware: Khadas VIM3 & update benchmarks (#39)	3 months ago

15-1-2 TensorFlow和TensorFlow Lite

- TensorFlow是Google公司的機器學習/深度學習框架，這是Google Brain Team所開發，在2005年底開放專案後，2017年推出第一個正式版本，之所以稱為TensorFlow，這是因為其輸入/輸出的運算資料是向量、矩陣等多維度的數值資料，稱為張量（Tensor），我們建立的機器學習模型需要使用流程圖來描述訓練過程的所有數值運算操作，稱為計算圖（Computational Graphs），Tensor張量就是經過這些流程Flow的數值運算來產生輸出結果，稱為：Tensor+Flow=TensorFlow。

15-1-2 TensorFlow和TensorFlow Lite

認識與安裝TensorFlow Lite

- TensorFlow Lite是輕量版TensorFlow，只提供部署的推論功能，可以在行動裝置和IoT裝置上部署和使用機器學習模型，這是開放原始碼的深度學習架構，能夠直接在裝置端載入模型來執行推論。
- 在Python開發環境安裝TensorFlow Lite需要使用.whl檔，請啟動瀏覽器開啟TensorFlow Lite網站，下載指定Python版本的.whl檔來安裝TensorFlow Lite，其網址如下所示：

https://github.com/google-coral/pycoral/releases/

15-1-2 TensorFlow和TensorFlow Lite

認識與安裝TensorFlow Lite

- 請捲動視窗找到Python 3.9版，即cp39，最後CPU是win_amd64的.whl檔：【tflite_runtime-2.5.0.post1-cp39-cp39-win_amd64.whl】，請使用滑鼠【右】鍵複製.whl檔的URL網址，接著在Python開發環境安裝TensorFlow Lite（也可以如同第12-4-1節的Dlib，下載.whl檔來進行安裝），其指令如下所示：

```
pip install <Wheel檔的URL網址> Enter
```

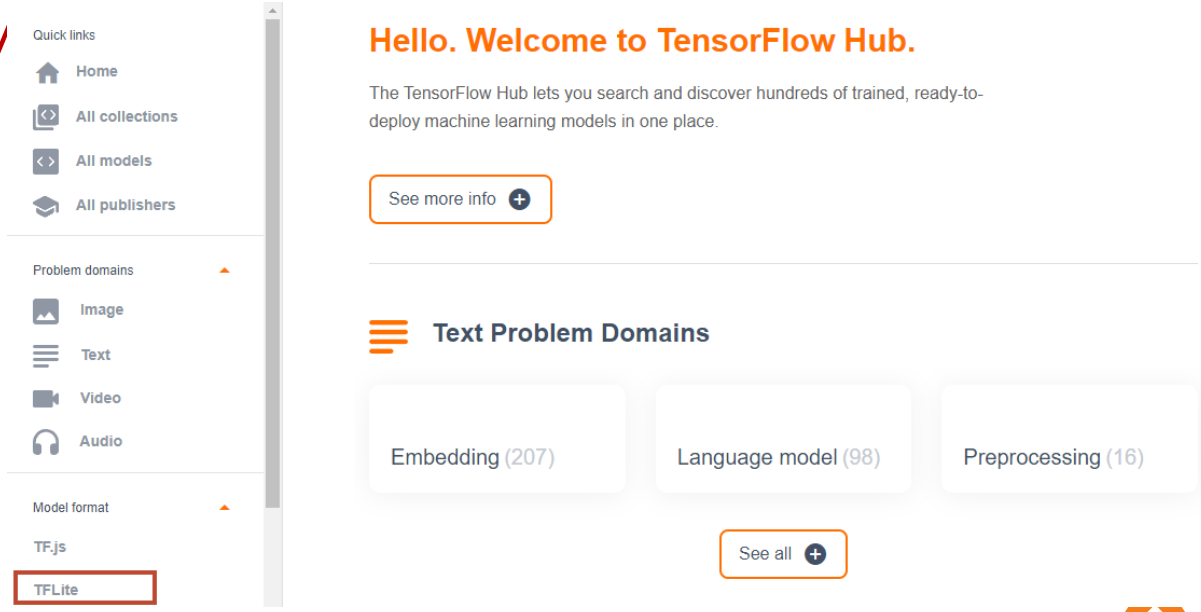
- 上述<Wheel檔的URL網址>請填入之前取得的URL網址。

15-1-2 TensorFlow和TensorFlow Lite

TensorFlow Lite的預訓練模型

•TensorFlow Lite提供多種預訓練模型，我們可以直接下載模型來建立Python人工智慧應用，在TensorFlow Hub提供支援TensorFlow.js和TensorFlow Lite的預訓練模型，其URL網址如下所示：

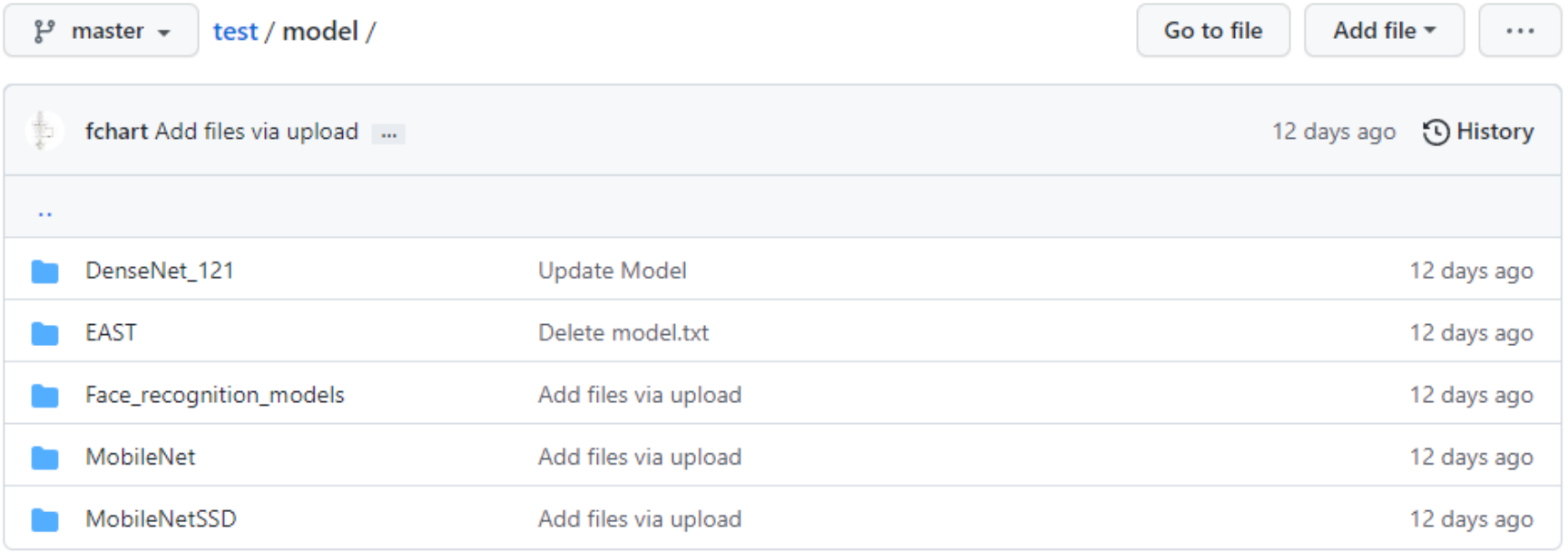
https://tfhub.dev/



15-1-3 下載本書使用的預訓練模型

- 為了方便讀者測試執行本章Python程式的人工智慧應用，筆者已經將本書使用的預訓練模型都上傳至GitHub，其URL網址如下所示：

  https://github.com/fchart/test/tree/master/model



15-2 影像分類

- 15-2-1 OpenCV DNN模組的影像分類
- 15-2-2 TensorFlow Lite的影像分類



15-2-1 OpenCV DNN模組的影像分類

- DenseNet是CNN卷積神經網絡的深度學習模型，OpenCV DNN模組可以直接使用DenseNet預訓練模型來執行影像分類。

下載DenseNet-Caffe預訓練模型

- 請進入本書預訓練模型的下載網頁後，下載【DenseNet_121】目錄下的DenseNet_121.caffemodel、DenseNet_121.prototxt.txt和classification_classes_ILSVRC2012.txt三個檔案至「ch15/models」目錄。
- 原預訓練模型檔案的下載網址：https://github.com/shicai/DenseNet-Caffe。

15-2-1 OpenCV DNN模組的影像分類

OpenCV DNN模組的影像分類：ch15-2-1.py

•Python程式在載入DenseNet-Caffe預訓練模型後，就可以分類影像，模型可以分類1000種不同的物體，影像尺寸是(224, 224)。Python程式的執行結果可以看到影像分類結果是一隻小獵犬（Beagle），可能性有67.43%，如下圖所示：



15-2-2 TensorFlow Lite的影像分類

- MobileNet是Google研究團隊提出的CNN模型，因為神經網路層的深度小很多，雖然準確度較差，但適合使用在計算能力較差的行動裝置來執行TensorFlow Lite影像分類。

下載MobileNet V1版預訓練模型

- 請進入本書預訓練模型的下載網頁後，下載【MobileNet】目錄下的mobilenet_v1_1.0_224_quantized_1_metadata_1.tflite和labels.txt二個檔案至「ch15/models」目錄。
- 原MobileNet V1版預訓練模型的下載網址（請下載內含標籤檔的Metadata版本）：https://tfhub.dev/tensorflow/lite-model/mobilenet_v1_1.0_224_quantized/1/metadata/1。

15-2-2 TensorFlow Lite的影像分類

TensorFlow Lite的影像分類：ch15-2-2.py

•Python程式在載入MobileNet V1預訓練模型後，就可以分類影像，模型可以分類1000種不同的物體，影像尺寸是(224, 224)。Python程式的執行結果可以看到影像分類結果是一隻小獵犬（Beagle），可能性有68.75%，如下圖所示：

```
>>> %Run ch15-2-2.py

  成功載入模型...
  影像尺寸: ( 224 , 224 )
  分類名稱 = beagle
  影像可能性 = 68.75 %
```

15-3 物體識別

- 15-3-1 OpenCV DNN模組的物體識別
- 15-3-2 TensorFlow Lite的物體識別



15-3 物體識別

- 物體識別（Object Recognition）是一種可以在影像的數位內容中，同時辨識出多種物體的電腦視覺應用領域，例如：在影像中辨識出人、車輛、椅子、石頭、建築物和各種動物等。事實上，物體識別就是在回答下列2個基本問題，如下所示：
  - 這是什麼東西？
  - 這個東西在哪裡？

15-3-1 OpenCV DNN模組的物體識別

- MobileNet是一種影像分類模型，MobileNet-SSD是基於MobileNet的SSD模型（Single Shot Multi-box Detector），可以在單一影像識別出多個物體名稱，並且一一取出其位置的方框。

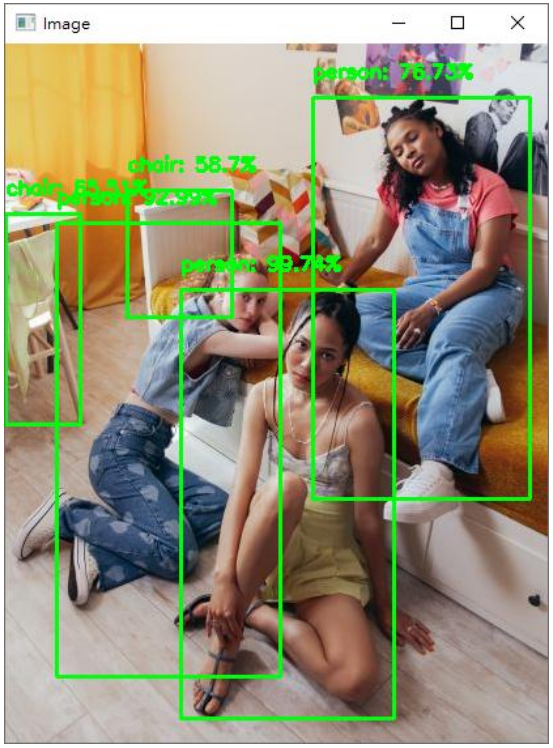
下載MobileNet-SSD預訓練模型

- 請進入本書預訓練模型的下載網頁後，下載【MobileNetSSD】目錄下的MobileNetSSD_deploy.caffemodel、models/MobileNetSSD_deploy.prototxt.txt和MobileNetSSD_labels.txt三個檔案至「ch15/models」目錄
- 原預訓練模型檔案的下載網址：https://github.com/PINTO0309/MobileNet-SSD-RealSense/tree/master/caffemodel/MobileNetSSD。

15-3-1 OpenCV DNN模組的物體識別

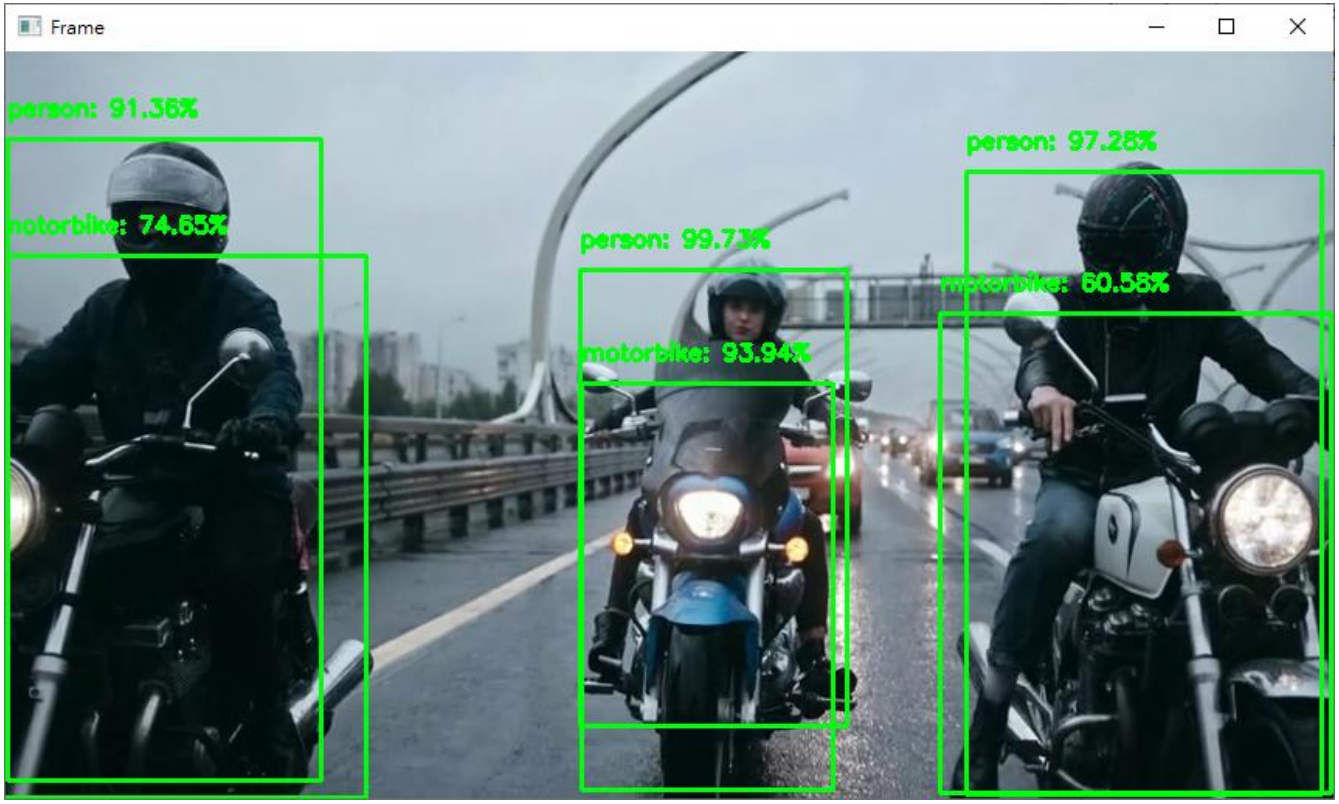
OpenCV DNN模組的物體識別：ch15-3-1.py

•Python程式是載入MobileNet-SSD預訓練模型來執行物體識別，可以辨識21種不同物體，影像尺寸是(300, 300)。Python程式的執行結果可以看到影像標示出識別物體的名稱和可能性，如下圖所示：



15-3-1 OpenCV DNN模組的物體識別

- Python程式：ch15-3-1a.py是使用OpenCV DNN模組的即時物體識別，可以在視訊的影格中識別出多個物體，如下圖所示：



15-3-2 TensorFlow Lite的物體識別

- TensorFlow Lite一樣可以使用MobileNet-SSD預訓練模型來建立物體識別，此模型可以辨識90種不同的物體。

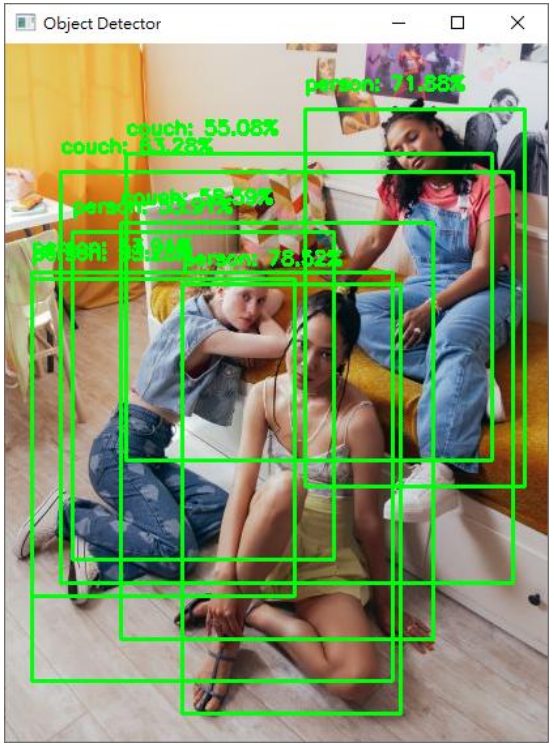
下載MobileNet-SSD預訓練模型

- 請進入本書預訓練模型的下載網頁後，下載【MobileNetSSD】目錄下的lite-model_ssd_mobilenet_v1_1_metadata_2.tflite和labelmap.txt二個檔案至「ch15/models」目錄。
- 原SSD MobileNet V1預訓練模型的下載網址（請下載內含標籤檔的Metadata版本）：https://tfhub.dev/tensorflow/lite-model/ssd_mobilenet_v1/1/metadata/2。

15-3-2 TensorFlow Lite的物體識別

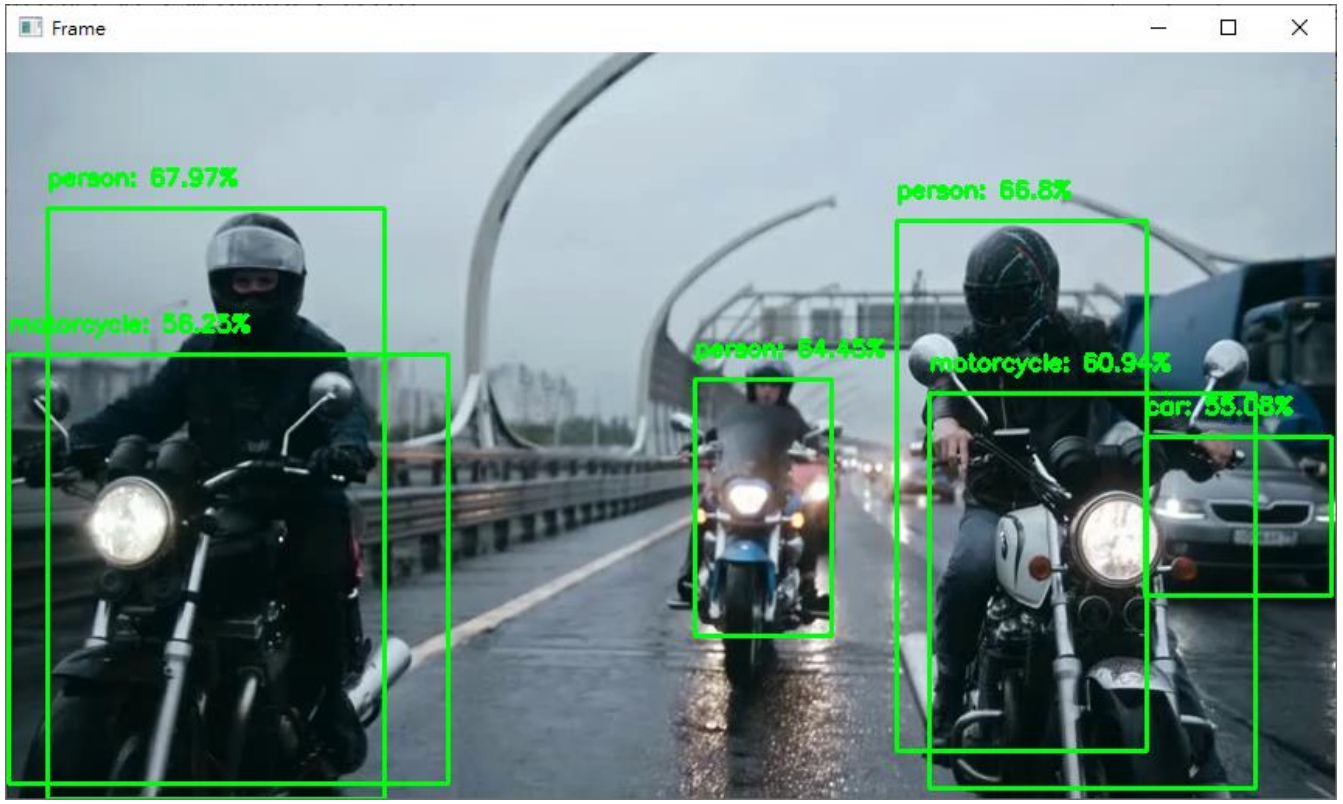
TensorFlow Lite的物體識別：ch15-3-2.py

•Python程式是載入SSD MobileNet V1預訓練模型來進行90種物體的物體識別，影像尺寸是(300, 300)。Python程式的執行結果可以看到使用方框標示出識別的物體、名稱和可能性百分比，如下圖所示：



15-3-2 TensorFlow Lite的物體識別

- Python程式：ch15-3-2a.py是使用TensorFlow Lite的即時物體識別，可以在視訊的影格中識別出多個物體（因為識別速度有些慢，視訊會有很明顯的停頓），如下圖所示：



15-4 文字偵測

- EAST（An Efficient and Accurate Scene Text Detector）是一種高效且正確率高的文字偵測預訓練模型，在這一節我們準備使用OpenCV DNN模組來建立EAST文字偵測。

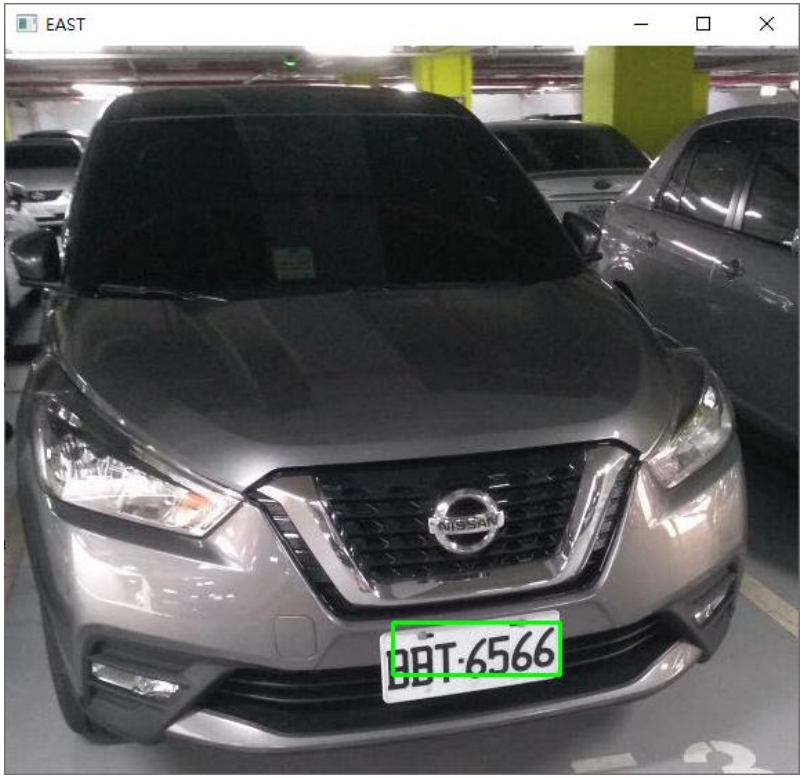
下載EAST預訓練模型

- 請進入本書預訓練模型的下載網頁後，下載【EAST】目錄下的frozen_east_text_detection.pb檔案至「ch15/models」目錄。
- 原預訓練模型的下載網址：https://github.com/ZER-0-NE/EAST-Detector-for-text-detection-using-OpenCV。

15-4 文字偵測

EAST文字偵測：ch15-4.py

•Python程式是載入EAST預訓練模型來進行文字偵測，可以在影像中使用方框標示偵測出的文字內容。Python程式的執行結果可以看到使用方框標示出的文字區域，以此例就是車牌的文字區域，如右圖所示：



15-5 Tesseract-OCR文字識別

- 15-5-1 下載安裝Tesseract-OCR和語言包
- 15-5-2 Tesseract-OCR基本使用

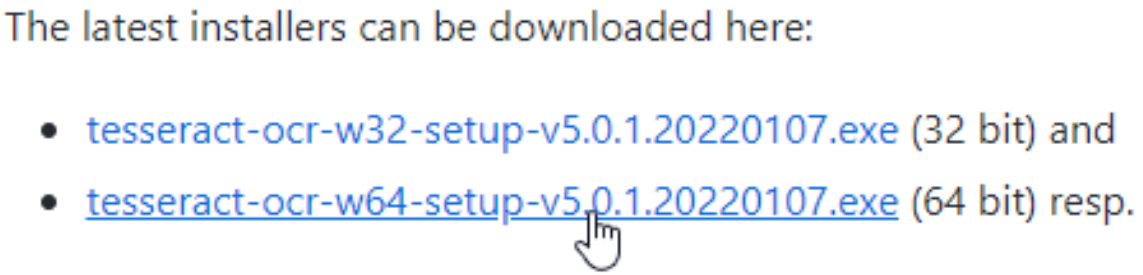


15-5 Tesseract-OCR文字識別

- OCR是Optical Character Recognition光學字元識別的縮寫，可以自動識別出影像中的文字將它轉換成字串。Tesseract-OCR引擎最早是HP實驗室在1985年開發，目前Tesseract已經是Google開源項目，提供一個命令列工具來執行OCR操作，讓我們從影像識別出之中的文字。

15-5-1 下載安裝Tesseract-OCR和語言包

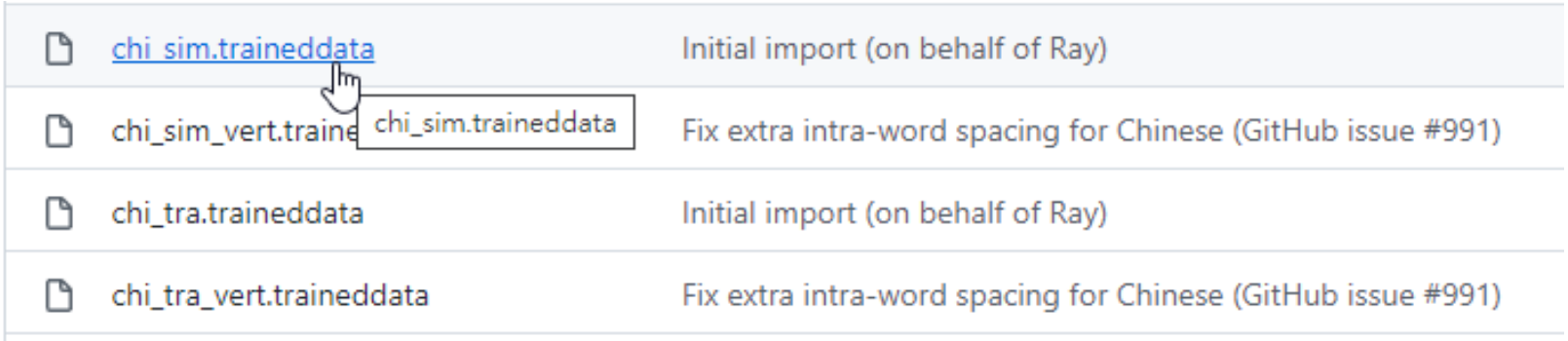
- Windows版Tesseract-OCR安裝程式的下載網址：https://github.com/UB-Mannheim/tesseract/wiki，如下圖所示：



- 在本書是下載安裝64位元版本，成功下載後請執行程式來安裝Tesseract，安裝過程首先請選【English】語言，然後使用預設值按【Next】鈕後，再按【I Agree】鈕同意授權後，按3次【Next】鈕，再按【Install】鈕開始安裝，最後按【Next】和【Finish】鈕完成安裝。

15-5-1 下載安裝Tesseract-OCR和語言包

- 因為Tesseract-OCR預設只提供英文語言包，我們需要自行至語言包網址：https://github.com/tesseract-ocr/tessdata_best下載繁體和簡體中文語言包，如下圖所示：



15-5-2 Tesseract-OCR基本使用

- Python程式需要使用pytesseract套件來執行Tesseract-OCR命令列工具。在Python開發環境安裝pytesseract套件的命令列指令，如下所示：

  pip install pytesseract==0.3.8 Enter

- 當成功安裝pytesseract後，在Python程式可以匯入此套件，如下所示：

  import pytesseract

15-5-2 Tesseract-OCR基本使用

使用pytesseract將影像轉文字：ch15-5-2.py

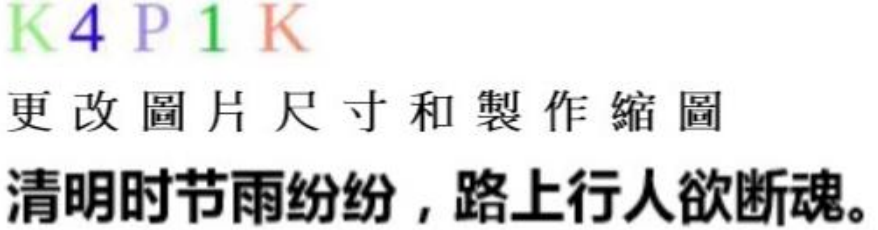
```
import cv2
import pytesseract

pytesseract.pytesseract.tesseract_cmd=
      "C:\\Program Files\\Tesseract-OCR\\tesseract.exe"
img = cv2.imread("images/number.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
text = pytesseract.image_to_string(img, lang="eng")
print(text.strip())
```

15-5-2 Tesseract-OCR基本使用

使用pytesseract將影像轉文字：ch15-5-2.py

```
img = cv2.imread("images/traditional.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
text = pytesseract.image_to_string(img, lang="chi_tra")
print(text.strip())
img = cv2.imread("images/simple.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
text = pytesseract.image_to_string(img, lang="chi_sim")
print(text.strip())
```



15-5-2 Tesseract-OCR基本使用

辨識垂直書寫的中文字：ch15-5-2a.py

•Python程式碼在載入圖檔後，使用pytesseract.image_to_string()方法將影像中的文字轉換成字串，lang參數是"chi_tra_vert"垂直書寫中文語言，如下所示：

```
pytesseract.pytesseract.tesseract_cmd=
    "C:\\Program Files\\Tesseract-OCR\\tesseract.exe"
img = cv2.imread("images/traditional2.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
text = pytesseract.image_to_string(img,
lang="chi_tra_vert")
print(text.strip())
```

15-5-2 Tesseract-OCR基本使用

在影像框出和顯示每一個字元內容：ch15-5-2b.py

```
img = cv2.imread("images/number.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
img_w = img.shape[1]
img_h = img.shape[0]
boxes = pytesseract.image_to_boxes(img)
print(boxes)
```

```
>>> %Run ch15-5-2b.py

K 16 52 42 80 0
4 49 52 70 80 0
P 87 53 107 79 0
1 125 52 140 80 0
K 159 52 187 80 0
```

15-5-2 Tesseract-OCR基本使用

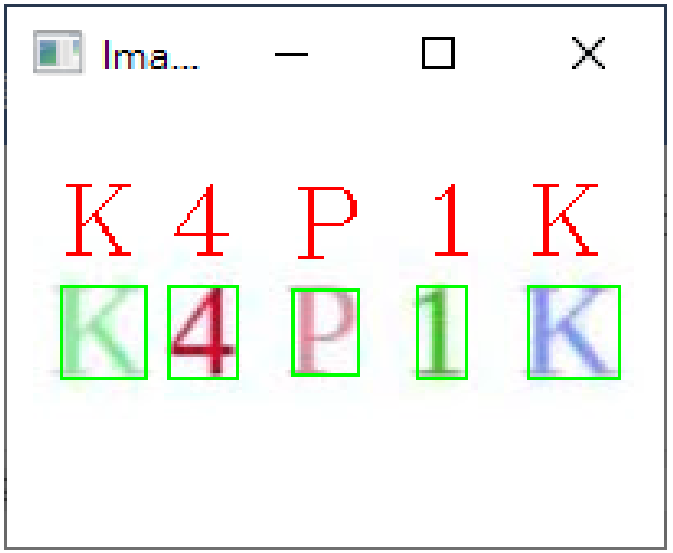
在影像框出和顯示每一個字元內容：ch15-5-2b.py

```
for box in boxes.splitlines():
    box = box.split(" ")
    character = box[0]
    x = int(box[1])
    y = int(box[2])
    x2 = int(box[3])
    y2 = int(box[4])
    cv2.rectangle(img, (x, img_h - y),
                  (x2, img_h - y2), (0, 255, 0), 1)
```

15-5-2 Tesseract-OCR基本使用

在影像框出和顯示每一個字元內容：ch15-5-2b.py

```
    cv2.putText(img, character, (x, img_h - y2 - 10),
                cv2.FONT_HERSHEY_COMPLEX, 1, (0, 0, 255), 1)
cv2.imshow("Image", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



第16章 AI整合實戰（二）：Teachable Machine訓練模型與車牌辨識

- 16-1 AI整合實戰：Teachable Machine訓練機器學習模型
- 16-2 AI整合實戰：車牌辨識



16-1 AI整合實戰：Teachable Machine訓練機器學習模型

- 16-1-1 使用Teachable Machine訓練機器學習模型
- 16-1-2 使用TensorFlow Lite識別剪刀、石頭或布



16-1 AI整合實戰：Teachable Machine訓練機器學習模型

- Teachable Machine是Google推出的網頁工具，你不需要專業知識和撰寫任何程式碼，就可以自行訓練所需的機器學習模型，支援影像分類、姿勢辨識和聲音分類。
- 在這一節我們準備使用Teachable Machine訓練一個機器學習模型，可以分類剪刀、石頭和布的影像。然後，建立Python程式使用TensorFlow Lite載入此機器學習模型進行推論來分類影像，和使用Webcam即時分類影像是剪刀、石頭或布。

16-1-1 使用Teachable Machine訓練機器學習模型

- 我們可以使用Teachable Machine網頁工具來訓練一個機器學習模型，能夠分類剪刀、石頭或布的影像。
  - 步驟一：新增專案和選擇機器學習模型的類型
  - 步驟二：建立分類和新增各分類的樣本影像
  - 步驟三：訓練模型
  - 步驟四：預覽、測試與優化模型
  - 步驟五：匯出TensorFlow Lite模型
  - 步驟六：儲存或下載專案

16-1-2 使用TensorFlow Lite識別剪刀、石頭或布

- 我們只需整合第15-2-2節的TensorFlow Lite影像分類，就可以改用Teachable Machine機器學習模型來辨識影像是剪刀、石頭或布。首先請將第16-1-1節匯出的TensorFlow Lite模型解壓縮後，複製解壓縮的2個檔案model.tflite和labels.txt至「ch16/models」目錄。
- Python程式：ch16-1-2.py在讀取影像後，使用TensorFlow Lite載入Teachable Machine機器學習模型來進行影像分類，可以辨識出影像是剪刀、石頭或布，其執行結果如右圖所示：



16-2 AI整合實戰：車牌辨識

- 16-2-1 Tesseract-OCR的AI 車牌辨識
- 16-2-2 EasyOCR的安裝與使用
- 16-2-3 EasyOCR的AI 車牌辨識



16-2 AI整合實戰：車牌辨識

- Python程式只需整合第15-4節的EAST文字偵測和第15-5節的Tesseract-OCR文字識別，就可以輕鬆打造AI車牌辨識，或是直接使用EasyOCR套件來快速建立AI 車牌辨識。

16-2-1 Tesseract-OCR的AI 車牌辨識

- Python 程式使用Tesseract-OCR建立AI 車牌辨識基本上分成兩部分：第一部分是EAST文字偵測；第二部分才是Tesseract-OCR文字識別。

下載EAST預訓練模型

- 請進入本書GitHub預訓練模型的下載網頁，然後下載【EAST】目錄下的frozen_east_text_detection.pb檔案至「ch16/models」目錄。原預訓練模型的下載網址，如下所示：

  https://github.com/ZER-0-NE/EAST-Detector-for-text-detection-using-OpenCV

16-2-1 Tesseract-OCR的AI 車牌辨識

Tesseract-OCR的AI 車牌辨識（一）：ch16-2-1.py

- Python程式是使用EAST預訓練模型進行車輛影像的文字偵測，即偵測出車牌所在位置（在影像中的車牌是斜的），在剪裁出車牌區域的影像後，再使用Tesseract-OCR文字識別來辨識出車牌文字，其執行結果如下所示：

```
>>> %Run ch16-2-1.py
偵測和辨識出車牌文字!
BBT:6566
```

- 上述執行結果可以看到辨識出的車牌文字BBT:6566，剪裁出的車牌影像，如下圖所示：



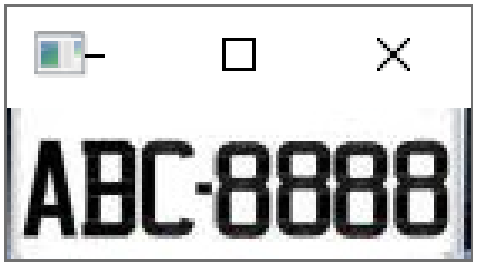
16-2-1 Tesseract-OCR的AI 車牌辨識

Tesseract-OCR的AI 車牌辨識（二）：ch16-2-1a.py

•Python程式是使用EAST預訓練模型進行車輛影像的文字偵測，即偵測出車牌所在位置（車牌影像是正的），在剪裁出車牌影像後，再使用Tesseract-OCR文字識別來辨識車牌文字，其執行結果如下所示：

```
>>> %Run ch16-2-1a.py
偵測和辨識出車牌文字!
ABC-8888
```

•上述執行結果可以看到辨識出的車牌文字ABC-8888，剪裁出的車牌影像，如下圖所示：



16-2-2 EasyOCR的安裝與使用

- EasyOCR是一套基於深度學習模型的文字偵測和OCR文字識別的Python套件，這是名為Jaided AI的OCR公司，使用PyTorch框架所開發的套件。目前EasyOCR已經支援超過70國語言的文字識別，大部分正常影像的文字識別，EasyOCR都有非常高的識別準確度。

16-2-2 EasyOCR的安裝與使用

在Python開發環境安裝EasyOCR套件

- 在安裝好OpenCV後，就可以安裝PyTorch框架，PyTorch框架需要安裝三個套件，其命令列指令如下所示：

```
pip install torch==1.10.2 Enter
pip install torchvision==0.11.3 Enter
pip install torchaudio==0.10.2 Enter
```

- 然後，可以安裝EasyOCR，其命令列指令如下所示：

```
pip install easyocr==1.4.1 Enter
```

- 當成功安裝EasyOCR後，在Python程式可以匯入此套件，如下所示：

```
import easyocr
```

16-2-2 EasyOCR的安裝與使用

EasyOCR的基本使用：ch16-2-2.py

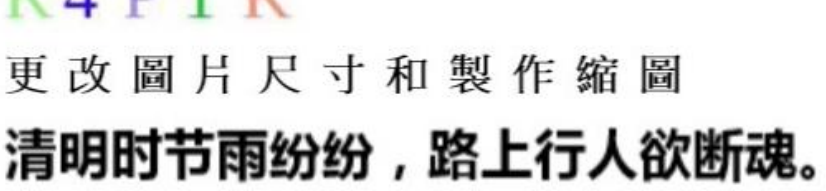
```
import easyocr
import cv2

reader = easyocr.Reader(["en"])
result = reader.readtext("images/number.jpg")
print(result)
reader = easyocr.Reader(["ch_sim", "en"])
img = cv2.imread("images/simple.jpg")
result = reader.readtext(img)
print(result)
```

16-2-2 EasyOCR的安裝與使用

EasyOCR的基本使用：ch16-2-2.py

```
reader = easyocr.Reader(["ch_tra", "en"])
with open("images/traditional.jpg", "rb") as f:
    img = f.read()
result = reader.readtext(img)
print(result)
```




```
>>> %Run ch16-2-2.py
CUDA not available - defaulting to CPU. Note: This module is much faster with a GPU.
[([[7, 47], [193, 47], [193, 89], [7, 89]], 'K4 P 1 K', 0.398470296910955)]
CUDA not available - defaulting to CPU. Note: This module is much faster with a GPU.
[([[5, 7], [239, 7], [239, 51], [5, 51]], '清明时节雨纷纷', 0.9478759638894352), ([[25
9, 7], [507, 7], [507, 51], [259, 51]], '路上行人欲断魂。', 0.9462621182651273)]
CUDA not available - defaulting to CPU. Note: This module is much faster with a GPU.
[([[33, 35], [449, 35], [449, 71], [33, 71]], '更改圖片尺寸和製作縮圖', 0.540436178672
3678)]
```

16-2-2 EasyOCR的安裝與使用

EasyOCR文字偵測：ch16-2-2a.py

```
import easyocr
import numpy as np
import cv2

img = cv2.imread("images/sample.jpg")
reader = easyocr.Reader(["ch_tra", "en"])
horizontal_list, free_list = reader.detect(img)
```

16-2-2 EasyOCR的安裝與使用

EasyOCR文字偵測：ch16-2-2a.py

```
for box in horizontal_list[0]:
    print(box)
    cv2.rectangle(img, (box[0], box[2]), (box[1], box[3]),
                  (0, 0, 255), 3)
cv2.imshow("Detection", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



16-2-2 EasyOCR的安裝與使用

EasyOCR文字識別：ch16-2-2c.py

```
import easyocr
import numpy as np
import cv2

boxes = [[32, 159, 8, 47],
         [6, 191, 43, 81],
         [30, 178, 86, 118]]
img = cv2.imread("images/sample.jpg")
reader = easyocr.Reader(["ch_tra", "en"])
```

16-2-2 EasyOCR的安裝與使用

EasyOCR文字識別：ch16-2-2b.py

```
results = reader.recognize(img, horizontal_list=boxes,
                           free_list=[])
for result in results:
    print(result[0])
    print(result[1])
    print(result[2])
```

```
>>> %Run ch16-2-2b.py
  CUDA not available - defaulting to CPU. Note: This module is much faster with a GPU.
  [[32, 8], [159, 8], [159, 47], [32, 47]]
  OpenCV
  0.9790066573923996
  [[6, 43], [191, 43], [191, 81], [6, 81]]
  Python程式設計
  0.8910885101124186
  [[30, 86], [178, 86], [178, 118], [30, 118]]
  DAT-4567
  0.6394378746862207
```

16-2-3 EasyOCR的AI 車牌辨識

- 因為EasyOCR套件同時支援文字偵測和文字識別，Python程式：ch16-2-3.py就是使用EasyOCR進行車牌文字偵測和OCR文字識別來辨識車牌文字，其執行結果可以看到辨識出的車牌文字BBT-6566，如下圖所示：

